

Parameter	Description	Example
ClientSecret	The client secret of the VCF PowerCLI OAuth client created in step 2	"<secret_value>"

When this script runs, VCF PowerCLI uses VMware Identity Services to send an authentication request to the authentication server. A Web browser opens and prompts the user to authorize the request of the client application. On confirmation, VMware Identity Services interacts with the authentication server to obtain the access and ID tokens, JSON Web Token (JWT). Based on this output VCF PowerCLI creates a new OAuth security context.

4. Exchange the OAuth 2.0 security context for a SAML security context.

```
$samlSecContext = New-VISamlSecurityContext -VCenterServer '<vcenter_fqdn>' -OAuthSecurityContext $oauthSecContext
```

5. Connect to your vCenter system by using the SAML security context.

```
Connect-VIServer -Server '<vcenter_fqdn>' -SamlSecurityContext $samlSecContext
```

Unattended Logins to a Federated vCenter System

Unattended logins to a federated vCenter system represent a security threat and must be allowed with great caution.

When you register the vCenter OAuth client with the OAuth provider, you can create a configuration that allows password grant type (Resource Owner Password Credentials grant type) for some users. Such users can then perform unattended logins to the federated vCenter system.

Note: This flow exposes the user credentials to the client application, VCF PowerCLI, and to vCenter. To minimize the security risk, allow access only to service users with limited permissions. For more information, see: <https://oauth.net/2/grant-types/password/>.

To connect to the vCenter system, you can directly use the `Connect-VIServer` cmdlet in the following way:

```
Connect-VIServer -Server '<vcenter_fqdn>' -User '<user_name>' -Password '<user_password>'
```

Create vSphere Inventory Objects

By using VCF PowerCLI cmdlets, you can automate creating different inventory objects on vSphere.

Verify that you are connected to a system.

1. Get the inventory root folder and create a new folder named `Folder` in it.

```
$folder = Get-Folder -NoRecursion | New-Folder -Name Folder
```

2. Create a new data center named `DC` in the `Folder` folder.

```
New-Datacenter -Location $folder -Name DC
```

3. Create a folder named `Folder1` under `DC`.

```
Get-Datacenter DC | New-Folder -Name Folder1
$folder1 = Get-Folder -Name Folder1
```

4. Create a new cluster `Cluster1` in the `Folder1` folder.

```
New-Cluster -Location $folder1 -Name Cluster1 -DrsEnabled -DrsAutomationLevel FullyAutomated
```

Distributed Resource Scheduler (DRS) is a feature that provides automatic allocation of cluster resources.

5. Add a host in the cluster by using the `Add-VMHost` command, and provide credentials when prompted.

```
$vmhost1 = Add-VMHost -Name 10.23.112.345 -Location (Get-Cluster Cluster1)
```

6. Create a resource pool in the root resource pool of the cluster.

```
$myClusterRootRP = Get-Cluster Cluster1 | Get-ResourcePool -Name Resources
New-ResourcePool -Location $myClusterRootRP -Name MyRP1 -CpuExpandableReservation $true -CpuReservation-
Mhz 500 -CpuSharesLevel high -MemExpandableReservation $true -MemReservationGB 1 -MemSharesLevel high
```

7. Create a virtual machine asynchronously.

```
$vmCreationTask = New-VM -Name VM2 -VMHost $vmhost1 -ResourcePool MyRP01 -DiskGB 100 -MemoryGB 2 -RunAsync
```

The `RunAsync` parameter indicates that the command runs asynchronously. This means that in contrast to a synchronous operation, you do not have to wait for the process to complete before supplying the next command at the command line.

vCenter and Host Management

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vCenter and host management.

Get a List of Hosts on a vCenter System and View Their Properties

With VCF PowerCLI, you can get information about all available hosts in a data center and view their properties.

Verify that you are connected to a system.

1. Get a list of all hosts that are part of a data center.

```
Get-Datacenter DC | Get-VMHost | Format-Custom
```

2. View the properties of the first host in the data center.

```
Get-Datacenter DC | Get-VMHost | Select-Object -First 1 | Format-Custom
```

3. View the Name and the OverallStatus properties of the hosts in the DC data center.

```
Get-Datacenter DC | Get-VMHost | Get-View | Format-Table -Property Name, OverallStatus
-AutoSize
```

4. View all hosts and their properties, and save the results to a file.

```
Get-Datacenter DC | Get-VMHost | Format-Custom | Out-File -FilePath hosts.txt
```

5. View a list of the hosts that are in maintenance mode and can be configured for vMotion operations.

```
Get-VMHost -State maintenance | Get-View | Where-Object -FilterScript { $_.capability -ne
$null -and $_.capability.vmotionSupported }
```

Add a Standalone Host to a vCenter System

You can add standalone hosts to a vCenter system by using the `Add-VMHost` cmdlet. After adding the hosts, you will be able to manage them through the vCenter system.

Verify that you are connected to a system.

1. View all hosts on the vCenter system that you have established a connection with.

```
Get-VMHost
```

2. Add the *Host* standalone host.

```
Add-VMHost -Name Host -Location (Get-Datacenter DC) -User root -Password pass
```

Set the License Key for a Host on vCenter

You can set the license key for a host on a vCenter system by using the `LicenseKey` parameter of the `Set-VMHost` cmdlet.

Verify that you are connected to a system.

1. Save the *Host* host object as a variable.

```
$vmhost = Get-VMHost -Name Host
```

2. Set the host to evaluation mode or provide a valid license key.

- Set the host to evaluation mode by providing the evaluation key.

```
Set-VMHost -VMHost $vmhost -LicenseKey 00000-00000-00000-00000-00000
```

- Provide a valid license key.

```
Set-VMHost -VMHost $vmhost -LicenseKeyYour_license_key
```

Activate Maintenance Mode for a Host on vCenter

To complete some specific administration tasks, you might need to activate maintenance mode for a host. On vCenter, you can activate maintenance mode by using the `Set-VMHost` cmdlet.

Verify that you are connected to a system.

1. Save the *Host* host object as a variable.

```
$vmhost = Get-VMHost -Name Host
```

2. Get the cluster to which *Host* belongs and save the cluster object as a variable.

```
$vmhostCluster = Get-Cluster -VMHost $vmhost
```

3. Start a task that activates maintenance mode for the *Host* host and save the task object as a variable.

```
$updateHostTask = Set-VMHost -VMHost $vmhost -State "Maintenance" -RunAsync
```

Note: If the host is not automated or is partially automated and has powered-on virtual machines running on it, you must use the `RunAsync` parameter and wait until all powered-on virtual machines are relocated or powered off before applying DRS recommendations.

4. Get and apply the recommendations generated by DRS.

```
Get-DrsRecommendation -Cluster $vmhostCluster | where {$_.Reason -eq "Host is entering maintenance mode"} | Invoke-DrsRecommendation
```

5. Get the task output object and save it as a variable.

```
$myUpdatedHost = Wait-Task $updateHostTask
```

Change the Host Advanced Configuration Settings on vCenter

You can modify host configuration, including advanced settings related to virtual machine migration, and apply them to another host.

Verify that you are connected to a system.

1. Change the migration timeout for the *ESXHost1* host.

```
Get-VMHost ESXHost1 | Set-VmHostAdvancedConfiguration -Name Migrate.NetTimeout -Value
( [system.int32] 10 )
```

2. Enable creation of a checksum of the virtual machines memory during the migration.

```
Get-VMHost ESXHost1 | Set-VmHostAdvancedConfiguration -Name Migrate.MemChksum -Value
( [system.int32] 1 )
```

3. Get the *ESXHost1* host migration settings.

```
$migrationSettings = Get-VMHost ESXHost1 | Get-VmHostAdvancedConfiguration -Name Migrate.*
```

4. Apply the migration settings to *ESXHost2*.

```
Set-VmHostAdvancedConfiguration -VMHost ESXHost2 -Hashtable $migrationSettings
```

Create a Host Profile on a vCenter System

The VMware Host Profiles feature enables you to create standard configurations for ESX hosts. With VCF PowerCLI, you can automate creation and modifying of host profiles.

Verify that you are connected to a host that runs 4.0 or later.

1. Get the host named *Host1* and store it in the *\$vmhost* variable.

```
$vmhost = Get-VMHost Host1
```

2. Create a profile based on the *Host1* host.

```
New-VMHostProfile -Name MyHostProfile01 -Description "This is my test profile based on
Host1." -ReferenceHost $vmhost
```

3. Get the newly created host profile.

```
$hpl = Get-VMHostProfile -Name MyHostProfile01
```

4. Change the description of the *HostProfile1* host profile.

```
Set-VMHostProfile -Profile $hpl -Description "This is my old test host profile based on
Host1."
```

Apply a Host Profile to a Host on vCenter

To simplify operational management of large-scale environments, you can apply standard configurations called host profiles to hosts on vCenter. If you want to set up a host to use the same host profile as a reference host, you can attach the host to a profile.

Verify that you are connected to a host that runs 4.0 or later.

1. Get the *Host2* host.

```
$vmhost2 = Get-VMHost Host2
```

2. Attach the *Host2* host to the *HostProfile1* host profile.

```
Set-VMHost -VMHost $vmhost2 -Profile HostProfile1
```

3. Verify that the *Host2* host is compliant with the *HostProfile1* profile.

```
Test-VMHostProfileCompliance -VMHost $vmhost2
```

The output of this command contains the noncompliant settings of the host, if any.

4. Apply the profile to the *Host2* host.

```
$neededVariables = Invoke-VMHostProfile -Entity $vmhost2 -Profile $hpl -Confirm:$false
```

The *\$neededVariables* variable contains the names of all required variables and their default or current values, as returned by the server. Otherwise, the *\$neededVariables* variable contains the name of the host on which the profile has been applied.

Create a vSphere Role and Assign Permissions to a User

With VCF PowerCLI, you can automate management of vSphere permissions, roles, and privileges.

Verify that you are connected to a system.

Note:

vSphere permissions determine your level of access to vCenter, and ESX hosts. Privileges define individual rights to perform actions and access object properties. Roles are predefined sets of privileges.

1. Get the privileges of the **Readonly** role.

```
$readOnlyPrivileges = Get-VIPrivilege -Role Readonly
```

2. Create a new role with custom privileges.

```
$role1 = New-VIRole -Privilege $readOnlyPrivileges -Name Role1
```

3. Add the **PowerOn** privileges to the new role.

```
$powerOnPrivileges = Get-VIPrivilege -Name "PowerOn"
$role1 = Set-VIRole -Role $role1 -AddPrivilege $powerOnPrivileges
```

4. Create a permission and apply it to a vSphere root object.

```
$rootFolder = Get-Folder -NoRecursion
$permission1 = New-VIPermission -Entity $rootFolder -Principal "user" -Role readonly -Propagate
```

The *Principal* parameter accepts both local and domain users and groups if the vCenter system is joined in AD.

5. Update the new permission with the custom role.

```
$permission1 = Set-VIPermission -Permission $permission1 -Role $role1
```

You created a new role and assigned permissions to a user.

Modify the vCenter Email Configuration

You can modify the email configuration settings of a vCenter.

Verify that you are connected to a system.

1. View the current email configuration settings of the vCenter from the *\$srv* variable.

```
Get-AdvancedSetting -Entity $srv -Name mail.*
```

2. Update the SMTP server name and port.

```
Get-AdvancedSetting -Entity $srv -Name mail.smtp.server | Set-AdvancedSetting -Value smtp-
p.vmware.com
Get-AdvancedSetting -Entity $srv -Name mail.smtp.port | Set-AdvancedSetting -Value 25
```

Modify the vCenter SNMP Configuration

To use SNMP, you must first configure the SNMP settings of the vCenter.

Verify that you are connected to a system.

1. View the current SNMP configuration settings of the vCenter from the `$srv` variable.

```
Get-AdvancedSetting -Entity $srv -Name snmp.*
```

2. Modify the SNMP receiver data.

```
Get-AdvancedSetting -Entity $srv -Name snmp.receiver.2.community | Set-AdvancedSetting
-Value public
Get-AdvancedSetting -Entity $srv -Name snmp.receiver.2.enabled | Set-AdvancedSetting -Value $true
Get-AdvancedSetting -Entity $srv -Name snmp.receiver.2.name | Set-AdvancedSetting -Value 192.168.1.10
```

Now you can use SNMP with vCenter.

Compute Cluster Administration

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in compute cluster administration.

Create and Modify Advanced Settings for a Cluster

You can customize the behavior of a cluster on a vCenter system by creating and modifying custom advanced settings for it.

Verify that you are connected to a system.

1. Create a new cluster named `Cluster`.

```
$cluster = New-Cluster -Name Cluster -Location (Get-Datacenter Datacenter)
```

2. Create two advanced settings for the new cluster.

```
$setting1 = New-AdvancedSetting -Type "ClusterHA" -Entity $cluster -Name 'das.defaultfailoverhost' -Value '192.168.10.1'
$setting2 = New-AdvancedSetting -Type "ClusterHA" -Entity $cluster -Name 'das.isolationaddress' -Value '192.168.10.2'
```

3. Modify the value of the advanced setting stored in the `$setting2` variable.

```
Get-AdvancedSetting -Entity $cluster -Name 'das.isolationaddress' | Set-AdvancedSetting
-Value '192.168.10.3' -Confirm:$false
```

4. Create another advanced setting.

```
New-AdvancedSetting -Entity $cluster -Name 'das.allowNetwork[Service Console]' -Value $true
-Type 'ClusterHA'
```

5. Get the Service Console setting and store it in a variable.

```
$setting3 = Get-AdvancedSetting -Entity $entity -Name 'das.allowNetwork `[Service Console`']'
```

The ``` character is used to escape the wildcard characters `[` and `]` in the advanced setting name.

Create a New VM-VM DRS Rule

You can create a VM-VM DRS affinity rule within a cluster.

- Verify that you are connected to a system.
- Verify that virtual machines and hosts exist within a cluster with enabled DRS in the vCenter environment.

1. Get the virtual machines for the VM-VM DRS rule.

```
$affinityVMs = Get-VM "VM1", "VM2"
```

2. Get the cluster where you want to create the rule.

```
$cluster = Get-Cluster "MyCluster"
```

3. Create the VM-VM DRS rule within the *MyCluster* cluster.

```
New-DrsRule -Cluster $cluster -Name "AffinityRule1" -KeepTogether $true -VM $affinityVMs
```

Create a New VM-VMHost DRS Rule

You can create a VM-VMHost DRS rule within a cluster after creating a VM DRS cluster group and a VMHost DRS cluster group.

- Verify that you are connected to a system.
- Verify that virtual machines and hosts exist within a cluster with enabled DRS in the vCenter environment.

1. Get the virtual machines for the VM DRS cluster group.

```
$vms = Get-VM "VM1", "VM2"
```

2. Get the hosts for the VMHost DRS cluster group.

```
$vmHosts = Get-VMHost "hostname1", "hostname2"
```

3. Get the cluster where you want to create the rule.

```
$cluster = Get-Cluster "MyCluster"
```

4. Create a VM DRS cluster group.

```
$vmGroup = New-DrsClusterGroup -Name "MyVmsDrsClusterGroup" -VM $vms -Cluster $cluster
```

5. Create a VMHost DRS cluster group.

```
$vmHostGroup = New-DrsClusterGroup -Name "MyVmHostsDrsClusterGroup" -VMHost $vmHosts -Cluster $cluster
```

6. Create the VM-VMHost DRS rule by using the newly created VM DRS cluster group and VMHost DRS cluster group.

```
New-DrsVMHostRule -Name "MyDrsRule" -Cluster $cluster -VMGroup $vmGroup -VMHostGroup $vmHostGroup -Type "MustRunOn"
```

Virtual Machine Administration

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in virtual machine administration.

Manage Virtual Machines on vSphere

With VCF PowerCLI, you can automate various administration tasks on virtual machines, for example retrieving information, shutting down and powering off virtual machines.

1. View all virtual machines on the target system.

```
Get-VM
```

2. Save the name and the power state properties of the virtual machines in the *ResourcePool* resource pool into a file named `myVMProperties.txt`.

```
$respool = Get-ResourcePool ResourcePool
Get-VM -Location $respool | Select-Object Name, PowerState > myVMProperties.txt
```

3. Start the *VM* virtual machine.

```
Get-VM VM | Start-VM
```

4. Get information of the guest OS of the *VM* virtual machine.

```
Get-VMGuest VM | fc
```

5. Shut down the OS of the *VM* virtual machine.

```
Stop-VMGuest VM
```

6. Power off the *VM* virtual machine.

```
Stop-VM VM
```

7. Move the virtual machine *VM* from the *Host01* host to the *Host02* host.

```
Get-VM -Name VM -Location Host01 | Move-VM -Destination Host02
```

Note: If the virtual machine you want to move across hosts is powered on, it must be located on a shared storage registered as a datastore on both the original and the new host.

Create Virtual Machines on vCenter Using an XML Specification File

You can use a specification provided in an XML file to automate the creation of virtual machines on vCenter.

Verify that you are connected to a system.

The `myVM.xml` file must be present with the following content:

```
<CreateVM>
<VM>
<Name>MyVM1</Name>
<HDDCapacity>100</HDDCapacity>
</VM>
<VM>
<Name>MyVM2</Name>
<HDDCapacity>100</HDDCapacity>
</VM>
</CreateVM>
```

1. Read the content of the `myVM.xml` file.

```
[xml]$s = Get-Content myVM.xml
```

2. Create the virtual machines.

```
$s.CreateVM.VM | foreach {New-VM -VMHost $vmHost1 -Name $_.Name -DiskGB $_.HDDCapacity}
```


Manage Virtual Machine Templates on vCenter

You can use VCF PowerCLI to create virtual machines templates and convert them to virtual machines on vCenter.

Verify that you are connected to a system.

Note: A virtual machine template is a reusable image created from a virtual machine. The template, as a derivative of the source virtual machine, includes virtual hardware components, an installed guest operating system, and software applications.

1. Create a template from the *VM1* virtual machine.

```
New-Template -VM VM1 -Name VM1Template -Location (Get-Datacenter DC )
```

2. Convert the *VM1Template* template for use by a virtual machine named *VM3*.

```
Get-Template VM1Template | Set-Template -ToVM -Name VM3
```

3. Create a template from the *VM2* virtual machine.

```
New-Template -VM VM2 -Name VM2Template -Location (Get-Datacenter DC )
```

4. Convert the *VM2Template* template to a virtual machine named *VM4*.

```
Get-Template VM2Template | Set-Template -ToVM -Name VM4
```

5. Convert the *VM4* virtual machine to a template.

```
Set-VM -VM VM4 -ToTemplate -Name "VM4Template"
```

6. Create a template called *VM3Template* by cloning *VM2Template*.

```
Get-Template VM2Template | New-Template -Name VM3Template -VMHost $targetVMHost
```

Create and Use Snapshots on vCenter

You can use the `Snapshot` parameter of `Get-VM` to take a snapshot of virtual machines and then revert the states of the virtual machines back to the snapshot.

Verify that you are connected to a system.

Note: A snapshot captures the memory, disk, and settings state of a virtual machine at a particular moment. When you revert to a snapshot, you return all these items to the state they were in at the time you took that snapshot.

1. Take a snapshot of all virtual machines in the *MyRP01* resource pool.

```
Get-ResourcePool MyRP01 | Get-VM | New-Snapshot -Name InitialSnapshot
```

The `Location` parameter takes arguments of the `VIContainer` type, on which `Cluster`, `Datacenter`, `Folder`, `ResourcePool`, and `VMHost` object types are based. Therefore, the `Location` parameter can use arguments of all these types.

2. Revert all virtual machines in the *MyRP01* resource pool to the *InitialSnapshot* snapshot.

```
$VMs = Get-ResourcePool MyRP01 | Get-VM
foreach( $vm in $VMs ) { Set-VM -VM $vm -Snapshot InitialSnapshot }
```

Update the Resource Configuration Settings of a Virtual Machine on vCenter

You can use the `Set-VMResourceConfiguration` cmdlet to modify the resource configuration properties of a virtual machine, including memory, CPU shares, and other settings.

Verify that you are connected to a system.

1. View the resource configuration for the *VM1* virtual machine.

```
Get-VMResourceConfiguration -VM VM1
```

2. View the disk share of the *VM1* virtual machine.

```
Get-VMResourceConfiguration -VM VM1 | Format-Custom -Property DiskResourceConfiguration
```

3. Change the memory share of the *VM1* virtual machine to low.

```
Get-VM VM1 | Get-VMResourceConfiguration | Set-VMResourceConfiguration -MemSharesLevel  
low
```

4. Change the CPU shares of the *VM1* virtual machine to high.

```
Get-VM VM1 | Get-VMResourceConfiguration | Set-VMResourceConfiguration -CpuSharesLevel  
high
```

5. Change the disk share of the *VM1* virtual machine to 100.

```
$vm1 = Get-VM VM1  
$vmldisk = Get-HardDisk $vm1  
Get-VMResourceConfiguration $vm1 | Set-VMResourceConfiguration -Disk $vmldisk -DiskSharesLevel cus-  
tom -NumDiskShares 100
```

Move a Virtual Machine to a Different Host Using VMware vSphere vMotion

You can migrate a virtual machine between vCenter hosts by using vSphere vMotion.

Verify that you are connected to a system.

The virtual machine must be stored on a datastore shared by the current and the destination host, and the vMotion interfaces on the two hosts must be configured.

Note: You can use vSphere vMotion to move a powered-on virtual machine from one host to another.

Get the *VM1* virtual machine and move it to a host named *ESXHost2*.

```
Get-VM VM1 | Move-VM -Destination (Get-VMHost ESXHost2)
```

Move a Virtual Machine to a Different Datastore Using VMware vSphere Storage vMotion

You can migrate a virtual machine between datastores using the VMware Storage vMotion feature of vCenter.

Verify that you are connected to a system.

The host on which the virtual machine is running must have access both to the datastore where the virtual machine is located and to the destination datastore.

Note: You can use Storage vMotion to move a powered-on virtual machine from one datastore to another.

Get the *VM1* virtual machine and move it to a datastore named *DS2*:

```
Get-VM VM1 | Move-VM -Datastore DS2
```

Move a Virtual Machine to a Different vCenter System

You can migrate a virtual machine from one vCenter system to another by using Cross vCenter vMotion.

You can move virtual machines between vCenter systems of vSphere version 6.0 and later by using the `Move-VM` cmdlet. When you move a virtual machine from one vCenter system to another, only datastores are supported as storage destinations.

1. Connect to the *myVC1* source vCenter system.

```
Connect-VIServer 'myVC1' -Username MyUser1 -Password MyPass1
```

2. Connect to the *myVC2* destination vCenter system.

```
Connect-VIServer 'myVC2' -Username MyUser2 -Password MyPass2
```

3. Store the *MyVM* virtual machine, its network adapters, the destination host, port group, and datastore in variables.

```
$vm = Get-VM 'myVM' -Location 'myVMhostOnVC1'
$destination = Get-VMHost 'MyVMhostOnVc2'
$networkAdapter = Get-NetworkAdapter -VM $vm
$destinationPortGroup = Get-VDPortgroup -VDSwitch 'myVDSwitchOnVC2' -Name 'myPortGroup'
$destinationDatastore = Get-Datastore 'MyDatastoreOnVc2'
```

4. Migrate the virtual machine to the specified destination host and datastore and attach the virtual machine network adapters to the destination port group.

```
Move-VM -VM $vm -Destination $destination -NetworkAdapter $networkAdapter -PortGroup $destinationPortGroup -Datastore $destinationDatastore
```

Add Passthrough Devices to a Host and Virtual Machine

You can get information about existing passthrough devices and add new SCSI and PCI devices to virtual machines and hosts.

Verify that you are connected to a system.

1. Get a list of the PCI passthrough devices of the *VMHost* host

```
$vmhost = Get-VMHost ESXHost
Get-PassthroughDevice -VMHost $vmhost -Type Pci
```

2. Get a list of the SCSI passthrough devices of the *VM* virtual machine

```
$vm = Get-VM VM
Get-PassthroughDevice -VM $vm -Type Scsi
```

3. Add a SCSI passthrough device to the *VM* virtual machine

```
$scsiDeviceList = Get-PassthroughDevice -VMHost ESXHost -Type Scsi
Add-PassthroughDevice -VM $vm -PassthroughDevice $scsiDeviceList[0]
```

Apply a Customization Object to a Cloned Virtual Machine

You can apply a custom configuration to a cloned virtual machine by using a customization object.

Verify that you are connected to a system.

Note: This feature runs only on a 32-bit process.

1. Get the *Spec* customization specification and clone it for temporary use.

```
Get-OSCustomizationSpec Spec | New-OSCustomizationSpec -Type NonPersistent -Name ClientSpec
```

2. Change the *NamingPrefix* property of the customization object to the name of the virtual machine you want to create.

```
Set-OSCustomizationSpec -Spec ClientSpec -NamingPrefix VM1
```

3. Create a virtual machine named *VM1* by cloning the existing *VM* virtual machine and applying the customization specification.

```
Get-VM VM | New-VM -VMHost Host -Datastore Storage1 -OSCustomizationSpec ClientSpec -Name VM1
```

Modify the Default NIC Mapping Object of a Customization Specification

You can modify the default NIC mapping object of a customization specification and apply the specification on a newly created virtual machine.

1. Create a nonpersistent customization specification for Windows operating systems.

```
New-OSCustomizationSpec -Type NonPersistent -Name Spec -OSType Windows -Workgroup Workgroup  
-OrgName Company -Fullname User -ProductKey "valid_key" -ChangeSid -TimeZone "Central European" -NamingScheme VM
```

2. View the default NIC mapping objects of the *Spec* specification.

```
Get-OSCustomizationNicMapping -Spec Spec | Set-OSCustomizationNicMapping -IpMode  
UseStaticIP -IpAddress 172.16.1.30 -SubnetMask 255.255.255.0 -DefaultGateway 172.16.1.1 -Dns 172.16.1
```

Each customization specification object has one default NIC mapping object.

3. Modify the default NIC mapping object of the *Spec* customization specification to use static IP.

```
Get-OSCustomizationNicMapping -Spec Spec | Set-OSCustomizationNicMapping -IpMode  
UseStaticIP -IpAddress 172.16.1.30 -SubnetMask 255.255.255.0 -DefaultGateway 172.16.1.1 -Dns 172.16.1.1
```

4. Create a new virtual machine named *VM1* from a template, and apply the static IP settings.

```
New-VM -Name VM1 -VMHost Host -Datastore Storage1 -OSCustomizationSpec Spec -Template Template
```

Modify Multiple NIC Mapping Objects of a Customization Specification

You can modify multiple NIC mapping objects of a customization specification and apply the specification to an existing virtual machine.

1. Get the network adapters of a virtual machine named *VM*.

```
Get-NetworkAdapter VM
```

When you apply a customization specification, each network adapter of the customized virtual machine must have a corresponding NIC mapping object. You can correlate network adapters and NIC mapping objects either by their position numbers, or by MAC address.

2. Create a customization specification named *Spec*.

```
New-OSCustomizationSpec -Type NonPersistent -Name Spec -OSType Windows -Workgroup Workgroup
-OrgName Company -Fullname User -ProductKey "valid_key" -ChangeSid -TimeZone "Central European" -Nam-
ingScheme VM
```

3. Add a new NIC mapping object that uses a static IP address.

```
New-OSCustomizationNicMapping -Spec Spec -IpMode UseStaticIP -IpAddress 172.16.1.30 -Sub-
netMask 255.255.255.0 -DefaultGateway 172.16.1.1 -Dns 172.16.1.1
```

4. View the NIC mapping objects and verify that two NIC mapping objects are available.

```
Get-OSCustomizationNicMapping -Spec Spec
```

The default NIC mapping object is DHCP enabled, and the newly added one uses a static IP address.

5. Apply the Spec customization specification to the *VM* virtual machine.

```
Get-VM VM | Set-VM -OSCustomizationSpec -Spec Spec
```

6. Associate a network adapter from the *VMNetwork* network with the NIC mapping object that uses DHCP mode.

```
$netAdapter = Get-NetworkAdapter VM | where { $_.NetworkName -eq 'VMNetwork' }
Get-OSCustomizationNicMapping -Spec Spec | where { $_.IPMode -eq 'UseDHCP' } | Set-OSCustomization-
NicMapping -NetworkAdapterMac $netAdapter.MacAddress
```

Create Multiple Virtual Machines that Use Static IP Addresses

You can deploy multiple virtual machines with a single network adapter and configure the deployed virtual machines to use static IP addresses by applying a customization specification.

Verify that you have defined a list of static IP addresses in a CSV file.

1. Define the naming convention for the virtual machines.

```
$vmNameTemplate = "VM-{0:D3}"
```

2. Save the cluster in which the virtual machines should be created into a variable.

```
$cluster = Get-Cluster MyCluster
```

3. Save the template on which the virtual machines should be based into a variable.

```
$template = Get-Template MyTemplate
```

4. Create the virtual machines.

```
$vmList = @()

for ($i = 1; $i -le 100; $i++) {
    $vmName = $vmNameTemplate -f $i
    $vmList += New-VM -Name $vmName -ResourcePool $cluster -Template $template
}
```

5. Save the static IP addresses from the stored CSV file into a variable.

```
$staticIpList = Import-CSV:\StaticIPs.csv
```

6. Create the customization specification.

```
$linuxSpec = New-OSCustomizationSpec -Name LinuxCustomization -Domain vmware.com -DomainUser-
name "your_domain_username" -DomainPassword "your_domain_password" -DnsServer "192.168.0.10",
"192.168.0.20" -NamingScheme VM -OSType Linux
```

7. Clone the customization specification to a nonpersistent type.

```
$specClone = New-OSCustomizationSpec -Spec $linuxSpec -Type NonPersistent
```

8. Apply the customization specification to each virtual machine.

```
for ($i = 0; $i -lt $vmList.Count; $i++) {
    # Acquire a new static IP from the list
    $ip = $staticIpList[$i].IP

    # The specification has a default NIC mapping - retrieve it and update it with the static IP
    $nicMapping = Get-OSCustomizationNicMapping -OSCustomizationSpec $specClone
    $nicMapping | Set-OSCustomizationNicMapping -IpMode UseStaticIP -IpAddress $ip -SubnetMask
    "255.255.252.0" -DefaultGateway "192.168.0.1"

    # Apply the customization
    Set-VM -VM $vmList[$i] -OSCustomizationSpec $specClone -Confirm:$false
}
```

Create Multiple Virtual Machines with Two Network Adapters

You can deploy multiple virtual machines with two network adapters each and configure each adapter to use specific network settings by applying a customization specification.

Verify that you have defined a list of static IP addresses in a CSV file.

You can configure each virtual machine to have one network adapter attached to a public network and one network adapter attached to a private network. You can configure the network adapters on the public network to use static IP addresses and the network adapters on the private network to use DHCP.

1. Define the naming convention for the virtual machines.

```
$vmNameTemplate = "VM-{0:D3}"
```

2. Save the cluster in which the virtual machines should be created into a variable.

```
$cluster = Get-Cluster MyCluster
```

3. Save the template on which the virtual machines should be based into a variable.

```
$template = Get-Template MyTemplate
```

4. Create the virtual machines.

```
$vmList = @()

for ($i = 1; $i -le 100; $i++) {
    $vmName = $vmNameTemplate -f $i
    $vmList += New-VM -Name $vmName -ResourcePool $cluster -Template $template
}
```

5. Save the static IP addresses from the stored CSV file into a variable.

```
$staticIpList = Import-CSVC:\StaticIPs.csv
```

6. Create the customization specification.

```
$linuxSpec = New-OSCustomizationSpec -Name LinuxCustomization -Domain vmware.com -DnsServer
    "192.168.0.10", "192.168.0.20" -NamingScheme VM -OSType Linux -Type NonPersistent
```

7. Apply the customization specification to each virtual machine.

```
for ($i = 0; $i -lt $vmList.Count; $i++) {
    # Acquire a new static IP from the list
    $ip = $staticIpList[$i].IP
```

```

# Remove any NIC mappings from the specification
$nicMapping = Get-OSCustomizationNicMapping -OSCustomizationSpec $linuxSpec
Remove-OSCustomizationNicMapping -OSCustomizationNicMapping $nicMapping -Confirm:$false

# Retrieve the virtual machine's network adapter attached to the public network named "Public"
$publicNIC = $vmList[$i] | Get-NetworkAdapter | where {$_.NetworkName -eq "Public"}

# Retrieve the virtual machine's network adapter attached to the private network named "Private"
$privateNIC = $vmList[$i] | Get-NetworkAdapter | where {$_.NetworkName -eq "Private"}

# Create a NIC mapping for the "Public" NIC that should use static IP
$linuxSpec | New-OSCustomizationNicMapping -IpMode UseStaticIP -IpAddress $ip -SubnetMask
"255.255.252.0" -DefaultGateway "192.168.0.1" -NetworkAdapterMac $publicNIC.MacAddress

# Create a NIC mapping for the "Private" NIC that should use DHCP
$linuxSpec | New-OSCustomizationNicMapping -IpMode UseDhcp -NetworkAdapterMac $privateNIC.MacAd-
dress

# Apply the customization
Set-VM -VM $vmList[$i] -OSCustomizationSpec $linuxSpec -Confirm:$false
}

```

vSphere vApp Administration

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere vApp administration.

Use vSphere vApp to package and manage virtual machines as a single entity. vApps are typically used to simplify the deployment and management of multi-tier applications.

Create a vApp on vCenter

With VCF PowerCLI, you can create and manage vApps.

Verify that you are connected to a system.

1. Create a new vApp named *VApp* on a host.

```
New-VApp -Name VApp -CpuLimitMhz 4000 -CpuReservationMhz 1000 -Location (Get-VMHost Host1)
```

2. Start the new virtual appliance.

```
Start-VApp VApp
```

Modify the Properties of a vApp

With VCF PowerCLI, you can start and stop vApps, and modify their properties.

Verify that you are connected to a system.

1. Get the vApp named *VApp* and stop it.

```
Get-VApp VApp | Stop-VApp -Confirm:$false
```

2. Change the name and memory reservation for the vApp.

```
Get-VApp VApp | Set-VApp -Name OldVApp -MemReservationGB 2
```

Export or Import vApps

You can import and export vApps to OVA and OVF files.

Verify that you are connected to a system.

1. Get the vApp you want to export.

```
$oldVApp = Get-VApp OldVApp
```

2. Export the *OldVApp* vApp to a local directory and name the exported appliance *WebApp*.

```
Export-VApp -VApp $oldVApp -Name WebApp -Destination D:\vapps\ -CreateSeparateFolder
```

3. Import the *WebApp* vApp from a local directory to the *Storage2* datastore.

```
Import-VApp -Source D:\vapps\WebApp\WebApp.ovf -VMHost (Get-VMHost Host1) -Datastore (Get-Datastore -VMHost MyHost01 -Name Storage2)
```

Using Tags

The vSphere Tag Service supports the definition of tags that you can associate with different types of vSphere objects, such as virtual machines, resource pools, datastores, and distributed switches. You can use VCF PowerCLI to create and manage tags and tag categories, and to retrieve specified groups of objects.

You create a vSphere tag to add metadata to objects in the vSphere inventory. Tags are grouped in categories and each tag must have at least one category related to it. After you create the tag, you can associate the tag with a vSphere object.

After you create a tag category and create a tag within the category, you can associate the tag with a vSphere managed object. An association is a simple link that contains no data of its own. You can enumerate objects that are attached to a tag or tags that are attached to an object.

Retrieve a Tag and Save It into a Variable

You can retrieve existing tags defined in vSphere and save a specific tag into a variable.

Verify that you are connected to a system.

1. Get the tag named *MyTag*.

```
Get-Tag -Name 'MyTag'
```

2. Save the tag into a variable.

```
$tag = Get-Tag -Name 'MyTag'
```

Retrieve a Tag Category and Save It into a Variable

You can retrieve existing tag categories defined in vSphere and save a specific tag category into a variable.

Verify that you are connected to a system.

1. Get the tag category named *MyTagCategory*.


```
Get-TagCategory -Name 'MyTagCategory'
```

2. Save the tag category into a variable.

```
$tagCategory = Get-TagCategory -Name 'MyTagCategory'
```

Create a Tag Category and a Tag

You can create a tag category and add a new tag in that category.

Verify that you are connected to a system.

1. Create a tag category named *Department*.

```
$departmentTagCategory = New-TagCategory -Name 'Department'
```

2. Create a new tag named *SalesDpt* in the *Department* category.

```
$salesDptTag = New-Tag -Name 'SalesDpt' -Category $departmentTagCategory
```

Assign a Tag to Virtual Machines

You can assign a tag to a group of virtual machines. For example, you can assign a custom tag to all virtual machines that belong to a specific department in your organization.

Verify that you are connected to a system.

1. Get the virtual machines of a department in your organization.

```
$vms = Get-VM sales-dpt*
```

2. Assign the custom tag to the group of virtual machines.

```
New-TagAssignment -Tag $salesDptTag -Entity $vms
```

Retrieve Objects by Tag

You can retrieve all objects that have a specific tag assigned to them.

Verify that you are connected to a system.

Get all virtual machines tagged with the *salesDptTag* tag.

```
Get-VM -Tag 'salesDptTag'
```

Note: You can only specify a tag filter parameter for the *VM*, *VMHost*, *Datastore*, and *VirtualPortGroup* object types.

Generate Tags Automatically by Using a Script

You can use a script to generate tags automatically. For example, you can create a virtual machine owner tag for each user account in a domain.

- Verify that you are connected to a vCenter system.
- Verify that the user accounts and the vCenter system are in the same domain.

You must use the `Get-VIAccount` cmdlet to retrieve user accounts. For more information, see the documentation of the cmdlet.

1. Create a new tag category and specify that tags in this category can only be assigned to entities of type `VirtualMachine`.

```
$ownerTagCategory = New-TagCategory -Name Owner -EntityType VirtualMachine
```

Note: If you do not specify an entity type, tags from this category can be assigned to all entity types.

2. Retrieve all domain user accounts and save them in a variable.

Note:

This cmdlet works only with the vCenter built-in identity provider that supports local accounts, Active Directory or OpenLDAP, Integrated Windows Authentication (IWA), and miscellaneous authentication mechanisms (smart card and RSA SecurID). You cannot retrieve user accounts for users who use external identity providers to authenticate with vCenter.

```
$accounts = Get-VIAccount -User -Domain 'DomainName' -Category | select -ExpandProperty Id
```

3. Create a tag for each user account.

```
$accounts | foreach { New-Tag -Category $ownerTagCategory -Name $_ }
```

4. Retrieve a specific tag from the *Owner* category, so that you can later assign it to a specific virtual machine.

```
$ownerTag = Get-Tag -Category $ownerTagCategory -Name 'John_Smith'
```

Add an Entity Type to a Tag Category

You can extend the list of entity types associated with a tag category.

Verify that you are connected to a system.

Add the *vApp* entity type to the *ownerTagCategory* tag category.

```
$ownerTagCategory | Set-TagCategory -AddEntityType vApp
```

Retrieve Tag Assignments

You can retrieve tag assignments by using category and entity filters.

Verify that you are connected to a system.

1. Retrieve all virtual machines that have a tag from the *ownerTagCategory* tag category assigned to them.

```
Get-TagAssignment -Category $ownerTagCategory
```

2. Retrieve the owner of the *MyVM* virtual machine.

```
Get-TagAssignment -Category $ownerTagCategory -Entity 'MyVM'
```

Using Content Libraries

Use content libraries with VCF PowerCLI to store and distribute various forms of content within your vSphere environment. Content libraries function as container entities within vSphere, allowing you to save and distribute items such as OVF and OVA packages, virtual machine templates, vApp templates, and various file types.

You can use content libraries to share VM and vApp templates, and other types of files, such as ISO images, text files, and so on, across your vCenter instances. Sharing templates across your virtual environment promotes consistency, compliance, efficiency, and automation in deploying workloads at scale.

A content library instance represents a container for a set of library items. A content library item instance represents the logical object stored in the content library, which might be one or more usable files.

You create and manage the content of a content library on a single vCenter instance, but you can distribute the content to other vCenter instances.

Depending on your needs, you can maintain two types of content libraries: local and subscribed. You can shape the contents of a library item and then combine several library items in a local content library. Furthermore, you can publish the library to make its content available to other users.

Create a Local Content Library

With VCF PowerCLI, you can create a local content library.

Verify that you are connected to a system.

1. Get the datastore that you want to use to store files for library items in this library.

```
$datastore = Get-Datastore myDatastore
```

2. Create the *My Local Content Library* published local content library that uses the *myDatastore* datastore.

```
New-ContentLibrary -Name 'My Local Content Library' -Description 'Local content library de-
scription.' -Datastore $Datastore1 -Published
```

Create a Subscribed Content Library

With VCF PowerCLI, you can create a subscribed content library.

Verify that you are connected to a system.

Create the *My Subscribed Content Library* subscribed content library that uses the *myDatastore* datastore.

```
New-ContentLibrary -Name 'My Subscribed Content Library' -Description 'Subscribed content li-
brary description.' -Datastore $Datastore1 -SubscriptionUrl $Url
```

Create a Content Library Item

With VCF PowerCLI, you can create content library items.

Verify that you are connected to a system.

1. Get the files that you want to upload to the content library.

```
$files = Get-ChildItem -File
```

2. Get the content library that you want to upload.

```
$contentLibrary = Get-ContentLibrary -name 'MyContentLibrary'
```

3. Create a content library item.

```
New-ContentLibraryItem -ContentLibrary $contentLibrary -name 'New item' -Files $files
```

Create a Virtual Machine from a Content Library Item

You can deploy a virtual machine from a content library template.

- Verify that you are connected to a system.

- Verify that you have a content library with virtual machine templates available.

Note: VMware Cloud Foundation PowerCLI cannot distinguish between OVF content library items of type virtual machine template and vApp template. As a result, `New-VM` creates a vApp if you specify a vApp template from the content library by using the `ContentLibraryItem` parameter of the cmdlet. If this happens, `New-VM` returns an error, notifying that the cmdlet produced an inventory item of the wrong type. Avoid creating vApps by using the `New-VM` cmdlet, as this behavior will be deprecated in future releases.

1. Get the virtual machine host.

```
$myVMHost = Get-VMHost myVMHost
```

2. Create the *MyVM* virtual machine from the *MyVMContentLibraryItemName* content library item.

```
Get-ContentLibraryItem -Name MyVMContentLibraryItemName | New-VM -Name MyVM -VMHost
$myVMHost
```

Create a vApp from a Content Library Item

You can deploy a vApp from a content library template.

- Verify that you are connected to a system.
- Verify that you have a content library with vApp templates available.

Note: VMware Cloud Foundation PowerCLI cannot distinguish between OVF content library items of type virtual machine template and vApp template. As a result, `New-VApp` creates a virtual machine if you specify a virtual machine template from the content library by using the `ContentLibraryItem` parameter of the cmdlet. If this happens, `New-VApp` returns an error, notifying that the cmdlet produced an inventory item of the wrong type. Avoid creating virtual machines by using the `New-VApp` cmdlet, as this behavior will be deprecated in future releases.

1. Get the virtual machine host.

```
$myVMHost = Get-VMHost myVMHost
```

2. Create the *MyVApp* vApp from the *MyVAppContentLibraryItemName* content library item.

```
Get-ContentLibraryItem -Name MyVAppContentLibraryItemName | New-VApp -Name MyVApp -VMHost
$myVMHost
```

Export Content Library Item's Files to a Local Machine

With VCF PowerCLI, you can export content library item's files to a local machine.

Verify that you are connected to a system.

1. Get a content library item.

```
$item = Get-ContentLibraryItem -Name 'myVM'
```

2. Export a content library item to a local machine.

```
Export-ContentLibraryItem -ContentLibraryItem $item -Destination ./myVM-files
```

vSphere Networking

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Networking.

Modify the Settings of the NIC Teaming Policy for a Virtual Switch

You can set the NIC teaming policy on a vSwitch. The NIC teaming policy determines the load balancing and failover settings of a virtual switch and lets you mark NICs as unused.

Verify that you are connected to a system.

1. Get a list of the physical NIC objects on the host network and store them in a variable.

```
$pn = Get-VMHost 10.23.123.128 | Get-VMHostNetwork | Select -Property physicalnic
```

2. Store the physical NIC objects you want to mark as unused in separate variables.

```
$pn5 = $pn.PhysicalNic[2]
$pn6 = $pn.PhysicalNic[3]
$pn7 = $pn.PhysicalNic[0]
```

3. View the NIC teaming policy of the *VSwitch01* virtual switch.

```
$policy = Get-VirtualSwitch -VMHost 10.23.123.128 -Name VSwitch01 | Get-NicTeamingPolicy
```

4. Change the policy of the switch to indicate that the *\$pn5*, *\$pn6*, and *\$pn7* network adapters are unused.

```
$policy | Set-NicTeamingPolicy -MakeNicUnused $pn5, $pn6, $pn7
```

5. Modify the load balancing and failover settings of the virtual switch NIC teaming policy.

```
$policy | Set-NicTeamingPolicy -BeaconInterval 3 -LoadBalancingPolicy 3 -NetworkFailoverDetectionPolicy
1 -NotifySwitches $false -FailbackEnabled $false
```

Network Management with vSphere Distributed Switches

The cmdlets provided in the `VMware.VimAutomation.VDS` module let you manage networking with vSphere distributed switches and port groups.

Create a Distributed Switch and Configure Networking

A vSphere distributed switch lets you handle networking traffic for all associated hosts in a data center. After you create a new vSphere distributed switch in VCF PowerCLI, you can add hosts and connect virtual machines to it.

Verify that you are connected to a system.

1. Get the data center where you want to create the vSphere distributed switch.

```
$myDatacenter = Get-Datacenter -Name "MyDatacenter"
```

2. Get all hosts in your data center.

```
$vmHosts = $myDatacenter | Get-VMHost
```

3. Create a new vSphere distributed switch.

```
$myVDSwitch = New-VDSwitch -Name "MyVDSwitch" -Location $myDatacenter
```

The distributed switch is created with no port groups.

4. Add the hosts in your data center to the distributed switch.

```
Add-VDSwitchVMHost -VDSwitch $myVDSwitch -VMHost $vmHosts
```

5. Get a physical network adapter from your hosts.

```
$hostsPhysicalNic = $vmHosts | Get-VMHostNetworkAdapter -Name "vmnic2"
```

6. Add the physical network adapter to the distributed switch that you created.

```
Add-VDSwitchPhysicalNetworkAdapter -VMHostNetworkAdapter $hostsPhysicalNic -Distribut-
edSwitch $myVDSwitch
```

7. Create a new distributed port group with 1000 ports and add it to the distributed switch.

```
$myVDPortGroup = New-VDPortgroup -Name "MyVMsPortGroup" -VDSwitch $myVDSwitch -NumPorts 1000
```

8. Connect all virtual machines running on the hosts in your data center to the distributed port group.

```
$vmHosts | Get-VM | Get-NetworkAdapter | Set-NetworkAdapter -PortGroup $myVDPortGroup
```

Adjust the settings of the distributed switch. See [Configure a Distributed Switch](#).

Configure a Distributed Switch

Based on your networking requirements, you can adjust the settings of a newly created or an existing distributed switch.

Verify that you are connected to a system.

- Modify the maximum MTU size setting for a distributed switch.

```
Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -Mtu 2000
```

- Modify the number of uplink ports on a distributed switch.

```
Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -NumUplinkPorts 4
```

- Modify the maximum number of ports on a distributed switch.

```
Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -MaxPorts 1000
```

- Modify the discovery protocol settings on a vSphere distributed switch.

```
Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -LinkDiscoveryProtocol LLDP -LinkDiscoveryProtocolOperation Both
```

Migrate Virtual Machine Networking Configuration from a vSphere Standard Switch to a vSphere Distributed Switch

To manage virtual machine networks on a data center level, you might need to migrate existing networks from vSphere standard switches to vSphere distributed switches.

Verify that you are connected to a system.

1. Get the source vSphere standard switch from which you want to migrate the virtual machine networking.

```
$virtualSwitch = Get-VirtualSwitch -Name 'MyVirtualSwitch'
```

2. Get the source standard port group to which the virtual machines are connected.

```
$vmsPortGroup = $virtualSwitch | Get-VirtualPortGroup -Name 'VM Network'
```

3. Get the target vSphere distributed switch to which you want to migrate the virtual machine networking.

```
$vdSwitch = Get-VDSwitch -Name 'MyTargetVDSwitch'
```

4. Get the target port group to which you want to connect the virtual machines.

```
$vdPortGroup = Get-VDPortGroup -VDSwitch $vdSwitch -Name 'DPortGroup'
```

5. Get the virtual machine network adapters connected to the source port group.

```
$vmsNetworkAdapters = Get-VM -RelatedObject $vmsPortGroup | Get-NetworkAdapter | where { $_.NetworkName -eq $vmsPortGroup.Name }
```

6. Disconnect the retrieved network adapters from the standard port group and connect them to the distributed port group.

```
Set-NetworkAdapter -NetworkAdapter $vmsNetworkAdapters -PortGroup $vdPortGroup
```

Migrate Physical and Virtual NICs to a vSphere Standard Switch

You can migrate both physical and virtual network adapters to a vSphere standard switch simultaneously.

Verify that you are connected to a system.

1. Get the physical network adapters that you want to migrate.

```
$pNics = Get-VMHostNetworkAdapter -VMHost $vmhost -Physical
```

2. Get the virtual network adapters that you want to migrate.

```
$vNicManagement = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk0
```

```
$vNicvMotion = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk1
```

3. Get the vSphere standard switch to which you want to migrate the network adapters.

```
$vSwitch = Get-VirtualSwitch -VMHost $vmhost -Name vSwitch0
```

4. Migrate all network adapters to the vSphere standard switch.

```
Add-VirtualSwitchPhysicalNetworkAdapter -VirtualSwitch $vSwitch -VMHostPhysicalNic $pNics  
-VMHostVirtualNic $vNicManagement,$vNicvMotion
```

Migrate Physical and Virtual NICs to a vSphere Distributed Switch

You can migrate both physical and virtual network adapters to a vSphere distributed switch simultaneously.

Verify that you are connected to a system.

1. Get the physical network adapters that you want to migrate.

```
$pNics = Get-VMHostNetworkAdapter -VMHost $vmhost -Physical
```

2. Get the virtual network adapters that you want to migrate.

```
$vNicManagement = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk0
```

```
$vNicvMotion = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk1
```

3. Get the port groups corresponding to the virtual network adapters that you want to migrate to the vSphere distributed switch.

```
$vdPortgroupManagement = Get-VDPortgroup -VDSwitch $vds -Name 'Management Network'
```

```
$vdPortgroupvMotion = Get-VDPortgroup -VDSwitch $vds -Name 'vMotion Network'
```

4. Migrate all network adapters to the vSphere distributed switch.

```
Add-VDSwitchPhysicalNetworkAdapter -DistributedSwitch $vds -VMHostPhysicalNic $pNics  
-VMHostVirtualNic $vNicManagement,$vNicvMotion -VirtualNicPortGroup $vdPortGroupManagement, $vdPort-  
GroupvMotion
```

You migrated the *\$vNicManagement* network adapter to the Management Network port group and the *\$vNicvMotion* network adapter to the vMotion Network port group.

Configure the Traffic Shaping Policy

You can modify the traffic shaping policy of a port group to limit the bandwidth of the incoming traffic and ensure that enough bandwidth is available for other port groups on the same vSphere distributed switch.

Verify that you are connected to a system.

1. Get the current traffic shaping policy of the port group.

```
$policy = Get-VDTrafficShapingPolicy -Direction In -VDPortGroup $myVDPortGroup
```

2. Set the peak bandwidth to 100 Mbps.

```
Set-VDTrafficShapingPolicy -Policy $policy -PeakBandwidth 104857600
```

Configure the Security Policy

You can modify the security policy of a port group to enable promiscuous mode, which allows monitoring of the traffic generated by virtual machines.

Verify that you are connected to a system.

1. Get the current security policy of the port group.

```
$policy = Get-VDSecurityPolicy -VDPortGroup $myVDPortGroup
```

2. Enable promiscuous mode for the port group.

```
Set-VDSecurityPolicy $policy -AllowPromiscuous $true
```

Managing Virtual Private Clouds in vCenter

A virtual private cloud (VPC) is a configurable pool of shared network resources allocated within a cloud environment, providing a certain level of isolation between the different organizations using the resources. A subnet is a range of IP addresses in VPC and a workload is connected to a subnet.

Create a VPC in vCenter

You can create a VPC in vCenter by using the `New-Vpc` cmdlet.

- Verify that you connected to a vCenter instance. See [Connect to a System](#).

1. Create the VPC.

```
New-Vpc -Name clivpc1 -PrivateIp 192.168.130.0/24 -Description "Create a VPC by using VCF  
PowerCLI."
```

2. Verify that the VPC is created correctly.

```
Get-Vpc clivpc1
```

A new VPC is created in vCenter.

Modify a VPC in vCenter

You can modify the settings of a VPC you created.

- Verify that you connected to a vCenter instance. See [Connect to a System](#).
- Verify that you have a VPC created.

1. Modify the VPC settings.

```
Set-Vpc -vpc clivpc1 -PrivateIps 192.168.131.0/24
```

2. Verify that the VPC private IP address is modified.

```
Get-Vpc clivpc1 | Format-List
```

Retrieve Information about a VPC in vCenter

You can view information about a specific VPC in vCenter

- Verify that you connected to a vCenter instance. See [Connect to a System](#).

- Verify that you have a VPC created.

1. Get information about the VPC projects in a vCenter instance.

For more information about NSX Projects and VPCs, see [NSX Projects and VPCs](#).

```
Get-VpcProject
```

2. Get a list all IP blocks for all VPCs in a vCenter instance.

The cmdlet returns detailed information about all IPs for all VPC projects.

```
Get-VpcIpBlock
```

3. Get the connectivity profiles of all VPCs in a vCenter instance.

A connectivity profile defines the common northbound connectivity configuration for VPCs.

The cmdlet returns detailed information about all connectivity profiles of all VPCs in a vCenter instance.

```
Get-VpcConnectivityProfile
```

4. Get the service profiles of all VPCs in a vCenter instance.

A VPC service profile is a collection of services (DHCP and subnet profiles). It simplifies configuration of common services on multiple VPCs.

The cmdlet returns detailed information about all service profiles of all VPCs in a vCenter instance.

```
Get-VpcServiceProfile
```

5. Get the statistics about all VPCs in a vCenter instance.

The cmdlet returns detailed information about the accumulated statistics about all VPCs in a vCenter instance.

```
Get-VpcStatistics
```

You retrieved detailed information about all the VPCs or a specific VPC in a vCenter instance.

Add a Subnet for a VPC in vCenter

You can add a subnet to a VPC in vCenter by using the `New-VpcSubnet` and DHCP options cmdlets `New-VpcDhcpGenericOption` and `New-VpcDhcpClasslessStaticRoute`.

- Verify that you connected to a vCenter instance. See [Connect to a System](#).
- Verify that you have a VPC created.

1. Optional: Create a DHCP configuration that is used by the VPC subnet.

Note:

You can set up a DHCP configuration only if the DHCP settings are configured in the Service Profile linked to the VPC.

```
$genericOptions = @((New-VpcDhcpGenericOption -Code MTUInterface -Value 1600), (New-VpcDhcpGenericOption -Code ARPTimeout -Value 60))
$staticRoutes = @((New-VpcDhcpClasslessStaticRoute -Network 192.168.133.0/28 -NextHop 192.168.133.1), (New-VpcDhcpClasslessStaticRoute -Network 192.168.133.0/28 -NextHop 192.168.133.2))
```

2. Add a subnet for a specific VPC in a vCenter instance.

```
New-VpcSubnet -Vpc clivpc3 -Name clivpc3sub1 -DhcpMode Server -DhcpGenericOption $genericOptions -DhcpClasslessStaticRoute $staticRoutes
```

3. Verify that the VPC subnet is created.

```
Get-VpcSubnet -Vpc cliVpc3
```

You added a subnet to a specific VPC in a vCenter instance.

Retrieve Information about the VPC Subnet in vCenter

You can retrieve a detailed information about a subnet of a VPC in a vCenter instance.

- Verify that you connected to a vCenter instance. See [Connect to a System](#).
- Verify that you have a VPC created and you added a subnet to it.

1. Get the accumulated statistics for a specific VPC subnet.

```
Get-VpcSubnetStatistics
```

2. Get the status of a specific VPC subnet.

```
Get-VpcSubnetStatus
```

3. Get the accumulated statistics of the DHCP server of the VPC subnet.

```
Get-VpcSubnetDhcpServerStatistics
```

4. Get the status of the DHCP server of the VPC subnet.

```
Get-VpcSubnetDhcpServerStatus
```

Modify the Subnet of a VPC in vCenter

You can modify the subnet configuration of a specific VPC in a vCenter instance.

- Verify that you connected to a vCenter instance. See [Connect to a System](#).
- Verify that you have a VPC created and you added a subnet to it.

1. Modify the subnet configuration of a VPC.

You can first check the IP address range used by the VPC subnet.

```
Set-VpcSubnet -Subnet cliVpc3sub1 -ReservedIpRange 192.168.133.10-192.168.133.20
```

2. Verify whether the subnet configuration is modified successfully.

```
Get-VpcSubnet cliVpc3sub1 | Format-List
```

Create a DHCP Static Bindings Configuration for the VPC Subnet in vCenter

You can create DHCP static bindings for a VPC subnet by using the `New-VpcDhcpStaticBinding` and DHCP options cmdlets `New-VpcDhcpGenericOption` and `New-VpcDhcpClasslessStaticRoute`.

- Verify that you connected to a vCenter instance. See [Connect to a System](#).
- Verify that you have a VPC created and you added a subnet to it.
- Verify that you have set the `ReservedIpRange` on the VPC subnet.

For more information about DHCP static bindings configuration, see [DHCP Static Bindings](#) section in the [DHCP Configuration Settings: Reference](#) article.

1. Optional: Create a DHCP configuration that is used by the VPC subnet.

Note:

You can set up a DHCP configuration only if the DHCP settings are configured in the Service Profile linked to the VPC.

```
$bindingGenericOptions = @((New-VpcDhcpGenericOption -Code MTUInterface -Value 1700), (New-VpcDhcp-
GenericOption -Code ARPTimeout -Value 120))
$bindingStaticRoutes = @((New-VpcDhcpClasslessStaticRoute -Network 192.168.133.0/28 -Nex-
tHop 192.168.133.3))
```

2. Configure the DHCP static bindings of the VPC subnet.

Note:

The `IpAddress` must be in the range of `ReservedIpRange` of the VPC subnet.

```
New-VpcDhcpStaticBinding -Subnet cliVpc3sub1 -Name binding1 -MacAddress 00:A0:C9:12:23:34 -
IpAddress 192.168.133.10 -Gateway 192.168.133.1 -HostName binding1.vpc.com -DhcpGenericOp-
tion $bindingGenericOptions -DhcpClasslessStaticRoute $bindingStaticRoutes
```

3. Verify that the DHCP static bindings of the VPC subnet are configured.

Modify the Configuration of the DHCP Static Bindings of the VPC Subnet in vCenter

You can modify the configuration of the DHCP static bindings of a VPC subnet by using the `Set-VpcDhcpStaticBinding`.

- Verify that you connected to a vCenter instance. See [Connect to a System](#).
- Verify that you have a VPC created and you added a subnet to it.
- Verify that you have set the `ReservedIpRange` on the VPC subnet.
- Verify that you configured the DHCP static bindings of the VPC subnet.

1. Modify the DHCP static bindings of the VPC subnet.

You can first check the IP address range used by the DHCP static bindings of the VPC subnet.

```
Set-VpcDhcpStaticBinding -Binding binding1 -IpAddress 192.168.133.11
```

2. Verify that your changes are applied correctly.

```
Get-VpcDhcpStaticBinding -Name binding1
```

Connect a Virtual Machine to a VPC Subnet in vCenter

You connect a virtual machine to a VPC subnet in a vCenter instance by using the `New-NetworkAdapter` or `Set-NetworkAdapter` cmdlets.

- Verify that you connected to a vCenter instance. See [Connect to a vCenter System](#).
- Verify that you have a VPC created and you added a subnet to it.

1. Connect a new network adapter to a specific VPC subnet.

```
New-NetworkAdapter -VM UBUNTU-vm1 -Type Vmxnet3 -Subnet cliVpc3sub1 -WakeOnLan -StartConnected
```

2. Verify that the virtual machine is connected to the VPC subnet.

```
Get-NetworkAdapter -VM UBUNTU-vm1
```

vSphere Storage

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Storage.

Create an iSCSI Host Storage

For a host, you can enable iSCSI, add iSCSI targets, and create new host storages.

Verify that you are connected to a system.

1. Enable software iSCSI on a host.

```
$vmhost = Get-VMHost ESXHost1
Get-VMHostStorage $myHost | Set-VMHostStorage -SoftwareIScsiEnabled $true
```

2. Get the iSCSI Host Bus Adapter (HBA) that is on the host.

```
$iscsiHba = Get-VMHostHba -Type iScsi
```

3. Add a new iSCSI target for dynamic discovery.

```
$iscsiHba | New-IScsiHbaTarget -Address 192.168.0.1 -Type Send
```

4. Rescan the HBAs on the host.

```
Get-VMHostStorage $vmhost -RescanAllHba
```

5. Get the path to the SCSI LUN.

```
$lunPath = Get-ScsiLun -VMHost $vmhost -CanonicalName ($iscsiHba.Device + "**") | Get-ScsiLunPath
```

You can provide the LUN path by using its canonical name beginning with the device name of the iSCSI HBA.

6. Create a new host storage.

```
New-Datastore -Vmfs -VMHost $vmhost -Path $lunpath.LunPath -Name iSCSI
```

Storage Policy Based Management with VMware Cloud Foundation PowerCLI

Manage your storage policies with VCF PowerCLI. Storage Policy Based Management (SPBM) is a storage policy framework that operates on the virtual machine level and serves as a single unified control panel across a broad range of data services and storage solutions.

Create a Tag-Based Storage Policy

You can create storage policies by using tags from vCenter.

- Verify that you are connected to a system.
- Verify that you have Profile-driven storage update privileges.
- Verify that a tag named *Tag1* exists in the vCenter environment.

1. Get the *Tag1* tag and store it in the *\$tag* variable.

```
$tag = Get-Tag -Name 'Tag1'
```

2. Create a rule with the *\$tag* tag and store the rule in the *\$rule* variable.

```
$rule = New-SpbmRule -AnyOfTags $tag
```

3. Create a rule set by using the *\$rule* rule and store the rule set in the *\$ruleset* variable.

```
$ruleset = New-SpbmRuleSet -AllOfRules $rule
```

4. Create a tag-based policy named *Tag-Based-Policy* by using the *\$ruleset* rule set and store the policy in the *\$policy* variable.

```
$policy = New-SpbmStoragePolicy -Name 'Tag-Based-Policy' -Description 'This policy is created by using a tag' -AnyOfRuleSets $ruleset
```

Create a Capability-Based Storage Policy

You can create storage policies by using vendor-exposed capabilities.

- Verify that you are connected to a system.
- Verify that you have Profile-driven storage update privileges.
- Verify that a storage provider is registered with the vCenter system.

1. Get the *VSAN.hostFailuresToTolerate* capability and store it in the *\$cap* variable.

```
$cap = Get-SpbmCapability -Name 'VSAN.hostFailuresToTolerate'
```

2. Create a rule with the *\$cap* capability and store the rule in the *\$rule* variable.

```
$rule = New-SpbmRule -Capability $cap -value 1
```

3. Create a rule set by using the *\$rule* rule and store the rule set in the *\$ruleset* variable.

```
$ruleset = New-SpbmRuleSet -AllOfRules $rule
```

4. Create a capability-based policy named *Capability-Based-Policy* by using the *\$ruleset* rule set and store the policy in the *\$policy* variable.

```
$policy = New-SpbmStoragePolicy -Name 'Capability-Based-Policy' -Description 'This policy is created by using capabilities' -AnyOfRuleSets $ruleset
```

Associate a Storage Policy with a Virtual Machine and Its Hard Disk

You can associate a storage policy with a virtual machine and its hard disk and check if they are compliant with the policy.

- Verify that you are connected to a system.
- Verify that a storage policy named *Str-Policy* exists in the vCenter environment.
- Verify that a virtual machine named *Target-VM* exists in the vCenter environment.

1. Get the *Str-Policy* storage policy and store it in the *\$policy* variable.

```
$policy = Get-SpbmStoragePolicy -Name 'Str-Policy'
```

2. Get the *Target-VM* virtual machine and store it in the *\$vm* variable.

```
$vm = Get-VM -Name 'Target-VM'
```

3. Get the hard disk associated with the *\$vm* virtual machine and store it in the *\$hd* variable.

```
$hd = Get-HardDisk -VM $vm
```

4. Assign the *\$policy* storage policy to the *\$vm* virtual machine and the *\$hd* hard disk.

```
Set-SpbmEntityConfiguration $vm, $hd -StoragePolicy $policy
```

5. View the *\$policy* storage policy's compliance with the *\$vm* virtual machine and the *\$hd* hard disk.

```
Get-SpbmEntityConfiguration $vm, $hd
```

Note: The storage policy can be compliant only if the datastore on which the virtual machine and hard disk are created is compliant with the storage policy.

Disassociate a Storage Policy Associated with a Virtual Machine and Its Hard Disk

You can disassociate a storage policy that is associated with a virtual machine and its hard disk.

- Verify that you are connected to a `system`.
- Verify that a virtual machine named `Target-VM` exists in the vCenter environment.
- Verify that a storage policy is associated with the `Target-VM` virtual machine.

1. Get the `Target-VM` virtual machine and store it in the `$vm` variable.

```
$vm = Get-VM -Name 'Target-VM'
```

2. Get the hard disk associated with the `$vm` virtual machine and store it in the `$hd` variable.

```
$hd = Get-HardDisk -VM $vm
```

3. Disassociate all storage policies that are associated with the `$vm` virtual machine and the `$hd` hard disk.

```
Set-SpbmEntityConfiguration $vm, $hd -StoragePolicy $null
```

Remove a Storage Policy

You can disassociate all entities associated with a storage policy and remove the policy completely.

- Verify that you are connected to a `system`.
- Verify that you have Profile-driven storage update privileges.
- Verify that a storage policy named `pol-tag` exists in the vCenter environment.

1. Get the `pol-tag` storage policy and store it in the `$policy` variable.

```
$policy = Get-SpbmStoragePolicy -Name 'pol-tag'
```

2. Disassociate all entities associated with the `$policy` storage policy.

```
Set-SpbmEntityConfiguration (Get-SpbmEntityConfiguration -StoragePolicy $policy) -StoragePolicy $null
```

3. Remove the `$policy` storage policy.

```
Remove-SpbmStoragePolicy -StoragePolicy $policy
```

Edit a Storage Policy

You can modify a storage policy to replace an existing rule set with a new rule set.

- Verify that you are connected to a `system`.
- Verify that you have Profile-driven storage update privileges.
- Verify that a storage provider is registered with the vCenter system.
- Verify that a storage policy named `pol-tag` exists in the vCenter environment.

1. Get the `pol-tag` storage policy and store it in the `$policy` variable.

```
$policy = Get-SpbmStoragePolicy -Name 'pol-tag'
```

2. Create a new rule and store it in the `$newRule` variable.

```
$newRule = New-SpbmRule -Capability (Get-SpbmCapability -Name 'VSAN.hostFailuresToTolerate') -Value 1
```

3. Create a new rule set by using the `$newRule` rule and store it in the `$newRuleset` variable.

```
$newRuleset = New-SpbmRuleSet -AllOfRules $newRule
```

4. Modify the `$policy` storage policy by replacing the existing rule set with the newly created `$newRuleset` rule set.

```
$modPolicy = Set-SpbmStoragePolicy -StoragePolicy $policy -AnyOfRuleSets $newRuleset
```

Export and Import a Storage Policy

You can back up a storage policy by exporting it as a file. You can later import the same storage policy.

- Verify that you are connected to a system.
- Verify that you have Profile-driven storage update privileges.
- Verify that you have read-write permissions for the directory in which the storage policy is saved.
- Verify that a storage policy named *pol-tag* exists in the vCenter environment.

1. Export the *pol-tag* storage policy.

```
Export-SpbmStoragePolicy -StoragePolicy 'pol-tag' -FilePath 'C:\Policy\pol-tag.xml'
```

2. Import the *pol-tag* storage policy and name it *Imported-Policy*.

```
Import-SpbmStoragePolicy -FilePath 'C:\Policy\pol-tag.xml' -Name 'Imported-Policy' -Description 'Imported policy description'
```

Create a Virtual Machine in a Datastore Compatible with Storage Policy

You can retrieve a datastore compatible with storage policy and create a virtual machine in the datastore.

- Verify that you are connected to a system.
- Verify that a tag-based storage policy named *Tag-Policy* exists in the vCenter environment.
- Verify that the tag of the *Tag-Policy* storage policy is associated with one of the available datastores in the vCenter environment.

1. Get the tag-based *Tag-Policy* storage policy and store it in the *\$policy* variable.

```
$policy = Get-SpbmStoragePolicy -Name 'Tag-Policy'
```

2. Get the tag used in the *Tag-Policy* storage policy and store it in the *\$tag* variable.

```
$tag = $($($policy.AnyOfRulesets).AllOfRules).AnyOfTags[0]
```

3. Get a datastore compatible with the *\$policy* storage policy and store it in the *\$ds* variable.

```
$ds = Get-SpbmCompatibleStorage -StoragePolicy $policy
```

4. Get the virtual machine host that contains the *\$ds* datastore and store it in the *\$vmhost* variable.

```
$vmHost = Get-VMHost -Datastore $ds
```

5. Create a virtual machine named *VM-Tag* in the *\$ds* datastore and store the virtual machine object in the *\$vm* variable.

```
$vm = New-VM -Name 'VM-Tag' -ResourcePool $vmHost -Datastore $ds -NumCPU 2 -MemoryGB 4 -DiskMB 1
```

6. Associate the *\$policy* storage policy with the *\$vm* virtual machine.

```
Set-SpbmEntityConfiguration $vm -StoragePolicy $policy
```

7. Verify that the *\$vm* virtual machine is compliant with the *\$policy* storage policy.

```
Get-SpbmEntityConfiguration $vm
```

The status should be *Compliant*.

8. Get the *Tag-Assignment* object for the *\$ds* datastore and store it in the *\$tagAs* variable.

```
$tagAs = Get-TagAssignment -Entity $ds
```

9. Remove the *\$tag* tag association from the *\$ds* datastore.

```
Remove-TagAssignment -TagAssignment $tagAs
```

10. Check the compliance of the *\$vm* virtual machine with the *\$policy* storage policy.

```
Get-SpbmEntityConfiguration $vm
```

The status should be `NonCompliant`.

Add a VASA Provider and Create a Policy

You can add a VASA provider to a vCenter system and create a storage policy.

- Verify that you are connected to a system.
- Verify that the datastore is mounted to the ESX host.

1. Add a VASA provider to the vCenter system.

```
$script:vasProvider = New-VasaProvider -Name 'name' -Url 'URL' -Username 'user_name' -Password 'password' -Description 'description' -Force
```

2. Get all SPBM capabilities exposed by the registered VASA provider.

```
Get-SpbmCapability
```

3. Create a new SPBM rule with the exposed capabilities of the registered VASA provider.

```
$rule = New-SpbmRule -Capability $capability -Value $value
```

4. Create a new SPBM rule set.

```
New-SpbmRuleSet -Name $ruleset -AllofRules @(($rule))
```

5. Create a storage policy.

```
New-SpbmStoragePolicy -Name $storagepolicy -RuleSet $ruleset
```

6. Refresh the VASA provider registered with the vCenter system.

```
$provider = Get-VasaProvider -Name $providername -Refresh
```

7. Verify the VASA storage array.

```
$vasaStorageArray = Get-VasaStorageArray -Provider $vasaProvider -Server $script:vcsvr
```

8. Refresh the VASA provider registered with the vCenter system.

```
$provider = Get-VasaProvider -Name $providername -Refresh
```

9. Get the VASA provider registered with the vCenter system.

```
$vasaProvider = Get-VasaProvider -Name $providername
```

10. Optional: Remove the VASA provider.

```
Remove-VasaProvider -Provider $provider -Confirm:$false
```

11. Optional: Verify that the VASA provider is removed.

```
$provider = Get-VasaProvider -Name $providername
```


Invoke a Planned Failover on a Replication Group and Reverse the Replication

You can invoke a planned failover on a target replication group and recover the devices on the target site. After that, you can reverse the direction of the replication and make the target site the source site.

- Verify that you are connected to the vCenter systems of the source and target sites.
- Verify that you have access to at least one virtual machine host on each site.
- Verify that you have registered a Virtual Volume VASA provider and have access to a Virtual Volume datastore on each site.

1. Create a storage policy with replication capability on the source site.

```
$replicationCapability = Get-SpbmCapability -Name *replication.RPO -Server $srcServer
$persistenceCapability = Get-SpbmCapability -Name *persistencel-readLatency -Server $srcServer
$replicationRule = New-SpbmRule -Capability $replicationCapability -Value (New-TimeSpan -Hours 4)
$persistenceRule = New-SpbmRule $persistenceCapability -Value 25
$ruleSet = New-SpbmRuleSet -AllOfRules $replicationRule, $persistenceRule
$replicationPolicy = New-SpbmStoragePolicy -Name cokeRep -AnyOfRuleSets $ruleSet -Server $srcServer
```

2. Get a datastore compatible with the created replication storage policy and store it in the *\$ds* variable.

```
$ds = Get-SpbmCompatibleStorage -StoragePolicy $replicationPolicy
```

3. Create a virtual machine named *MyVM* with a hard disk in the *\$ds* datastore.

```
$vm = New-VM -Name 'MyVM' -VMHost 'Host-Source' -DiskMB 512 -Datastore $ds
$hd = Get-HardDisk -VM $vm
```

4. Get a replication group for the *\$ds* datastore and the *\$replicationPolicy* storage policy, and store the replication group in the *\$rg* variable.

```
$rg = Get-SpbmReplicationGroup -Datastore $ds -StoragePolicy $replicationPolicy
```

5. Associate the *\$vm* virtual machine and its hard disk with the *\$replicationPolicy* storage policy, and put them in the *\$rg* replication group.

```
Set-SpbmEntityConfiguration -Configuration $vm, $hd -StoragePolicy $replicationPolicy
-ReplicationGroup $rg
```

6. Check the compliance of the *\$vm* virtual machine and *\$hd* hard disk with the *\$replicationPolicy* storage policy.

```
Get-SpbmEntityConfiguration $vm, $hd
```

7. Get the replication pair corresponding to the *\$rg* source replication group, and store that pair in the *\$rgPair* variable.

```
$rgPair = Get-SpbmReplicationPair -Source $rg
```

8. Synchronize the target replication group.

```
Sync-SpbmReplicationGroup $rgPair.Target
```

9. Power off the *\$vm* virtual machine and unregister it.

```
Stop-VM $vm
Remove-VM $vm
```

10. Prepare the failover on the source replication group.

```
Start-SpbmReplicationPrepareFailover $rgPair.Source
```

11. Synchronize the target replication group again, to get the latest state of the source devices.

```
Sync-SpbmReplicationGroup $rgPair.Target
```

12. Invoke the planned failover on the source replication group and store the virtual machine file path on the target site in the `$vmFilePath` variable.

```
$vmFilePath = Start-SpbmReplicationFailover $rgPair.Target
```

13. Register the virtual machine on the `Host-Target` host and power on the virtual machine.

```
$vm = New-VM -VMFilePath $vmFilePath -VMHost 'Host-Target'
Start-VM $vm
```

14. Reverse the direction of the replication.

```
Start-SpbmReplicationReverse $rgPair.Target
```

Attach a Flat VDisk to a Virtual Machine

You can create a flat `VDisk` object and attach it to a virtual machine as a hard disk. After that, you can verify whether the operation was completed successfully.

- Verify that you are connected to a system.
- Verify that you have access to at least one virtual machine host.
- Verify that there is at least one datastore mounted on the virtual machine host.

1. Mount the datastore on the virtual machine host and store the datastore in the `$ds` variable.

```
$vmHost = Get-VMHost 'Host-A'
$ds = Get-Datastore -RelatedObject $vmHost
```

2. Create a flat, thin-provisioned virtual disk with 2-GB capacity on the `$ds` datastore.

```
$vDisk = New-VDisk -Name 'VirtualDisk' -DiskType Flat -StorageFormat Thin -CapacityGB 2 -Datastore $ds
```

3. Create a virtual machine named `VirtualMachine` with one hard disk and store this virtual machine in the `$vm` variable.

```
$vm = New-VM -Name 'VirtualMachine' -VMHost 'Host-A' -Datastore $ds -DiskMB 512
```

4. Power on the `$vm` virtual machine.

```
Start-VM -VM $vm
```

5. Attach the `VDisk` object to the `$vm` virtual machine.

```
New-HardDisk -VM $vm -VDisk $vDisk
```

6. Verify that the `VDisk` object has been attached and the virtual machine now has two hard disks.

```
Get-HardDisk -VM $vm
```

Create an NFS 4.1 Datastore

You can create an NFS 4.1 datastore with Kerberos authentication and multipathing.

- Verify that you are connected to a system.
- Verify that the remote NFS share supports multipathing and Kerberos authentication.

1. Get the virtual machine host where you want to create the NFS 4.1 datastore.

```
$vmhost = Get-VMHost 'hostname'
```

2. Set NTP servers for the virtual machine host.

```
Add-VMHostNtpServer -VMHost $vmhost -NtpServer 'ntp_server_ip'
```

3. Set a DNS server and search the domain for the virtual machine host.

```
$vmhostnetwork = Get-VMHostNetwork -VMHost $vmhost
Set-VMHostNetwork -Network $vmhostnetwork -DnsFromDhcp $false -DnsAddress 'dns_server_ip' -DomainName
'domain_name' -SearchDomain 'search_domain'
```

4. Add the virtual machine to the Active Directory domain.

```
$vmhost | Get-VMHostAuthentication | Set-VMHostAuthentication -JoinDomain -Domain 'AD_domain_name'
-Username 'AD_user_name' -Password 'AD_password'
```

5. Create an NFS user on the virtual machine host for Kerberos-based authentication for the NFS 4.1 datastore.

```
New-NfsUser -VMHost $vmhost -Username 'NFS_user_name' -Password 'password'
```

6. Create an NFS 4.1 datastore with Kerberos authentication and multipathing.

```
New-Datastore -Name 'NFS_datastore_name' -Nfs -FileSystemVersion '4.1' -VMHost $vmhost -Nf-
sHost @('remote_host_1_ip', 'remote_host_2_ip') -Path 'NFS_datastore_remote_path' -Kerberos
```

7. Optional: Retrieve the datastore.

```
$ds = Get-Datastore 'NFS_datastore_name'
```

8. Optional: Remove the datastore.

```
Remove-Datastore $ds -VMHost $vmhost
```

9. Optional: Get the NFS user from the virtual machine host.

```
$user = Get-NfsUser -VMHost $vmhost
```

10. Optional: Update the password of the NFS user.

```
$user = Set-NfsUser -NfsUser $user -Password 'new_password'
```

11. Optional: Remove the NFS user.

```
Remove-NfsUser -NfsUser $user
```

Managing Certificates

You can use VMware Cloud Foundation PowerCLI to manage certificates for both vCenter and ESX no matter if you are using Hybrid, Subordinate CA, or Full Custom Mode.

Starting from VMware Cloud Foundation PowerCLI 12.4.0, you can do various certificate management operations in your vSphere environment. The certificate management functionality is included in the `VMware.PowerCLI.VCenter` module.

Add a Root Certificate to vCenter and ESX

You can use VCF PowerCLI to add a root certificate or certificate chain to the trusted root store of vCenter and to the certificate stores of the connected ESX hosts.

- Verify that you are connected to a vCenter system.
 - Verify that the root certificate or certificate chain you want to add is available on your machine.
1. Optional: List the trusted root certificates on your vCenter system and the connected ESX hosts.

```
Get-VITrustedCertificate
```

2. Create a variable with the root certificate or certificate chain you want to add.

```
$trustedCertChain = Get-Content "C:\Users\jdoe\Downloads\ca-chain.cert.pem" -Raw
```

3. Add the root certificate or certificate chain to vCenter and to the connected ESX hosts

```
Add-VITrustedCertificate -PemCertificateOrChain $trustedCertChain
```

4. Optional: Verify that your new root certificate or certificate chain has been added to the trusted certificate stores.

```
Get-VITrustedCertificate
```

Change the Machine SSL Certificate of vCenter

You can use VCF PowerCLI to change the Machine SSL certificate of a vCenter system. For a custom certificate, you must generate a certificate signing request (CSR) and send it to the certificate authority (CA) of your choice.

- Verify that you are connected to a vCenter system.
- Verify that the root certificate of the CA you are going to use is added to the trusted root store of vCenter.

1. Optional: Retrieve the current Machine SSL certificate of the vCenter system.

```
Get-VIMachineCertificate -VCenterOnly
```

2. Generate a CSR.

```
$csrParams = @{
    Country="US"
    Email="jdoe@vmware.com"
    Locality="San Francisco"
    Organization="My Company"
    OrganizationUnit="PowerCLI"
    StateOrProvince="California"
}
$csr = New-VIMachineCertificateSigningRequest @csrParams
```

3. Save the CSR to your system.

```
$csr.CertificateRequestPEM | Out-File "C:\Users\jdoe\Downloads\vc.csr.pem" -Force
```

4. Send the CSR to the CA of your choice.
5. Save the issued custom certificate to your system.
6. Set the new custom certificate to the vCenter system.

```
$vcCert = Get-Content "C:\Users\jdoe\Downloads\vc.cert.jdoe.pem" -RawSet-VMachineCertificate -PemCertificate $vcCert
```

Note:

Starting with vSphere 8.0 Update 2, restart of vCenter services after the certificate change is no longer necessary. The certificate replacement is completed seamlessly and all your sessions remain active.

For older vSphere versions, the change of the Machine SSL certificate triggers a restart of vCenter. Wait for the system to reboot and log in when available.

Change the Machine SSL Certificate of an ESX Host

You can use VCF PowerCLI to change the Machine SSL certificates of one or more ESX hosts in your vSphere environment.

- Verify that the root certificate of the CA you are going to use is added to the trusted root store of vCenter and to the connected ESX hosts.

1. Connect to the vCenter system.

```
$vCenterConnection = Connect-VIServervc1.example.com `
    -User 'My User' `
    -Password 'My Password'
```

2. In the vCenter system, retrieve the setting for the ESX host certificate management mode.

```
$certModeSetting = Get-AdvancedSetting "vpzd.certmgmt.mode" -Entity $vCenterConnection
$certModeSetting.Value
```

3. Change the setting to *custom*.

```
Set-AdvancedSetting $certModeSetting -Value "custom"
```

4. For the new certificate management mode to take effect, reboot your vCenter system.
5. Connect to the vCenter system.

```
$vCenterConnection = Connect-VIServervc1.example.com `
    -User 'My User' `
    -Password 'My Password'
```

6. Set the ESX host you want to manage to *Maintenance* mode.

```
$vmhost = Get-VMHost 'MyESXHost' `
Set-VMHost -VMHost $vmhost -StateMaintenance
```

7. Generate a certificate signing request (CSR) for the ESX host.

```
$esxRequest = New-VMachineCertificateSigningRequest `
    -VMHost $vmhost `
    -Country "US" `
    -Locality "San Francisco" `
    -Organization "My Company" `
    -OrganizationUnit "PowerCLI" `
    -StateOrProvince "California" `
```

```
-CommonName <ESX host's FQDN> or <ESX host's IP address>
```

Note:

For `CommonName`, you must use either the ESX host's FQDN or IP address. The common name must match the identifier you use to add the host to the vCenter system.

8. Save the CSR to your system.

```
$esxRequest.CertificateRequestPEM | Out-File "C:\Users\jdoe\Downloads\esx.csr.pem" -Force
```

9. Send the CSR to the CA of your choice.**10. Save the issued custom certificate to your machine.****11. Create a variable with your issued custom certificate.**

```
$esxCertificatePem = Get-Content "C:\Users\jdoe\downloads\myesxcert.pem" -Raw
```

12. Remove the ESX host from the vCenter system.

```
Remove-VMHost $vmhost
```

13. Disconnect from the vCenter system.

```
Disconnect-VIServer $vCenterConnection
```

14. Connect directly to the ESX host.

```
$esxConnection = Connect-VIServer $vmhost.Name `
    -User 'My User' `
    -Password 'My Password' `
    -Force
```

15. Set the custom Machine SSL certificate to the ESX host.

```
$targetEsxHost = Get-VMHost $vmhost.Name
Set-VIMachineCertificate -PemCertificate $esxCertificatePem -VMHost $targetEsxHost | Out-Null
```

16. To apply the change, restart the ESX host.

```
Restart-VMHost $targetEsxHost
```

17. Disconnect from the ESX host.

```
Disconnect-VIServer $esxConnection
```

18. Connect to the vCenter system.

```
$vCenterConnection = Connect-VIServervc1.example.com `
    -User 'My User' `
    -Password 'My Password'
```

19. Add the ESX host to the vCenter system.

```
$vmhost = Add-VMHost -Name<ESX host's FQDN> or <ESX host's IP address> `
    -Location (Get-Datacenter "My Datacenter") `
    -User "My User" `
    -Password "My Password"
```

20. Set the ESX host to the *Connected* mode.

```
$vmhost = Set-VMHost -VMHost $vmhost -StateConnected
```

21. Optional: Verify that the Machine SSL certificate of the ESX host is changed.

```
Get-VMachineCertificate -VMHost $vmhost
```

Using Get-View

Use `Get-View` to access vSphere Management API view objects with VCF PowerCLI.

The `Get-View` cmdlet operates on the low level and binds directly to the vSphere Management API objects. You can use this cmdlet for high-performance automation as it overrides the VCF PowerCLI high-level architecture.

You can use `Get-View` in combination with `Get-VIObjectByVIView` to retrieve the VCF PowerCLI variant of a vSphere Management API view object.

Filter vSphere Objects with Get-View

You can use the `Get-View` cmdlet to filter vSphere objects before performing various actions on them.

Verify that you are connected to a system.

The filter parameter is a `HashTable` object containing one or more pairs of filter criteria. Each of the criteria consists of a property path and a value that represents a regular expression pattern used to match the property.

1. Create a filter by the power state and the guest operating system name of the virtual machines.

```
$filter = @{"Runtime.PowerState" = "poweredOn"; "Config.GuestFullName" = "Windows XP"}
```

2. Get a list of the virtual machines by using the created filter and call the `ShutdownGuest` method for each virtual machine in the list.

```
Get-View -ViewType "VirtualMachine" -Filter $filter | foreach{$_ .ShutdownGuest() }
```

The filter gets a list of the powered-on virtual machines whose guest OS names contain the string `Windows XP`. The `Get-View` cmdlet then initiates shutdown for each guest operating system in the list.

Populate a View Object with Get-View

To save time and efforts, you can use `Get-View` to retrieve VCF PowerCLI views from previously retrieved view objects.

Verify that you are connected to a system.

1. Get a view of the `VM2` virtual machine by name.

```
$vm2 = Get-View -ViewType VirtualMachine -Filter @{"Name" = "VM2"}
```

2. Populate the `$vmhostView` object.

```
$vmhostView = Get-View -Id $vm2.Runtime.Host
```

3. Retrieve the runtime information for the `$vmhostView` object.

```
$vmhostView.Summary.Runtime
```

Update the State of a Server-Side Object

You can use the `Get-View` cmdlet to update server-side objects.

Verify that you are connected to a system.

1. Get the *VM2* virtual machine by name.

```
$vm2 = Get-View -ViewType VirtualMachine -Filter @{"Name" = "VM2"}
$vmhostView = Get-View -Id $vm2.Runtime.Host
```

2. View the current power state.

```
$vm2.Runtime.PowerState
```

3. Power off the virtual machine.

```
If ($vm2.Runtime.PowerState -ne "PoweredOn") {
    $vm.PowerOnVM($vm2.Runtime.Host)
} else {
    $vm2.PowerOffVM()
}
```

4. View the value of the *\$vm2* power state.

```
$vm2.Runtime.PowerState
```

The power state is not updated yet because the virtual machine property values are not updated automatically.

5. Update the view object.

```
$vm2.UpdateViewData()
```

6. Obtain the actual power state of the virtual machine.

```
$vm2.Runtime.PowerState
```

Reboot a Host with Get-View

You can reboot a host by using its corresponding view object.

Verify that you are connected to a `system`.

1. Use the `Get-VMHost` cmdlet to get a host by its name, and pass the result to the `Get-View` cmdlet to get the corresponding view object.

```
$vmhostView = Get-VMHost -Name Host | Get-View
```

2. Call the reboot method of the host view object to reboot the host.

```
$vmhostView.RebootHost()
```

Modify the CPU Levels of a Virtual Machine with Get-View and Get-VIObjectByVIView

You can use `Get-View` in combination with `Get-VIObjectByVIView` to invoke a vSphere Management API view object and retrieve its VCF PowerCLI variant.

Verify that you are connected to a `system`.

You can modify the CPU levels of a virtual machine by using a combination of the `Get-View` and `Get-VIObjectByVIView` cmdlets.

1. Get the *VM2* virtual machine, shut it down, and pass it to the `Get-View` cmdlet to view the virtual machine view object.

```
$vmView = Get-VM VM2 | Stop-VM | Get-View
```

2. Create a *VirtualMachineConfigSpec* object to modify the virtual machine CPU levels and call the `ReconfigVM` method of the virtual machine view managed object.

```
$spec = New-Object VMware.Vim.VirtualMachineConfigSpec;
$spec.CPUAllocation = New-Object VMware.Vim.ResourceAllocationInfo;
$spec.CpuAllocation.Shares = New-Object VMware.Vim.SharesInfo;
$spec.CpuAllocation.Shares.Level = "normal";
$spec.CpuAllocation.Limit = -1;
```



```
$vmView .ReconfigVM_Task($spec)
```

3. Get the virtual machine object by using the `Get-VIObjectByVIView` cmdlet and start the virtual machine.

```
$vm = Get-VIObjectByVIView $vmView | Start-VM
```

Creating Custom Properties

You can use VCF PowerCLI to create custom properties for different PowerCLI object types. Then, you can use the custom properties to retrieve objects based on specified criteria.

Create a Custom Property Based on an Extension Data Property

You can create custom properties to add more information to vSphere objects. Custom properties based on extension data properties correspond directly to the property of the corresponding .NET view object.

Verify that you are connected to a system.

1. Create a new custom property based on the `Guest.ToolsVersion` property.

```
New-VIProperty -ObjectType VirtualMachine -Name ToolsVersion -ValueFromExtensionProperty
'Guest.ToolsVersion'
```

2. View the `ToolsVersion` properties of the available virtual machines.

```
Get-VM | Select Name, ToolsVersion
```

You have created a custom property named `ToolsVersion` for `VirtualMachine` objects.

Create a Script-Based Custom Property for a vSphere Object

You can create a custom property by writing a script and providing a name for the property. The script evaluates when the custom property is called for the first time.

Verify that you are connected to a system.

1. Create a new custom property named `NameOfHost` that stores the name of the host on which a virtual machine resides.

```
New-VIProperty -Name NameOfHost -ObjectType VirtualMachine -Value { return
$args[0].VMHost.Name }
```

2. View the `NameOfHost` properties of the available virtual machines.

```
Get-VM | select Name, NameOfHost | Format-Table -AutoSize
```

You created a custom script property named `NameOfHost` for `VirtualMachine` objects.

Using the VCF PowerCLI Inventory Provider

Use the VCF PowerCLI Inventory Provider to interact with the VMware vSphere inventory through a file system interface, so that it is easier to navigate, manipulate, and automate tasks.

The VCF PowerCLI Inventory Provider allows navigation and file-style management of the VMware vSphere inventory. By creating a PowerShell drive based on a managed object (such as a data center), you can obtain a view of its contents and

the relationships between the items. In addition, you can move, rename, or delete objects by running commands from the PowerShell console.

When you connect to a server with `Connect-VIServer`, the cmdlet builds two default inventory drives: `vi` and `vis`.

To access the inventory of the last connected server, run: `cd vi:\`.

To access the inventory of all vSphere servers connected within the current VCF PowerCLI session, run `cd vis:\`.

Use the `dir` and `cd` commands to navigate the inventories.

Browse the Default Inventory Drive

You can browse the default inventory drive and view its contents.

Verify that you are connected to a system.

Note: For more information about the Inventory Provider and the default inventory drive, see [Using the VCF PowerCLI Inventory Provider](#).

1. Navigate to the `vi` inventory drive.

```
cd vi:
```

2. View the drive content.

```
dir
```

`dir` is an alias of the `Get-ChildItem` cmdlet.

Create a New Custom Inventory Drive

In addition to the default drive, you can create new custom inventory drives by using the `New-PSDrive` cmdlet.

Verify that you are connected to a system.

Note: An alternative to creating an inventory drive is to map an existing inventory path. For example, run: `New-PSDrive -Name myVi -PSProvider VimInventory -Root "vi:\Folder01\Datacenter01"`.

1. Get the root folder of the server.

```
$root = Get-Folder -NoRecursion
```

2. Create a PowerShell drive named `myVi` in the server root folder.

```
New-PSDrive -Location $root -Name myVi -PSProvider VimInventory -Root '\'
```

Note: You can use the `New-InventoryDrive` cmdlet, which is an alias of `New-PSDrive`. This cmdlet creates a new inventory drive using the `Name` and `Datastore` parameters. For example: `Get-Folder -NoRecursion | New-VIInventoryDrive -Name myVi`.

Manage Inventory Objects Through Inventory Drives

You can use the VCF PowerCLI Inventory Provider to browse, modify, and remove inventory objects from inventory drives.

Verify that you are connected to a system.

1. Navigate to a host in your server inventory by running the `cd` command with the full path to the host.

```
cd Folder01\DataCenter01\host\Web\Host01
```

2. View the content of the host using the `ls` command.

```
ls
```

`ls` is the UNIX style alias of the `Get-ChildItem` cmdlet.

This command returns the virtual machines and the root resource pool of the host.

3. View only the virtual machines on the host.

```
Get-VM
```

When called within the inventory drive, `Get-VM` gets a list only of the virtual machines on the current drive location.

4. Delete a virtual machine named `VM1`.

```
del VM1
```

5. Rename a virtual machine, for example, from `VM1New` to `VM1`.

```
ren VM1New VM1
```

6. Start all virtual machines with names that start with `VM`.

```
dir VM* | Start-VM
```

Using the VCF PowerCLI Datastore Provider

Use the VCF PowerCLI Datastore Provider to access the contents of one or more datastores through a file system interface, so that it is easier to navigate, manipulate, and automate tasks.

The items in a datastore are files that contain configuration, virtual disk, and other data associated with a virtual machine.

When you connect to a server with `Connect-VIServer`, the cmdlet builds two default datastore drives: `vmstore` and `vmstores`.

To access the datastores of the last connected vCenter or ESX, run: `cd vmstore:\`.

To access the datastores available on all vCenter and ESX systems you are currently connected to, run: `cd vmstores:\`.

Use the `dir` and `cd` commands to navigate the inventories.

Browse the Default Datastore Drives

You can use the VCF PowerCLI Datastore Provider to browse the default datastore drives: `vmstore` and `vmstores`.

Verify that you are connected to a system.

Note: For more information about default datastore drives, see [Using the VCF PowerCLI Datastore Provider](#).

1. Navigate to the `vmstore` drive.

```
cd vmstore:
```

2. View the drive content.

```
dir
```

Create a New Custom Datastore Drive

You can use the VCF PowerCLI Datastore Provider to create custom datastore drives.

Verify that you are connected to a system.

1. Get a datastore by its name and assign it to the `$datastore` variable.

```
$datastore = Get-Datastore Storage1
```

2. Create a new PowerShell drive `ds:` in `$datastore`.

```
New-PSDrive -Location $datastore -Name ds -PSProvider VimDatastore -Root '\'
```

Note: You can use the `New-PSDrive` cmdlet, which is an alias of `New-DatastoreDrive`. It creates a new datastore drive using the `Name` and `Datastore` parameters. For example: `Get-Datastore Storage1 | New-DatastoreDrive -Name ds`.

Manage Datastores Through Datastore Drives

You can use the VCF PowerCLI Datastore Provider to browse datastores from datastore drives.

Verify that you are connected to a system.

1. Navigate to a folder on the `ds:` drive.

```
cd VirtualMachines\XPVirtualMachine
```

2. View the files of the folder by running the `ls` command.

```
ls
```

`ls` is the UNIX style alias of the `Get-ChildItem` cmdlet.

3. Rename a file by running the `Rename-Item` cmdlet or its alias `ren`.

For example, to change the name of the `vmware-3.log` file to `vmware-3old.log`, run:

```
ren vmware-3.log vmware-3old.log
```

All file operations apply only on files in the current folder.

4. Delete a file by running the `Remove-Item` cmdlet or its alias `del`.

For example, to remove the `vmware-3old.log` file from the `XPVirtualMachine` folder, run:

```
del ds:\VirtualMachines\XPVirtualMachine\vmware-2.log
```

5. Copy a file by running the `Copy-Item` cmdlet or its alias `copy`.

```
copy ds:\VirtualMachines\XPVirtualMachine\vmware-3old.log ds:\VirtualMachines\vmware-3.log
```

6. Copy a file to another datastore by running the `Copy-Item` cmdlet or its alias `copy`.

```
copy ds:\Datacenter01\Datastore01\XPVirtualMachine\vmware-1.log ds:\Datacenter01\Datastore02\XPVirtualMachine02\vmware.log
```

7. Create a new folder by running the `New-Item` cmdlet or its alias `mkdir`.

```
mkdir -Path ds:\VirtualMachines -Name Folder01 -Type Folder
```

8. Download a file from the datastore drive to the local machine by running the `Copy-DatastoreItem` cmdlet.

```
Copy-DatastoreItem ds:\VirtualMachines\XPVirtualMachine\vmware-3.log C:\Temp\vmware-3.log
```

9. Upload a file from the local machine by running the `Copy-DatastoreItem` cmdlet.

```
Copy-DatastoreItem C:\Temp\vmware-3.log ds:\VirtualMachines\XPVirtualMachine\vmware-3new.log
```

Managing vSphere Lifecycle Manager Images and Baselines with VMware Cloud Foundation PowerCLI

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Lifecycle Manager administration.

You can use VCF PowerCLI to manage the lifecycle of all hosts in a cluster collectively or of standalone hosts with vSphere Lifecycle Manager images.

You can use cluster-level and host-level administration starting from the following vSphere and VCF PowerCLI product versions:

Table 1895: vSphere Lifecycle Manager Availability

	Available from vSphere version	Available from PowerCLI version
Cluster-level administration	7.0	12.1.0
Host-level administration	8.0	13.1.0

Understanding vSphere Lifecycle Manager

With VMware vSphere Lifecycle Manager, you can manage ESX clusters and hosts by using images.

Important:

In vSphere Foundation 9.1, you cannot import into your vCenter instance a custom depot ZIP that you exported with `New-OfflineBundle` cmdlet of VCF PowerCLI 9.0 and earlier.

Starting with vSphere 7.0 and VMware PowerCLI 12.1.0, you can use high-level cmdlets for operations on the cluster level as a part of the `VMware.VimAutomation.Core` module. Starting with vSphere 7.0 and VMware PowerCLI 12.4.0, you can also use low-level cmdlets for cluster-level operations.

Starting with vSphere 8.0 and VMware PowerCLI 13.1.0, you can use high-level and low-level cmdlets for vSphere Lifecycle Manager host-level operations.

The low-level cmdlets map directly to the syntax of the [ESX REST APIs](#) and empower you to use the full API functionality within your PowerShell console.

You can construct low-level SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the `ESX REST API` data structures or operations.

The [ESX REST API Reference](#) on VMware Developer features ready-to-use vSphere Lifecycle Manager SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.

Lifecycle Management in vSphere

Lifecycle management refers to the process of installing software, maintaining it through updates and upgrades, and decommissioning it.

In the context of maintaining a vSphere environment, your clusters and hosts in particular, lifecycle management refers to tasks such as installing and firmware on new hosts, and updating or upgrading the version and firmware when required.

What Is vSphere Lifecycle Manager

vSphere Lifecycle Manager is a service that operates within vCenter, by using the built-in PostgreSQL database. No additional setup is required to use vSphere Lifecycle Manager. When you deploy the vCenter appliance, the vSphere Lifecycle Manager user interface is automatically activated in the HTML5-based vSphere Client. You can also access vSphere Lifecycle Manager functions through VCF PowerCLI.

vSphere Lifecycle Manager encompasses the functionality of Update Manager from previous vSphere versions and enhances it by introducing new features and options for managing the ESX lifecycle at a cluster and host level.

Up to vSphere 7.0, the Update Manager allowed the use of baselines and baseline groups for patching and upgrading hosts. With vSphere 7.0, vSphere Lifecycle Manager introduced the alternative approach of using vSphere Lifecycle Manager images to manage the life cycle of hosts and clusters in your environment. It also makes possible the upgrade of virtual machine hardware and VMware Tools versions.

vSphere Lifecycle Manager is compatible with an Internet-connected environment, either directly or through a proxy server. It can also operate in a secure network without Internet access. In such cases, you can use the Update Manager Download Service (UMDS) to download updates to the vSphere Lifecycle Manager depot, or import updates manually.

vSphere Lifecycle Manager and VCF PowerCLI

Starting with vSphere 7.0 and VCF PowerCLI 12.1.0, you can create and manage vSphere Lifecycle Manager clusters. For more information, see [Creating and Managing vSphere Lifecycle Manager Clusters with VCF PowerCLI](#).

Starting with vSphere 8.0 and VCF PowerCLI 13.1.0, you can manage standalone ESX hosts with vSphere Lifecycle Manager images. For more information, see [Managing vSphere Lifecycle Manager Standalone Hosts with VCF PowerCLI](#).

Creating and Managing vSphere Lifecycle Manager Clusters with VCF PowerCLI

Starting with vSphere 7.0 and VMware Cloud Foundation PowerCLI 12.1.0, you can create and manage vSphere Lifecycle Manager clusters with VCF PowerCLI.

Cluster-level Functionality Starting from vSphere 7.0 and VMware Cloud Foundation PowerCLI 12.1.0

Starting from vSphere 7.0 and VMware Cloud Foundation PowerCLI 12.1.0, some of the cluster-level operations that you can perform are:

- Create a vSphere Lifecycle Manager cluster with a desired state.
- View desired state elements:
 - ESX version
 - Vendor add-ons
 - Firmware/ drivers
 - Components
- View/ check recommended images.
- Add hosts to a cluster.
- Retrieve a cluster with a vLCM desired state.
- Update a cluster's ESX base image.

- Add, remove, replace, upgrade, or downgrade a vendor add-on on a cluster.
- Update a cluster's vSphere Lifecycle Manager component list.
- Add, remove, replace, upgrade, or downgrade a firmware add-on to a cluster.
- Modify the desired state of a vSphere Lifecycle Manager cluster.
- Convert a Update Manager cluster into a vSphere Lifecycle Manager cluster.
- Check compliance of the ESX hosts on a cluster.
- Apply the desired state on hosts in a cluster, whose current state is different from the desired specification (remediate cluster).
- Export a desired state from a cluster.
- Import a desired state to a cluster.
- View hardware compatibility.

Cluster-level Functionality Starting from vSphere 8.0 and VMware Cloud Foundation PowerCLI 13.1.0

Starting from vSphere 8.0 and VMware Cloud Foundation PowerCLI 13.1.0, some of the new cluster-level operations that you can perform are:

- Update hardware compatibility list.
- View parallel remediation settings.
- Activate/ deactivate parallel remediation.
- Create or modify a cluster by importing an ESX base image.
- Retrieve pre-check results.
- Stage a cluster.

Sample Scripts for vSphere Lifecycle Manager Cluster-Level Operations

The following scripts demonstrate how you can use vSphere Lifecycle Manager on the cluster level with VCF PowerCLI.

Retrieve vSphere Lifecycle Manager (vLCM) Image Profiles

You can retrieve vLCM images from a vCenter system based on different filters by using the `Get-LcmImage` cmdlet. You can pass IDs or a combination of name and version filters to the `Get-LcmImage` parameters.

List all types of vLCM images

```
Get-LcmImage
```

List only `BaseImage` (ESX) vLCM images

```
Get-LCMImage -Type 'BaseImage'
```

List only `VendorAddOn` vLCM images

```
Get-LCMImage -Type 'VendorAddOn'
```

List only `Component` vLCM images

```
Get-LCMImage -Type 'Component'
```

List only `Package` (FirmwareAddOn) vLCM images

```
Get-LCMImage -Type 'Package'
```

Create a New Cluster with a vLCM Desired State

You can create a new vSphere Lifecycle Manager cluster in the following ways:

- By specifying parameters from the VMware depots:

```
$clusterName= Read-Host -Prompt 'Provide the cluster Name'
$vlcmBaseImage = Get-LCMImage -Version '7.0 GA - 15843807' -Type BaseImage
```

```
$vendorAddon = Get-LcmImage -Name 'NEC-addon-GEN' -Version '7.0.2-02'
New-Cluster -Location Datacenter -Name $clusterName -BaseImage $vLCMBaseImage -VendorAddon $vendorAddon -
HAEnabled -DrsEnabled
```

- By importing a vLCM desired state spec.

```
New-Cluster -Name 'MyCluster' -Location <MyDatacenter> | Import-LcmClusterDesiredState -LocalSpecLocation
F:\Image\desired-state-spec.json
```

Retrieve a Cluster's Desired State

```
Get-Cluster -Name $clusterName | Select-Object -Property Name, BaseImage, @{n='BaseImageVersion'; e={$_.BaseI-
mage.Version}}, Components, VendorAddon, FirmwareAddons
```

Update a Cluster's vLCM Base Image

You cannot remove or downgrade an ESX base image - you can only upgrade it. The following example demonstrates an upgrade of a cluster's desired state to ESX 7.0 U2.

```
$vLCMBaseImageu2 = Get-LcmImage -Version '7.0 U2a - 17867351' -Type BaseImage
Get-Cluster -Name $clusterName | Set-Cluster -BaseImage $vLCMBaseImageu2
```

Remove a vLCM Vendor Add-on from a Cluster

```
Get-Cluster -Name $clusterName | Set-Cluster -VendorAddon $null
```

Note: You can add, replace, remove, upgrade, or downgrade a vendor add-on.

Update a Cluster's vLCM Component List

Replace a Component list with a new one.

```
$components = Get-LcmImage -Id 'Component-<Component1 Name>/<Component1 Version>', 'Component-<Component2
Name>/<Component2 Version>'
Get-Cluster -Name $clusterName | Set-Cluster -Component $components
```

Note: To remove all components, use Set-Cluster -Component \$null

Add a vLCM Firmware Add-on

```
$firmwareAddon = Get-LcmImage -Id '<FirmwareAddOn_Id>'
Get-Cluster -Name $clusterName | Set-Cluster -FirmwareAddOn $firmwareAddon
```

Note: You can add, replace, remove, upgrade, or downgrade a firmware add-on.

Modify a Cluster's vLCM Desired State

```
$vLCMBaseImage = Get-LCMImage -Version '7.0 U3f - 20036589'
$vendorAddon = get-lcmimage -type vendoraddon -Version '703.0.0.10.9.0-11'
Set-Cluster 'vlcm-cluster-pcli' -BaseImage $vLCMBaseImage -VendorAddon $vendorAddon
```

Convert a Update Manager Cluster to a vSphere Lifecycle Manager Cluster

Convert a Update Manager cluster to a vSphere Lifecycle Manager cluster.

```
$vLCMBaseImage = Get-LCMImage -Version '7.0 U3f - 20036589'
$vendorAddon = get-lcmimage -type vendoraddon -Version '703.0.0.10.9.0-11'
Set-Cluster 'vum-cluster' -BaseImage $vLCMBaseImage -VendorAddon $vendorAddon
```


Check Cluster Compliance

Check the compliance of the hosts on the cluster against the cluster's desired state.

```
Get-Cluster -Name $clusterName | Test-LcmClusterCompliance
```

Remediate a Cluster

Apply the desired state to the hosts in the cluster, whose current state is different from the desired software specification.

```
Get-Cluster -Name $clusterName | Set-Cluster -Remediate -AcceptEULA
```

Export a vLCM Desired State

```
Get-Cluster -Name $clusterName | Export-LcmClusterDesiredState -Destination 'F:\Image' -ExportOfflineBundle -
ExportIsoImage
```

Import a vLCM Desired State

```
Import-LcmClusterDesiredState -Cluster 'ExistingCluster' -LocalSpecLocation F:\Image\desired-state-spec.json -
Verbose
```

View / Check Recommended Images

Generate a desired state recommendation of a vSphere Lifecycle Manager cluster.

- Based on the latest ESX base image version.

```
Get-LcmClusterDesiredStateRecommendation 'lcm-cluster' -Latest
```

- Based on the latest patch or update of the currently installed ESX base image version.

```
Get-LcmClusterDesiredStateRecommendation 'lcm-cluster' -Current
```

View Hardware Compatibility

```
Get-LcmHardwareCompatibility -Cluster 'lcm-cluster'
```

Update Hardware Compatibility List

```
Invoke-DownloadHclCompatibilityDataAsync
```

View Parallel Remediation Settings

```
Invoke-GetDefaultsClustersPoliciesApply
Invoke-GetClusterPoliciesApply -Cluster $clusterId
```

Activate or Deactivate Parallel Remediation

Activate or deactivate parallel remediation with the maximum number of concurrent remediations.

```
$ParallelRemediationSetting = Initialize-SettingsDefaultsClustersPoliciesParallelRemediationAction -Enabled
$true -MaxHosts $maxHostCount
$PolicySpec = Initialize-SettingsDefaultsClustersPoliciesApplyConfiguredPolicySpec -ParallelRemediationAction
$ParallelRemediationSetting
Invoke-SetDefaultsClustersPoliciesApply -Cluster $clusterID -SettingsClustersPoliciesApplyConfiguredPolicySpec
$PolicySpec
```

Create a Cluster by Importing an Image from a Standalone Host

Extract a software specification from an ESX host and import it into the software depot.

```
$SettingsDepotsOfflineHostCredentials = Initialize-SettingsDepotsOfflineHostCredentials -HostName "MyHostName"
-UserName "MyUserName" -Password "MyPassword" -Port 0 -SslThumbPrint "MySslThumbPrint"
$SettingsDepotsOfflineConnectionSpec = Initialize-SettingsDepotsOfflineConnectionSpec -AuthType "USER-
NAME_PASSWORD" -HostCredential $SettingsDepotsOfflineHostCredentials
Invoke-CreateFromHostDepotsOfflineAsync -SettingsDepotsOfflineConnectionSpec $SettingsDepotsOfflineConnec-
tionSpec
```

Then use `New-Cluster` or `Set-Cluster` to create or modify a cluster's desired state from the returned software spec.

Retrieve Pre-check Results

Return the last remediation pre-check results.

```
$cluster = Get-Cluster -Name <cluster_name>
Invoke-GetClusterSoftwareReportsLastCheckResult -Cluster $cluster.ExtensionData.MoRef.Value
```

Stage a Cluster

Stage a desired software specification on a vSphere Lifecycle Manager cluster.

```
$SettingsClustersSoftwareStageSpec = Initialize-SettingsClustersSoftwareStageSpec -Commit <myCommit> -Hosts
<MyHosts>
Invoke-StageClusterSoftwareAsync -Cluster <MyCluster> -SettingsClustersSoftwareStageSpec $spec
```

Managing vSphere Lifecycle Manager Standalone Hosts with VCF PowerCLI

Starting with vSphere 8.0 and VMware Cloud Foundation PowerCLI 13.1.0, you can manage vSphere Lifecycle Manager standalone hosts with VCF PowerCLI.

Host-level Functionality in VCF PowerCLI

In VCF PowerCLI 13.1.0 and later, you can use both high-level and low-level cmdlets that bind directly to the [ESX REST APIs](#) to manage vSphere Lifecycle Manager hosts.

Starting with VMware Cloud Foundation PowerCLI 13.1.0, some of the host-level operations you can perform are:

- Activate vSphere Lifecycle Manager on a standalone host.
- Retrieve vSphere Lifecycle Manager image profiles:
 - ESX version
 - Vendor add-ons
 - Firmware/ drivers
 - Components
- View/ check recommended images.
- Retrieve a host with a vSphere Lifecycle Manager desired state.
- Modify the desired state of a vSphere Lifecycle Manager host.
- Convert a Update Manager host into a vSphere Lifecycle Manager host.
- Update the ESX base image of a standalone host.
- Add, remove, replace, upgrade, or downgrade a vendor add-on on a standalone host.
- Add remove, replace, upgrade, or downgrade a vSphere Lifecycle Manager firmware add-on on a standalone host.
- Check compliance of a standalone ESX host.
- Pre-check a host.
- Stage a host.

- Apply the desired state on a standalone host, whose current state is different from the desired specification (remediate host).

Sample Scripts for vSphere Lifecycle Manager Host-Level Operations

The following scripts demonstrate how you can use vSphere Lifecycle Manager on the host level with VCF PowerCLI.

Activate vSphere Lifecycle Manager on a Standalone Host

```
$hostName = <host_ip_address_or_fqdn>
$vmhost = Get-VMhost $hostName
$spec=Initialize-SettingsHostsEnablementSoftwareEnableSpec -SkipSoftwareCheck $false
Invoke-SetHostEnablementSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -SettingsHostsEnablementSoftwareEnableSpec $spec
```

Retrieve vSphere Lifecycle Manager (vLCM) Image Profiles

You can retrieve vLCM images from a vCenter system based on different filters by using the `Get-LcmImage` cmdlet. You can pass IDs or a combination of name and version filters to the `Get-LcmImage` parameters.

List all types of vLCM images

```
Get-LcmImage
```

List only BaseImage (ESX) vLCM images

```
Get-LCMImage -Type 'BaseImage'
```

List Only VendorAddOn vLCM images

```
Get-LCMImage -Type 'VendorAddOn'
```

List only Component vLCM images

```
Get-LCMImage -Type 'Component'
```

List only Package (Firmware) vLCM images

```
Get-LCMImage -Type 'Package'
```

View / Check Recommended Images

```
$vmhost = Get-VMHost -Name $hostName
Invoke-GetHostSoftwareRecommendations -Host $vmhost.ExtensionData.MoRef.Value
```

Retrieve a Host's Desired State

```
Get-VMHost -Name $hostName | Select-Object -Property Name, BaseImage, @{n='BaseImageVersion'; e={$_.BaseImage.Version}}, Components, VendorAddOn, FirmwareAddons
```

Modify a Host's vLCM Desired State

```
$vLCMBaseImageu2 = Get-LCMImage -Version '7.0 GA - 15843807' -Type BaseImage
$vendorAddOn = Get-LcmImage -Name 'NEC-addon-GEN' -Version '7.0.2-02'
$components = $components = Get-LcmImage -Id 'Component-<Component1 Name>/<Component1 Version>'
$firmwareAddOn = Get-LcmImage -Id '<FirmwareAddOn_Id>'
Get-VMHost -Name $hostName | Set-VMHost -BaseImage $vLCMBaseImageu2 -VendorAddOn $vendorAddOn -Component $components -FirmwareAddOn $firmwareAddOn
```

Update a Host's vLCM Base Image

You cannot remove or downgrade an ESX base image - you can only upgrade it. The following example demonstrates an upgrade of a host's desired state to ESX 7.0 U2.

```
$vLCMBaseImageu2= Get-LcmImage -Version '7.0 U2a - 17867351' -Type BaseImage
Get-VMHost -Name $hostName | Set-VMHost -BaseImage $vLCMBaseImageu2
```

Remove a vLCM Vendor Add-on from a Host

```
Get-VMHost -Name $hostName | Set-VMHost -VendorAddOn $null
```

Note: You can add, replace, remove, upgrade, or downgrade a vendor add-on.

Update a Host's vLCM Component List

Replace a Component list with a new one.

```
$components = Get-LcmImage -Id 'Component-<Component1 Name>/<Component1 Version>', 'Component-<Component2
Name>/<Component2 Version>'
Get-VMHost -Name $hostName | Set-VMHost -Component $components
```

Note: To remove all components, use `Set-Cluster -Component $null`

Add a vLCM Firmware Add-on

```
$firmwareAddon = Get-LcmImage -Id '<FirmwareAddOn_Id>'
Get-VMHost -Name $hostName | Set-VMHost -FirmwareAddOn $firmwareAddon
```

Note: You can add, replace, remove, upgrade, or downgrade a firmware add-on.

Check Compliance of a Host

Compare the current image on the standalone host against the desired image that you specified and define the compatibility status of the host.

```
$vmhost = Get-VMHost -Name $hostName
Invoke-ScanHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value
```

Pre-check a Host

Run a remediation pre-check on the host to ensure software and hardware compatibility with the desired software specification.

```
$spec = Initialize-SettingsHostsSoftwareCheckSpec -Commit <MyCommit>
$vmhost = Get-VMHost -Name $hostName
Invoke-CheckHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -SettingsHostsSoftwareCheckSpec $spec
```

Stage a Host

Stage the desired state on a standalone ESX host.

```
$spec = Initialize-SettingsHostsSoftwareStageSpec -Commit <myCommit>
$vmhost = Get-VMHost -Name $hostName
Invoke-StageHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -SettingsHostsSoftwareStageSpec $spec
```

Remediate a Host

Apply the desired state on a standalone host, whose current state is different from the desired software specification.

```
$spec = Initialize-SettingsHostsSoftwareApplySpec -AcceptEula $true
```

```
$vmhost = Get-VMHost -Name $hostName
Invoke-ApplyHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -SettingsHostsSoftwareApplySpec $spec
```

Managing vSphere Lifecycle Manager Baselines with VMware Cloud Foundation PowerCLI

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Lifecycle Manager.

The vSphere Lifecycle Manager module provides a set of cmdlets for downloading software patches, creating and modifying baselines, and for scanning and remediating virtual machines or hosts.

Create Patch Baselines

You can apply patch baselines to hosts. Depending on the patch criteria you select, patch baselines can be either dynamic or fixed.

Verify that you are connected to a system.

Patch data in dynamic baselines changes depending on the criteria you specify each time Update Manager downloads new patches. Fixed baselines contain only the patches you have selected, regardless of new patch downloads.

1. Retrieve all host patches released after 1 Jan 2015 for ESX products, and create a fixed baseline named *Static Baseline*, containing the retrieved patches.

```
$patches = Get-Patch -After "1 Jan 2015" -Product "ESX"
$staticBaseline = New-PatchBaseline -Static -Name "Static Baseline" -IncludePatch $patches
```

2. Create a critical dynamic baseline named *Dynamic Baseline* by using a fetch-all query.

```
$criticalPatchBaseline = New-PatchBaseline -Dynamic -Name "Dynamic Baseline" -SearchPatchSeverity Critical
```

3. Create an extension baseline that contains all available extensions.

```
$extensions = Get-Patch -BundleType Extension
New-PatchBaseline -Static -Name "Extension Baseline" -Extension -IncludePatch $extensions
```

Attach and Detach Baselines

You can attach baselines to individual objects and to container objects, such as folders, hosts, clusters, and data centers. Attaching a baseline to a container object attaches the baseline to all objects in the container.

Verify that you are connected to a system.

1. Attach the host patch baselines stored in the provided variables to the host named *Host*.

```
Add-EntityBaseline -Baseline $staticBaseline, $criticalPatchBaseline -Entity Host
```

2. Detach the two baselines from the host.

```
Remove-EntityBaseline -Baseline $dynamicBaseline, $staticBaseline -Entity Host
```

Scan a Virtual Machine

You can scan a virtual machine against the baselines attached to it or inherited by its parent object.

Verify that you are connected to a system.

1. Initialize scanning on a virtual machine that is named *VM* against baselines containing virtual machine hardware upgrades and VMware Tools upgrades.

```
$task = Test-Compliance -Entity VM -UpdateType VmHardwareUpgrade, VmToolsUpgrade -RunAsync
```

The command initializes a task on the server, returns a snapshot object of the initial state of the task, and saves it in the *\$task* variable.

2. View the initial status of the scanning task.

```
$task
```

Note: The task object is not updated with the actual state of the task process running on the server. Even after the task is completed, the *\$task* variable value is running. To view the actual status of the tasks running on the server, use the `Get-Task` cmdlet.

3. Optional: Run the `Wait-Task` cmdlet to monitor the process progress and wait for the task to complete before running other commands.

```
Wait-Task -Task $task
```

Check Virtual Machine Baseline Status

You can check whether a virtual machine has any baselines with unknown compliance status attached to it and start a scan.

Verify that you are connected to a system.

1. Retrieve the compliance statuses with the value `Unknown` for the baselines attached to the *VM* virtual machine and store them in the *\$statuses* variable.

```
$statuses = Get-Compliance -Entity VM -ComplianceStatus Unknown
```

2. Check whether the virtual machine has any baselines with unknown compliance status attached to it and start a scan.

```
if ($statuses.Count -gt 0) {
    Test-Compliance -Entity VM -RunAsync"
}
```

Stage Patches

Staging allows you to download patches and extensions from the Update Manager server to the ESX hosts without applying the patches and extensions immediately.

Verify that you are connected to a system.

Note: Staging can be performed only for hosts, clusters, and data centers.

1. Retrieve a host and store it in the *\$vmHost* variable.

```
$vmHost = Get-VMHost -Name 10.23.112.233
```

2. Stage the patches for upgrading the host.

```
Stage-Patch -Entity $vmHost
```

Remediate a Virtual Machine

You can retrieve all baselines attached to a virtual machine and remediate the virtual machine.

Verify that you are connected to a system.

1. Retrieve all baselines attached to the *VM* virtual machine.

```
$baselines = Get-Baseline -Entity VM
```

2. Remediate the virtual machine.

```
Update-Entity -Entity VM -Baseline $baselines
```

Upgrade Virtual Machine Hardware

You can upgrade virtual machine hardware and VMware Tools for all virtual machines in a data center.

Verify that you are connected to a system.

1. Retrieve all virtual machines in the *Datacenter* data center.

```
$vms = Get-VM -Location Datacenter
```

2. Retrieve all virtual machine upgrade baselines.

```
$upgradeBaselines = Get-Baseline -TargetType VM -BaselineType Upgrade
```

3. Remediate all virtual machines against the virtual machine upgrade baselines.

```
foreach ($vm in $vms) {
    Update-Entity -Entity $vm -Baseline $upgradeBaselines
}
```

Remediate a Cluster

You can retrieve all baselines attached to a cluster and remediate the cluster.

Verify that you are connected to a system.

Note: Before remediation, you must temporarily deactivate the Distributed Power Management (DPM), High Availability (HA) admission control, and Fault Tolerance (FT) features of the clusters you want to remediate. After remediation, Update Manager automatically activates the deactivated features.

1. Retrieve all baselines attached to the *Cluster* cluster.

```
$baselines = Get-Baseline -Entity Cluster
```

2. Remediate the cluster.

```
Update-Entity -Entity Cluster -Baseline $baselines -ClusterDisableDistributedPowerManagement
$true -ClusterDisableHighAvailability $true -ClusterDisableFaultTolerance $true
```

Remediate a Host

You can retrieve all baselines attached to a host and remediate the host.

Verify that you are connected to a system.

Note: When remediating a host, you can configure the maintenance mode settings. You can temporarily disconnect any removable media devices that might prevent the host from entering maintenance mode as well.

1. Retrieve all baselines attached to the *Host* host.

```
$baselines = Get-Baseline -Entity Host
```

2. Remediate the host.

```
Update-Entity -Entity Host -Baseline $baselines -HostFailureAction Retry -HostNumberOfRetries
2 -HostDisableMediaDevices $true
```

Download Patches and Scan Objects

You can download patches from a previously defined location.

Verify that you are connected to a system.

1. Retrieve all entities from the *Datacenter* data center and store the result in a variable.

```
$entities = Get-Inventory -Location Datacenter
```

2. Download all available patches and store the result in a variable.

```
$result = Sync-Patch
```

3. Check whether new patches are downloaded and start scanning the entities in the *Datacenter* data center.

```
if ($result.Count > 0) {
    Test-Compliance -Entity $entities
}
```

vSphere Monitoring and Performance

You can monitor your virtual environment with VCF PowerCLI to locate the source of current and potential issues. Some monitoring and performance tools you can use with VCF PowerCLI are the vCenter alarms, and the *esxtop* functionality. You can also use VCF PowerCLI to retrieve valuable statistics about your virtual infrastructure.

Use Esxtop to Get Information on the Virtual CPUs of a Virtual Machine

You can use the *Get-Esxtop* cmdlet to retrieve real-time data for troubleshooting performance problems on your ESX hosts.

Verify that you are connected to a server that runs ESX 4.0, 5.0 or later.

The *Get-Esxtop* cmdlet exposes the *esxtop* functionality that provides real-time monitoring and performance analysis of an ESX host's resources. You can use *esxtop* to gain insights into the performance and resource utilization of virtual machines and the underlying hardware.

1. Get the group to which the virtual machine belongs and save it as a variable.

```
$group = Get-Esxtop -CounterName SchedGroup | where {$_.VMName -eq $vm.Name}
```

2. Get the IDs of all virtual CPUs of the virtual machine and store them in an array.

```
$gr = Get-Esxtop -TopologyInfo -Topology SchedGroup | %{$_.Entries} | where {$group.GroupID -contains
    $_.GroupID} $group.GroupID
$cpuIds = @()
$gr.CpuClient | %{$cpuIds += $_.CPUClientID}
```

3. Get the CPU statistics for the virtual machine.

```
$cpuStats = Get-Esxtop -CounterName VCPU | where {$cpuIds -contains $_.VCPUID}
```

4. Calculate the used and ready for use percentage by using the *UsedTimeInUsec* and *ReadyTimeInUsec* stats.

```
$result = @()
$cpuStats | %{ `
$row = "" | select VCPUID, Used, Ready; `
$row.VCPUID = $_.VCPUID; `
$row.Used = [math]::Round((([double]$_.UsedTimeInUsec/[double]$_.UpTimeInUsec)*100, 2); `
$row.Ready = [math]::Round((([double]$_.ReadyTimeInUsec/[double]$_.UpTimeInUsec)*100, 2); `
$result += $row
}
```

5. View the used and ready for use percentage for each virtual CPU of the virtual machine.

```
$result | Format-Table -AutoSize
```


Manage Statistics and Statistics Intervals on vCenter

You can use the VCF PowerCLI cmdlets to automate tasks for viewing and managing statistics for vCenter inventory objects.

Verify that you are connected to a system.

You can modify the properties of a statistics interval and view statistics for a selected cluster.

1. Increase the amount of time for which statistics of the previous day are stored.

```
Set-StatInterval -Interval "past day" -StorageTimeSecs 700000
```

2. View the available memory metric types for the *Cluster1* cluster.

```
$cluster = Get-Cluster Cluster1
$statTypes = Get-StatType -Entity $cluster -Interval "past day" -Name mem.*
```

3. View the cluster statistics collected for the day.

```
Get-Stat -Entity $cluster -Start ([System.DateTime]::Now.AddDays(-1)) -Finish ([System.DateTime]::Now) -Stat $statTypes
```

Alarm Management

You can use a set of preconfigured alarms in vCenter to monitor the operations of vSphere inventory objects. This section provides sample scripts with basic and advanced tasks for vCenter alarm management.

View Actions and Triggers for an Alarm on vCenter

You can see which actions and triggers are configured for an alarm.

Verify that you are connected to a system.

1. Get all VCF PowerCLI supported alarm actions for the *Host Processor Status* alarm.

```
Get-AlarmDefinition -Name "Host Processor Status" | Get-AlarmAction -ActionType "ExecuteScript", "SendSNMP", "SendEmail"
```

2. Get all the triggers for the first alarm definition found.

```
Get-AlarmDefinition | select -First 1 | Get-AlarmTrigger
```

Create Alarm Definitions on vCenter

With VCF PowerCLI, you can create vCenter alarm definitions.

Verify that you are connected to a system.

1. Create an alarm action trigger.

```
$actionTrigger = New-AlarmActionTrigger -StartStatus Green -EndStatus Yellow
```

2. Create an alarm action email.

```
$emailAction = New-AlarmAction -Email -To 'test@vmware.com' -CC @('test1@vmware.com', 'test2@vmware.com') -Body 'Email text' -Subject 'Email subject' -AlarmActionTrigger $actionTrigger
```

3. Create an event-based alarm trigger.

```
$vmReconfigEvt = Get-EventType | Where-Object {$_.Description -eq "VM reconfigured"}
```

```
$eventTrigger = New-AlarmTrigger -EventType $vmReconfigEvt -EntityStatus Yellow -EntityType "VirtualMachine"
```

4. Create an alarm definition.

```
$vm = Get-VM | Select-Object -First 1
New-AlarmDefinition -Name "MyAlarm" -Description "Description" -AlarmTrigger $eventTrigger -AlarmAction $emailAction -Entity $vm
```

Modify Alarm Definitions on vCenter

With VCF PowerCLI, you can modify vCenter alarm definitions.

Verify that you are connected to a system.

1. For all host alarms, modify the interval after the action repeats.

```
Get-AlarmDefinition -Entity (Get-VMHost) | foreach { $_ | Set-AlarmDefinition -ActionRepeatMinutes ($_.ActionRepeatMinutes + 1)}
```

2. Modify the name and the description of a selected alarm definition, and enable the alarm.

```
Get-AlarmDefinition -Name AlarmDefinition | Set-AlarmDefinition -Name AlarmDefinitionNew -Description 'Alarm Definition Description' -Enabled:$true
```

Create Alarm Actions and Triggers on vCenter

With VCF PowerCLI, you can create vCenter alarm actions and triggers.

Verify that you are connected to a system.

1. Create an alarm action email for the renamed alarm definition.

```
Get-AlarmDefinition -Name AlarmDefinitionNew | New-AlarmAction -Email -To 'test@vmware.com' -CC @('test1@vmware.com', 'test2@vmware.com') -Body 'Email text' -Subject 'Email subject'
```

2. Create an snmp alarm action.

```
Get-AlarmDefinition -Name AlarmDefinitionNew | New-AlarmAction -Snmp
```

3. Create a script alarm action.

```
Get-AlarmDefinition -Name AlarmDefinitionNew | New-AlarmAction -Script -ScriptPath 'c:\test.ps1'
```

4. Create a metric-based alarm trigger.

```
$metric = Get-Metric -MetricGroup CPU -Name Usage | Where-Object {$_.Key -eq 2}
$metricTrigger = New-AlarmTrigger -Metric $metric -Red 9000 -RedIntervalSeconds 3000 -EntityType "VirtualMachine" -MetricAlarmOperator Above
```

5. Create a state-based alarm trigger.

```
$stateTrigger = New-AlarmTrigger -StatePath "runtime.powerState" -Value "poweredOff" -EntityStatus Red -EntityType "VirtualMachine" -StateAlarmOperator Equal
```

Remove Alarm Definitions and Actions

In some cases, you might want to remove obsolete alarm definitions and actions.

Verify that you are connected to a system.

1. Remove all the actions for an alarm definition.

```
Get-AlarmDefinition -Name AlarmDefinition | Get-AlarmAction | Remove-AlarmAction -Confirm:$false
```

2. Remove an alarm definition.

```
Get-AlarmDefinition -Name AlarmDefinition | Remove-AlarmDefinition
```

Managing the VMware Cloud Foundation API with VMware Cloud Foundation PowerCLI

You can manage the VMware Cloud Foundation API with VCF PowerCLI by using the vSphere Automation SDK for PowerShell. Starting from VCF PowerCLI version 12.4.0, the VMware Cloud Foundation SDK provides low-level cmdlets for all available VMware Cloud Foundation API services.

Understanding the vSphere Automation SDK for PowerShell

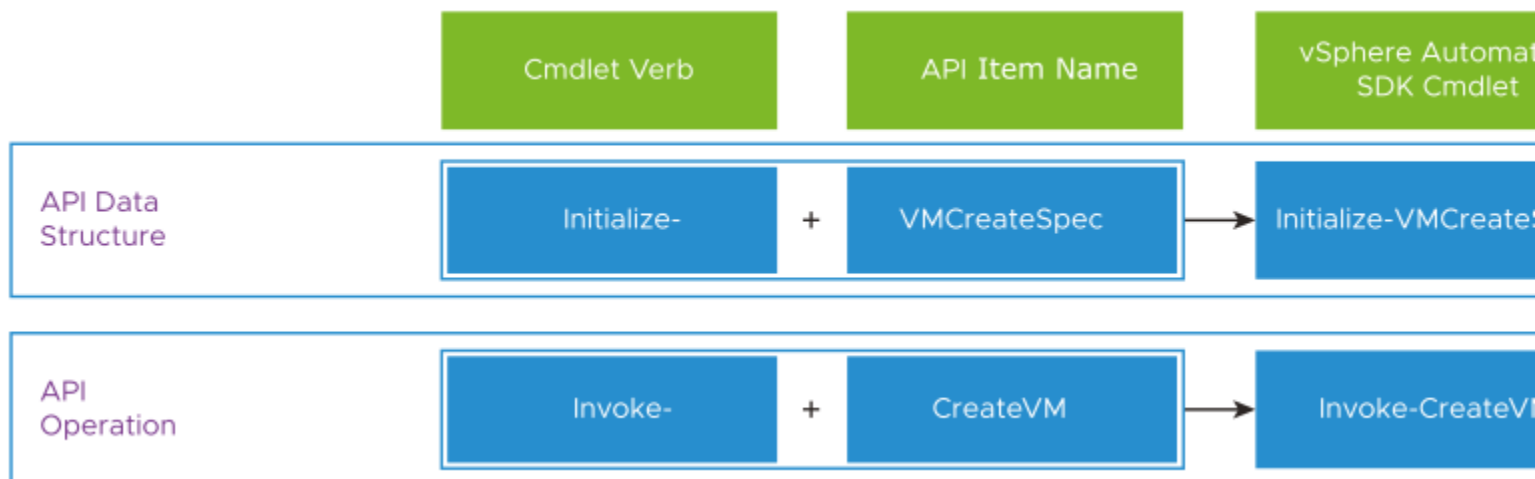
You can manage the VMware Cloud Foundation API with VMware Cloud Foundation PowerCLI by using the vSphere Automation SDK for PowerShell.

Starting from VMware Cloud Foundation PowerCLI 12.4.0, you can use the vSphere Automation SDK for PowerShell to communicate with all available vSphere Automation REST APIs.

The modules that start with the `VMware.vSphere.Sdk` prefix contain automatically-generated SDK cmdlets that bind directly to the vSphere Automation REST APIs. The modules function on a low level and communicate directly with the vSphere Automation API server.

You can construct VMware Cloud Foundation SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the vSphere Automation API data structures or operations.

Figure 656: Constructing VMware Cloud Foundation SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the API server.
- Use `Invoke-` cmdlets to call the VMware Cloud Foundation API server and invoke the operations.

Navigating the vSphere Automation SDK for PowerShell

You can navigate the vSphere Automation SDK for PowerShell by using the VMware Developer website, and the `Get-Help` and `Get-Command` cmdlets.

To use the vSphere Automation SDK for PowerShell effectively, you must be familiar with the [vSphere Automation API reference documentation](#).

How to Find a VMware Cloud Foundation SDK Cmdlet

To search for VMware Cloud Foundation SDK cmdlets:

- In your browser, navigate the VMware Cloud Foundation API reference documentation.
The VMware Cloud Foundation API reference on VMware Developer features ready-to-use VMware Cloud Foundation SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.
- In PowerShell, run `Get-Command -Module VMware.Sdk.vSphere.*` to list all available VMware Cloud Foundation SDK cmdlets.

How to Get Information About a Cmdlet

To get information about a VMware Cloud Foundation SDK cmdlet, run `Get-Help` with the cmdlet name. For example:

```
Get-Help
Invoke-CreateVm -full
```

To open the online information in a browser, run `Get-Help` with the cmdlet name and the `Online` parameter. For example:

```
Get-Help
Invoke-CreateVm -Online
```

Connecting to a VMware Cloud Foundation API Server

You can connect to the VMware Cloud Foundation API with VMware Cloud Foundation PowerCLI through the `Connect-VIServer` cmdlet or through a configuration object.

You can access a VMware Cloud Foundation API server in two ways with VCF PowerCLI.

- Through the `Connect-VIServer` cmdlet.
- Through a configuration object.

Connect Through Connect-VIServer

Use the `Connect-VIServer` cmdlet to connect to a vCenter system and access the vSphere Automation API.

- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).

- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Note: If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

You can have more than one connection to the same server. For more information, see [Managing Default Server Connections in VCF PowerCLI](#).

If your login credentials contain non-alphanumeric characters, you might need to escape them. For more information, see [Processing Non-alphanumeric Characters in VCF PowerCLI](#).

Run `Connect-VIServer` with the server name and valid credentials.

```
Connect-VIServer -Server vc3.example.com -User 'MyUser' -Password 'MyPassword'
```

Connect Through a Configuration Object

You can access the vSphere Automation API by connecting to a vCenter system through a configuration object.

You can create a configuration object and pass information about the vCenter system on which you want to use the vSphere Automation SDK for PowerShell. You can use this connection method if you don't want to use the PowerCLI high-level cmdlets or if you don't have the `VMware.VimAutomation.Core` module installed on your system.

1. Create a server configuration for a vCenter system.

```
$Server = 'vCenter address'
$User = 'username'
$Password = Read-Host -AsSecureString -Prompt "Password:"
$serverConfig = New-vSphereServerConfiguration -Server $Server -User $User -Password $Password
-SkipCertificateCheck
```

2. Create a vSphere Automation API session.

```
$apiSession = Invoke-CreateSession -WithHttpInfo
```

3. Update the server configuration with the created API session key.

```
$serverConfig | Set-vSphereServerConfigurationApiKey -SessionResponse $apiSession
```

To learn more about using server configuration objects with the VMware Cloud Foundation SDK, see [Managing vCenter Connections Through Configuration Objects](#).

Managing vCenter Connections Through Configuration Objects

The vSphere Automation SDK for PowerShell features special configuration object cmdlets to manage vCenter connections. A configuration object is a client-side PowerShell object that holds information about a server that the `Invoke-` cmdlets can interact with.

Table 1896: Configuration Object Cmdlets

Cmdlet	Description
<code>New-vSphereServerConfiguration</code>	Creates a vCenter configuration object.
<code>Get-vSphereServerConfiguration</code>	Retrieves the existing vCenter configuration objects.
<code>Set-vSphereServerConfiguration</code>	Updates a vCenter configuration object.
<code>Set-vSphereServerConfigurationApiKey</code>	Updates a vCenter configuration object with an API key for authorization.

Cmdlet	Description
Remove-vSphereServerConfiguration	Removes a vCenter configuration object.

Working with Configuration Objects

You can create multiple vCenter configuration objects and manage all of them simultaneously with the `Invoke-` cmdlets. For example, `Invoke-DeleteSession` deletes all existing vSphere Automation API sessions.

If you want to apply the `Invoke-` cmdlets to a specific configuration object, you must specify it in the command. For example,

```
Invoke-ListFolder -Server $serverConfig
```

Example Use Cases with vCenter Configuration Objects

- You can create a session and authenticate to a vSphere Automation API server - see [Connect Through a Configuration Object](#).
- You can retrieve the active server configurations.

```
Get-vSphereServerConfiguration
```

- You can remove a specific server configuration.

```
$serverConfig | Remove-vSphereServerConfiguration
```

Create a Local User Account in vCenter

You can create a local user account in vCenter by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Prepare the data structures for the API operation.

```
$LocalAccountsConfig = Initialize-LocalAccountsConfig -Password '$trongPa$$w0rd' -Roles "superAdmin"
$LocalAccountsCreateRequestBody = Initialize-LocalAccountsCreateRequestBody -Username "MyUsername" -Config
$LocalAccountsConfig
```

2. Invoke the API operation.

```
Invoke-CreateLocalAccounts -LocalAccountsCreateRequestBody $LocalAccountsCreateRequestBody
```

3. Optional: Retrieve the local account you created.

```
Invoke-GetUsernameLocalAccounts -Username "MyUsername"
```

Update the Local Accounts Global Password Policy in vCenter

You can update the local accounts global password policy in vCenter by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Retrieve the current local accounts global password policy.

```
Invoke-GetLocalAccountsGlobalPolicy
```

2. Prepare the data structures for the operation.

```
$LocalAccountsPolicyInfo = Initialize-LocalAccountsPolicyInfo -MaxDays 352 -MinDays 1 -WarnDays 7
```

3. Invoke the API operation.

```
Invoke-SetLocalAccountsGlobalPolicy -LocalAccountsPolicyInfo $LocalAccountsPolicyInfo
```

4. Optional: Retrieve the new local accounts global password policy.

```
Invoke-GetLocalAccountsGlobalPolicy
```

Create a Virtual Machine

You can create virtual machines in vCenter by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Retrieve a target datastore ID.

```
$targetDatastore = Invoke-ListDatastore -Names 'local-0'
```

2. Retrieve a target folder ID.

```
$targetFolder = Invoke-ListFolder -Names 'vm'
```

3. Retrieve a target host ID.

```
$targetHost = Invoke-ListHost -Names '10.185.41.87'
```

4. Prepare the virtual machine placement specification.

```
$placementSpec = Initialize-VMPlacementSpec -Folder $targetFolder.folder -Datastore $targetDatastore.datastore -VarHost $targetHost.host
```

5. Prepare the virtual machine create specification.

```
$vmCreateSpec = Initialize-VMCreateSpec -Name 'TestVM' -GuestOS WIN_31 -Placement $placementSpec
```

6. Invoke the CreateVM API operation.

```
Invoke-CreateVM -VMCreateSpec $vmCreateSpec
```

Create Tag Category, Tag, and Tag Association

You can create a tag category, tag, and tag association by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Create a tag category.

```
$categoryCreateSpec = Initialize-TaggingCategoryCreateSpec -Cardinality 'MULTIPLE' -AssociableTypes 'VirtualMachine' -Name 'TestCategory' -Description 'TestDescription'
$categoryId = Invoke-CreateCategory -TaggingCategoryCreateSpec $categoryCreateSpec
```

2. Create a tag.

```
$tagCreateSpec = Initialize-TaggingTagCreateSpec -Name 'TestVMTag' -Description 'This is a test tag for vms' -CategoryId $categoryId
$tag = Invoke-CreateTag -TaggingTagCreateSpec $tagCreateSpec
```

3. Create a tag association.

```
$vmId = Invoke-ListVM -Names 'MyVM'
$vmDynamicId = Initialize-StdDynamicID -Type 'VirtualMachine' -Id $vmId
```

```
$tagAssociationBody = Initialize-TaggingTagAssociationAttachRequestBody -ObjectId $vmDynamicId Invoke-AttachTagIdTagAssociation -TagId $tag -TaggingTagAssociationAttachRequestBody $tagAssociationBody
```

Create Content Library and Content Library Item

You can create a content library and content library item by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Specify a datastore for storage backing.

```
$backingDatastore = Invoke-ListDatastore -Names 'local-0'
```

2. Create a content library.

```
$storageBacking = Initialize-LibraryStorageBacking -DatastoreId $backingDatastore.datastore -Type 'DATAS-TORE'
$libraryModel = Initialize-LibraryModel -Name 'TestLocalContentLibrary' -StorageBackings $storageBacking
$contentLibraryId = Invoke-CreateContentLocalLibrary -LibraryModel $libraryModel
```

3. Create a content library item.

```
$libraryItemModel = Initialize-LibraryItemModel -Name 'TestContentLibraryItem' -LibraryId $contentLibraryId
$libraryItemId = Invoke-CreateContentLibraryItem -LibraryItemModel $libraryItemModel
```

Managing VMware vSAN™ with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of sample scripts that illustrate basic and advanced tasks in VMware vSAN™ management.

Create a vSAN Datastore

You can create vSAN disk groups on standalone hosts and add the hosts to a vSAN-enabled cluster to form a vSAN datastore. You can then create a virtual machine on the vSAN datastore and assign a storage policy to the virtual machine and its hard disk.

- Verify that you are connected to a system.
- Verify that you have access to at least three virtual machine hosts.
- Verify that each of the virtual machine hosts has at least one SSD and one HDD.
- Verify that the virtual machine hosts are in maintenance mode.

1. Create a vSAN enabled cluster with manual disk claim mode.

```
$vsanCluster = New-Cluster -Name 'VsanCluster' -Location (Get-Datacenter) -VsanEnabled -VsanDiskClaim-Mode 'Manual'
```

2. Configure a vSAN VMkernel port on each of the three hosts.

```
New-VMHostNetworkAdapter -VMHost 'Host-A' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0' -VsanTrafficEnabled $true
New-VMHostNetworkAdapter -VMHost 'Host-B' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0' -VsanTrafficEnabled $true
New-VMHostNetworkAdapter -VMHost 'Host-C' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0' -VsanTrafficEnabled $true
```

3. Create a vSAN disk group on each of the three hosts.


```
New-VsanDiskGroup -DataDiskCanonicalName 'HDD1-CanonicalName' -SsdCanonicalName 'SSD1-CanonicalName' -VMHost 'Host-A'
New-VsanDiskGroup -DataDiskCanonicalName 'HDD1-CanonicalName' -SsdCanonicalName 'SSD1-CanonicalName' -VMHost 'Host-B'
New-VsanDiskGroup -DataDiskCanonicalName 'HDD1-CanonicalName' -SsdCanonicalName 'SSD1-CanonicalName' -VMHost 'Host-C'
```

4. Add each of the three hosts to the vSAN cluster to create a vSAN datastore.

```
Move-VMHost -VMHost 'Host-A' -Destination $vsanCluster
Move-VMHost -VMHost 'Host-B' -Destination $vsanCluster
Move-VMHost -VMHost 'Host-C' -Destination $vsanCluster
```

5. Revert the virtual machine hosts to the Connected state.

```
Set-VMHost -VMHost 'Host-A','Host-B','Host-C' -State 'Connected'
```

6. Create a virtual machine on the vSAN datastore.

```
$vsanDS = Get-Datastore -Name 'vsanDatastore'
$vm = New-VM -Name 'newVM' -DiskMB 1024 -Datastore $vsanDS -VMHost 'Host-A'
```

7. Create a storage policy by using any of the vSAN capabilities.

```
$scap = Get-SpbmCapability -Name vSAN*
$rule = New-SpbmRule $scap[1] $true
$ruleset = New-SpbmRuleSet $rule
$policy = New-SpbmStoragePolicy -Name 'vsan policy' -RuleSet $ruleset -Description 'vSAN-based storage policy'
```

8. Assign the storage policy to the virtual machine and its hard disk.

```
$vmHdd = Get-HardDisk -VM $vm
Set-SpbmEntityConfiguration $vm, $vmHdd -StoragePolicy $policy
```

9. Check the compliance of the virtual machine and its hard disk with the storage policy.

```
Get-SpbmEntityConfiguration $vm, $vmHdd
```

The status should be Compliant.

Modify a vSAN Datastore

You can add or remove local disks from existing vSAN disk groups or remove entire vSAN disk groups.

- Verify that you are connected to a system.
- Verify that at least one vSAN disk group exists in the cluster.

1. Get the vSAN disk group from a cluster.

```
$dgs = Get-VsanDiskGroup -Cluster 'VsanCluster'
```

2. Get all vSAN disks from the vSAN disk group.

```
$dg = $dgs[0]
Get-VsanDisk -VsanDiskGroup $dg
```

3. Add a hard disk to the vSAN disk group.

```
$disk = New-VsanDisk -CanonicalName 'HDD-CanonicalName' -VsanDiskGroup $dg
```

4. Remove a hard disk from the vSAN disk group.

```
Remove-VsanDisk -VsanDisk $disk
```

5. Remove the entire vSAN disk group.

```
Remove-VsanDiskGroup -VsanDiskGroup $dg
```

Create a vSAN Stretched Cluster

You can create a vSAN stretched cluster with a witness node. You can then create a vSAN storage policy and enable performance service on the vSAN cluster.

- Verify that you are connected to a system.
- Verify that you have access to at least two virtual machine hosts.
- Verify that each of the virtual machine hosts has at least one SSD and one HDD.
- Verify that the virtual machine hosts are in maintenance mode.
- Verify that you have access to an ESX host that can be used as a witness host or deploy a witness appliance on any node. Ensure that the witness host or appliance is outside the vSAN cluster.

1. Configure a vSAN VMkernel port on each of the two hosts.

```
New-VMHostNetworkAdapter -VMHost 'Host-A' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0'
-VsanTrafficEnabled $true
New-VMHostNetworkAdapter -VMHost 'Host-B' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0' -VsanTrafficEnabled $true
```

2. Create a vSAN enabled cluster with automatic disk claim mode.

```
$vsanCluster = New-Cluster -Name 'VsanCluster' -Location (Get-Datacenter) -VsanEnabled -VsanDiskClaimMode 'Automatic'
```

3. Add the two hosts to the vSAN cluster to create a vSAN datastore.

```
Move-VMHost -VMHost 'Host-A' -Destination $vsanCluster
Move-VMHost -VMHost 'Host-B' -Destination $vsanCluster
```

4. Revert the virtual machine hosts to the Connected state.

```
Set-VMHost -VMHost 'Host-A','Host-B' -State 'Connected'
```

5. Create two fault domains in the vSAN cluster.

```
$primaryFd = New-VsanFaultDomain -Name 'Primary' -VMHost 'Host-A'
$secondaryFd = New-VsanFaultDomain -Name 'Secondary' -VMHost 'Host-B'
```

6. Enable stretched cluster.

```
Set-VsanClusterConfiguration -Configuration $vsanCluster -StretchedClusterEnabled $true
-PreferredFaultDomain $primaryFd -WitnessHost 'Witness-Virtual-Appliance-IP'
```

7. Create a storage policy by using any vSAN capability.

```
$cap = Get-SpbmCapability -Name vSAN*
$rule = New-SpbmRule $cap[1] $true
$ruleset = New-SpbmRuleSet $rule
$policy = New-SpbmStoragePolicy -Name 'vsan policy' -RuleSet $ruleset -Description 'vSAN-based storage policy'
```

8. Enable performance service on the vSAN cluster.

```
Set-VsanClusterConfiguration -Configuration $vsanCluster -PerformanceServiceEnabled $true
-StoragePolicy $policy
```

Enable a vSAN File Service

With VCF PowerCLI, you can enable a vSAN file service.

Verify that you are connected to a system.

1. Add a vSAN file service OVF to the OVF repository of the vCenter system.

```
Add-VsanFileServiceOvf
```

2. Verify that the vSAN file service OVF is successfully added.

```
Get-VsanFileServiceOvfInfo
```

3. Increase the VCF PowerCLI timeout for the current session, so that the operation does not stop.

```
Set-PowerCLIConfiguration -WebOperationTimeoutSeconds 3600 -Scope Session
```

4. Retrieve the network that you want to use for the vSAN file service.

```
$net = Get-VirtualNetwork -Name 'MyNetwork'
```

5. Modify the vSAN cluster configuration that you want to use to enable the vSAN file service.

```
$config = Get-VsanClusterConfiguration
Set-VsanClusterConfiguration $config -FileServiceEnabled
$true -FileServiceNetwork $net
```

Create a vSAN File Service Domain

With VCF PowerCLI, you can create a vSAN file service domain.

- Verify that you are connected to a system.
- Verify that the vSAN file service is enabled.

1. Configure a vSAN file server IP configuration. You can use the IPs as an IP pool for the file service domain.

```
$ipConfig = New-VsanFileServerIpConfig -Fqdn "h101.vmware.com" -Gateway "192.2.8.3" -IpAddress
"192.2.8.4" -SubnetMask "255.255.254.0" -IsPrimary
```

2. Create a vSAN file service domain.

```
New-VsanFileServiceDomain -DnsServerAddress "10.172.199.241" -VsanFileServerIpConfig $ip-
Config -Name "MyFileServiceDomain" -DnsSuffix "vmware.com" -Cluster $config.Cluster
```

Create a vSAN File Share

With VCF PowerCLI, you can create a vSAN file share.

- Verify that you are connected to a system.
- Verify that the vSAN file service is enabled.
- Verify that the vSAN file service domain is created.

1. Get a vSAN file service domain.

```
$domain = Get-VsanFileServiceDomain -Name "MyFileServiceDomain"
```

2. Create a vSAN file share.

```
New-vsanfileshare -FileServiceDomain $domain -Name "MyVsanFileShare"
```

Create a vSAN ESA-Enabled Cluster

Starting from VMware Cloud Foundation PowerCLI 13.0.0 and vSAN 8.0, you can activate vSAN ESA (Express Storage Architecture) and manage storage pools on your clusters.

- Verify that you are connected to a vCenter 8.0 or later system.
- Verify that you have access to at least three virtual machine hosts.
- Verify that each of the virtual machine hosts has at least one SSD.

Starting from VMware Cloud Foundation PowerCLI 13.0.0 and vSAN 8.0, you can use vSAN ESA and the new storage format called storage pool disks. With vSAN ESA, you can boost performance with more predictable I/O latencies and optimized space efficiency. Each host that contributes storage contains a single storage pool of flash devices. Each flash device provides both caching and capacity to the vSAN ESA cluster. This is different from a disk group, which has dedicated devices in different tiers of cache and capacity.

1. Create a vSAN ESA-enabled cluster.

```
$vsanEsaCluster = New-Cluster -Name 'VsanESACluster' -Location (Get-Datacenter <MyDatacenter>) -VsanEn-
abled -VsanEsaEnabled
```

2. Retrieve the available storage pool disks to claim on your cluster.

```
Get-VsanEsaEligibleDisk -Cluster $vsanEsaCluster
```

The system returns the `ScsiLun` type of the available storage pool disks.

3. Add disks to the storage pool on the specified host.

For `DiskCanonicalName`, use the canonical name from `ScsiLun`.

```
Add-VsanStoragePoolDisk -VMHost (Get-VMHost <MyVMHost>) -VsanStoragePoolDiskType "singleTier" -DiskCanoni-
calName ("mpx.vmhba0:C0:T11:L0", "mpx.vmhba0:C0:T12:L0")
```

4. Optional: You can remove a storage pool disk from the specified host.

```
$disks = Get-VsanStoragePoolDisk -VMHost <MyVMHost>
Remove-VsanStoragePoolDisk -VsanStoragePoolDisk $disks -VsanDataMigrationMode "nodatamigration" -Purpose
"test" -Confirm:$false
```

Mount and Unmount Remote vSAN Datastores

Starting from VMware Cloud Foundation PowerCLI 12.7 and vSAN 7.0 Update 1, you can mount and unmount remote vSAN datastores to your vSAN-enabled clusters.

You can mount a remote datastore:

- From a vSAN or vSAN ESA-enabled cluster on the same vCenter system.
- From a vSAN stretched cluster.
- From a vSAN cluster on a different vCenter system.

Mount and Unmount a Remote vSAN Datastore from the Same vCenter System

Starting from VMware Cloud Foundation PowerCLI 12.7 and vSAN 7.0 Update 1, you can mount a remote vSAN datastore from another vSAN-enabled cluster residing on the same vCenter system.

To mount a datastore:

1. Retrieve the vSAN cluster configuration of the cluster to which you want to mount the remote datastore.

```
$config = Get-VsanClusterConfiguration -Cluster "MyCluster"
```

2. Mount the remote datastore.

```
Set-VsanClusterConfiguration -Configuration $config -MountRemoteDatastore (Get-Datastore -Name "MyDatastore")
```

3. Retrieve the mounted datastore.

```
$newVsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
$newVsanConfiguration.RemoteDatastore
```

To unmount a datastore:

1. Retrieve the new vSAN cluster configuration of the cluster from which you want to unmount the remote datastore.

```
$vsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
```

2. Unmount the remote datastore.

```
Set-VsanClusterConfiguration -Configuration $vsanConfiguration -UnMountRemoteDatastore $vsanConfiguration.RemoteDatastore
```

Important: Starting from VMware Cloud Foundation PowerCLI 13.1.0 and vSAN 8.0 Update 1, you can mount remote datastores from vSAN ESA-enabled clusters. You can follow the same steps as for vSAN clusters.

Mount and Unmount a Remote vSAN Datastore from a vSAN Stretched Cluster

Starting from VMware Cloud Foundation PowerCLI 13.1.0 and vSAN 8.0 Update 1, you can mount a remote vSAN datastore to a vSAN cluster from a vSAN stretched cluster.

To mount a datastore:

1. Configure the remote vSAN server cluster configuration.

It depends on the network topology between the client and server clusters in your HCI Mesh configuration.

a. In a symmetric HCI Mesh configuration:

```
$serverconfig = New-RemoteVsanServerClusterConfig -cluster $clusterServer -networktopology "Symmetric"
```

b. In an asymmetric HCI Mesh configuration with one stretched cluster.

```
$serverconfig = New-RemoteVsanServerClusterConfig -cluster $clusterServer -networktopology "Asymmetric" -ServerSiteName "Secondary"
```

c. In an asymmetric configuration where both the client and server cluster are stretched clusters.

```
$serverconfig = New-RemoteVsanServerClusterConfig -cluster $clusterServer -networktopology "Asymmetric" -ServerSiteName "Secondary" -ClientSiteName "Preferred"
```

2. Retrieve the vSAN cluster configuration of the cluster to which you want to mount the remote datastore.

```
$clusterClient = Get-VsanClusterConfiguration -Cluster "MyCluster"
```

3. Mount the remote datastore.

```
Set-VsanClusterConfiguration -configuration $clusterClient -MountRemoteDatastore (Get-Datastore -Name "My-Datastore") -RemoteVsanServerClusterConfig $serverconfig
```

To unmount a datastore:

1. Retrieve the new vSAN cluster configuration of the cluster from which you want to unmount the remote datastore.

```
$newVsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
```

2. Unmount the datastore.

```
Set-VsanClusterConfiguration -configuration $newVsanConfiguration -UnmountRemoteDatastore $newVsanConfiguration.RemoteDatastore
```

Mount and Unmount a Remote vSAN Datastore from a Different vCenter System

Starting from VMware Cloud Foundation PowerCLI 13.1.0 and vSAN 8.0 Update 1, you can mount a remote vSAN datastore to a vSAN cluster from a vSAN cluster residing on a different vCenter system.

To mount a datastore:

1. Add the different vCenter as a datastore source to your current vCenter system.

```
$dsSource = New-VsanHCIMeshDatastoreSource -VCHost <RemoteVCAddress> -User <MyVCUser> -Password <MyVC-Password>
```

2. Retrieve the available remote datastores.

```
$ds = Get-VsanHCIMeshDatastore -VsanHCIMeshDatastoresource $dsSource
```

3. Retrieve the vSAN cluster configuration of the cluster to which you want to mount the remote datastore.

```
$clusterClient = Get-VsanClusterConfiguration -Cluster "MyCluster"
```

4. Mount the remote datastore.

```
Set-VsanClusterConfiguration -Configuration $clusterClient -MountXVCDDatastore $ds
```

5. Retrieve the mounted remote datastore.

```
$vsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
$vsanConfiguration.VsanXVCDDatastore
```

To unmount the datastore:

```
Set-VsanClusterConfiguration -Configuration $vsanConfiguration -UnmountXVCDDatastore $ds
```

Managing the vSphere Replication API with VMware Cloud Foundation PowerCLI

Use the vSphere Replication SDK for PowerShell to access the vSphere Replication API with VCF PowerCLI. Starting from VMware Cloud Foundation PowerCLI version 13.1.0, the vSphere Replication SDK provides low-level cmdlets for all available vSphere Replication API operations.

Understanding the vSphere Replication SDK for PowerShell

You can access the vSphere Replication API with VMware Cloud Foundation PowerCLI by using the vSphere Replication SDK for PowerShell.

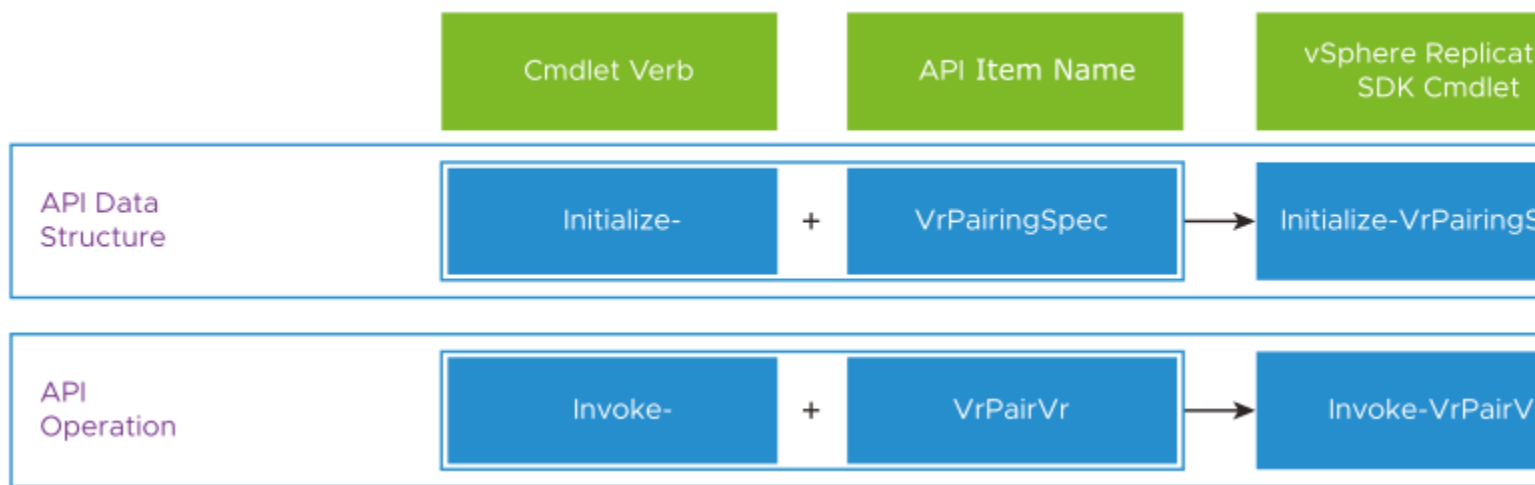
How Does the vSphere Replication SDK for PowerShell Work

Starting from VMware Cloud Foundation PowerCLI 13.1.0, you can use the vSphere Replication SDK for PowerShell to communicate with all available vSphere Replication REST APIs.

The `VMware.Sdk.Vr` module contains automatically-generated SDK cmdlets that bind directly to the vSphere Replication REST APIs. The module functions on a low level and communicates directly with the vSphere Replication API server.

You can construct vSphere Replication SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the vSphere Replication API data structures or operations.

Figure 657: Constructing vSphere Replication SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the vSphere Replication API server.
- Use `Invoke-` cmdlets to call the vSphere Replication API server and invoke the operations.

Navigating the vSphere Replication SDK for PowerShell

You can navigate the vSphere Replication SDK for PowerShell by using the VMware Developer website, and the `Get-Help`, `Get-Command`, and `Get-VrOperation` cmdlets.

To use the vSphere Replication SDK for PowerShell effectively, you must be familiar with the [vSphere Replication API reference documentation](#).

How to Find a vSphere Replication Cmdlet

There are several ways to find the vSphere Replication cmdlets you need.

- In your browser, navigate the vSphere Replication API reference documentation. The vSphere Replication API Reference on VMware Developer features ready-to-use vSphere Replication SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.

- In PowerShell, run `Get-Command -Module VMware.Sdk.Vr` to list all available vSphere Replication cmdlets.
- If you want to use information from the REST API specification to find a cmdlet, you can run `Get-VrOperation` with the `Method` and `Path` parameters.

For example:

```
Get-VrOperation -Method GET -Path '/replication-servers'
```

The command returns an object with the operation details.

```
Name           : GetAllVrServers
CommandInfo     : Invoke-VrGetAllVrServers
ApiName         : ServerApi
Path            : /replication-servers
Tags            : {server}
RelatedCommandInfos : {}
Method          : GET
Parameters      : {filterProperty, filter, sortBy, orderBy...}
```

The cmdlet name is listed in the `CommandInfo` property.

How to Get Information About a Cmdlet

To get information about a vSphere Replication cmdlet, run `Get-Help` with the cmdlet name. For example:

```
Get-Help
Invoke-VrConfigureReplication -full
```

Connect to a Local vSphere Replication Server

For replications within the same vCenter system, you must first connect to a local vSphere Replication server. You can do this by using the `Connect-VrServer` cmdlet.

- Verify that a vSphere Replication appliance is deployed at the local vCenter system.
- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Note: If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

1. Run `Connect-VrServer` and create a variable for the connection. Use the local vSphere Replication server address with the local vCenter user credentials.

```
$vrl = Connect-VrServer -Server <local_vr_address_or_fqdn> -User 'MyVCUser' -Password 'MyVCPassWord'
```

2. Optional: Print the connection variable to see the `VrServerConnection` object you created.

```
$vrl
```

The `VrServerConnection` object contains the `ConnectedPairings` property with the pairing you created. You might need this property to extract information required by some vSphere Replication operations.

Connect to a Local and to a Remote vSphere Replication Server

For replications from a source to a target vCenter system, you use the `Connect-VrServer` cmdlet to connect to the local and to the remote vSphere Replication server.

- Verify that vSphere Replication appliances are deployed at the local and at the remote vCenter systems.
- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Note: If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

1. Run `Connect-VrServer` and create a variable for the connection. Use the local vSphere Replication server address with the local vCenter user credentials. For the remote server connection, use the remote vCenter address with the remote vCenter user credentials.

```
$vr2 = Connect-VrServer -Server <local_vr_address_or_fqdn> -User 'MyVCUser' -Password 'MyVCPassWord' -RemoteServer <remote_vc_server_name> -RemoteUser 'MyRemoteVCUser' -RemotePassword 'MyRemoteVCPassWord'
```

2. Optional: Print the connection variable to see the `VrServerConnection` object you created.

```
$vr2
```

The `VrServerConnection` object contains the `ConnectedPairings` property with the pairing you created. You might need this property to extract information required by some vSphere Replication operations.

Use the ConnectedPairings Property

Use the `ConnectedPairings` property to extract connection information that you might need for vSphere Replication SDK operations.

Verify that you have a valid `VrServerConnection`. This example uses the `$vr2` connection variable from [Connect to a Local and to a Remote vSphere Replication Server](#).

The `ConnectedPairings` property is a key-value dictionary that contains information about all available vSphere Replication pairings. Use `ConnectedPairings` to extract information such as `PairingID` and `VcServerID`, which is required for some vSphere Replication SDK operations.

1. To list the connected pairings for a vSphere Replication connection, run:

```
$vr2.ConnectedPairings
```

The system lists a key-value map with your connected pairings.

2. To list a pairing, run:

```
$vr2.ConnectedPairings[<vc_server_name>].Pairing
```

The system returns the `Pairing` object with all its properties. You can copy the `PairingID` property as it is required by some vSphere Replication API operations.

3. To list the `LocalVcServer` object, run:

```
$vr2.ConnectedPairings[<vc_server_name>].Pairing.LocalVcServer
```

The system returns a `ServerInfo` object that contains a server ID that is required by some vSphere Replication API operations.

4. To list the `RemoteVcServer` object, run:

```
$vr2.ConnectedPairings[<vc_server_name>].Pairing.RemoteVcServer
```

The system returns a `ServerInfo` object that contains a server ID that is required by some vSphere Replication API operations.

Replicate a VM to a Datastore on the Local vCenter System

You can use the vSphere Replication SDK for PowerShell to replicate a VM from a source to a target datastore on the local vCenter system.

- Verify that the vSphere Replication appliance is deployed on the vCenter system.
- Verify that you are connected to the vCenter system. This example uses the `$vrl` connection variable from [Connect to a Local vSphere Replication Server](#).
- You must have the required privileges to configure replications and manage datastores on the vCenter system.

This example shows how to replicate a single VM from a source to a target datastore on the same vCenter system.

1. Get the local VM that you want to replicate.

```
$pairing = $vrl.ConnectedPairings["<local_vc_address_or_fqdn>"].Pairing
$replicatedVm = Invoke-VrGetLocalVms -PairingId $pairing.PairingId `
    -VcenterId $pairing.LocalVcServer.Id `
    -FilterProperty "Name" `
    -Filter "<my_vm_name>" `
    -Server $vrl
```

2. Get the VM hard disks that you want to replicate.

```
$replicatedHdds = Invoke-VrGetLocalVmDisks -PairingId $pairing.PairingId `
    -VcenterId $pairing.LocalVcServer.Id `
    -VmId $replicatedVm.List[0].Id `
    -Server $vrl
```

3. Get the target datastore for the replication.

```
$targetDatastore = Invoke-VrGetVrCapableTargetDatastores -PairingId $pairing.PairingId `
    -VcenterId $pairing.RemoteVcServer.Id `
    -FilterProperty "Name" `
    -Filter "<target_datastore_name>" `
    -Server $vrl
```

4. Create the configuration for disks replication.

```
$replicationVmDisks = @()
$replicatedHdds.List | ForEach-Object {
    $replicationVmDisks += Initialize-VrConfigureReplicationVmDisk -VmDisk $_ `
        -EnabledForReplication:$true `
        -DestinationDatastoreId $targetDatastore.List[0].Id `
        -DestinationDiskFormat 'SAMEASSOURCE'
```

5. Create the replication spec.

```
$replicationSpec = Initialize-VrConfigureReplicationSpec `
    -Rpo 60 -NetworkCompressionEnabled:$true `
    -MpitEnabled:$true -AutoReplicateNewDisks:$true `
    -LwdEncryptionEnabled:$false `
    -Disks $replicationVmDisks `
    -MpitInstances 1 `
    -MpitDays 1 `
    -TargetVcId $pairing.RemoteVcServer.Id `
    -VmId $replicatedVm.List[0].Id
```

6. Invoke the ConfigureReplication operation.

```
$task = Invoke-VrConfigureReplication -PairingId $pairing.PairingId `
```

```
-ConfigureReplicationSpec $replicationSpec -Server $vr1
```

vSphere Replication starts replicating the virtual machine files to the target datastore.

7. Monitor the task progress.

```
$t = Invoke-VrGetTaskInfo -TaskId $task.List[0].Id -Server $vr1
$t
```

Eventually, the task status changes to SUCCESS .

8. Optional: Get the replication on the target datastore.

```
$repl = Invoke-VrGetAllReplications -PairingId $pairing.PairingId `
-FilterProperty "Name" -Filter <my_vm_name> -Server $vr1
```

9. Sync your virtual machine to the configured replication on the target datastore.

```
Invoke-VrSyncReplication -PairingId $pairing.PairingId `
-ReplicationId $repl.List[0].Id -Server $vr1
```

Replicate a VM to a Datastore on a Remote vCenter System

You can use the vSphere Replication SDK for PowerShell to replicate a VM from a source to a target vCenter system.

- Verify that the vSphere Replication appliance is deployed on the source and target sites.
- Verify that your `VrServerConnection` is authenticated against the source and target vCenter systems. This example uses the `$vr2` connection variable from [Connect to a Local and to a Remote vSphere Replication Server](#).
- You must have the required privileges to configure replications and manage datastores on the source and target sites.

This example shows how to replicate a single VM from a datastore on the local vCenter system to a datastore on a remote vCenter system.

1. Get the local VM that you want to replicate.

```
$pairing = $vr2.ConnectedPairings["<remote_vc_server_name>"].Pairing
$replicatedVm = Invoke-VrGetLocalVms -PairingId $pairing.PairingId `
-VcenterId $pairing1.LocalVcServer.Id `
-FilterProperty "Name" `
-Filter "<my_vm_name>" `
-Server $vr2
```

2. Get the VM hard disks that you want to replicate.

```
$replicatedHdds = Invoke-VrGetLocalVmDisks -PairingId $pairing.PairingId `
-VmId $replicatedVm.List[0].Id `
-VcenterId $pairing.LocalVcServer.Id `
-Server $vr2
```

3. Get the target datastore on the remote site.

```
$targetDatastore = Invoke-VrGetVrCapableTargetDatastores -PairingId $pairing.PairingId `
-VcenterId $pairing.RemoteVcServer.Id `
-FilterProperty "Name" -Filter "<remote_datastore_name>" -Server $vr2
```

4. Create the configuration for disks replication.

```
$replicatedHdds.List | ForEach-Object {
    $replicationVmDisks += Initialize-VrConfigureReplicationVmDisk -VmDisk $_ `
-EnabledForReplication:$true `
-DestinationDatastoreId $targetDatastore.List[0].Id `
}
```

```

        -DestinationDiskFormat 'SAMEASSOURCE'
    }

```

5. Create the replication spec.

```

$replicationSpec = Initialize-VrConfigureReplicationSpec `
    -Rpo 60 -NetworkCompressionEnabled:$true `
    -MpitEnabled:$true -AutoReplicateNewDisks:$true `
    -LwdEncryptionEnabled:$false `
    -Disks $replicationVmDisks `
    -MpitInstances 1 `
    -MpitDays 1 `
    -TargetVcId $pairing.RemoteVcServer.Id `
    -VmId $replicatedVm.List[0].Id

```

6. Invoke the ConfigureReplication operation.

```

$task = Invoke-VrConfigureReplication -PairingId $pairing.PairingId `
    -ConfigureReplicationSpec $replicationSpec -Server $vr2

```

vSphere Replication starts replicating the virtual machine files to the target site datastore.

7. Monitor the task progress.

```

$t = Invoke-VrGetTaskInfo -TaskId $task.List[0].Id -Server $vr2
$t

```

Eventually, the task status must change to `SUCCESS`.

8. Optional: Get the replication on the remote site.

```

$rep2 = Invoke-VrGetAllReplications -PairingId $pairing.PairingId `
    -FilterProperty "Name" -Filter <my_vm_name> -Server $vr2

```

9. Optional: Sync your virtual machine to the configured replication on the target site.

```

Invoke-VrSyncReplication -PairingId $pairing.PairingId `
    -ReplicationId $rep2.List[0].Id -Server $vr2

```

Managing VMware Live Site Recovery™ with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of sample scripts that illustrate basic and advanced tasks in VMware Live Site Recovery™ (SRM) administration.

Connect to an SRM Server

To use the SRM API, you must establish a connection to an SRM server.

- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Note: If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

Some of the objects returned by the SRM API are objects from the vSphere API. To use those objects in integration with the vSphere API through PowerCLI, you can connect to the vCenter system that the SRM server is registered with.

1. To connect to the vCenter system that the SRM server is registered with, run `Connect-VIServer` with the server name and valid credentials.

```
Connect-VIServer -Server vc3.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
```

2. To connect to the SRM server registered with the connected vCenter system, run `Connect-SrmServer`.

```
$srmConnection = Connect-SrmServer
```

Note: If you have previously connected to other vCenter systems configured with SRM server support, this cmdlet invocation establishes a connection to their corresponding SRM servers as well.

3. Optional: To use the SRM API, you can call methods of the root object and instances of the objects that those calls return.

```
$srmApi = $srmConnection.ExtensionData
```

Note: The root SRM API object is the `ExtensionData` property of the `SrmServer` object.

Protect a Virtual Machine

You can protect a virtual machine by replicating it to a remote SRM site.

1. Connect to the vCenter system that the SRM server is registered with.

```
Connect-VIServer -Server vc3.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
```

2. Establish a connection to the local SRM server by providing credentials to the remote SRM site.

```
$srmConnection = Connect-SrmServer -RemoteUser 'MyRemoteUser' -RemotePassword 'MyRemotePassword'
```

3. List all protection groups associated with the SRM server.

```
$srmApi = $srmConnection.ExtensionData
$protectionGroups = $srmApi.Protection.ListProtectionGroups()
```

4. Associate the `TestVM` virtual machine with the `ProtGroup1` protection group and enable the protection for that virtual machine.

```
$vmToAdd = Get-VM "TestVM"

$targetProtectionGroup = $protectionGroups | where {$_.GetInfo().Name -eq "ProtGroup1" }

$targetProtectionGroup.AssociateVms(@( $vmToAdd.ExtensionData.MoRef))

# Enable protection for that virtual machine
$protectionSpec = New-Object VMware.VimAutomation.Srm.Views.SrmProtectionGroupVmProtectionSpec
$protectionSpec.Vm = $vmToAdd.ExtensionData.MoRef
$protectTask = $targetProtectionGroup.ProtectVms($protectionSpec)
while(-not $protectTask.IsComplete()) { sleep -Seconds 1 }
```

Create a Report of the Protected Virtual Machines

You can create a simple report containing information about the protected virtual machines associated with an SRM server.

- Verify that you are connected to a system.
- Verify that you are connected to an SRM server.

1. List all protection groups associated with the SRM server.

```
$srmApi = $srmConnection.ExtensionData
$protectionGroups = $srmApi.Protection.ListProtectionGroups()
```

2. Generate a report of the protected virtual machines.

```
$protectionGroups | % {
    $protectionGroup = $_

    $protectionGroupInfo = $protectionGroup.GetInfo()

    # The following command lists the virtual machines associated with a protection group
    $protectedVms = $protectionGroup.ListProtectedVms()
    # The result of the above call is an array of references to the virtual machines at the vSphere API
    # To populate the data from the vSphere connection, call the UpdateViewData method on each virtual machine view object
    $protectedVms | % { $_.Vm.UpdateViewData() }
    # After the data is populated, use it to generate a report
    $protectedVms | %{
        $output = "" | select VmName, PgName
        $output.VmName = $_.Vm.Name
        $output.PgName = $protectionGroupInfo.Name
        $output
    }
} | Format-Table @{Label="VM Name"; Expression={$_.VmName} }, @{Label="Protection group name"; Expression={$_.PgName} }
```

Create a Report of the Virtual Machines Associated with All Protection Groups

You can create a simple report containing information about the virtual machines associated with all protection groups.

- Verify that you are connected to a system.
- Verify that you are connected to an SRM server.

1. List all protection groups associated with the SRM server.

```
$srmApi = $srmConnection.ExtensionData
$protectionGroups = $srmApi.Protection.ListProtectionGroups()
```

2. Generate a report of the virtual machines associated with all protection groups.

```
$protectionGroups | % {
    $protectionGroup = $_

    $protectionGroupInfo = $protectionGroup.GetInfo()

    # The following command lists the virtual machines associated with a protection group
    $vms = $protectionGroup.ListAssociatedVms()
    # The result of the above call is an array of references to the virtual machines at the vSphere API
    # To populate the data from the vSphere connection, call the UpdateViewData method on each virtual machine view object
    $vms | % { $_.UpdateViewData() }
    # After the data is populated, use it to generate a report
    $vms | %{
        $output = "" | select VmName, PgName
        $output.VmName = $_.Name
    }
}
```

```

        $output.PgName = $protectionGroupInfo.Name
    }
    $output
}
} | Format-Table @{Label="VM Name"; Expression={$_.VmName} }, @{Label="Protection group name"; Expression={$_.PgName} }

```

Managing the VMware Live Site Recovery API with VMware Cloud Foundation PowerCLI

Use the VMware Live Site Recovery SDK for PowerShell to access the VMware Live Site Recovery API with VCF PowerCLI. Starting from VMware Cloud Foundation PowerCLI version 13.1.0, the VMware Live Site Recovery SDK provides low-level cmdlets for all available VMware Live Site Recovery API operations.

Understanding the VMware Live Site Recovery SDK for PowerShell

You can access the VMware Live Site Recovery API with VMware Cloud Foundation PowerCLI by using the VMware Live Site Recovery SDK for PowerShell.

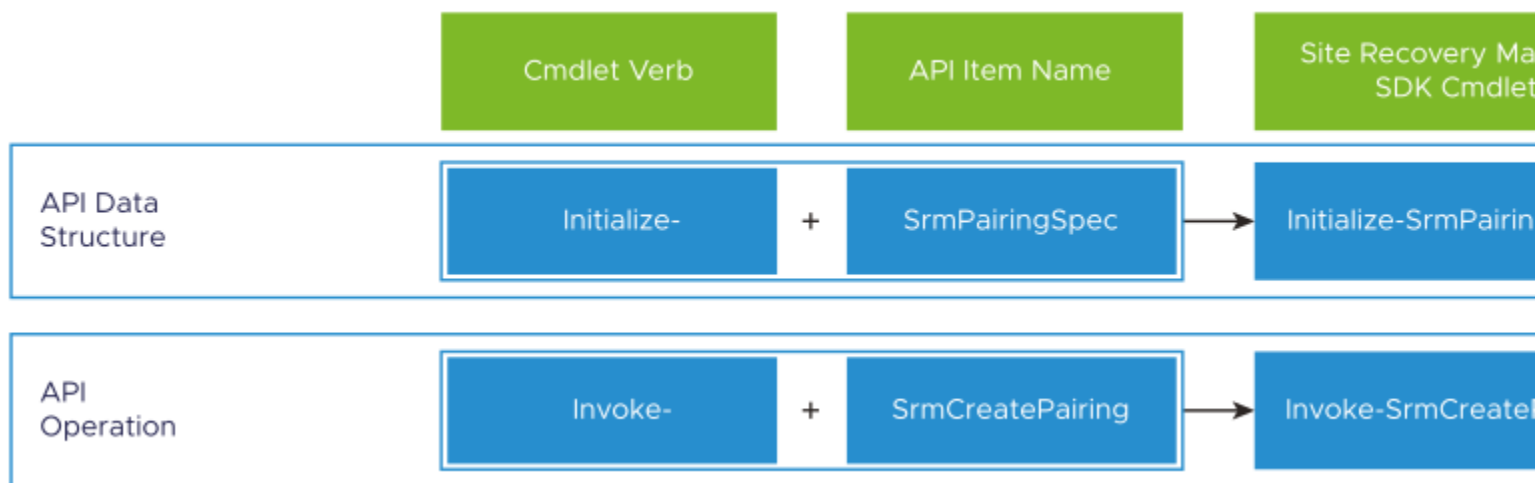
How Does the VMware Live Site Recovery SDK for PowerShell Work

Starting from VMware Cloud Foundation PowerCLI 13.1.0, you can use the VMware Live Site Recovery SDK for PowerShell to communicate with the VMware Live Site Recovery REST APIs.

The `VMware.Sdk.Srm` module contains automatically-generated SDK cmdlets that bind directly to the VMware Live Site Recovery REST APIs. The module functions on a low level and communicates directly with the VMware Live Site Recovery API server.

You can construct VMware Live Site Recovery SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the VMware Live Site Recovery API data structures or operations.

Figure 658: Constructing VMware Live Site Recovery SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the VMware Live Site Recovery API server.
- Use `Invoke-` cmdlets to call the VMware Live Site Recovery API server and invoke the operations.

Navigating the VMware Live Site Recovery SDK for PowerShell

You can navigate the VMware Live Site Recovery SDK for PowerShell by using the VMware Developer website, and the `Get-Help`, `Get-Command`, and `Get-SrmOperation` cmdlets.

To use the VMware Live Site Recovery SDK for PowerShell effectively, you must be familiar with the [VMware Site Recovery Manager REST API reference documentation](#).

How to Find a VMware Live Site Recovery SDK Cmdlet

There are several ways to find the VMware Live Site Recovery SDK cmdlets you need.

- In your browser, navigate the VMware Live Site Recovery API reference documentation. The VMware Live Site Recovery API Reference on VMware Developer features ready-to-use VMware Live Site Recovery SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.
- In PowerShell, run `Get-Command -Module VMware.Sdk.Srm` to list all available vSphere Replication cmdlets.
- If you want to use information from the REST API specification to find a cmdlet, you can run `Get-SrmOperation` with the `Method` and `Path` parameters.

For example:

```
Get-SrmOperation -Method GET -Path '/pairings/{pairing_id}/protection-management/groups'
```

The command returns an object with the operation details.

```
Name                : GetAllGroups
CommandInfo         : Invoke-SrmGetAllGroups
ApiName             : ProtectionApi
Path                : /pairings/{pairing_id}/protection-management/groups
Tags                : {protection}
RelatedCommandInfos : {}
Method              : GET
Parameters          : {pairingId, filterProperty, filter, sortBy...}
```

The cmdlet name is listed in the `CommandInfo` property.

How to Get Information About a Cmdlet

To get information about a VMware Live Site Recovery SDK cmdlet, run `Get-Help` with the cmdlet name. For example:

```
Get-Help
Invoke-SrmGetAllGroups -full
```

Connect to a Local and to a Remote VMware Live Site Recovery Server

To invoke operations on the VMware Live Site Recovery API, you must first connect to the VMware Live Site Recovery instances on the local (protected) and the remote (recovery) sites.

- Verify that VMware Live Site Recovery™ is deployed to the local (protected) and to the remote (recovery) vCenter systems.
- Verify that the VMware Live Site Recovery instances on the local and the remote sites are connected.
- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).

- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Note: If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

1. Connect to the local and to the remote VMware Live Site Recovery servers.

```
$localSrm = <local_srm_address_or_fqdn>
$srnConnection = Connect-SrmSdkServer -Server $localSrm `
    -User 'MyLocalVCUser' `
    -Password 'MyLocalVCPassWord' `
    -RemoteUser 'MyRemoteVCUser' `
    -RemotePassword 'MyRemoteVCPassWord'
```

2. Optional: Print the `ConnectedPairing` property of the connection you created.

```
$srnConnection.ConnectedPairing
```

The command prints the `Pairing` object that represents the pair between the local and the remote VMware Live Site Recovery sites.

You will need the `Pairing` object and its properties for various VMware Live Site Recovery API operations.

Host-Based Replication Scenarios

You can use the VMware Live Site Recovery SDK for PowerShell to manage scenarios for host-based replication (HBR) with VMware Cloud Foundation PowerCLI. HBR operates at the hypervisor level and allows you to replicate virtual machine data across different hosts or storage systems.

Create Protection Group for Host-Based Replication

To activate the protection of virtual machines, you first create a protection group for host-based replication (HBR).

- Verify that your SRM connection is authenticated against the protected (local) and recovery (remote) vCenter systems. This example uses the `$srnConnection` variable from [Connect to a Local and to a Remote VMware Live Site Recovery Server](#).
- The VMs that you want to protect must be replicated by using vSphere Replication.
- You must have the required privileges to create protection groups on the recovery site.

You create protection groups to enable VMware Live Site Recovery to protect virtual machines. This example shows how to create a protection group for host-based replication by using the VMware Live Site Recovery SDK for PowerShell.

1. Get the VM that you want to add to a protection group.

```
$replicatedVm = Invoke-SrmGetReplicatedVms `
    -PairingId $srnConnection.ConnectedPairing.PairingId `
    -VcenterId $srnConnection.ConnectedPairing.LocalVcServer.Id `
    -FilterProperty "Name" `
    -Filter "<myVmName>"
```

2. Prepare the HBR protection group spec.

```
$hbrSpec = Initialize-SrmHbrProtectionGroupSpec -Vms $replicatedVm.List[0].Id
```

3. Prepare the protection group creation spec.

For the `ReplicationType` parameter, specify `HBR`.

```
$protectionGroupSpec = Initialize-SrmProtectionGroupCreateSpec -Name "<myProtectionGroupName>" `
    -ReplicationType HBR `
    -ProtectedVcGuid $srnConnection.ConnectedPairing.LocalVcServer.Id `
```

```
-HbrSpec $hbrSpec
```

4. Create the protection group task.

```
$task = Invoke-SrmCreateGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
    -ProtectionGroupCreateSpec $protectionGroupSpec
```

5. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

`$task.Status` eventually changes to `SUCCESS`.

6. Optional: Retrieve the protection group you created.

```
$protectionGroup = Invoke-SrmGetGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
    -GroupId $task.Result
```

Create a recovery plan for your protection group.

Create Recovery Plan

To ensure business continuity and facilitate efficient disaster recovery for your virtual machines, you create a recovery plan for a protection group in a given pairing.

- Verify that your VMware Live Site Recovery connection is authenticated against the protected (local) and recovery (remote) vCenter systems. This example uses the `$srmConnection` variable from [Connect to a Local and to a Remote VMware Live Site Recovery Server](#).
- You must have created one or more protection groups on the protected (local) site. This example uses the `$protectionGroup` variable from [Create Protection Group for Host-Based Replication](#).
- You must have the required privileges to create recovery plans on the recovery site.

1. Create the recovery plan spec.

```
$recPlanSpec = Initialize-SrmRecoveryPlanCreateSpec `
    -Name "<myRecoveryPlanName>" `
    -ProtectedVcGuid $srmConnection.ConnectedPairing.LocalVcServer.Id `
    -ProtectionGroups $protectionGroup.Id
```

2. Create the recovery plan task.

```
$task = Invoke-SrmCreatePlan -PairingId $srmConnection.ConnectedPairing.PairingId `
    -RecoveryPlanCreateSpec $recPlanSpec
```

3. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

Eventually, the task status changes to `SUCCESS`.

4. Optional: Retrieve the recovery plan you created.

```
$recoveryPlan = Invoke-SrmGetRecoveryPlan -PairingId $srmConnection.ConnectedPairing.PairingId `
    -PlanId $task.Result
```

Array-Based Replication Scenarios

You can use the VMware Live Site Recovery SDK for PowerShell to manage scenarios for array-based replication (ABR) with VMware Cloud Foundation PowerCLI. With ABR, storage arrays at the protected site replicate data to peer arrays at the recovery site, typically at the block level.

Configure Array Managers for an Array Pair

Before creating a replicated array pair, you must configure array managers for the protected (local) and recovery (remote) SRM sites.

- Verify that your SRM connection is authenticated against the protected (local) and recovery (remote) vCenter systems. This example uses the `$srmConnection` variable from [Connect to a Local and to a Remote VMware Live Site Recovery Server](#).
- Verify that the SRM instances on the protected (local) and recovery (remote) sites are connected. This is known as site pairing.
- Verify that storage replication adapters (SRAs) are installed at both sites.
- You must have the required privileges to create array managers on the protected and recovery sites.

Configure array managers for the protected and recovery sites to enable the creation of an array pair for array-based replication (ABR). Array managers must be configured so that VMware Live Site Recovery can discover replicated devices, compute datastore groups, and initiate storage operations.

1. Retrieve the storage replication adapter (SRA) for the local array manager.

```
$localAdapter = (Invoke-SrmGetStorageAdapters -PairingId $srmConnection.ConnectedPairing.PairingId `
  -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
  -FilterProperty "Name" `
  -Filter "<myAdapterName>").List[0]
```

2. Prepare the connection parameters for the local SRA.

a) Retrieve the `ConnectionParamGroup` object for the local SRA.

```
$localConnectionParamGroup = (Invoke-SrmGetStorageAdapterConnectionParams `
  -PairingId $srmConnection.ConnectedPairing.PairingId `
  -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
  -StorageAdapterId $localAdapter.Id).List[0]
```

b) Provide values to the connection parameters, available in the

`$localConnectionParamGroup.ConnectionParams` list.

The set of connection parameters is specific to the used SRA. This example uses parameters specific to the NetApp SRA.

```
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "username"}).Value = "<myNetAppUser>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "password"}).Value = "<myNetAppPass>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "VolumeInclusion"}).Value = "<myVolume>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "SVM Name"}).Value = "<myLocalSvmName>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "mgmtIP"}).Value = "<myNetAppMgmtIP>"
```

3. Prepare the local array manager creation spec.

```
$localArrayManagerSpec = Initialize-SrmArrayManagerSpec -Name "<myLocalArrayManagerName>" `
  -StorageAdapter $localAdapter.Id `
  -ConnectionParams $localConnectionParamGroup
```

4. Create the local array manager.

```
$task = Invoke-SrmCreateArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
  -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
  -ArrayManagerSpec $localArrayManagerSpec
```

5. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id

$task.Status eventually changes to SUCCESS .
```

6. Optional: Retrieve the local array manager.

The `Result` property of the `$localManagerTask` object contains the ID of the created array manager.

```
$localManagerTask = Invoke-SrmGetTaskInfo -TaskId $task.Id

$localArrayManager = Invoke-SrmGetArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
    -ArrayManagerId $localManagerTask.Result
```

7. Create the remote array manager by using the same approach.

```
$remoteAdapter = (Invoke-SrmGetStorageAdapters -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
    -FilterProperty "Name" `
    -Filter "<myAdapterName>").List[0]

$remoteConnectionParamGroup = (Invoke-SrmGetStorageAdapterConnectionParams `
    -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
    -StorageAdapterId $remoteAdapter.Id).List[0]

($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "username"}).Value = "<myNetApp-
pUser>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "password"}).Value = "<myNetApp-
Pass>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "VolumeInclusion"}).Value =
"<myVolume>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "SVM Name"}).Value = "<myRe-
moteSvmName>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "mgmtIP"}).Value = "myNetApp-
MgmtIP"

$remoteArrayManagerSpec = Initialize-SrmArrayManagerSpec -Name "<myRemoteArrayManagerName>" `
    -StorageAdapter $remoteAdapter.Id `
    -ConnectionParams $remoteConnectionParamGroup

$task = Invoke-SrmCreateArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
    -ArrayManagerSpec $remoteArrayManagerSpec

#Wait for the create array manager task to complete.
$remoteManagerTask = Invoke-SrmGetTaskInfo -TaskId $task.Id

$remoteArrayManager = Invoke-SrmGetArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
    -ArrayManagerId $remoteManagerTask.Result
```

Create a replicated array pair for array-based replication.

Create Replicated Array Pair

Before you can create protection groups and recovery plans for ABR, you must create a replicated array pair for the protected (local) and recovery (remote) SRM sites.

- Verify that your SRM connection is authenticated against the protected and recovery vCenter systems. This example uses the `$srmConnection` variable from [Connect to a Local and to a Remote VMware Live Site Recovery Server](#).
- Verify that you configured array managers for the protected (local) and recovery (remote) SRM sites. This example uses the variables from [Configure Array Managers for an Array Pair](#).
- You must have the required privileges to create replicated array pairs on the protected and recovery sites.

1. Prepare the replicated array pair create spec.

You must specify the local array manager ID, and the storage array keys of the local (protected) and remote (recovery) array managers.

```
$replicatedArrayPairSpec = Initialize-SrmReplicatedArrayPairSpec -ArrayManagerId $localArrayManager.Id `
    -StorageArrayKey $localArrayManager.StorageArrays[0].StorageArrayKey `
    -PeerStorageArrayKey $remoteArrayManager.StorageArrays[0].StorageArrayKey
```

2. Create the replicated array pair.

```
$task = Invoke-SrmCreateReplicatedArrayPair -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
    -ReplicatedArrayPairSpec $replicatedArrayPairSpec
```

3. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

`$task.Status` eventually changes to SUCCESS .

4. Optional: Retrieve the replicated array pair.

The `Result` property of the `$replicatedArrayPairTask` object contains the ID of the replicated array pair.

```
$replicatedArrayPairTask = Invoke-SrmGetTaskInfo -TaskId $task.Id

$replicatedArrayPair = Invoke-SrmGetReplicatedArrayPair -PairingId $srmConnection.ConnectedPairing.Pair-
ingId `
    -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
    -ArrayPairId $replicatedArrayPairTask.Result
```

Create protection group and recovery plan by using the replicated array pair.

Create Protection Group and Recovery Plan for ABR

To ensure business continuity and facilitate efficient disaster recovery for your virtual machines with array-based replication, you create protection groups and recovery plans.

- Verify that your SRM connection is authenticated against the protected and recovery vCenter systems. This example uses the `$srmConnection` variable from [Connect to a Local and to a Remote VMware Live Site Recovery Server](#).
- Verify that you created a replicated array pair for the protected (local) and recovery (remote) SRM sites. This example uses the variables from [Create Replicated Array Pair](#)
- You must have the required privileges to create protection groups and recovery plans on the recovery site.

1. Discover the storage devices of the replicated array pair.

```
$task = Invoke-SrmDiscoverStorageDevices -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
    -ArrayPairId $replicatedArrayPair.Id
```

2. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

`$task.Status` eventually changes to `SUCCESS`.

3. Retrieve the storage devices from the replicated array pair.

```
$storageDevices = Invoke-SrmGetStorageDevices -PairingId $srmConnection.ConnectedPairing.PairingId `
    -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
    -ArrayPairId $replicatedArrayPair.Id
```

4. Prepare the ABR protection group spec.

This example uses the replicated array pair with a single storage device and a single datastore created in [Create Replicated Array Pair](#).

```
$abrProtectionGroupSpec = Initialize-SrmAbrProtectionGroupSpec `
    -ReplicatedArrayPair $replicatedArrayPair.Id `
    -Datastores $storageDevices.StorageDevicesData[0].Datastores[0].Id
```

5. Prepare the protection group create spec.

For the `ReplicationType` parameter, specify `ABR`.

```
$protectionGroupSpec = Initialize-SrmProtectionGroupCreateSpec -Name "<MyAbrProtectionGroup>" `
    -ReplicationType ABR `
    -ProtectedVcGuid $srmConnection.ConnectedPairing.LocalVcServer.Id `
    -AbrSpec $abrProtectionGroupSpec
```

6. Create the ABR protection group.

```
$task = Invoke-SrmCreateGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
    -ProtectionGroupCreateSpec $protectionGroupSpec
```

7. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

`$task.Status` eventually changes to `SUCCESS`.

8. Optional: Retrieve the ABR protection group.

```
$protectionGroup = Invoke-SrmGetGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
    -GroupId $task.Result
```

9. Prepare the recovery plan create spec.

```
$recPlanSpec = Initialize-SrmRecoveryPlanCreateSpec `
    -Name "<MyAbrRecoveryPlan>" `
    -ProtectedVcGuid $srmConnection.ConnectedPairing.LocalVcServer.Id `
    -ProtectionGroups @($protectionGroup.Id)
```

10. Create the recovery plan.

```
$task = Invoke-SrmCreatePlan -PairingId $srmConnection.ConnectedPairing.PairingId `
    -RecoveryPlanCreateSpec $recPlanSpec
```

11. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

`$task.Status` eventually changes to `SUCCESS`.

Managing the NSX Policy API with VMware Cloud Foundation PowerCLI

Manage your networking automation with VCF PowerCLI and the NSX Policy SDK for PowerShell. Starting from VCF PowerCLI 12.6, you can use low-level SDK cmdlets to communicate with all available NSX Policy REST APIs.

Understanding the NSX Policy SDK for PowerShell

You can manage the NSX Policy API with VMware Cloud Foundation PowerCLI by using the NSX Policy SDK for PowerShell.

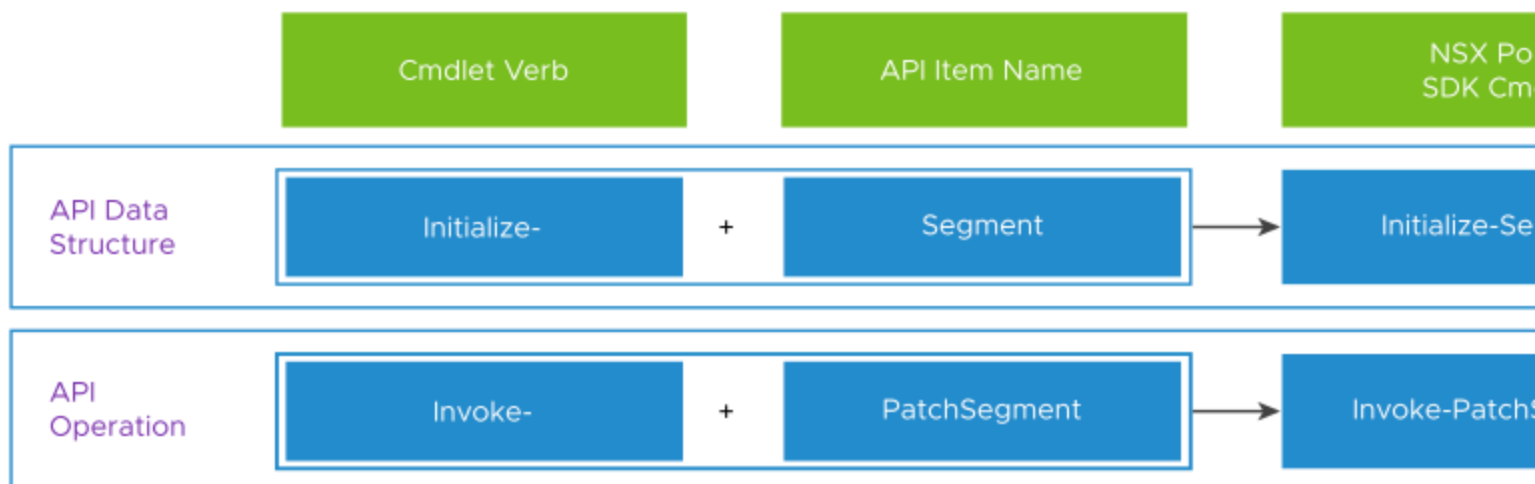
How Does the NSX Policy SDK for PowerShell Work

Starting from VMware Cloud Foundation PowerCLI 12.6, you can use the NSX Policy SDK for PowerShell to communicate with all available NSX Policy REST APIs.

The `VMware.Sdk.Nsx.Policy` module contains SDK cmdlets that are automatically generated and provide PowerShell bindings to the NSX REST APIs. The VCF PowerCLINSX module functions on a low level and communicates directly with the NSX Policy API server.

You can construct NSX Policy SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the API data structures or operations.

Figure 659: Constructing NSX Policy SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the API server.
- Use `Invoke-` cmdlets to call the NSX Policy API server and invoke the operations.

Navigate the NSX Policy SDK for PowerShell

You can navigate the NSX Policy SDK for PowerShell by using the `Get-NsxOperation` and `Get-Help` cmdlets.

To navigate the NSX Policy SDK for PowerShell, you must be familiar with the NSX Policy API and its reference documentation.

You can access two NSX REST API references on the VMware Developer website:

- The [NSX REST API reference](#) that contains all NSX networking services, for both on-prem and VMware Cloud on AWS.

- The [NSX VMC Policy API reference](#) that contains only the networking services for VMware Cloud on AWS. Here, you can find ready-to-use VCF PowerCLI samples that you can paste directly into your PowerShell console.

The following procedure is based on the NSX REST API reference.

1. In the NSX REST API reference, navigate to the NSX Policy API operation you want to use.

For example, go to **Segments (Fixed) > Create or update a segment**.

2. Copy the method and the path of the operation.

3. In PowerShell, run `Get-NsxOperation` with the `Method` and `Path` parameters.

For example:

```
Get-NsxOperation -Method PUT -Path '/policy/api/v1/infra/tier-1s/<tier-1-id>/segments/<segment-id>'
```

The command retrieves a custom object with the NSX operation details.

```
Name                : CreateOrReplaceSegment
CommandInfo         : Invoke-CreateOrReplaceSegment
ApiName             : PolicyConnectivityApi
Path                : /infra/tier-1s/{tier-1-id}/segments/{segment-id}
Tags                : {Policy, Networking, Connectivity, Segments...}
RelatedCommandInfos : {Initialize-Segment}
Method              : PUT
Parameters          : {tier1Id, segmentId, segment}
```

4. Copy the VCF PowerCLI cmdlet that corresponds to the API operation.

It is listed in the `CommandInfo` property.

5. To retrieve information about the cmdlet, run `Get-Help`.

For example:

```
Get-Help Invoke-CreateOrReplaceSegment -full
```

You can see the full information about the cmdlet such as its description, parameters, and code examples.

Connect to an On-Prem NSX Server

You can connect to an on-prem NSX server by using the `Connect-NsxServer` cmdlet.

- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Run `Connect-NsxServer` with the NSX server IP address or FQDN and valid user credentials.

```
Connect-NsxServer -Server <nsx_server_ip_address_or_fqdn> -User <my_username> -Password <my_password>
```

Connect to an NSX Server on VMware Cloud on AWS

You can connect to an NSX server on VMware Cloud on AWS by using the `Connect-NsxVmcServer` cmdlet.

- Verify that you have a valid API token from VMware Cloud services.
- Verify that you know the name of the SDDC, in which the server you want to connect to resides.

1. Create a variable with the VMware Cloud on AWS server connection.

```
$vmcServer = Connect-VmcServer -ApiToken $apiToken
```

2. Create a variable with the SDDC name.

```
$sddc = Get-VmcSddc -Name $sddcName
```

3. Connect to the NSX server on VMware Cloud on AWS.

```
Connect-NsxVmcServer -VmcConnection $vmcServer -VmcSddc $sddc
```

Create a Tier-0 Gateway

You can create a Tier-0 gateway with the NSX Policy SDK for PowerShell by using the `Initialize-Tier0` and `Invoke-PatchTier0` cmdlets.

Verify that you are connected to an NSX server system.

1. Prepare the input for the operation.

```
$t0 = Initialize-Tier0 -ArpLimit 5000 -DisplayName $T0GatewayName
```

2. Invoke the operation.

```
Invoke-PatchTier0 -Tier0Id $T0GatewayName -Tier0 $t0
```

Create a Tier-1 Gateway

You can create a Tier-1 gateway with the NSX Policy SDK for PowerShell by using the `Initialize-Tier1` and `Invoke-PatchTier1` cmdlets.

- Verify that you are connected to an NSX server system.
- Verify that your system has Tier-0 gateway(s).

1. Retrieve the Tier-0 paths.

```
$t0s = Invoke-ListTier0s
$t0 = $t0s.Results | where {$_.DisplayName -eq $T0GatewayName}
```

2. Prepare the input for the operation.

```
$t1 = Initialize-Tier1 -ArpLimit 5000 -DisplayName $T1GatewayName -Tier0Path $t0.Path
```

3. Invoke the operation.

```
Invoke-PatchTier1 -Tier1Id $T1GatewayName -Tier1 $t1
```

Add an Existing Tier-1 Gateway to a Specific Edge Cluster

You can add an existing Tier-1 gateway to a specific Edge Cluster with the NSX Policy SDK for PowerShell by using the `Invoke-PatchTier1LocaleServices` cmdlet.

- Verify that you are connected to an NSX server system.
- Verify that you know the Edge Cluster name of the Edge Cluster to which you want to add the Tier-1 gateway.

1. Retrieve the Edge Cluster information.

```
$edgeClusters = Invoke-ListEdgeClustersForEnforcementPoint -SiteId default -EnforcementPointId default
```

```
$edgeCluster = $edgeClusters.Results | where {$_.Displayname -eq $edgeClusterName}
```

2. Create a client-side object for locale service with the Edge Cluster path.

```
$localeService = Initialize-LocaleServices -EdgeClusterPath $edgeCluster.Path -
DisplayName default -Id default
```

3. Invoke the operation.

```
Invoke-PatchTier1LocaleServices -Tier1Id $T1.Id -LocaleServices $localeService -LocaleServicesId default
```

Create a Segment (On-Prem)

You can create a segment on an on-prem NSX server system by using the `Invoke-PatchInfraSegment` cmdlet.

- Verify that you are connected to an on-prem NSX server system.
- Verify that your system has Tier-1 gateway(s).

To create an on-prem (flexible) segment, you must specify a transport zone and assign a subnet.

1. Retrieve transport zone details.

```
$tzs = Invoke-ListTransportZonesForEnforcementPoint -EnforcementpointId default -SiteId default
$tz = $tzs.Results | where {$_.DisplayName -eq $TransportZone}
```

2. Prepare the input for the segment subnet.

```
$segmentSubnet = Initialize-SegmentSubnet -GatewayAddress $GatewayCIDR
```

3. Retrieve the Tier-1 paths.

```
$t1s = Invoke-ListTier1
$t1 = $t1s.Results | where {$_.DisplayName -eq $T1GatewayName}
```

4. Prepare the input for the segment.

```
$segment = Initialize-Segment -DisplayName $SegmentName -TransportZonePath $tz.Path `
-Subnets $segmentSubnet -ConnectivityPath $t1.Path
```

5. Invoke the operation.

```
Invoke-PatchInfraSegment -Segment $segment -SegmentId $SegmentName
```

Create a Segment (VMware Cloud on AWS)

You can create a segment for an NSX server system on VMware Cloud on AWS by using the `Invoke-CreateOrReplaceSegment` cmdlet.

- Verify that you are connected to an NSX server system on VMware Cloud on AWS.
- Verify that your system has Tier-1 gateway(s).

1. Prepare the input for the segment subnet.

```
$segmentSubnet = Initialize-SegmentSubnet -GatewayAddress $GatewayCIDR
```

2. Prepare the input for the segment.

```
$segment = Initialize-Segment -DisplayName $SegmentName -Subnets @($segmentSubnet)
```

3. Invoke the operation.

```
Invoke-CreateOrReplaceSegment -Tier1Id $T1GatewayName -SegmentId $SegmentName -Segment $segment
```

Create a Distributed Firewall Policy

You can create a distributed firewall policy for an NSX server system by using the `Invoke-PatchSecurityPolicyForDomain` cmdlet.

- Verify that you are connected to an NSX server system.

1. Create lookup object variables.

```
$serviceList = @("SSH", "HTTP")
$sourceGroups = @("ANY")
$destinationGroups = @("MyGroupName")
```

2. Look up groups and services.

```
$allServices = Invoke-ListServicesForTenant
$servicePathList = @()
foreach ($serv in $serviceList) {
    $s = $allServices.Results | where {$_.DisplayName -eq $serv}
    $servicePathList += $s.Path
}

$allGroups = Invoke-ListGroupForDomain -DomainId default
$sourceGroupList = @()
foreach ($gp in $sourceGroups) {
    if ($gp -eq "ANY") {
        $sourceGroupList += "ANY"
    } else {
        $g = $allGroups.Results | where {$_.DisplayName -eq $gp}
        $sourceGroupList += $g.Path
    }
}

$destinationGroupList = @()
foreach ($gp in $destinationGroups) {
    if ($gp -eq "ANY") {
        $destinationGroupList += "ANY"
    } else {
        $g = $allGroups.Results | where {$_.DisplayName -eq $gp}
        $destinationGroupList += $g.Path
    }
}
```

3. Prepare the input for the policy rule.

```
$rule = Initialize-Rule -DisplayName $ruleName -Id $ruleName -SourceGroups $sourceGroupList -Destination-
Groups $destinationGroupList -Services $servicePathList -Action "ALLOW"
```

4. Prepare the input for the security policy.

```
$securityPolicy = Initialize-SecurityPolicy -DisplayName $policyName -Rules @($rule)
```

5. Invoke the operation.

```
Invoke-PatchSecurityPolicyForDomain -DomainId default -SecurityPolicyId $policyName -SecurityPolicy $secu-
rityPolicy
```

Managing VMware Cloud Director with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of scripts that illustrate basic and advanced tasks in cloud administration.

Connect to a VMware Cloud Director Server

To run cmdlets on a VMware Cloud Director server and perform administration or monitoring tasks, you must establish a connection to the server.

- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Note: If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

You can have more than one connection to the same server. For more information, see [Managing Default Server Connections in VCF PowerCLI](#).

If your login credentials contain non-alphanumeric characters, you might need to escape them. For more information, see [Processing Non-alphanumeric Characters in VCF PowerCLI](#).

Run `Connect-CIServer` with the server name and valid credentials.

```
Connect-CIServer -Server cloud.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
```

Create and Manage Organizations

Organizations provide resources to a group of users and set policies that determine how users can consume those resources. Create and manage organizations for each group of users that requires its own resources, policies, or both.

Verify that you are connected to a server as a provider administrator.

1. Generate a customized report for all organizations on the server.

```
Get-Org | Select Name, Enabled, StoredVMQuota, DeployedVMQuota
```

2. Add a new organization on the server and provide a name and a full name for it.

```
New-Org -Name 'MyOrg1' -FullName 'My Organization 1'
```

By default, the new organization is enabled. Enabling the organization lets users log in.

3. Add a description for the new organization.

```
Get-Org -Name 'MyOrg1' | Set-Org -Description "This organization provides resources to John Doe."
```

4. Deactivate and remove the new organization.

```
Get-Org -Name 'MyOrg1' | Set-Org -Enabled $false | Remove-Org
```

Create and Manage Organization Virtual Data Centers

To allocate resources to an organization, you need to create an organization virtual data center (vDC). When the demands of the organization change, you can modify or remove the organization vDC.

- Verify that you are connected to a server as a provider administrator.
 - Verify that at least one enabled provider vDC is available on the server.
1. Create a new organization vDC using the Pay As You Go model for resource allocation.

```
$myOrg = Get-Org -Name 'MyOrg1'
$myPVdc = Get-ProviderVdc -Name 'MyProvidervDC'
New-OrgVdc -Name 'MyOrgvDC' -AllocationModelPayAsYouGo -Org $myOrg -ProviderVdc $myPVdc -VMCPUCoreMhz
1000
```

To create the organization vDC, VMware Cloud Foundation Automation Provider Management PowerCLI uses a default configuration based on the selected resource allocation model.

- VMMaxCount is set to 100
 - NetworkMaxCount is set to 1024
 - The vDC is automatically enabled
 - Thin provisioning is deactivated
 - Fast provisioning is deactivated
 - NicMaxCount is set to \$null (unlimited)
 - MemoryGuaranteedPercent is set to 100
 - CpuGuaranteedPercent is set to 0
2. Modify the vCPU speed setting for the virtual machines in the organization vDC.

```
Get-OrgVdc -Name 'MyOrgVdc' | Set-OrgVdc -VMCpuCoreMhz 2000
```

3. Enable fast provisioning for the virtual machines in the organization vDC.

```
Get-OrgVdc -Name 'MyOrgVdc' | Set-OrgVdc -UseFastProvisioning $true
```

4. Deactivate and remove the new organization vDC.

```
Get-OrgVdc -Name 'MyOrgVdc' | Set-OrgVdc -Enabled $false | Remove-OrgVdc
```

Filter and Retrieve Organization Virtual Data Center Networks

To generate reports about organization vDC networks, you need to retrieve the respective organization vDC networks. You can use search criteria to filter the results returned by `Get-OrgVdcNetwork`.

Verify that you are connected to a server.

- Get all organization vDC networks for the organization named *MyOrgVdc*.

```
Get-OrgVdc -Name 'MyOrgVdc' | Get-OrgVdcNetwork
```

- Get the organization vDC network that is named *MyOrgVdcNetwork*.

```
Get-OrgVdc -Name 'MyOrgVdc' | Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork'
```

Import a vApp Template from the Local Storage

To make an OVF package from your local storage available to other cloud users, you can import the package and save it as a vApp template in a catalog.

Verify that you are connected to a server.

1. Retrieve the catalog to which you want to add the imported vApp template.

```
$myCatalog = Get-Catalog -Name 'MyCatalog'
```

2. Retrieve the organization virtual data center (vDC) to which you want to add the imported vApp template.

```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```

3. Import a virtual machine from your local storage and save it as a vApp template in the cloud.

```
Import-CIVAppTemplate -SourcePath 'C:\OVFs\WindowsXP\WindowsXP.ovf' -Name 'MyWin-  
dowsXPVAppTemplate' -OrgVdc $myOrgVdc -Catalog $myCatalog
```

Create a vApp Template from a vApp

Creating vApp templates from vApps in the cloud might minimize future efforts for cloning vApps. You can use the templates later to create vApps that are based on the source vApp.

Verify that you are connected to a server.

1. Retrieve the source vApp for the vApp template that you want to create.

```
$myVApp = Get-CIVApp -Name 'MyVApp'
```

2. If the source vApp is running, stop it.

```
$myVApp = Stop-CIVApp -VApp $myVApp
```

3. Retrieve the catalog to which you want to add the new vApp template.

```
$myCatalog = Get-Catalog -Name 'MyCatalog'
```

4. Retrieve the organization vDC to which you want to add the new vApp template.

```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```

5. Create a vApp template.

```
New-CIVAppTemplate -Name 'MyVAppTemplate' -VApp $myVApp -OrgVdc $myOrgVDC -Catalog $myCata-  
log
```

6. Start the source vApp.

```
$myVApp = Start-CIVApp -VApp $myVApp
```

Create a vApp from the template and modify the vApp. See [Create and modify a vApp](#).

Import a vApp from vSphere

To make a virtual machine from the underlying vSphere infrastructure available to your VMware Cloud Director server, you can import it and save it as a vApp.

- Verify that you are connected to a server as a provider administrator.
- Verify that you are connected to a system.

1. Retrieve the vSphere virtual machine that you want to import.

```
$myVm = Get-VM -Name 'MyVMToImport'
```

2. Retrieve the organization vDC to which you want to import the virtual machine.

```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```

3. Import the virtual machine and store it as a vApp.

```
Import-CIVApp -VM $myVm -OrgVdc $myOrgVdc
```

Create and Modify a vApp

You can use vApp templates to instantiate vApps. After creating the vApp, you can modify its settings to minimize the consumption of computing and storage resources.

Verify that you are connected to a server.

1. Retrieve the organization vDC to which you want to add the new vApp.

```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```

2. Retrieve the source vApp template for your new vApp.

```
$myVAppTemplate = Get-CIVAppTemplate -Name 'MyVAppTemplate'
```

3. Create your new vApp.

```
$myVApp = New-CIVApp -Name 'MyVApp' -VAppTemplate $myVAppTemplate -OrgVdc $myOrgVDC
```

By default, the vApp is powered off.

4. Renew the runtime lease for the new vApp and set it to 12 hours.

```
Set-CIVApp -VApp $myVApp -RuntimeLease "12:0:0" -RenewLease
```

To set leases, you can use the *days.hours:minutes:seconds* syntax.

5. Start the new vApp.

```
Start-CIVApp -VApp $myVApp
```

Manage Virtual Machines with vApps

For a large-scale approach to administration, you can start, stop, or restart virtual machines or their guest operating systems by running cmdlets on the associated vApps.

Verify that you are connected to a server.

1. Power on all virtual machines in all vApps with names starting with *MyVApp*.

```
Get-CIVApp -Name 'MyVApp*' | Start-CIVApp
```

2. Suspend all virtual machines in all vApps with names starting with *YourVApp*.

```
Get-CIVApp -Name 'YourVApp*' | Suspend-CIVApp
```

3. Power off all virtual machines in the vApp named *MyVApp1*.

```
Get-CIVApp -Name 'MyVApp1' | Stop-CIVApp
```

4. Shut down the guest operating systems of all virtual machines in the vApp named *MyVApp2*.

```
Get-CIVApp -Name 'MyVApp2' | Stop-CIVAppGuest
```

- Restart the guest operating systems of all virtual machines in the vApp named *MyVApp3*.

```
Get-CIVApp -Name 'MyVApp3' | Restart-CIVAppGuest
```

- Reset all virtual machines in the vApp.

```
Get-CIVApp -Name 'MyVApp4' | Restart-CIVApp
```

Manage Virtual Machines and Their Guest Operating Systems

For a targeted approach to administration, you can use the `CIVM` and `CIVMGuest` cmdlets to handle lifecycle operations for one or more virtual machines.

Verify that you are connected to a server.

- Retrieve all virtual machines with names starting with *MyVM* and power them on.

```
Get-CIVM -Name 'MyVM*' | Start-CIVM
```

- Suspend all virtual machines with names starting with *YourVM*.

```
Get-CIVM -Name 'YourVM*' | Suspend-CIVM
```

- Power off the virtual machine named *MyVM1*.

```
Get-CIVM -Name 'MyVM1' | Stop-CIVM
```

- Shut down the guest operating system of the virtual machine named *MyVM2*.

```
Get-CIVM -Name 'MyVM2' | Stop-CIVMGuest
```

- Restart the guest operating system of the virtual machine named *MyVM3*.

```
Get-CIVM -Name 'MyVM3' | Restart-CIVMGuest
```

- Reset the nonresponsive virtual machine named *MyVM4*.

```
Get-CIVM -Name 'MyVM4' | Restart-CIVM
```

Retrieve a List of the Internal and External IP Addresses of Virtual Machines in vApps

When managing vApps in the cloud, you might need to obtain information about the NIC settings of the associated virtual machines.

Verify that you are connected to a server.

- Retrieve the organization for which you want to generate the report.

```
$myOrg = Get-Org -Name 'MyOrg'
```

- Retrieve all vApps in the organization.

```
$vApps = Get-CIVApp -Org $myOrg
```

- Populate an array with the information that you want to report.

```
$vAppNetworkAdapters = @()
foreach ($vApp in $vApps) {
    $vms = Get-CIVM -VApp $vApp
```



```

foreach ($vm in $vms) {
    $networkAdapters = Get-CINetworkAdapter -VM $vm
    foreach ($networkAdapter in $networkAdapters) {
        $vAppNicInfo = New-Object "PSCustomObject"
        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name VAppName -Value $vApp-
p.Name
        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name VMName -Value $vm.Name
        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name NIC -Value ("NIC" +
$networkAdapter.Index)
        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name ExternalIP -Value $net-
workAdapter.ExternalIpAddress
        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name InternalIP -Value $net-
workAdapter.IpAddress

        $vAppNetworkAdapters += $vAppNicInfo
    }
}
}

```

Running this script retrieves the names of the virtual machines and their associated vApp, the IDs of the NICs of the virtual machines, and external, and internal IP addresses of the NICs.

4. Display the report on the screen.

```
$vAppNetworkAdapters
```

Create and Manage Access Control Rules

By defining access control rules you can assign levels of access to separate users, user groups, or everyone in the organization. You can define access control rules for catalogs and vApps.

Verify that you are connected to a server.

1. Create a new rule for accessing the vApp named *MyVApp*.

```
New-CIAccessControlRule -Entity 'MyVApp' -EveryoneInOrg -AccessLevel "Read"
```

All users in the organization have read-only access to the vApp.

2. Modify the access rule for a trusted user who needs full control over *MyVApp*.

```
New-CIAccessControlRule -Entity 'MyVApp' -User "MyAdvancedUser" -AccessLevel "FullControl"
```

3. Restrict the full control access of *MyAdvancedUser* to read/write access.

```
$accessRule = Get-CIAccessControlRule -Entity 'MyVApp' -User 'MyAdvancedUser'
$accessRule | Set-CIAccessControlRule -AccessLevel "ReadWrite"
```

4. Remove the custom rule that you created earlier for *MyAdvancedUser*.

```
$accessRule | Remove-CIAccessControlRule
```

Filter and Retrieve vApp Networks

To generate reports about vApp networks, you need to retrieve the respective vApp networks. You can use search criteria to filter the results returned by `Get-CIVAppNetwork`.

Verify that you are connected to a server.

- Get the vApp network named *MyVAppNetwork*.

```
Get-CIVAppNetwork -Name 'VAppNetwork'
```

- Get all vApp networks for the vApp named *MyVApp*.

```
Get-CIVApp -Name 'MyVApp' | Get-CIVAppNetwork
```

- Get all vApp networks of connection type direct and direct fenced.

```
Get-CIVAppNetwork -ConnectionType Direct, DirectFenced
```

- Get all direct vApp networks that connect to the organization vDC network named *MyOrgVdcNetwork*.

```
Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork' | Get-CIVAppNetwork -ConnectionType Direct
```

Create vApp Networks for a Selected vApp

To define how the virtual machines in a vApp connect to each other and access other networks, you need to create a vApp network. When creating the vApp network, you can select the settings for the network, or adopt them from an organization policy.

To address multiple networking scenarios for a vApp, you can create multiple vApp networks.

Create an Isolated vApp Network

When you do not want the virtual machines in a vApp to connect to objects outside the vApp, you must create an isolated vApp network.

Verify that you are connected to a server.

1. Retrieve the vApp for which you want to create a vApp network.

```
$myVApp = Get-CIVApp -Name 'MyVApp'
```

2. Create the new vApp network with a selected gateway and network mask.

```
New-CIVAppNetwork -VApp $myVApp -Name 'MyVAppInternalNetwork' -Routed -Gateway '192.168.2.1'
-Netmask '255.255.255.0' -ParentOrgVdcNetwork $null
```

By default, the vApp network has an enabled firewall.

Create an NAT Routed vApp Network

To provide a vApp network with DHCP, firewall, NAT, and VPN services, you must create it as an NAT routed vApp network.

Verify that you are connected to a server.

1. Retrieve the vApp for which you want to create a vApp network.

```
$myVApp = Get-CIVApp -Name 'MyVApp'
```

2. Retrieve the organization vDC network to which you want to connect the vApp network.

```
$myOrgVdcNetwork = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork'
```

3. Create the new vApp network with a gateway and network mask, defined pool of static IP addresses, and a deactivated firewall.

```
New-CIVAppNetwork -VApp $myVApp -ParentOrgVdcNetwork $myOrgVdcNetwork -Name 'MyVAppInternalNetwork' -Routed -Gateway '192.168.2.1' -Netmask '255.255.255.0' -DisableFirewall -StaticIPPool "192.168.2.100 - 192.168.2.199"
```

If you do not run `New-CIVAppNetwork` with the `DisableFirewall` parameter, the new vApp network has an active firewall by default.

Create a Direct vApp Network

To establish a network connection between the virtual machines in a vApp and an organization network, you need to create a direct vApp network.

Verify that you are connected to a server.

1. Retrieve the vApp for which you want to create a vApp network.

```
$myVApp = Get-CIVApp -Name 'MyVApp'
```

2. Retrieve the organization vDC network that you want to connect to.

```
$myOrgVdcNetwork = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork'
```

3. Create a direct vApp network that connects to the selected organization vDC network.

```
New-CIVAppNetwork -VApp $myVApp -Direct -ParentOrgVdcNetwork $myOrgVdcNetwork
```

By default, the new vApp network has an enabled firewall.

Modify or Remove vApp Networks

Based on the type of the vApp network, you can configure various network settings, such as DNS, static IP pools, and firewalls. If you no longer need a vApp network, you can remove it.

Verify that you are connected to a server.

1. Retrieve the vApp for which you want to modify vApp networks.

```
$myVApp = Get-CIVApp -Name 'MyVApp'
```

2. Modify the settings for DNS and static IP pool for the vApp network named *MyVAppNetwork*.

```
Get-CIVAppNetwork -VApp $myVApp -Name 'MyVAppNetwork' | Set-CIVAppNetwork -PrimaryDns 10.17.0.94 -SecondaryDns 10.17.0.95 -DnsSuffix 'my.domain.com' -StaticIPPool "10.151.168.1 - 10.151.169.240"
```

3. Optional: Remove *MyVAppNetwork*.

```
$myVApp | Get-CIVAppNetwork -Name 'MyVAppNetwork' | Remove-CIVAppNetwork
```

4. Optional: Remove all isolated vApp networks for the vApp named *MyVApp*.

```
$myVApp | Get-CIVAppNetwork -ConnectionType Isolated | Remove-CIVAppNetwork
```

5. Retrieve the organization vDC network named *MyOrgVdcNetwork1*.

```
$myOrgVdcNetwork1 = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork1'
```

6. Retrieve the organization vDC network named *MyOrgVdcNetwork2*.

```
$myOrgVdcNetwork2 = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork2'
```

7. Redirect all vApp networks that connect to *MyOrgVdcNetwork1* to connect to *MyOrgVdcNetwork2*.

```
Get-CIVAppNetwork -ParentOrgVdcNetwork $myOrgVdcNetwork1 | Set-CIVAppNetwork -ParentOrgVdcNetwork $myOrgVdcNetwork2 -NatEnabled $false -FirewallEnabled $false
```

The operation deactivates the firewall and NAT routing for all vApp networks that are connected to *MyOrgVdcNetwork1*.

Managing VMware Cloud Foundation Operations with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of sample scripts that illustrate basic and advanced tasks in VMware Cloud Foundation Operations.

Connect to a VMware Cloud Foundation Operations Server

To run VMware Cloud Foundation Operations cmdlets, you must establish a connection to a VMware Cloud Foundation Operations server and a vCenter system that is monitored by the VMware Cloud Foundation Operations instance.

- If the certificate of the server you want to connect to is not trusted, verify that your invalid server certificate action settings are configured properly. For more information, see [Configuring VCF PowerCLI Response to Untrusted Certificates](#).
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Note: If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

You can have more than one connection to the same server. For more information, see [Managing Default Server Connections in VCF PowerCLI](#).

If your login credentials contain non-alphanumeric characters, you might need to escape them. For more information, see [Processing Non-alphanumeric Characters in VCF PowerCLI](#).

1. Run `Connect-OMServer` with the server name and valid credentials.

```
Connect-OMServer -Server vrops3.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
```

2. Run `Connect-VIServer` with the server name and valid credentials.

```
Connect-VIServer -Server vc3.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
```

Check Memory Waste Levels

You can check the memory waste levels of a virtual machine host for a specific period of time. For example, you can check the memory waste levels in the last month.

- Verify that you are connected to a instance.
 - Verify that you are connected to the system that is monitored by the instance.
1. Browse the vCenter inventory and select a virtual machine host for which you want to check the memory waste levels.

```
$vmHost = Get-VMHost 'MyHost'
```

2. Get the VMware Cloud Foundation Operations resource that refers to this virtual machine host.

```
$hostResource = $vmHost | Get-OMResource
```

3. Check the defined metrics for this VMware Cloud Foundation Operations resource type.

```
Get-OMStatKey -AdapterKind $hostResource.AdapterKind -ResourceKind $hostResource.ResourceKind
```

4. Get data for a specific metric.

```
$hostResource | Get-OMStat -Key "mem|waste"
```

Note: This command retrieves all available metric data with the highest available granularity.

5. Get metric data for the last month aggregated on a daily basis.

```
$hostResource | Get-OMStat -Key "mem|waste" -From ([datetime]::Now.AddMonths(-1)) -IntervalType Days -IntervalCount 1 -RollupType Avg
```

Get Remediation Recommendations

You can get remediation recommendations for a specific resource, such as a problematic virtual machine.

- Verify that you are connected to a `instance`.
- Verify that you are connected to the `system` that is monitored by the `instance`.
- Verify that at least one alert is triggered for the virtual machine.

1. Get the virtual machine you want to check for alerts.

```
$myVm = Get-VM 'MyVM'
```

2. Get the associated VMware Cloud Foundation Operations resource and its associated active alerts.

```
$myVmAlerts = $myVm | Get-OMResource | Get-OMAlert -Status Active
```

3. List the remediation recommendations for the obtained alerts.

```
$myVmAlerts | Get-OMRecommendation
```

Change Alert Ownership

You can retrieve all active alerts for a specific datastore and assign the alert ownership to your user profile.

- Verify that you are connected to a `instance`.
- Verify that you are connected to the `system` that is monitored by the `instance`.

1. Get all active alerts for the datastore.

```
$alerts = Get-Datastore 'shared' | Get-OMResource | Get-OMAlert -Status Active | where { ($_.AssignedUser -eq $null) -and ($_.ControlState -eq 'Open') }
```

2. Assign the obtained alerts to the user profile you are currently using.

```
$alerts | Set-OMAlert -TakeOwnership
```

Create a Report for Problematic Hosts

You can create a report for virtual machine hosts that have problematic health status.

- Verify that you are connected to a `instance`.
- Verify that you are connected to the `system` that is monitored by the `instance`.

1. Get all problematic host resources in VMware Cloud Foundation Operations that have red or yellow health status.

```
$hosts = Get-OMResource | where { $_.ResourceKind -eq 'HostSystem' -and $_.Health -in ('Red', 'Yellow') }
```

2. Get the virtual machine hosts that cause the problem.

```
$hosts | Get-VmHost
```

Managing VMware Cloud on AWS with VMware Cloud Foundation PowerCLI

To familiarize yourself with VMware Cloud Foundation PowerCLI, you can use a set of scripts that illustrate basic and advanced tasks in the VMware Cloud on AWS administration.

Connecting to VMware Cloud on AWS

You can use VMware Cloud on AWS to provision and manage vCenter virtual data centers in the cloud. VMware Cloud on AWS is a cloud service that belongs to the VMware Cloud services platform.

The VMware Cloud services platform is a web-based application that supports any number of VMware Cloud services. When you sign up for VMware Cloud services, you access your services and manage all your users, organizations, and payment methods through this platform. The VMware Cloud services platform manages common requirements for all cloud services, such as authentication, user roles, and so on.

When connecting to a VMware Cloud on AWS service, you need to authenticate with the corresponding VMware Cloud services platform.

Connect to VMware Cloud on AWS by Using an API Token

To authenticate with VMware Cloud on AWS, you can use an API token from the VMware Cloud service web portal.

Use the VMware Cloud service web portal to generate an API token for authentication with VCF PowerCLI.

- To connect to a VMware Cloud on AWS server by using the API token, run the following command.

```
Connect-Vmc -ApiToken "Your API token"
```

Connect to VMware Cloud on AWS by Using an OAuth Security Context

To connect to VMware Cloud on AWS through an OAuth 2.0 authentication mechanism, you need an OAuth security context for the VMware Cloud services platform.

Use the VMware Cloud service web portal to generate an API token for authentication with VCF PowerCLI.

You can create an OAuth security context by using the `New-VcsOAuthSecurityContext` cmdlet.

1. Create an OAuth security context for the VMware Cloud services platform by using an API token.

```
$oauthSecContext = New-VcsOAuthSecurityContext -ApiToken "Your API token"
```

2. Connect to VMware Cloud on AWS by using an OAuth security context.

```
Connect-Vmc -OAuthSecurityContext $oauthSecContext
```

View the Available Software-Defined Data Centers

With VCF PowerCLI, you can view the available Software-Defined Data Centers (SDDCs) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

View the available SDDCs in your VMware Cloud on AWS organization.

```
Get-VmcSddc
```

Connect to a vCenter on VMware Cloud on AWS

You can use PowerCLI to connect to a vCenter system running on VMware Cloud on AWS.

- Verify that you are connected to VMware Cloud on AWS.
- Configure the VMware Cloud on AWS networking to provide access to the vCenter system running on the cloud.

1. Create a variable with the SDDC you want to connect to.

```
$SDDC = Get-VmcSddc 'Your SDDC'
```

2. To connect to the vCenter system, provide the vCenter host name and credentials.

```
Connect-VIServer $sddc.VCenterHostName -Credential $sddc.VCenterCredentials
```

Connect to a vCenter on VMware Cloud on AWS by Using an OAuth 2.0 Authentication

If the vCenter system has an OpenID authentication enabled, you can authenticate with the vCenter system by using an OAuth security context for the VMware Cloud services platform.

- Verify that you are connected to VMware Cloud on AWS.
- Configure the VMware Cloud on AWS networking to provide access to the vCenter system running on the cloud.

1. Create an OAuth security context for the VMware Cloud service by using an API token.

```
$oauthSecContext = New-VcsOAuthSecurityContext -ApiToken "Your API token"
```

2. Create a variable with the SDDC you want to connect to.

```
$SDDC = Get-VmcSddc 'Your SDDC'
```

3. Exchange the OAuth security context for an SAML security context.

```
$samlSecContext = New-VISamlSecurityContext -VCenterServer $sddc.VCenterHostName -OAuthSecurityContext $oauthSecContext
```

4. Connect to a vCenter system by using an SAML security context.

```
Connect-VIServer -Server $sddc.VCenterHostName -SamlSecurityContext $samlSecContext
```

Create a Software-Defined Data Center

With VCF PowerCLI, you can create a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

1. Get a VMware Cloud on AWS account.

```
$awsAccount = Get-AwsAccount
```

2. Get the VMware Cloud on AWS Virtual Private Cloud (VPC) subnets.

```
$awsVpcSubnets = Get-AwsVpcSubnet -AwsAccount $awsAccount -Region "US_EAST_1"
```

3. Create an SDDC.

```
New-VmcSddc -Name "MySDDC" -HostCount 3 -Region "US_EAST_1" -AwsAccount $awsAccount  
-AwsVpcSubnet $awsVpcSubnets[0]
```

Create a Cluster in a Software-Defined Data Center

With VCF PowerCLI, you can create a cluster in a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

1. Retrieve the SDDC in which you want to create a cluster.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Create the cluster.

```
New-VmcSddcCluster -Sddc$sddc -HostCount3
```

Set the Elastic Distributed Resource Scheduler (EDRS) Policy of a Cluster

With VCF PowerCLI, you can set the Elastic Distributed Resource Scheduler (EDRS) policy of clusters.

Verify that you are connected to VMware Cloud on AWS.

1. Create a variable with the SDDCs of the clusters you want to manage.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Create a variable with the clusters you want to manage.

```
$cluster = Get-VmcSddcCluster -Name 'Your Cluster' -Sddc$sddc
```

3. Set the EDRS policy.

```
Set-VmcClusterEdrsPolicy -Cluster$cluster -PolicyTypecost -MinHostCount6 -MaxHostCount15
```

Remove a Cluster from a Software-Defined Data Center

With VCF PowerCLI, you can remove clusters from Software-Defined Data Centers (SDDCs) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

1. Create a variable with the SDDCs from which you want to remove clusters.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Create a variable with the clusters you want to remove.

```
$cluster = Get-VmcSddcCluster -Name 'Your Cluster' -Sddc$sddc
```

3. Remove the clusters.

```
Remove-VmcSddcCluster -VmcSddcCluster$cluster
```

Add Hosts to a Software-Defined Data Center

With VCF PowerCLI, you can add hosts to a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

Add hosts to an SDDC with VCF PowerCLI. You can use the `Cluster` parameter to indicate in which cluster you want to add the hosts. If you do not specify a cluster, the hosts are added to the default cluster of the SDDC.

1. Create a variable with the SDDCs in which you want to add hosts.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Add the hosts to the default cluster of the specified SDDCs.

```
Add-VmcSddcHost -Sddc$sddc -HostCount2
```

Remove Hosts from a Software-Defined Data Center

With VCF PowerCLI, you can remove hosts from a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

Remove hosts from an SDDC with VCF PowerCLI. You can use the `Cluster` parameter to indicate from which cluster you want to remove hosts. If you do not specify a cluster, the hosts are removed from the default cluster of the SDDC.

1. Create a variable with the SDDCs from which you want to remove hosts.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Remove hosts from the default cluster of the specified SDDCs.

```
Remove-VmcSddcHost -Sddc$sddc -HostCount2
```

Generate a VCF PowerCLI Support Bundle

Generate a VCF PowerCLI support bundle and send it to Broadcom support to help them resolve your problem.

If you encounter a problem in VCF PowerCLI that you cannot resolve, you can open a support request for Broadcom.

Generate and download a VCF PowerCLI support bundle in the form of a ZIP file and attach it to your Broadcom support request.

1. Create a variable with the script that produced the error.

```
$errorcode = {
    $vm = Get-VM vCLS*
    Start-VM $vm
}
```

2. Generate and download the support bundle by running the `Get-ErrorReport` cmdlet.

```
Get-ErrorReport -ProblemScript $errorcode -Destination .\Downloads -ProblemDescription "VM would not power on"
```

Optional: To limit the size of your support bundle, include the `MaxDataDepth` parameter which specifies the object data depth for the report. For example, to limit object data depth to one level, add `-MaxDataDepth1` to the preceding example.

The ZIP file that contains environmental information about the error is saved at the specified destination.

Attach or upload the ZIP file and submit your Broadcom support request.

Understanding the VMware Cloud Foundation Operations API

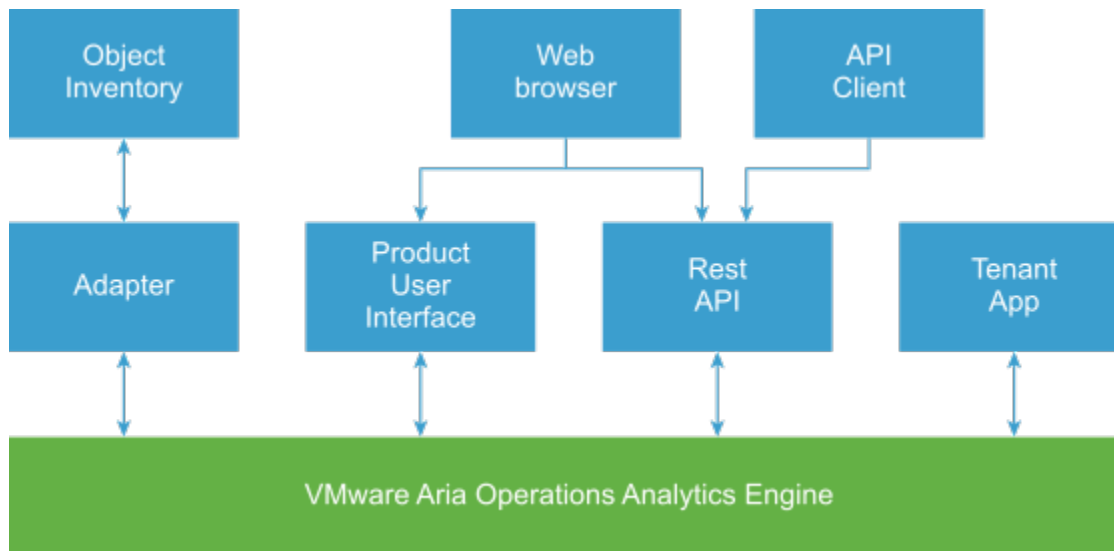
Developers can use the API to build interactive clients of VMware Cloud Foundation Operations. The API follows the REST style and is available to all licensed users.

VCF Operations clients communicate with the server over HTTP, exchanging representations of VCF Operations objects. These representations take the form of JSON or XML elements. You use HTTP GET requests to retrieve the current representation of an object, HTTP POST and PUT requests to create or modify an object, and HTTP DELETE requests to delete an object.

How the VMware Cloud Foundation Operations API Works

You use a Web browser to communicate with the VCF Operations analytics engine, either through the product user interface or through API calls.

Figure 660: VCF Operations Simplified Architecture



The adapter instance collects data from objects in your monitored environment. The VCF Operations analytics engine processes the data and displays the complete model in the graphical interface.

Why Use the VMware Cloud Foundation Operations API

The API is most useful when there is a need to automate a well-defined workflow, such as repeating the same tasks to configure access control for new users. The API is also useful when performing queries on the VCF Operations data repository, such as retrieving data for particular assets in your virtual environment. In addition, you can use the API to extract all data from the VCF Operations data repository and load it into a separate analytics system.

VMware Cloud Foundation Operations API Terminology

The XML syntax you use to describe the objects for an adapter corresponds to the API code syntax but differs from what you find in the user interface. The following terms appear in the user interface. Included with the description of each term is the corresponding XML syntax used in an API call.

Adapter types	Defines the adapter used to discover particular object types. For example, the vCenter adapter discovers objects connected to vSphere datacenters. The AWS adapter discovers AWS services and objects. XML syntax: <code>adapterkinds</code> .
Object types	The class of entities that represent objects or information sources. Objects report data to the VCF Operations analytics engine. Virtual machines, datastores, and host systems are examples of object types defined in a vCenter adapter model.

XML syntax: resourcekinds.

Client Workflow Overview

VCF Operations API clients implement a REST workflow, making HTTP requests to the server and retrieving the information they need from the server's responses.

About REST

REST, an acronym for Representational State Transfer, describes an architectural style characteristic of programs that use the Hypertext Transfer Protocol (HTTP) to exchange serialized representations of objects between a client and a server. In the VCF Operations API, these representations are JSON or XML documents.

In a REST workflow, representations of objects are passed back and forth between a client and a server with the explicit assumption that neither party need know anything about an object other than what is presented in a single request or response. The URLs at which these documents are available often persist beyond the lifetime of the request or response that includes them.

REST API Workflows

Application programs written to use a REST API use HTTP requests that are often executed by a script or other higher-level language to make remote procedure calls that create, retrieve, update, or delete objects that the API defines. In the VCF Operations REST API, these objects are defined by a collection of XML schemas. The operations themselves are HTTP requests, and so are generic to all HTTP clients.

To write a REST API client application, you must understand only the HTTP protocol, and the semantics of JSON or XML, the transfer format that the VCF Operations API uses. To use the API effectively in such a client, you must become familiar with the following concepts.

- The set of objects that the API supports, and what they represent.
- How the API represents these objects.
- How a client refers to an object on which it wants to operate.

The API reference includes a complete list of API requests. See [Accessing Swagger Documentation for Schema Reference](#) and [About the VCF Operations API Examples](#).

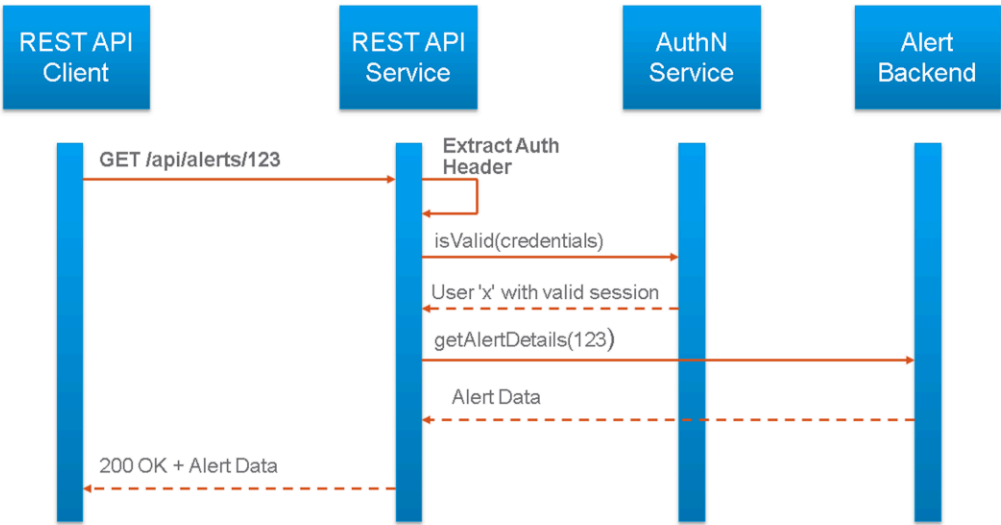
VMware Cloud Foundation Operations API REST Requests

To retrieve object representations, clients make HTTP requests to object references.

Security

The HTTP link between an API client and server is established using SSL. API clients configure token-based authentication to communicate with the server.

Figure 661: Scenario: Provide user credentials to obtain details about alert with ID 123



With token-based authentication, you POST a login request to the VCF Operations API server, supplying valid user credentials to obtain an authentication token. The following example presents a token-based authentication scenario.

- 1. You obtain valid user credentials for your VCF Operations instance. This is applicable for non-SSO authentication.
- 2. POST a request to the REST endpoint for authentication.
`https://RESTendpoint.example.com/suite-api/api/auth/token/acquire`
The request body includes the user name, password, and authentication source.
- 3. In the response body, the endpoint returns the token, expiry date, and time.
- 4. For further communication, you include the token object in the Authorization header with the format :
`Authorization: OpsToken <vROps_token>`

Note: The old format,
`Authorization: vRealizeOpsToken <vROps_token>`
continues to be supported in VCF Operations.

If you acquired the token externally from an SSO source (without using `/suite-api/api/auth/token/acquire` API), the Authorization header is of the format:
`Authorization: SSO2Token <SSO_SAML_TOKEN>`

- 5. You can invalidate the token before the expiration date and time by sending a POST request to the logout endpoint.
`POST https://RESTendpoint.example.com/suite-api/api/auth/token/release`

Request Headers

The following HTTP headers are typically included in API requests:

Accept-Language	To specify the language desired in responses, use the <code>Accept-Language</code> request header. Message strings in <code>ErrorType</code> responses are localized. To request a response with message strings localized to French, use the following header: <code>Accept-Language: fr-FR</code>
-----------------	--

Authorization	All requests to create an API session must include an <code>Authorization</code> header of the form prescribed by the identity provider that your organization uses
Content-Type	Requests that include a body must include an appropriate <code>HTTP Content-Type</code> header. - For a request body in XML, the header must include <code>Content-Type: application/xml</code> - For a request body in JSON, the header must include <code>Content-Type: application/json</code>
Accept	To specify the desired response format, include the <code>Accept</code> request header. - For a response in XML, the header must include <code>Accept: application/xml</code> - For a response in JSON, the header must include <code>Accept: application/json</code>

Request Bodies in XML

For a request body written in XML, VCF Operations uses a validating XML parser that requires elements in a request body to agree with the schema in order and number. Request bodies are rejected as invalid unless they meet the following criteria:

- XML namespace attributes must be supplied for all namespaces represented by elements in the request.
- If multiple namespaces are represented in the request, XML namespace attributes must include an identifying prefix, and that prefix must be used with all elements from that namespace.
- All required elements must appear in request bodies. All elements that appear in request bodies must appear in the order that the schema establishes, and with content that conforms to the type constraint that the schema specifies.

VCF Operations API REST Responses

All responses include an HTTP status code and, unless the status code is 204 (No Content), an `Accept` header. Response content depends on the request. Some responses include a document body, some include only a URL, and some are empty.

HTTP Response Codes

An API client can expect a subset of HTTP status codes in a response.

Table 1897: HTTP Status Codes that the API Returns

Status Code	Status Description
200 OK	The request is valid and was completed. The response includes a document body.
201 Created	The request is valid. The requested object was created and can be found at the URL specified in the <code>Location</code> header.
202 Accepted	The request is valid and a task was created to handle it. This response is usually accompanied by a <code>TaskStatus</code> element.
204 No Content	The request is valid and was completed. The response does not include a body.

Status Code	Status Description
400 Bad Request	The request body is malformed, incomplete, or otherwise invalid.
401 Unauthorized	Login failed or authentication token has expired.
403 Forbidden	The user is not authenticated or does not have adequate privileges to access one or more objects specified in the request.
404 Not Found	The object specified in the request could not be found.
405 Method Not Allowed	The HTTP method specified in the request is not supported for this object.
406 Not Acceptable	The resource identified by the request is not capable of generating a response of the type specified in the request's <code>Accept</code> header.
415 Unsupported Media Type	The resource identified by the request does not support a request of the specified <code>Content-Type</code> and HTTP method.
422 Not Found	Usually indicates a malformed request URL or request body.
429 Too Many Requests	A client has sent too many requests or multiple clients are sending too many simultaneous requests and the server is unable to process them due to rate limits. To work around the problem, try sending the request again later.
500 Internal Server Error	The request was received but could not be completed because of an internal error on the server.
503 Service Unavailable	The server is currently unable to handle the request due to a temporary condition such as resource exhaustion or server maintenance.
504 Gateway Timeout	The server, while acting as a gateway or proxy, did not receive a timely response from the upstream server specified by the request URL.

Using the API with VCF Operations

You can use a browser or an HTTP client program to send requests and receive responses.

REST Client Programs

Any client application that can send HTTPS requests is an appropriate tool for developing REST applications with the VCF Operations API. REST client plug-ins are available for most browsers and many IDEs. The following open-source programs are commonly used:

- cURL. <http://curl.haxx.se>
- Postman application. <http://www.getpostman.com>

In addition, VMware provides language-specific client bindings for the VCF Operations API. See [Accessing Swagger Documentation for Schema Reference](#) and [About the VCF Operations API Examples](#).

Accessing Swagger Documentation for Schema Reference

The VCF Operations REST API documentation includes reference material for all elements, types, queries, and operations in the VCF Operations API. It also includes the schema definition files.

Swagger based API documentation is available with the product, with the capability of making REST API calls right from the landing page.

To access the API documentation, you must first log into VCF Operations at the URL of your VCF Operations instance.

For example, if the URL of your instance is `https://operations.example.com`, the API reference is available from: `https://operations.example.com/suite-api/doc/swagger-ui.html`

Language-specific client bindings are available from: `https://operations.example.com/suite-api/`

About the VCF Operations API Examples

All examples include HTTP requests and responses. These examples show the workflow and content associated with operations such as creating and querying for information about objects in your monitored environment.

Example request bodies are in JSON. Request headers required by the VCF Operations API are included in example requests that are not fragments of a larger example.

Most example responses show only those elements and attributes that are relevant to the operation being discussed. Ellipses (...) indicate omitted content within response bodies.

Getting Started with the API

API clients and VCF Operations servers communicate over HTTPS, exchanging XML representations of API objects.

This simple example of a REST API workflow shows how to obtain a list of all metrics for a virtual machine object type that is included in the model definition of the vCenter adapter. Using the API, you can obtain the complete list of available metrics for any object type.

Acquire an Authentication Token

VCf Operations requires API requests to be authenticated. The first step in this workflow is to obtain an authentication token.

- Secure a channel between the web browser and the VCF Operations server. Open a browser and enter the URL of a VCF Operations instance such as:

```
https://vrealize.example.com
```

The system warns that your connection is not private. Click through to confirm the security exception and establish an SSL handshake.

- Verify that you can access the APIs. Enter the URL of your VCF Operations instance with `suite-api/docs/rest/index.html` added to the end, such as:

```
https://vrealize.example.com/suite-api/docs/rest/index.html
```

- Verify that you have the login credentials for a user of your VCF Operations instance.

To obtain an authentication token, the login request supplies the user credentials in a form that Basic HTTP authentication requires. In this example, the user is logging in to a VCF Operations instance with URL `https://vrealize.example.com/`.

Note: This example uses token-based authentication. For more information regarding authentication mechanisms, see [Security](#).

Using `authSource`, you can import and authenticate users and user group information that reside on another machine. For example, you can authenticate users from LDAP, Active Directory, VMware Identity Manager, Single Sign-On and so on. When you import user account information that resides on another machine, you must define the criteria used to import the user accounts from the source machine.

After creating an auth source you can use it for acquiring a token by specifying the name. The default auth source type is `LOCAL`.

1. POST a request to the login URL to acquire a token.

See [Login Request and Response](#).

1. You obtain valid user credentials for your VCF Operations instance.
2. POST a request to the REST endpoint for authentication (non-SSO).

```
https://RESTendpoint.example.com/suite-api/api/auth/token/acquire
```

The request body includes the user name, password, and authentication source.

3. In the response body, the endpoint returns the token, expiry date, and time.
4. For further communication, you include the token object in the Authorization header with the format :

```
Authorization: OpsToken <vROps_token>
```

Note: The old format,

```
Authorization: vRealizeOpsToken <vROps_token>
```

continues to be supported in VCF Operations.

If you acquired the token externally from an SSO source (without using /suite-api/api/auth/token/acquire API), the Authorization header is of the format:

```
Authorization: SSO2Token <SSO_SAML_TOKEN>
```

2. Examine the response.

A successful request returns an `ops` authorization token, which you must include in subsequent API requests.

Login Request and Response

This example shows a request and response for a user with the login username: `vRealize-user` and password: `vRealize-dummy-password`.

Request header:

```
POST https://vrealize.example.com/suite-api/api/auth/token/acquire
Content-Type: application/json
Accept: application/json
```

Request body in JSON format:

```
{
  "username" : "vRealize-user",
  "password" : "vRealize-dummy-password"
}
```

Response in JSON:

```
200 OK
{
  "token": "8f868cca-27cc-43d6-a838-c5467e73ec45::77cea9b2-1e87-490e-b626-e878beaaa23b",
  "validity": 1470421325035,
  "expiresAt": "Friday, August 5, 2016 6:22:05 PM UTC",
  "roles": []
}
```

The response code indicates whether the request succeeded, or how it failed.

- If the request is successful, the server returns HTTP response code 200 (OK) and re-usable `ops` authorization token that expires after six hours. This token must be included in each subsequent API request.
- If the authorization header is missing for the request, the server returns HTTP response code 403.
- If the credentials supplied in the Authorization header are invalid, the server returns HTTP response code 401.

Find the Adapter Type and Object Type

Your VCF Operations instance includes multiple adapter types. To find the adapter type for the vCenter adapter, you make a GET request to retrieve a list of all adapter types. The API response includes all the object types that the adapter monitors.

Verify that you are logged in to the VCF Operations instance.

1. Make a GET request for all adapter types.

```
GET https://operations.example.com/suite-api/api/adapterkinds
```

2. Examine the response to find the vCenter adapter and list of monitored object types.

See the response portion of [Determine the Adapter Type and Object Types for the vCenter Adapter](#).

Determine the Adapter Type and Object Types for the vCenter Adapter

This example finds the adapter type for the vCenter adapter and all the object types included in the adapter model definition.

Request header:

```
GET https://operations.example.com/suite-api/api/adapterkinds
Content-Type: application/json
Authorization: OpsToken <vROps_token>
Accept: application/json
```

Note: The old format,

```
Authorization: vRealizeOpsToken <vROps_token>
```

continues to be supported in VCF Operations.

Where vROps_token is the token that you obtained from the response in [Acquire an Authentication Token](#).

Snippet of the response in JSON for the vCenter Adapter:

```
200 OK
{
  "key": "VMWARE",
  "name": "vCenter Adapter",
  "description": "Provides the connection information and credentials required...",
  "adapterKindType": "GENERAL",
  "describeVersion": 573,
  "identifiers": [],
  "resourceKinds": [
    "ClusterComputeResource",
    "ComputeResource",
    "CustomDatacenter",
    "Datacenter",
    "Datastore",
    "StoragePod",
    "DatastoreFolder",
    "VM Entity Status",
    "Folder",
    "HostFolder",
    "HostSystem",
    "NetworkFolder",
    "ResourcePool",
    "VMwareAdapter Instance",
    "VirtualMachine",
```

```

    "VMFolder",
    "DistributedVirtualPortgroup",
    "VmwareDistributedVirtualSwitch",
    "vSphere World"
  ],
  ...
}

```

For the vCenter adapter, the `adapter-kind` key is `VMWARE`. The `resourceKinds` are the object types that the vCenter adapter monitors. For virtual machine object type, the `resourceKinds` is `VirtualMachine`.

Generate a List of All Metrics for the Object

To generate a complete list of metrics for any virtual machine defined in the vCenter adapter model, you make a GET request to the URL with the adapter type and the object type.

Verify that the following requirements are met:

- You are logged in to the VCF Operations instance.
- You know the `adapterKind` value for the vCenter adapter and the `resourceKinds` value for the virtual machine. See [Determine the Adapter Type and Object Types for the vCenter Adapter](#).

1. Make a GET request to obtain the metadata for metrics.

```
GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds/VirtualMachine/statkeys
```

2. Compare the metrics listed in the response to metrics displayed in the user interface. See [Virtual Machine Metrics from the API and in the User Interface](#).

Virtual Machine Metrics from the API and in the User Interface

This example shows how the virtual machine metrics listed in the XML response compare to the metrics displayed in the VCF Operations user interface.

Request:

```

GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds/VirtualMachine/statkeys
Content-Type: application/json
Authorization: OpsToken <vROps_token>
Accept: application/json

```

Note: The old format,

```
Authorization: vRealizeOpsToken <vROps_token>
```

continues to be supported in VCF Operations.

Where:

- `VMWARE` is the `adapterKindKey`.
- `VirtualMachine` is the `resourceKindKey`.
- `vROps_token` is the token that you obtained from the response in [Acquire an Authentication Token](#).

Snippet of the response in JSON:

```

200 OK

{
  "resourceTypeAttributes": [
    ...

```

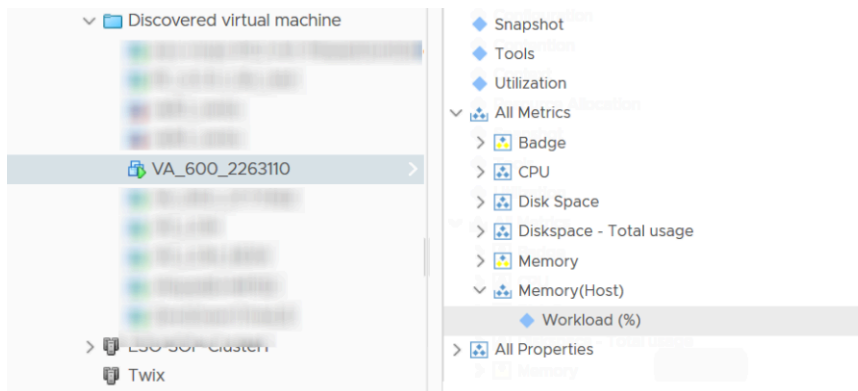
```

{
  "key": "mem|host_workload",
  "name": "Memory|Host Workload",
  "description": "Host Workload (%)",
  "defaultMonitored": false,
  "rollupType": "AVG",
  "instanceType": "INSTANCED",
  "unit": "%",
  "dataType2": "FLOAT",
  "monitoring": false,
  "property": false
},
...
]
}

```

Every `resourceTypeAttribute` in the response is a metric with metadata for a virtual machine object. The `name` corresponds to text displayed in the VCF Operations user interface. In this example, the snippet lists metrics for Memory and Host Workload.

To compare metrics in the response with metrics in the user interface, log in to the VCF Operations interface and navigate to the metrics for a virtual machine. The following example shows where you find metrics for Memory(Host) and Workload.



The example shows how to retrieve metrics for the virtual machine object type. To retrieve metrics for other object types, replace `VirtualMachine` in the GET request with other `resourceKinds`.

Configuring an Adapter Instance

After installing a solution that includes a management pack with an adapter, you must configure an adapter instance to collect data from the objects in the adapter model definition. You can use the VCF Operations API to configure an adapter instance.

This use case example shows how to configure an adapter instance for a vSphere Solution and includes:

- summary of operations with request, request body, and response for each
- specific procedure for each operation

Summary of Configuring an Adapter Instance Requests

You make sequential API requests to configure an adapter instance. Responses from earlier requests yield information required for a subsequent requests.

Table 1898: Summary of Requests

Operation	Request	Request Body	Response
Get all solutions registered with the product and identify the adapter types	GET <API-URL>/suite-api/api/solutions	None	adapterkindkeys
Get all the object types for a particular adapter type.	GET <API-URL>/suite-api/api/adapterkinds/{key}/resourcekinds	None	resourceIdentifierTypes
Create an adapter instance object	POST <API-URL>/suite-api/api/adapters	Values for resourceIdentifiers and credential	adapterid
Patch an adapter instance to acknowledge the presented certificate	PATCH <API-URL>/suite-api/api/adapters	API response of POST <API-URL>/suite-api/api/adapters	API response of POST <API-URL>/suite-api/api/adapters without adapter-cerfiticates
Start adapter monitoring	PUT <API-URL>/suite-api/api/adapters/{adapterid}/monitoringstate/start	None	200 OK

Identify the Solution and Its Adapters

Your VCF Operations instance may have several solutions installed. To find the vSphere solution and its adapter types, you make a GET request to retrieve a list of all solutions. The response includes all the adapters included with the solution.

Verify that you can log in to the API URL for a VCF Operations instance. See [Acquire an Authentication Token](#).

For this example use case, the *API-URL* for the VCF Operations instance is operations.example.com.

1. Make a GET request to list all the solutions.

```
GET https://operations.example.com/suite-api/api/solutions
```

2. Examine the response to find the vSphere solution and its adapter types.

See the response portion of [Adapter Types for the vSphere Solution](#).

Adapter Types for the vSphere Solution

This example lists all the installed solutions and the adapter types for each.

Request header:

```
GET https://operations.example.com/suite-api/api/solutions
```

The response in JSON:

```
200 OK
{
  "solution":
  {
    "id": "MPforLogInsight",
    "name": "VMware vRealize Operations Management Pack for Log Insight",
    "version": "6.0.3171089",
    "description": "VMware vRealize Operations Management Pack for Log Insight... ",
    "vendor": "VMware Inc.",
    "adapterKindKeys": [
      "LogInsightAdapter"
    ]
  }
}
```

```

    ]
  },
  {
    "id": "ep-ops-os-and-availability",
    "name": "Operating Systems / Remote Service Monitoring",
    "version": "1.0.4071095",
    "description": "The End Point Operations Management Solution for Operating... ",
    "vendor": "VMware Inc.",
    "adapterKindKeys": [
      "ep-ops-os-and-availability-kind"
    ]
  },
  {
    "id": "VMware vSphere",
    "name": "VMware vSphere",
    "version": "6.0.7496664",
    "description": "Manages vSphere objects such as Clusters, Hosts...",
    "vendor": "VMware Inc.",
    "adapterKindKeys": [
      "VMWARE",
      "PythonRemediationVcenterAdapter"
    ]
  }
]
}

```

The response shows three solutions installed:

- Management Pack for Log Insight solution
- End Point Operations solution
- vSphere solution

The vSphere solution has two adapter types:

- VMWARE
- PythonRemediationVcenterAdapter

For the vCenter adapter, the adapter type is VMWARE.

Identify the Object Types Required for the Adapter

After you determine that you want to create an instance of the vCenter adapter, you must identify the required object types for that adapter. You make a GET request to retrieve a list of all object types for the vCenter adapter.

Verify that you know the adapter type for the vCenter adapter.

1. Make a GET request to list all the object types for the vCenter adapter.

```
GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds
```

2. Examine the response to identify the required object types..

See the response portion of [Object Types Required for the vCenter Adapter](#).

Object Types Required for the vCenter Adapter

This example finds all the object types for the vCenter adapter.

Request header:

```
GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds
```

Snippet of the response in JSON:

```

200 OK
{
  "key": "VMwareAdapter Instance",
  "name": "vCenter",
  "adapterKind": "VMWARE",
  "resourceKindType": "ADAPTER_INSTANCE",
  "resourceKindSubType": "NONE",
  "resourceIdentifierTypes": [
    {
      "name": "AUTODISCOVERY",
      "dataType": "STRING",
      "isPartOfUniqueness": false
    },
    {
      "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
      "dataType": "STRING",
      "isPartOfUniqueness": false
    },
    {
      "name": "DV_PORT_GROUP_DISABLED",
      "dataType": "STRING",
      "isPartOfUniqueness": false
    },
    {
      "name": "DVS_DISABLED",
      "dataType": "STRING",
      "isPartOfUniqueness": false
    },
    {
      "name": "PROCESSCHANGEEVENTS",
      "dataType": "STRING",
      "isPartOfUniqueness": false
    },
    {
      "name": "VCURL",
      "dataType": "STRING",
      "isPartOfUniqueness": true
    },
    ...
    {
      "name": "VM_LIMIT",
      "dataType": "INTEGER",
      "isPartOfUniqueness": false
    }
  ],
  ...
}

```

This snippet shows the Resource Kind with the attribute `"resourceKindType": "ADAPTER_INSTANCE"`. Any object type that has the resource identifier `"isPartOfUniqueness": true` requires a value for that object type with the API request to create the adapter instance.

An adapter instance of the vCenter adapter requires a value for `VCURL` or the URL of the vCenter.

Create the Adapter Instance

After you identify the object types required for the adapter, you provide parameter values for the object types to create an adapter instance. Your POST request includes a request body with the required parameters.

Verify that you have an IP address and credentials for a vCenter.

To create an adapter instance, the `VCURL` setting is mandatory.

1. Make a POST request to create the adapter instance.

```
POST https://operations.example.com/suite-api/api/adapters
```

2. Examine the response to find the name for the vSphere Solution and its adapter types.

See the response portion of [Adapter Instance](#).

Adapter Instance

This example creates the adapter instance for a vCenter with the following parameters:

- Display Name: VC Adapter Instance
- Description: A vCenter Adapter Instance for VC 12.345.678.9
- vCenter IP address: https://12.345.678.9
- Credential name: VC-Credential-1
- User Name: administrator@vsphere.local
- Password: VC-dummy-passwd

`AUTODISCOVERY` and `PROCESSCHANGEEVENTS` are optional, but are included to show additional examples of resource identifiers in the request body and in the response.

Request header:

```
POST https://operations.example.com/suite-api/api/adapters
```

Request body in JSON format:

```
{
  "name" : "VC Adapter Instance",
  "description" : "A vCenter Adapter Instance for VC 12.345.678.9",
  "collectorId" : "1",
  "adapterKindKey" : "VMWARE",
  "resourceIdentifiers" : [
    {
      "name" : "AUTODISCOVERY",
      "value" : "true"
    },
    {
      "name" : "PROCESSCHANGEEVENTS",
      "value" : "true"
    },
    {
      "name" : "VCURL",
      "value" : "https://12.345.678.9"
    }
  ],
  "credential" : {
    "id" : null,
    "name" : "VC-Credential-1",
```

```

    "adapterKindKey" : "VMWARE",
    "credentialKindKey" : "PRINCIPALCREDENTIAL",
    "fields" : [
      {
        "name" : "USER",
        "value" : "administrator@vsphere.local"
      },
      {
        "name" : "PASSWORD",
        "value" : "VC-dummy-passwd"
      }
    ],
  },
}

```

Snippet of the response in JSON:

201 Created

```

{
  "resourceKey": {
    "name": "VC Adapter Instance",
    "adapterKindKey": "VMWARE",
    "resourceKindKey": "VMwareAdapter Instance",
    "resourceIdentifiers": [
      {
        "identifierType": {
          "name": "AUTODISCOVERY",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
      },
      {
        "identifierType": {
          "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "DV_PORT_GROUP_DISABLED",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "DVS_DISABLED",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },

```



```

        "value": ""
    },
    {
        "identifierType": {
            "name": "PROCESSCHANGEEVENTS",
            "dataType": "STRING",
            "isPartOfUniqueness": false
        },
        "value": "true"
    },
    {
        "identifierType": {
            "name": "VCURL",
            "dataType": "STRING",
            "isPartOfUniqueness": true
        },
        "value": "https://12.345.678.9"
    },
    {
        "identifierType": {
            "name": "VM_FOLDER_DISABLED",
            "dataType": "STRING",
            "isPartOfUniqueness": false
        },
        "value": ""
    },
    {
        "identifierType": {
            "name": "VM_LIMIT",
            "dataType": "STRING",
            "isPartOfUniqueness": false
        },
        "value": ""
    }
]
},
"description": "A vCenter Adapter Instance for VC 12.345.678.9",
"collectorId": 1,
"collectorGroupId": "909c2fbf-2c2c-4957-9a75-21bf2a887d31",
"credentialInstanceId": "65081a8d-d462-43b2-b4e0-596eaf3d497e",
"monitoringInterval": 5,
"adapter-certificates": [
    {
        "thumbprint": "2520fb4351bc91ee7b82ef7cc54a8d88fa893da9",
        "certificateDetails": "[
            Version: V3 Subject: C=US, CN=12.345.678.9
            Signature Algorithm: SHA256withRSA, OID = 1.2.840.113549.1.1.11
            Key: Sun RSA public key, 2048 bits modulus: ...
            Validity: [From: Wed Jul 15 19:26:51 UTC 2015, To: Tue Jul 08 11:26:30 UTC 2025]
            Issuer: O=W12R2UINanduVC, C=US, DC=local, DC=vsphere, CN=CA ...
            ...
        ]"
    }
]
}

```

```

],
...
"id": "a97bd204-e3e5-404b-a219-e2b20cf158d2"
}

```

The API creates a new adapter with an internally generated UUID that uniquely identifies the object. The API response includes the certificates that vCenter 12.345.678.9 presents. The value of the adapter instance ID is used to start monitoring and collecting data.

Provide Proof of Certificate Validity

Before VCF Operations can connect to the vCenter and start collecting data, it needs to verify that data sources discovered by the adapter instance are presenting valid certificates. Your PATCH request provides the proof that the certificates are valid by including a request body that is the response from the POST request used to create the adapter.

Verify that you have the response from the POST request used to create the adapter. See the response in [Adapter Instance](#).

Make a PATCH request to notify the system that the user has accepted the certificate presented by the vCenter.

```
PATCH https://operations.example.com/suite-api/api/adapters
```

Certificate Validation

In this example, the request body for the PATCH request is the same as the response from the POST request used to create the adapter instance.

Request header:

```
PATCH https://operations.example.com/suite-api/api/adapters
```

Request body in JSON format:

```

{
  "resourceKey": {
    "name": "VC Adapter Instance",
    "adapterKindKey": "VMWARE",
    "resourceKindKey": "VMwareAdapter Instance",
    "resourceIdentifiers": [
      {
        "identifierType": {
          "name": "AUTODISCOVERY",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
      },
      {
        "identifierType": {
          "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "DV_PORT_GROUP_DISABLED",
          "dataType": "STRING",

```

```

        "isPartOfUniqueness": false
    },
    "value": ""
},
{
    "identifierType": {
        "name": "DVS_DISABLED",
        "dataType": "STRING",
        "isPartOfUniqueness": false
    },
    "value": ""
},
{
    "identifierType": {
        "name": "PROCESSCHANGEEVENTS",
        "dataType": "STRING",
        "isPartOfUniqueness": false
    },
    "value": "true"
},
{
    "identifierType": {
        "name": "VCURL",
        "dataType": "STRING",
        "isPartOfUniqueness": true
    },
    "value": "https://12.345.678.9"
},
{
    "identifierType": {
        "name": "VM_FOLDER_DISABLED",
        "dataType": "STRING",
        "isPartOfUniqueness": false
    },
    "value": ""
},
{
    "identifierType": {
        "name": "VM_LIMIT",
        "dataType": "STRING",
        "isPartOfUniqueness": false
    },
    "value": ""
}
]
},
"description": "A vCenter Adapter Instance for VC 12.345.678.9",
"collectorId": 1,
"collectorGroupId": "909c2fbf-2c2c-4957-9a75-21bf2a887d31",
"credentialInstanceId": "65081a8d-d462-43b2-b4e0-596eaf3d497e",
"monitoringInterval": 5,
"adapter-certificates": [
    {

```

```

    "thumbprint": "2520fb4351bc91ee7b82ef7cc54a8d88fa893da9",
    "certificateDetails": "[
      Version: V3 Subject: C=US, CN=12.345.678.9
      Signature Algorithm: SHA256withRSA, OID = 1.2.840.113549.1.1.11
      Key: Sun RSA public key, 2048 bits modulus: ...
      Validity: [From: Wed Jul 15 19:26:51 UTC 2015, To: Tue Jul 08 11:26:30 UTC 2025]
      Issuer: O=W12R2UINanduVC, C=US, DC=local, DC=vsphere, CN=CA ...
    ]"
  }
],
...
"id": "a97bd204-e3e5-404b-a219-e2b20cf158d2"
}

```

Response in JSON:

```

{
  "resourceKey": {
    "name": "VC Adapter Instance",
    "adapterKindKey": "VMWARE",
    "resourceKindKey": "VMwareAdapter Instance",
    "resourceIdentifiers": [
      {
        "identifierType": {
          "name": "AUTODISCOVERY",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
      },
      {
        "identifierType": {
          "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "DV_PORT_GROUP_DISABLED",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "DVS_DISABLED",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      }
    ]
  }
}

```

```

    },
    {
      "identifierType": {
        "name": "PROCESSCHANGEEVENTS",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      "value": "true"
    },
    {
      "identifierType": {
        "name": "VCURL",
        "dataType": "STRING",
        "isPartOfUniqueness": true
      },
      "value": "https://12.345.678.9"
    },
    {
      "identifierType": {
        "name": "VM_FOLDER_DISABLED",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      "value": ""
    },
    {
      "identifierType": {
        "name": "VM_LIMIT",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      "value": ""
    }
  ]
},
"description": "A vCenter Adapter Instance for VC 12.345.678.9",
"collectorId": 1,
"collectorGroupId": "909c2fbf-2c2c-4957-9a75-21bf2a887d31",
"credentialInstanceId": "65081a8d-d462-43b2-b4e0-596eaf3d497e",
"monitoringInterval": 5,
...
"id": "a97bd204-e3e5-404b-a219-e2b20cf158d2"
}

```

The response is same as the request body without the `adapter-certificates` section.

Start Monitoring the New Adapter Instance

After the creating the adapter instance and configuring VCF Operations to recognize a valid certificate, start monitoring and collecting data. Your PUT request provides the UUID of the adapter instance used to discover new objects.

Verify that you have the UUID of the newly created adapter instance. See the response in [Adapter Instance](#).

Make a PUT request to start monitoring with the new adapter instance.

```
PUT https://operations.example.com/suite-api/api/adapters/<adapter_UUID>/monitoringstate/start
```

Discover Objects and Collect Data

This example starts the adapter monitoring process using the adapter instance ID from the PUT request that created the adapter instance.

Request header:

```
PUT https://operations.example.com/suite-api/api/adapters/a97bd204-e3e5-404b-a219-e2b20cf158d2/monitoringstate/start
```

VMware Cloud Foundation Operations for Networks API Guide

VMware Cloud Foundation Operations for Networks API Programming Guide

The VCF Operations for networks API Programming Guide provides information about the VCF Operations for networks REST APIs, including how to use the Representational State Transfer (REST) API resources, authenticate, and construct REST API calls.

Intended Audience

This information is intended for administrators and programmers who want to configure and manage VCF Operations for networks programmatically using the VCF Operations for networks REST API. The guide focuses on common use cases.

API Reference

For more in-depth information on the objects available in the VCF Operations for networks REST API, go to the [Developer Portal](#).

Understanding the REST APIs

You can use APIs to automate workflows in VCF Operations for networks. The APIs follow the REST style and are available to all licensed users.

Overview of Rest APIs

The clients of VCF Operations for networks API implement a REST workflow making HTTP requests to the server and retrieving the information they need from the server's responses.

About REST

REST, an acronym for Representational State Transfer, describes an architectural style characteristic of applications that use the Hypertext Transfer Protocol (HTTP) to exchange serialized representations of objects between a client and a server. In the VCF Operations for networks API, these representations are JSON documents. In a REST workflow, object representations are passed back and forth between a client and a server with the explicit assumption that neither party need know anything about an object other than what is presented in a single request or response. The URLs at which these documents are available often persist beyond the lifetime of the request or response that includes them.

VCF Operations for networks API Reference is available at:

<https://developer.broadcom.com/xapis/vcf-operations-for-networks-api/latest/>

The API Reference is also available in the product at:

https://operations-for-networks.your_domain.com/doc-api/index.html

REST API Workflows

Applications that use a REST API send HTTP requests. A script or other higher-level language runs the HTTP requests to make remote procedure calls. These procedure calls create, retrieve, update, or delete objects that the API defines. In the VCF Operations for networks REST API, these objects are defined by a collection of JSON schemas. The operations themselves are HTTP requests, and so are generic to all HTTP clients. To write a REST API client application, you must understand only the HTTP protocol, and the semantics of JSON, the transfer format that the VCF Operations for networks API uses. To use the API effectively in such a client, you must become familiar with the following concepts.

- The set of objects that the API supports, and what they represent.
- How the API represents these objects.
- How a client refers to an object on which it wants to operate. The API reference includes a complete list of API requests and model objects.

API Request

The following HTTP headers are typically included in API requests:

Request Header	Description
Authorization	All requests must include an Authorization header with the authentication token generated from VCF Operations for networks auth API.
Content-Type	Requests that include a body must include HTTP Content-Type: application/json header.

Request Body

Ensure that the content in the request body conforms to the type constraint in the model objects in API Reference.

API Response

VCf Operations for networks uses conventional HTTP response codes to indicate the success or failure of an API request.

- 2xx range indicates success.
- 4xx range indicates an error when the information provided is incorrect. For example, a required parameter was omitted.
- 5xx range indicates an error with VCF Operations for networks server. However, these errors are rare.

An API client can expect a subset of HTTP status codes in a response.

Table 1899: HTTP Response Codes

Status Code	Status Description
200 OK	The request is valid and was completed. The response includes a document body.
201 Created	The request is valid. The requested object was created.
204 No Content	The request is valid and was completed. The response does not include a body.
400 Bad Request	The request body is malformed, incomplete, or otherwise invalid.
401 Unauthorized	Login failed or authentication token has expired.

Status Code	Status Description
403 Forbidden	The user is not authenticated or does not have adequate privileges to access one or more objects specified in the request.
404 Not Found	The object specified in the request cannot be found.
405 Method Not Allowed	The HTTP method specified in the request is not supported for this object.
412 Precondition Failed	The maximum limit for unique active sessions is reached.
415 Unsupported Media Type	The resource identified by the request does not support a request of the specified Content-Type and HTTP method.
429 Too Many Requests	A client has sent too many requests or multiple clients are sending too many simultaneous requests and the server is unable to process them due to rate limits. To work around the problem, try sending the request again later.
500 Internal Server Error	The request was received but cannot be completed because of an internal error on the server.
503 Service Unavailable	The server is unable to handle the request due to a temporary condition such as resource exhaustion or server maintenance.

API Request Limits

- The maximum limit for unique active sessions is 100. When you reach the limit, you get a response code 412.
- The overall system limit for the maximum number of API requests per second is 100. When you reach the system limit, you get a response code 429.

In addition to above limits there are separate rate limits for different categories of API, which are as follows:

Category	Request per second limit
Applications APIs	20
Applications Flows APIs	1
Applications Flows APIs	20
Data Sources APIs	20
Entitites APIs	50
Search APIs	50
Microsegmentation APIs	20
Authentication APIs	20
Schema APIs	20
Metrics APIs	20
Default for all other APIs	20

These limits are configurable and can be changed by contacting Broadcom Technical Support.

REST API Services

This topic provides a summary of REST API services and their functions.

Table 1900: REST API Services

Service	Description
Authentication	Acquire or delete an authentication token based on the user credentials provided.
Data Source Management	Retrieve, update, add, and delete various data sources supported by VCF Operations for networks.
Entities	Retrieve entities from the VCF Operations for networks inventory.
Applications	Create Applications and Tiers based on virtual machine membership criteria and/or IP address/subnet criteria.
Micro Segmentation	Determine the firewall rules recommended by VCF Operations for networks for the different tiers, security groups, and applications.
Search	Search VCF Operations for networks entities.

Rest API Authentication and Authorization

VCF Operations for networks supports token based authentication. Tokens are non-modifiable identifiers returned by the system when the user has successfully authenticated using valid credentials.

VCF Operations for networks requires API requests to be authenticated. The first step in this workflow is to obtain an authentication token. To obtain an authentication token, the login request supplies the user credentials. In this example, the user is logging in to a VCF Operations for networks instance with URL <https://operations-for-networks.example.com/>.

Use an authentication token

Each API request must contain the authorization token in Authorization Header in the following format:

Authorization: NetworkInsight *auth-token*

Acquire an Authentication Token

The authorization token is valid for 5 hours after generation. If the token expires, you must regenerate it.

- Secure a channel between the web browser and the VCF Operations for networks server. Open a browser and enter the URL of a VCF Operations for networks instance such as <https://operations-for-networks.example.com>. The system warns that your connection is not private. Click through to confirm the security exception and establish an SSL handshake
- Verify that you have the login credentials for a user of your VCF Operations for networks instance.
 - POST a request to the authentication URL to acquire a token.
 - Examine the response. A successful request returns an authorization token that you must include in subsequent API requests.

```
Request header:
POST https://operations-for-networks.example.com/api/ni/auth/token
Content-Type: application/json
Accept: application/json
Request body in JSON format:
{
  "username": "testuser@local",
  "password": "testpassword",
  "domain": {
    "domain_type": "LOCAL"
```

```

    }
  }
  Response:
  {
    "token": "6QfAhSQ7/ivmvDkHP0EvTw==",
    "expiry": 1508767809240
  }

```

Delete an Authorization Token

You can also delete a token before it expires using the following http request.

Make a DELETE request.

```

Request Header
DELETE https://operations-for-networks.example.com/api/ni/auth/token
Authorization: NetworkInsight auth-token

```

Authentication using VMware Identity Management

Starting release 4.1.0 users can authenticate public APIs using VMware Identity Manager

Workflow for authentication using VMware Identity Manager.

Attention:

This feature is not available from 9.0 onward.

Request Client Id of Password grant OAuth client registered at VMware Identity Manager

Password grant OAuth client is automatically created by VCF Operations for networks after VMware Identity Manager is successfully configured. This client-id along with user credentials is required while making an access token request to VMware Identity Manager.

VMware Identity Manager must be configured & enabled in VCF Operations for networks.

To retrieve the client Id use the GET request.

```

Request:
GET https://operations-for-networks.example.com/api/ni/auth/vidm/client-id
Response:
{
  "vidm_appliance": "identity.domain.com",
  "client_id": "g9xUFRr84K_pwd_grant"
}

```

Get Access Token from VMware Identity Manager

Get the access token from VMware Identity Manager using "client_id" and "vidm_appliance". The response contains the access token which is further used to get the authentication token for accessing VCF Operations for networks public APIs.

1. "client_id" and "vidm_appliance". (Refer previous section)
2. User credentials.

To retrieve the access token use the POST request.

```

Request:
POST https://identity.domain.com/SAAS/auth/oauthtoken?grant_type=password&client_id=g9xUFRr84K_pwd_grant&username=<username>&password=<password>

Response:
{

```

```

    "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJqdGkiOiJkMDkxZTlmOS0xZDVlLTQ0ODctODk5YS03YTZhMGJjNjBkYmUiLCJwcm4iOiJlc2VybUBWUk5JLVZJRE0tVEVOQU5UMSIsImRvbWpbiI6InZpZG1sYWJzLmNvbSIsInVzZXJfaWQiOiIxMyIsImF1dGhfdGltZSI6MTU1MjU4Mjgxc0swiaXNzIjoiaHR0cHM6Ly92cm5pLXZpZG0tdGVuYy-W50MS5lbmcudm13YXJlLmNvbS9TQUFTL2F1dGgiLCJhdWQiOiJodHRwczovL3ZybmktZmlkbS10ZW5hbnQxLmVuZy52bXdhcmUuY29tL1NBQVMvYXV0aC9vYXV0aHRva2VuIiwiaW3R4Ijoiw3tIm10ZFwiOlwidXJuOm9hc2lzOm5hbWVzOnR-jOlNBtUw6Mi4wOmFjOmNsYXNzZXMM6UGFzc3dvcnRQcm90ZWN0ZWRUcmFuc3BvcnRcIixcImlhdFwiOjE1NTI1ODI4MTk-sXCJpZFwiOjE1fV0iLCJzY3AiOiJvcGVuaWQgcHJvZmlsZSB1c2VyaGVtYWlsIiwiaWRwIjoimcIsImVtbCI6InVz-ZXIXQHhtd2FyZS5jb20iLCJjaWQiOiJnOXhVRLJyODRLX3B3ZF9ncmFudCIsImRpZCI6IiIsIndpZCI6IiIsImV4cCI6MTU1MjU5MzYxOSwiaWF0IjoxNTUyNTgyODE5LCJzdWIiOiIzMjhkN2E0ZS0zYTU4LTRiOTktOGQwOC04MWQzMMDmZmRkZjUiL-CJwcm5fdHlwZSI6IlVTRViifQ.i5QWkrsrjReLJiKnkxqrZ-4CAcjqWn0NjDWBDHxMPtTW4CEV_Hz_8izwIC7mG5w-goy5zlzj7Ur9h08j3XpylsaI3ZqRUNZ20F9ffqI1bUDOR-4yUhvUmonRgRblbql6qr5Q_CxDu_tHa65u0SbBcJCwhF-PoS7H5r0Ue7ZoofZYFzc0yh2exkDC00GzcuVgTtysLKt117d3-2zF0K6U6ut-5L80f8LQSm590A3LFRMHYyWk-tFFD5-jS5TvBU4jElWeZoZICFpxaA6Jk9UIVoAcz4SG0thkyZUSmfPi8S_Ty61G_Ll3M-3RxEXdmJWBLCfzzSmNWJar-e8Hft-mvqg",
    "token_type": "Bearer",
    "expires_in": 10799,
    "scope": "user"
  }
}

```

Acquire an Authentication Token for User mapped through VMware Identity Manager

1. VMware Identity Manager must be configured & enabled in VCF Operations for networks.
2. Access token must be obtained from VMware Identity Manager.

POST a request to the authentication URL to acquire token for VMware Identity Manager users

```

Request:
POST https://operations-for-networks.example.com/api/ni/auth/token/vidm
Content-type : application/json
Accept : application/json
Body:
{
  "vidm_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJqdGkiOiJkMDkxZTlmOS0xZDVlLTQ0ODctODk5YS03YTZhMGJjNjBkYmUiLCJwcm4iOiJlc2VybUBWUk5JLVZJRE0tVEVOQU5UMSIsImRvbWpbiI6InZpZG1sYWJzLmNvbSIsInVzZXJfaWQiOiIxMyIsImF1dGhfdGltZSI6MTU1MjU4Mjgxc0swiaXNzIjoiaHR0cHM6Ly92cm5pLXZpZG0tdGVuYy-W50MS5lbmcudm13YXJlLmNvbS9TQUFTL2F1dGgiLCJhdWQiOiJodHRwczovL3ZybmktZmlkbS10ZW5hbnQxLmVuZy52bXdhcmUuY29tL1NBQVMvYXV0aC9vYXV0aHRva2VuIiwiaW3R4Ijoiw3tIm10ZFwiOlwidXJuOm9hc2lzOm5hbWVzOnR-jOlNBtUw6Mi4wOmFjOmNsYXNzZXMM6UGFzc3dvcnRQcm90ZWN0ZWRUcmFuc3BvcnRcIixcImlhdFwiOjE1NTI1ODI4MTk-sXCJpZFwiOjE1fV0iLCJzY3AiOiJvcGVuaWQgcHJvZmlsZSB1c2VyaGVtYWlsIiwiaWRwIjoimcIsImVtbCI6InVz-ZXIXQHhtd2FyZS5jb20iLCJjaWQiOiJnOXhVRLJyODRLX3B3ZF9ncmFudCIsImRpZCI6IiIsIndpZCI6IiIsImV4cCI6MTU1MjU5MzYxOSwiaWF0IjoxNTUyNTgyODE5LCJzdWIiOiIzMjhkN2E0ZS0zYTU4LTRiOTktOGQwOC04MWQzMMDmZmRkZjUiL-CJwcm5fdHlwZSI6IlVTRViifQ.i5QWkrsrjReLJiKnkxqrZ-4CAcjqWn0NjDWBDHxMPtTW4CEV_Hz_8izwIC7mG5w-goy5zlzj7Ur9h08j3XpylsaI3ZqRUNZ20F9ffqI1bUDOR-4yUhvUmonRgRblbql6qr5Q_CxDu_tHa65u0SbBcJCwhF-PoS7H5r0Ue7ZoofZYFzc0yh2exkDC00GzcuVgTtysLKt117d3-2zF0K6U6ut-5L80f8LQSm590A3LFRMHYyWk-tFFD5-jS5TvBU4jElWeZoZICFpxaA6Jk9UIVoAcz4SG0thkyZUSmfPi8S_Ty61G_Ll3M-3RxEXdmJWBLCfzzSmNWJar-e8Hft-mvqg"
}
Response:
{
  "token": "KxlrQZeVHhPsBbDOTu5Xzw==",
  "expiry": 1552600841923
}

```

Configuring VMware Identity Manager with VMware Cloud Foundation Operations for Networks

VCF Operations for networks supports SSO authentication through VMware Identity Manager. To authenticate against a particular VMware Identity Manager appliance, it must be configured & enabled in VCF Operations for networks.

1. Register VCF Operations for networks as an OAuth client (Service client) to VMware Identity Manager appliance
2. The user performing this action must hold administrator privileges.

1. POST a request to the settings URL for VMware Identity Manager
2. sha_thumbprint is an optional field (must be kept as an empty string if thumbprint validation is not required)

```
Request:
POST https://operations-for-networks.example.com/api/ni/settings/vidm
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "vidm_appliance": "identity.domain.com",
  "client_id": "operations_for_networks_client",
  "client_secret": "operations_for_networks_secret",
  "sha_thumbprint": "",
  "enable": "true"
}
Response:
{
  "vidm_appliance": "identity.domain.com",
  "client_id": "operations_for_networks_client",
  "enable": true
}
```

Updating VMware Identity Manager Configuration

1. VMware Identity Manager must be configured in VCF Operations for networks.
 2. The user performing this action must hold administrator privileges.
1. PUT request to the settings URL for VMware Identity Manager.
 2. sha_thumbprint is an optional field (must be kept as an empty string if thumbprint validation is not required)

```
Request:
PUT https://operations-for-networks.example.com/api/ni/settings/vidm
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "vidm_appliance": "identity.domain.com",
  "client_id": "operations_for_networks_client",
  "client_secret": "operations_for_networks_secret",
  "sha_thumbprint": "",
  "enable": "true"
}
Response:
{
  "vidm_appliance": "identity.domain.com",
  "client_id": "operations_for_networks_client",
  "enable": true
}
```

Fetching VMware Identity Manager configuration

1. VMware Identity Manager must be configured in VCF Operations for networks.

2. The user performing this action must hold administrator privileges.

1. GET request to the settings URL for VMware Identity Manager

2. sha_thumbprint is an optional field (must be kept as an empty string if thumbprint validation is not required)

```
Request:
GET https://operations-for-networks.example.com/api/ni/settings/vidm
Authorization : NetworkInsight {auth-token}
```

```
Response:
{
  "vidm_appliance": "identity.domain.com",
  "client_id": "operations_for_networks_client",
  "enable": true
}
```

Deleting the Existing VMware Identity Manager Configuration in Operations for Networks

Deleting the VMware Identity Manager configuration eventually removes corresponding users & user-groups (those mapped through VMware Identity Manager) from VCF Operations for networks.

1. VMware Identity Manager must be configured in VCF Operations for networks.

2. The user performing this action must have administrator privileges.

DELETE request to the settings URL for VMware Identity Manager.

```
Request:
DELETE https://operations-for-networks.example.com/api/ni/settings/vidm
Authorization : NetworkInsight {auth-token}
```

```
Response:
HTTP 204 (No-content)
```

Integrating VMware Identity Manager in VMware Cloud Foundation Operations for Networks

1. VMware Identity Manager must be configured in VCF Operations for networks.

2. The user performing this action must hold administrator privileges.

POST request to the settings URL for VMware Identity Manager enable operation.

```
Request:
POST https://operations-for-networks.example.com/api/ni/settings/vidm/enable
Authorization : NetworkInsight {auth-token}
```

```
Response:
HTTP 204 (No-content)
```

Disabling the Integration of VMware Identity Manager in VMware Cloud Foundation Operations for Networks

1. VMware Identity Manager must be configured in VCF Operations for networks.

2. The user performing this action must hold administrator privileges.

POST request to the settings URL for VMware Identity Manager disable operation

```
Request:
POST https://operations-for-networks.example.com/api/ni/settings/vidm/disable
Authorization : NetworkInsight {auth-token}
```

```
Response:
HTTP 204 (No-content)
```

Adding a User Mapped through VMware Identity Manager

1. VMware Identity Manager must be configured in VCF Operations for networks.
 2. The user performing this action must hold administrator privileges to manage user and user-groups.
1. POST request to the users settings URL for VMware Identity Manager
 2. Expected values for "role" = {"ADMIN", "MEMBER"}

Adding a user

```
Request:
PUT https://operations-for-networks.example.com/api/ni/settings/users/vidm
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "username": "testuser",
  "domain": "vidmlabs.com",
  "display_name": "test user",
  "role": "MEMBER"
}
Response:
{
  "user_type": "VIDM",
  "id": "testuser@vidmlabs.com",
  "role": "MEMBER",
  "username": "testuser",
  "domain": "vidmlabs.com",
  "display_name": "test user"
}
```

Updating Role for User Mapped through VMware Identity Manager

1. VMware Identity Manager must be configured in VCF Operations for networks.
 2. The user performing this action must hold administrator privileges to manage user and user-groups.
 3. User (whose role is getting updated) is already added in VCF Operations for networks.
1. PUT request to the users settings URL for VMware Identity Manager
 2. Expected values for "role" = {"ADMIN", "MEMBER"}

Update user roles

```
Request:
PUT https://operations-for-networks.example.com/api/ni/settings/users/vidm
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "username": "testuser",
  "domain": "vidmlabs.com",
  "display_name": "test user",
  "role": "MEMBER"
}
Response:
{
```

```

    "user_type": "VIDM",
    "id": "testuser@vidmlabs.com",
    "role": "MEMBER",
    "username": "testuser",
    "domain": "vidmlabs.com",
    "display_name": "test user"
  }

```

Listing all the User Details

User performing this action must have adequate permissions (admin user) to manage users and user-groups.

1. GET request to the user settings URL
2. Mandatory query param : "type", expected values = {"vidm", "local", "ldap"}
3. Optional query param : "size", "cursor"

Listing all the users

```

Request:
GET https://operations-for-networks.example.com/api/ni/settings/users?type=vidm
Authorization : NetworkInsight {auth-token}
Response:
{
  "results": [
    {
      "user_type": "VIDM",
      "id": "testuser1@vidmlabs.com",
      "role": "MEMBER",
      "username": "testuser1",
      "domain": "vidmlabs.com",
      "display_name": "test user1"
    },
    {
      "user_type": "VIDM",
      "id": "testuser2@vidmlabs.com",
      "role": "MEMBER",
      "username": "testuser2",
      "domain": "vidmlabs.com",
      "display_name": "test user2"
    }
  ],
  "cursor": "Mg==",
  "total_count": 3
}

```

Fetching User Details

According to the current version, the API only return details of user mapped through VMware Identity Manager. Details of local/ldap user is not yet supported.

User performing this action must have administrator permissions to manage users and user-groups.

1. GET request to the user settings URL.
2. The user's 'id' must be included as a URL-encoded path parameter.

Fetch user details

```

Request:
GET https://operations-for-networks.example.com/api/ni/settings/users/testuser1@system%20domain

```

```

Authorization : NetworkInsight {auth-token}
Response:
{
  "user_type": "VIDM",
  "id": "testuser1@system domain",
  "role": "MEMBER",
  "username": "testuser1",
  "domain": "system domain",
  "display_name": "test user1"
}

```

Deleting an Existing User

The API is currently restricted to deletion of users mapped through VMware Identity Manager. Local/ldap users deletion is not yet supported.

User performing this action must have administrator permissions to manage users and user-groups.

1. DELETE request to the users settings URL.
2. The user's 'id' must be included as a URL-encoded path parameter.

Delete an existing user.

```

Request:
DELETE https://operations-for-networks.example.com/api/ni/settings/users/testuser1@sytem%20domain
Authorization : NetworkInsight {auth-token}
Response:
HTTP 204 (No-content)

```

Adding a User Group through VMware Identity Manager

1. VMware Identity Manager must be configured in VCF Operations for networks.
 2. User performing this action must have adequate permissions (admin user) to manage users and user-groups.
1. POST request to the user-groups settings URL for VMware Identity Manager.
 2. Expected values for "role" = {"ADMIN", "MEMBER"} .

Adding a User

```

Request:
POST https://operations-for-networks.example.com/api/ni/settings/user-groups/vidm
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "group_name": "testgroup1",
  "domain": "vidmlabs.com",
  "role": "MEMBER"
}
Response:
{
  "group_type": "VIDM",
  "id": "testgroup1@vidmlabs.com",
  "group_name": "testgroup1",
  "domain": "vidmlabs.com",
  "role": "MEMBER"
}

```


Updating the Role for User Group mapped through VMware Identity Manager

1. VMware Identity Manager must be configured in VCF Operations for networks.
2. User performing this action must have adequate permissions (admin user) to manage users and user-groups.
3. User-group (whose role is getting updated) is already added in VCF Operations for networks.

PUT request to the user-groups settings URL for VMware Identity Manager.

Update role for user group

```
Request:
PUT https://operations-for-networks.example.com/api/ni/settings/user-groups/vidm
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "group_name": "testgroup1",
  "domain": "vidmlabs.com",
  "role": "ADMIN"
}
Response:
{
  "group_type": "VIDM",
  "id": "testgroup1@vidmlabs.com",
  "group_name": "testgroup1",
  "domain": "vidmlabs.com",
  "role": "ADMIN"
}
```

Listing Details of all the User Groups

Currently this API only return results for user-groups mapped through VMware Identity Manager. Results for other user-groups is not yet supported.

User performing this action must have adequate permissions (admin user) to manage users and user-groups.

1. GET request to the user-groups settings URL.
2. Mandatory query param : "type", expected values = {"vidm", "local", "ldap"}.
3. Optional query param : "size", "cursor".

Listing Details of all the User Groups

```
Request:
GET https://operations-for-networks.example.com/api/ni/settings/user-groups?type=vidm
Authorization : NetworkInsight {auth-token}
Response:
{
  "results": [
    {
      "group_type": "VIDM",
      "id": "testgroup1@vidmlabs.com",
      "group_name": "testgroup1",
      "domain": "vidmlabs.com",
      "role": "ADMIN"
    },
    {
      "group_type": "VIDM",
```

```

        "id": "testgroup2@vidmlabs.com",
        "group_name": "testgroup2",
        "domain": "vidmlabs.com",
        "role": "MEMBER"
    }
],
"cursor": "Mg==",
"total_count": 3
}

```

Fetching Details of a User Group

Currently this API only return details of user-group mapped through VMware Identity Manager. Retrieving details of other user-groups is not yet supported.

User performing this action must have adequate permissions (admin user) to manage users and user-groups.

1. GET request to the user-groups settings URL
2. The user's 'id' must be included as a URL-encoded path parameter.

Fetching details of a User Group

```

Request:
GET https://operations-for-networks.example.com/api/ni/settings/users/sysadmins@sytem%20domain
Authorization : NetworkInsight {auth-token}
Response:
{
  "group_type": "VIDM",
  "id": "sysadmins@system domain",
  "group_name": "sysadmins",
  "domain": "vidmlabs.com",
  "role": "ADMIN"
}

```

Deleting an Existing User Group

This API is currently restricted to deletion of user-groups mapped through VMware Identity Manager.

User performing this action must have adequate permissions (admin user) to manage users and user-groups.

1. DELETE request to the user-groups settings URL
2. The user's 'id' must be included as a URL-encoded path parameter.

Delete an existing User group

```

Request:
DELETE https://operations-for-networks.example.com/api/ni/settings/users/sysadmins@sytem%20domain
Authorization : NetworkInsight {auth-token}
Response:
HTTP 204 (No-content)

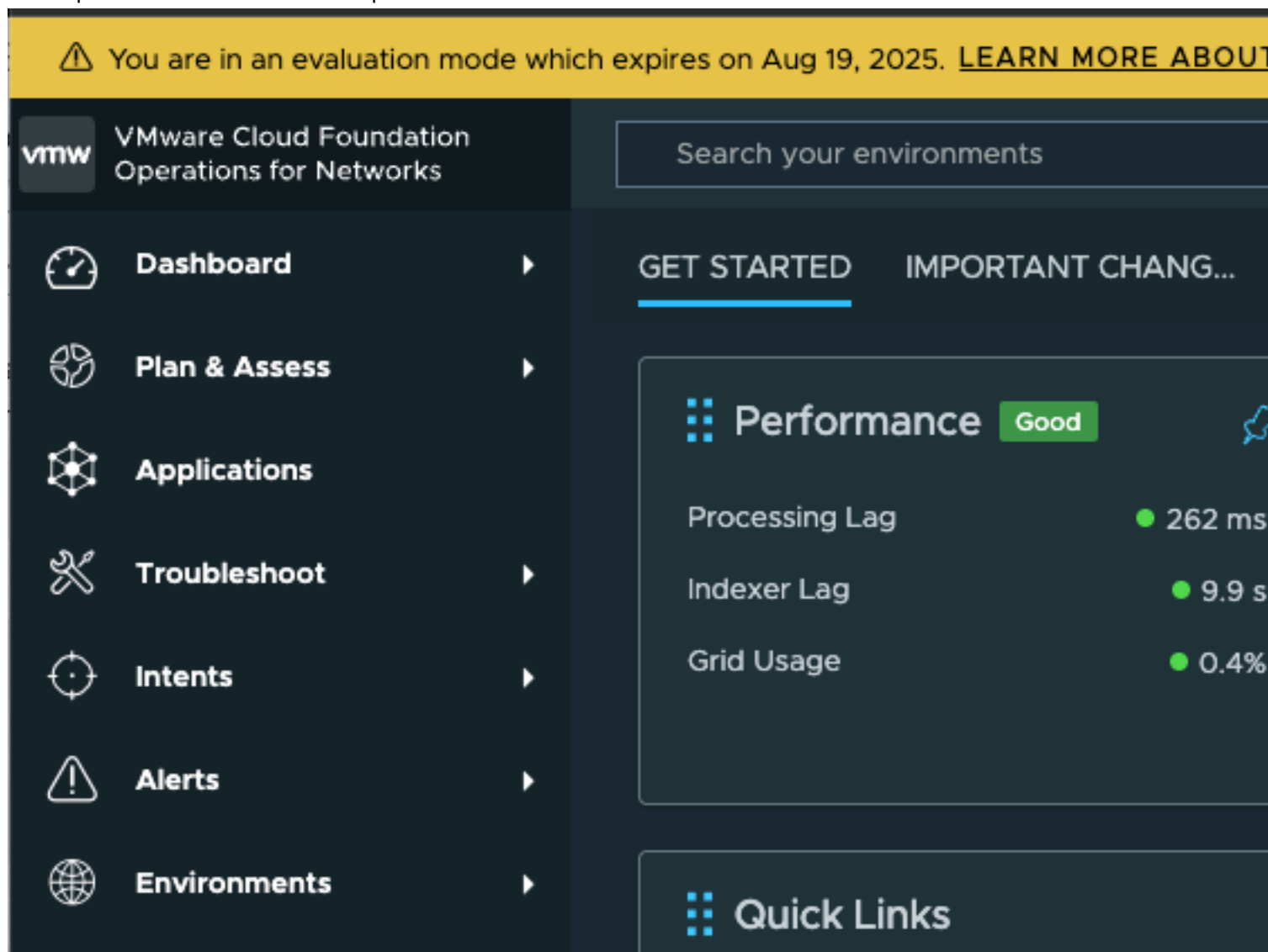
```

Using API Explorer

VCF Operations for networks API Explorer is available in product which is used to explore APIs and their responses.

Steps to use API Explorer.

- VCF Operations for networks API Explorer can be accessed from API Documentation link as shown below.



- Generate auth token using Authentication API.

Authentication Authentication APIs

POST**/auth/token** Create an auth token

VCF Operations for Networks supports token based authentication. Tokens are non-r
Authorization : NetworkInsight {token}
If a token is invalid or expired, 401-Unauthorized error gets returned in the response
There is limit of 100 valid tokens per user and further requests will return 401-Unauth
Expired tokens are cleaned periodically by the system.

Parameters

No parameters

Request body required

User Credentials

```
{  
  "username": "admin@vrni.com",  
  "password": "password"
```

- Click on the **Authorize** button to enter the auth token as shown below.

VMware Cloud Foundation Operations for Networks API Reference

VCF Operations for Networks API Reference

Servers

/api/ni

▼

Filter by tag

Authentication Authentication APIs

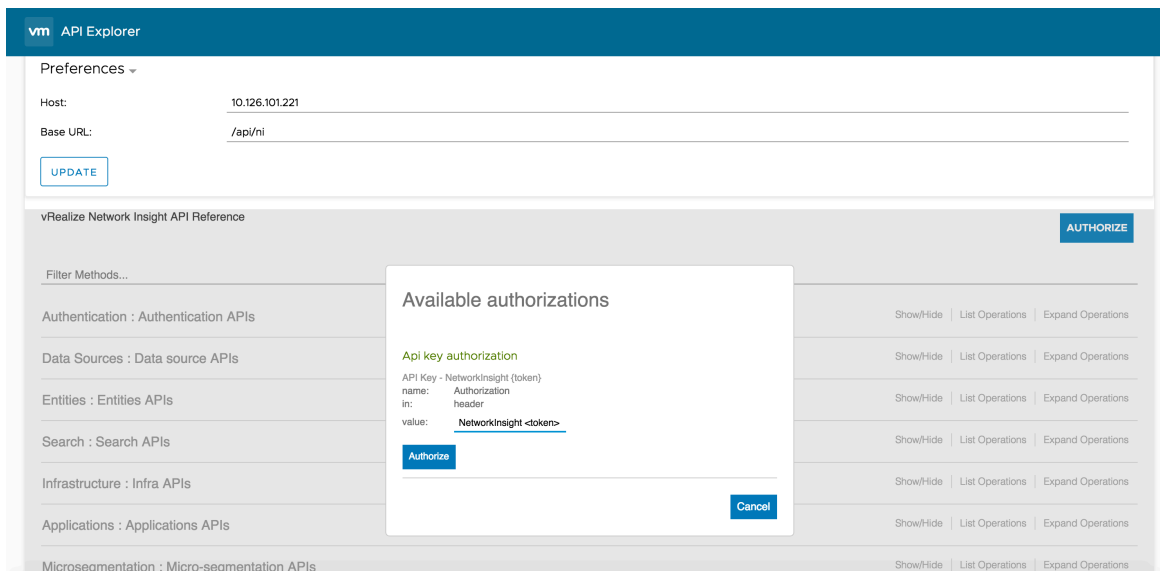
Data Sources Data Source APIs

Entities Entities APIs

Search Search APIs

Infrastructure Infra APIs

- Click on the **Authorize** button to enter the auth token as shown below.



- Now you can try any API using **Try it out** button.

Managing Data Sources

You can manage data sources using APIs. Data sources can be added, updated, deleted, enabled, or disabled using APIs.

Data Sources URL Prefix:

/data-sources/

Data sources are grouped on the basis of the data source type. URL Prefix corresponding to data sources types are listed in this table.

Table 1901: Data Source Type to URL Prefix

Data Source Type	URL Prefix
VCenterDataSource	/data-sources/vcenters
NSXVManagerDataSource	/data-sources/nsxv-managers
NSXTManagerDataSource	/data-sources/nsxt-managers
CiscoSwitchDataSource	/data-sources/cisco-switches
CiscoACIDataSource	/data-sources/cisco-aci
AristaSwitchDataSource	/data-sources/arista-switches
DellSwitchDataSource	/data-sources/dell-switches
BrocadeSwitchDataSource	/data-sources/brocade-switches
JuniperSwitchDataSource	/data-sources/juniper-switches
UCSManagerDataSource	/data-sources/ucs-managers
HPOneViewDataSource	/data-sources/hpov-managers
HPVCMManagerDataSource	/data-sources/hpvc-managers
CheckpointFirewallDataSource	/data-sources/checkpoint-firewalls

Data Source Type	URL Prefix
PanFirewallDataSource	/data-sources/panorama-firewalls
KubernetesDataSource	/data-sources/kubernetes-clusters
PolicyManagerDataSource	/data-sources/vmc-nsxmanagers
PKSDataSource	/data-sources/pks
ServiceNowDataSource	/data-sources/servicenow-instances

List Data Sources

Get the data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).

Make a GET request to the URL corresponding to the data source type.

```
Request
GET https://operations-for-networks.example.com/api/ni/data-sources/vcenters
Response Body
{
  "results": [
    {
      "entity_id": "10000:902:627340223",
      "entity_type": "VCenterDataSource"
    },
    {
      "entity_id": "10000:902:993642840",
      "entity_type": "VCenterDataSource"
    },
    {
      "entity_id": "10000:902:738162743",
      "entity_type": "VCenterDataSource"
    },
    {
      "entity_id": "10000:902:627340998",
      "entity_type": "VCenterDataSource"
    },
    {
      "entity_id": "10000:902:390269772",
      "entity_type": "VCenterDataSource"
    }
  ],
  "total_count": 5
}
```

Fetch Data Source Details

1. Get the data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).
2. Get entity ID using the `list` command.

Make a GET request for the specific data source entity ID.

```
Request
GET https://operations-for-networks.example.com/api/ni/data-sources/vcenters/10000:902:627340223
Response
{
```

```

    "entity_id": "10000:902:993642895",
    "entity_type": "VCenterDataSource",
    "ip": "10.197.17.68",
    "proxy_id": "10000:901:1586035958",
    "nickname": "aa",
    "enabled": true,
    "notes": "ecmp lab aa",
    "credentials": {
      "username": "administrator@vSphere.local",
      "password": ""
    }
  }
}

```

Update Data Source Credentials

1. Get the data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).
2. Get entity ID using the `list` command.
1. Make a GET request for the specific data source entity ID.
2. Make a PUT request with request body after changing the credentials in the response retrieved in step 1.

Request

PUT <https://operations-for-networks.example.com/api/ni/data-sources/vcenters/10000:902:627340223>

Request Body

```

{
  "entity_id": "10000:902:993642895",
  "entity_type": "VCenterDataSource",
  "ip": "10.197.17.68",
  "proxy_id": "10000:901:1586035958",
  "nickname": "aa",
  "enabled": true,
  "notes": "ecmp lab aa",
  "credentials": {
    "username": "newuser",
    "password": "newpassword"
  }
}

```

Response

```

{
  "entity_id": "10000:902:993642895",
  "entity_type": "VCenterDataSource",
  "ip": "10.197.17.68",
  "proxy_id": "10000:901:1586035958",
  "nickname": "aa",
  "enabled": true,
  "notes": "ecmp lab aa",
  "credentials": {
    "username": "newuser",
    "password": ""
  }
}

```

Enable a Data Source

1. Get the data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).
2. Get entity_id from the `list` command.

To enable a data source, make a POST request.

```
Request
POST https://operations-for-networks.example.com/api/ni/data-sources/vcenters/10000:902:627340223/enable
Response
200 OK
```

Disable a Data Source

1. Get the data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).
2. Get entity_id from the `list` command.

To disable a data source, make a POST request.

```
Request
POST https://operations-for-networks.example.com/api/ni/data-sources/vcenters/10000:902:627340223/disable
Response
200 OK
```

Add a Data Source

1. Get the data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).
2. You must have IP Address or the FQDN of the data source and its credentials.
 1. Get the proxy id for adding data sources using the `infra/nodes` APIs.
 2. Make a POST request conforming to the API specification. For example, to add a vCenter data source.

```
Request
POST https://operations-for-networks.example.com/api/ni/data-sources/vcenters/
Request Body
{
  "ip": "10.197.17.68",
  "fqdn": "",
  "proxy_id": "10000:901:1586035958",
  "nickname": "aa",
  "enabled": true,
  "notes": "ecmp lab aa",
  "credentials": {
    "username": "administrator@vsphere.local",
    "password": "password"
  }
}

Response
{
  "entity_id": "10000:902:993642895",
  "entity_type": "VCenterDataSource",
  "ip": "10.197.17.68",
```

```

    "proxy_id": "10000:901:1586035958",
    "nickname": "aa",
    "enabled": true,
    "notes": "ecmp lab aa",
    "credentials": {
      "username": "administrator@vsphere.local",
      "password": ""
    }
  }
}

```

Delete a Data Source

1. Get the data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).
2. Get entity_id from the `list` command.

Make a DELETE request for the specific entity_id.

```

Request DELETE https://operations-for-networks.example.com/api/ni/data-sources/vcen-
ters/10000:902:627340223/
Response
204 No Content

```

Configure SNMP on a Switch Data Source

1. Get the switch data source URL for the data source type. For more information, for URL Prefix see [Table: DataSourceType](#).
2. Get entity_id from the `list` command.

Make a PUT request conforming to the API specification.

```

Request
PUT https://operations-for-networks.example.com/api/ni/data-sources/cisco-switch-
es/10000:903:627340223
Request Body
{
  "snmp_enabled": true,
  "snmp_version": "v3",
  "config_snmp_3": {
    "username": "nilesh",
    "authentication_type": "SHA",
    "authentication_password": "authentication_password",
    "privacy_type": "NO_PRIV"
  }
}

Response Body
{
  "snmp_enabled": true,
  "snmp_version": "v3",
  "config_snmp_3": {
    "username": "nilesh",
    "authentication_type": "SHA",
    "authentication_password": "authentication_password",
    "privacy_type": "NO_PRIV"
  }
}

```

```
}

```

Bulk Edit Data Sources

This endpoint allows users to submit a request for bulk operations on data sources. Bulk operations enable users to perform multiple operations simultaneously, streamlining the process and improving efficiency. Users can perform enable, disable, edit and delete operations on data sources in bulk using this API.

Get `entity_id` of data sources from the `list` command.

1. Make a POST request conforming to the API specification
2. Response provides a unique `request_id`, which serves as an identifier to track the progress of the operation. To monitor the status of the bulk operation, you can utilize the `Get Bulk Operation Details API` by passing the received `request_id` as a parameter.

Bulk Enable Data Sources

Bulk DISABLE Data Sources

Bulk UPDATE username and password for Data Sources

Bulk UPDATE SNMP credentials for Data Sources

Bulk DELETE Data Sources

Request

POST `https://operations-for-networks.example.com/api/ni/data-sources/bulk`

Content-type : `application/json`

Accept : `application/json`

Authorization : `NetworkInsight {auth-token}`

Body:

```
{
  "action_type": "ENABLE",
  "data_sources": [
    {
      "entity_id": "10071:907:4230117030541228376"
    },
    {
      "entity_id": "10071:907:8650548508942367585"
    }
  ]
}
```

Response

```
{
  "request_id": "195d283e-30ff-4eef-865b-629851eff7"
}
```

Request

POST `https://operations-for-networks.example.com/api/ni/data-sources/bulk`

Content-type : `application/json`

Accept : `application/json`

Authorization : `NetworkInsight {auth-token}`

Body:

```
{
  "action_type": "DISABLE",
  "data_sources": [

```

```

    {
      "entity_id": "10071:907:4230117030541228376"
    },
    {
      "entity_id": "10071:907:8650548508942367585"
    }
  ]
}

```

Response

```

{
  "request_id": "195d283e-30ff-4eef-865b-629851effbb7"
}

```

Request

POST https://operations-for-networks.example.com/api/ni/data-sources/bulk

Content-type : application/json

Accept : application/json

Authorization : NetworkInsight {auth-token}

Body:

```

{
  "action_type": "UPDATE",
  "data_sources": [
    {
      "entity_id": "10071:907:4230117030541228376",
      "fields": [
        {
          "key": "USER",
          "value": "username"
        },
        {
          "key": "PWD",
          "value": "pwd"
        }
      ]
    },
    {
      "entity_id": "10071:907:8650548508942367585",
      "fields": [
        {
          "key": "USER",
          "value": "username"
        },
        {
          "key": "PWD",
          "value": "pwd"
        }
      ]
    }
  ]
}

```

Response

```

{

```

```

    "request_id": "993f42b1-010b-4811-8873-6bb3bebb90d7"
  }
Request
POST https://operations-for-networks.example.com/api/ni/data-sources/bulk
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "action_type": "UPDATE",
  "data_sources": [
    {
      "entity_id": "10071:907:4230117030541228376",
      "fields": [
        {
          "key": "snmp_metric_enabled",
          "value": "true"
        },
        {
          "key": "snmp_version",
          "value": "_2c"
        },
        {
          "key": "snmp_community",
          "value": "community_string"
        }
      ]
    }
  ]
}

```

```

Response
{
  "request_id": "993f42b1-010b-4811-8873-6bb3bebb90d7"
}

```

Other fields which can be used in bulk update

- * snmp_version, allowed values are _2c or _3
- * snmp_username
- * context_name
- * authentication_type, allowed values are MD5 or SHA
- * authentication_password
- * privacy_type, allowed values are DES, AES128, AES192, AES256, _3DES
- * privacy_password
- * configPollingInterval
- * configPollingIntervalType
- * notes

```

Request
POST https://operations-for-networks.example.com/api/ni/data-sources/bulk
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}

```

```

Body:
{
  "action_type": "DELETE",
  "data_sources": [
    {
      "entity_id": "10071:907:4230117030541228376"
    },
    {
      "entity_id": "10071:907:8650548508942367585"
    }
  ]
}

Response
{
  "request_id": "993f42b1-010b-4811-8873-6bb3bebb90d7"
}

```

Get Bulk Operation Details

This endpoint allows users to retrieve details of a specific bulk operation by providing the `request_id`. The `request_id` is a unique identifier assigned to each submitted bulk operation request. The `request_id` is received when user submits a bulk enable, disable, update or delete operation on data sources.

Make a GET request conforming to the API specification.

```

Request
GET https://operations-for-networks.example.com/api/ni/data-sources/bulk/view-de-
tails/31094378-2261-4046-b02d-7a7851a88c99
Response
{
  "total_count": 2,
  "success_count": 2,
  "failed_count": 0,
  "successful_data_sources": [
    "GENERICSWITCH_DELLOS10_10.79.221.185",
    "GENERICSWITCH_DELLOS10_10.79.221.183"
  ],
  "failed_data_sources": []
}

```

Tagging IP Addresses

IP Addresses/Subnets can be tagged with EAST_WEST/INTERNET tags with this API.

List All Available Tags

Following API returns all the available tags in system.

```

Request
GET https://operations-for-networks.example.com/api/ni/ip-tags
Response
{
  "tag_ids": [
    "INTERNET",

```



```

    "EAST_WEST"
  ]
}

```

Add IP Address or Subnet to a Tag

Use the following request to add a subnet or IP address (es) to a tag. In the following example, Subnet and IP addresses are being tagged with EAST_WEST tag.

```

Request
POST https://operations-for-networks.example.com/api/ni/settings/ip-tags/EAST_WEST/add
Request Body
{
  "tag_id": "EAST_WEST",
  "subnets": [
    "192.168.10.0/24"
  ],
  "ip_address_ranges": [
    {
      "start_ip": "192.168.20.1",
      "end_ip": "192.168.20.5"
    }
  ]
}
Response 200 OK.

```

Remove IP Address or Subnet from a Tag

Use the following request to remove the subnet or IP address (es) for a tag. In the following example, IP address ranges (192.168.20.1-192.168.20.5) are being removed from EAST_WEST tag

```

Request
POST https://operations-for-networks.example.com/api/ni/settings/ip-tags/EAST_WEST/remove
Request Body
{
  "tag_id": "EAST_WEST",
  "ip_address_ranges": [
    {
      "start_ip": "192.168.20.1",
      "end_ip": "192.168.20.5"
    }
  ]
}
Response 200 OK.

```

Get Tag Details

Use the following request to GET tag details.

```

Request
GET https://operations-for-networks.example.com/api/ni/settings/ip-tags/EAST_WEST
Response
{
  "tag_id": "EAST_WEST",

```

```

    "ip_address_ranges": [
      {
        "start_ip": "192.168.20.1",
        "end_ip": "192.168.20.5"
      }
    ]
  }
}

```

Performing Search

You can search for entities using the entity type and search criteria. You can define a search criteria using filter expressions based on entity properties as defined in the API reference. Search results return a paginated list of entity IDs that match the filter criteria.

Search Request

Format of the Search Request body

```

{
  "entity_type": "string",
  "filter": "string",
  "sort_by": {
    "field": "string",
    "order": "ASC"
  },
  "size": 0,
  "cursor": "string",
  "time_range": {
    "start_time": 0,
    "end_time": 0
  }
}

```

Entity Type -

In the Entity Type field, use the entity types defined in AllEntityType enum in the API Reference.

Filter Expression

A filter expression defines search criteria in one of the following formats.

A filter expression defines search criteria in one of the following formats:

- *{field_name} {binary operator} {field_value}*
- *{field_name} {unary_operator}*

Optionally, complex expressions can be designed using {logical operators} and parenthesis (...).

Considerations

- You can use spaces in filter expressions.
- Use quotes with strings. You need not use quotes with numbers.

{field_name}

The name of a field in an Object. For example, VirtualMachine has a field named `vendor_id`.

{binary operator}

Used to compare a field to a value creating an expression.

{unary_operator}	Unary operator is used to test the field_name against the operator check. For example, IS SET is used to determine if a field is non NULL.
{field_value}	Represents the value of a field and can either be a number, string , list of numbers or list of strings.
{logical operator}	Logical operators are used to create complex filters. For example: (name = 'my_vm') OR (vendor_id = 'vm-101')
parentheses (...)	Parentheses are used to group expressions. They also determine the order in which the components of the expression are evaluated. For example: ((name = 'my_vm') AND (default_gateway = '10.1.1.1'))

Building Filter Expression

Components of a filter expression are: *FIELD_NAME**OPERATOR**VALUE*

FIELD_NAME

Field name can be a property defined in the model object as per the API Reference or it can be a property of a related object in the response. For example, a subset of the VirtualMachine object properties appears as follows:

```
{
  "entity_id": "18230:1:1158969162",
  "name": "NSX_Controller_9e80ec74-57ce-4671-8fd7-b5884a997535",
  "entity_type": "VirtualMachine",
  "ip_addresses": [
    {
      "ip_address": "10.197.17.74",
      "netmask": "255.255.255.0",
      "network_address": "10.197.17.0/24"
    }
  ],
  "default_gateway": "10.197.17.1",
  "cluster": {
    "entity_id": "18230:66:1293137396",
    "entity_type": "Cluster"
  },
  "host": {
    "entity_id": "18230:4:652218965",
    "entity_type": "Host"
  }
}
```

From the preceding model, you can use the following field names:

- Direct primitive properties
 - name
 - default_gateway

- Properties inside the structure.

For example:

- ip_addresses.ip_address.
- host.entity_id

- Properties of a related object.

For example:

- host.name
- cluster.name

Aggregation Search API

Format of the aggregation search request body :

```
{
  "entity_type": "{ENTITY_TYPE}",
  "filter": "{FILTER}",
  "aggregations": [
    {
      "field": "{FIELD}",
      "aggregation_type": "{AGGREGATION_TYPE}"
    }
  ],
  "time_range": {
    "start_time": {
      START_TIME_IN_SECONDS
    },
    "end_time": {
      END_TIME_IN_SECONDS
    }
  }
}
```

Aggregation search request contains following fields:

1. **entity_type** : Use the entity types defined in AllEntityType enum in the API Reference.
2. **filter** : Format of the filter expression is defined above.
3. **aggregations**: It contains array of elements. Each element consists of a field and aggregation type.
 - a. **field**: must be a valid property of the entity.
 - b. **aggregation_type**: can be one of the following.
 - a. SUM
 - b. MAX
 - c. MIN
 - d. AVG
4. **time_range**: Time range consists of start_time and end_time. Both must be POSIX epochs (The number of seconds elapsed since Jan 1st 1970).
 - a. **start_time**
 - b. **end_time**

Finding total number of CPU cores and overall total memory of all the Virtual Machines

```
{
  "entity_type": "VirtualMachine",
```

```

    "aggregations": [
      {
        "field": "cpu_count",
        "aggregation_type": "SUM"
      },
      {
        "field": "memory",
        "aggregation_type": "SUM"
      }
    ]
  }
}

```

Finding total memory of the Virtual Machines containing 'app' in their name

```

{
  "entity_type": "VirtualMachine",
  "filter": "name like 'app'",
  "aggregations": [
    {
      "field": "cpu_count",
      "aggregation_type": "SUM"
    },
    {
      "field": "memory",
      "aggregation_type": "SUM"
    }
  ]
}

```

Find sum of bytes of all flows from 10th Nov 2018 to 20th Nov 2018

```

{
  "entity_type": "Flow",
  "aggregations": [
    {
      "field": "flow.totalBytes.delta.summation.bytes",
      "aggregation_type": "SUM"
    }
  ],
  "time_range": {
    "start_time": 1541808000,
    "end_time": 1542672000
  }
}

```

Find sum of bytes of all flows from 1.1.1.1 to 2.2.2.2 from 10th Nov 2018 to 20th Nov 2018.

```

{
  "entity_type": "Flow",
  "filter": "source_ip.ip_address = '1.1.1.1' and destination_ip.ip_address = '2.2.2.2'",
  "aggregations": [
    {
      "field": "flow.totalBytes.delta.summation.bytes",
      "aggregation_type": "SUM"
    }
  ]
}

```

```

    ],
    "time_range": {
      "start_time": 1541808000,
      "end_time": 1542672000
    }
  }
}

```

Find max, min and sum memory of virtual machines

```

{
  "entity_type": "VirtualMachine",
  "aggregations": [
    {
      "field": "cpu_count",
      "aggregation_type": "MAX"
    },
    {
      "field": "cpu_count",
      "aggregation_type": "SUM"
    }
  ]
}

```

Group By

Group By feature in search query

URL: /api/ni/search/groupby

```

{
  "entity_type": "{ENTITY_TYPE}",
  "filter": "{FILTER}",
  "aggregations": [
    {
      "field": "{FIELD}",
      "aggregation_type": "{AGGREGATION_TYPE}"
    }
  ],
  "group_by": [
    String
  ],
  "sort_by": [
    {
      "field": "{FIELD}",
      "aggregation_type": "{AGGREGATION_TYPE}",
      "order": {
        ASC
      }
    }
  ],
  "time_range": {
    "start_time": {
      START_TIME_IN_SECONDS
    },

```

```

    "end_time": {
        END_TIME_IN_SECONDS
    }
},
"size": 0,
"cursor": string
}

```

Group By Request contains following fields:

entity_type: Use the entity types defined in AllEntityType enum in the API Reference.

filter: Format of the filter is same as above.

aggregations: It contains array of elements. Each element consists of a field and aggregation type. Field must be a valid property of the entity and aggregation type can be one of the following.

```

. SUM
. MAX
. MIN
. AVG

```

group_by: It contains array of elements. Each element is field which is valid property of entity.

sort_by: It contains array of elements. Each element consists of field, aggregation type and optional order (ASC / DESC).

time_range: Time range consist of start_time and end_time. Both must be POSIX epochs (The number of seconds elapsed since JAN 1st 1970).

```

. start_time
. end_time

```

Examples:

Find avg of CPU cores group by sum of CPU cores in ascending order

```

{
  "entity_type": "VirtualMachine",
  "aggregations": [
    {
      "field": "cpu_count",
      "aggregation_type": "AVG"
    }
  ],
  "group_by": [
    "cpu_count"
  ],
  "sort_by": [
    {
      "field": "cpu_count",
      "aggregation_type": "SUM",
      "order": "ASC"
    }
  ],
  "time_range": {

```

```

    "start_time": 1570612016,
    "end_time": 1570612016
  }
}

```

Find sum of bytes of all flows from 1.1.1.1 to 2.2.2.2 from 10th Nov 2018 to 20th Nov 2018 group by source ip and destination ip order by maximum of flow in ascending order

```

{
  "entity_type": "Flow",
  "filter": "source_ip.ip_address = '1.1.1.1' and destination_ip.ip_address = '2.2.2.2'",
  "aggregations": [
    {
      "field": "flow.totalBytes.delta.summation.bytes",
      "aggregation_type": "SUM"
    }
  ],
  "sort_by": [
    {
      "field": "flow.totalBytes.delta.summation.bytes",
      "aggregation_type": "MAX",
      "order": "ASC"
    }
  ],
  "group_by": [
    "destination_ip",
    "source_ip"
  ],
  "time_range": {
    "start_time": 1541808000,
    "end_time": 1542672000
  }
}

```

Operators

Operators used in search expression

Operator	Description
=	Equals
!=	Not Equals
>	Greater Than
<	Less Than
>=	Greater Than OR Equals
<=	Less Than OR Equals
LIKE	Value contains the the field value
IN	Value matches one of the field values in list
NOT IN	Value does not match any field value in list
SET	Value is not null
NOT SET	Value is null

FIELD_VALUE

The search value can be a string, integer, list of strings or list of integers.

For example:

- String: 'securitygroup-10'
- Integer : 211

Examples of Search

Sample Search Requests

Find VMs with a specific ip_address, such as 192.168.10.1

Find all VMs on a host with name 'host-a'

Find all VMs where the name is 'vm-a' or 'vm-b' or 'vm-c'

Find all VMs where the default gateway is set

Find all VMs in host 'host-a' and security group 'sg-1'

Find all VMs in security group sg-1 and that do not have network address '192.168.10.0/24'

```
Request
POST https://operations-for-networks.example.com/api/ni/search
Request Body
```

```
{
  "entity_type": "VirtualMachine",
  "filter": "ip_addresses.ip_address = '192.168.10.1'",
}
```

```
Request
POST https://operations-for-networks.example.com/api/ni/search
Request Body
```

```
{
  "entity_type": "VirtualMachine",
  "filter": "host.name = 'host-a'",
}
```

```
Request
POST https://operations-for-networks.example.com/api/ni/search
Request Body
```

```
{
  "entity_type": "VirtualMachine",
  "filter": "name IN ('vm-a', 'vm-b', 'vm-c')",
}
```

```
Request
POST https://operations-for-networks.example.com/api/ni/search
Request Body
```

```
{
  "entity_type": "VirtualMachine",
  "filter": "default_gateway is set",
}
```

```
Request
POST https://operations-for-networks.example.com/api/ni/search
Request Body
```

```

{
  "entity_type": "VirtualMachine",
  "filter": "((security_groups.name = 'sg-1') and (host.name = 'host-a'))",
}
Request
POST https://operations-for-networks.example.com/api/ni/search
Request Body
{
  "entity_type": "VirtualMachine",
  "filter": "((security_groups.name = 'sg-1') and (ip_addresses.network_address !=
'192.168.10.0/24'))",
}

```

Working with Entities

Each entity in VCF Operations for networks has an associated entity ID that is unique and an entity type. You can fetch an entity using the entity ID from the URL corresponding its entity type. You can get the list of all entity IDs using a paginated list request.

You can determine the entity ID using a generic search based on the properties of the entity. For more information, see the [Performing Search](#) topic.

Table 1902: Table 3: Entity Type to URI

EntityType	URI
VirtualMachine	/entities/vms
EC2Instance	/entities/vms
Host	/entities/hosts
Vnic	/entities/vnics
Vmknics	/entities/vmknics
VxlanLayer2Network	/entities/layer2-networks
VlanLayer2Network	/entities/layer2-networks
Cluster	/entities/clusters
SecurityTag	/entities/security-tags
ResourcePool	/entities/resouce-pools
NSXIPSet	/entities/ipsets
EC2IPSet	/entities/ipsets
NSXSecurityGroup	/entities/security-groups
EC2SecurityGroup	/entities/security-groups
Flow	/entities/flows
ProblemEvent	/entities/problems
NSXFirewallRule	/entities/firewall-rules
EC2SGFirewallRule	/entities/firewall-rules
NSXRedirectRules	/entities/firewall-rules
VCenterManager	/entities/vcenter-managers
NSXVManager	/entities/nsxv-managers
NSXService	/entities/services

EntityType	URI
EC2Service	/entities/services
NSXDistributedFirewall	/entities/firewalls
EC2Firewall	/entities/firewalls
NSXServiceGroup	/entities/service-groups
DistributedVirtualSwitch	/entities/distributed-virtual-switches
DistributedVirtualPortgroup	/entities/distributed-virtual-portgroups
VCDatcenter	/entities/vc-datacenters
Datastore	/entities/datastores
Folder	/entities/folders
PolicyManagerSegment	/entities/layer2-networks

List All Entities for an Entity Type

Listing all entities of an entity type requires issuing a GET request on the URL corresponding to the entity type. The response contains a paginated list of entities with time corresponding to the version of the entity that has been retrieved. Response contains the cursor that can be used as a query parameter to fetch next page of results. You can add the *size* query parameter to request to set the number of entities that server must send in response. It can be set to a maximum of 10000

Obtain the URL corresponding to the entity type in the search result.

Issue a GET request using the URL.

Response contains a paginated list with the cursor to be passed in the next request as a parameter to retrieve the next set of VMs.

```
Request
GET https://operations-for-networks.example.com/api/ni/entities/vms/
Response
{
  "results": [
    {
      "entity_id": "10000:1:875335999",
      "entity_type": "VirtualMachine",
      "time": 1508745899
    },
    {
      "entity_id": "10000:1:1206990361",
      "entity_type": "VirtualMachine",
      "time": 1508745899
    },
    ...
  ],
  "cursor": "MTA=",
  "total_count": 78,
  "start_time": 1508972172,
  "end_time": 1508972172
}
```

Retrieve an Entity

1. Obtain the entity ID of the entity by using a search or list command
2. Obtain the URL corresponding to the entity type in the search result. For more information, see [Entity Type to URI](#).

Run a GET request using the entity ID and the URL.

This example uses entity_id 1000:1:123121 and entity_type is VirtualMachine

```
Request
GET https://operations-for-networks.example.com/api/ni/entities/vms/1000:1:123121
Response
{
  "entity_id": "10000:1:1207046967",
  "name": "pashan-69-1",
  "entity_type": "VirtualMachine",
  "ip_addresses": [
    {
      "ip_address": "192.168.24.40",
      "netmask": "255.255.255.0",
      "network_address": "192.168.24.0/24"
    }
  ],
  "default_gateway": "192.168.24.10",
  "vnics": [
    {
      "entity_id": "10000:18:928014219",
      "entity_type": "Vnic"
    }
  ],
  "security_groups": [
    {
      "entity_id": "10000:82:1509031646",
      "entity_type": "NSXSecurityGroup"
    }
  ],
  "source_firewall_rules": [
    {
      "rules": [
        {
          "entity_id": "10000:87:1310503280",
          "entity_type": "NSXFirewallRule"
        },
        {
          "entity_id": "10000:87:1310503311",
          "entity_type": "NSXFirewallRule"
        }
      ]
    },
    {
      "entity_id": "10000:39:306795770",
      "entity_type": "NSXDFirewall"
    }
  ],
  "rule_set_type": "NSX_STANDARD"
}
```

```

],
"destination_firewall_rules": [
  {
    "rules": [
      {
        "entity_id": "10000:87:1310621266",
        "entity_type": "NSXFirewallRule"
      },
      {
        "entity_id": "10000:87:1310503280",
        "entity_type": "NSXFirewallRule"
      }
    ],
    "firewall": {
      "entity_id": "10000:39:306795770",
      "entity_type": "NSXDistributedFirewall"
    },
    "rule_set_type": "NSX_STANDARD"
  }
],
"ip_sets": [
  {
    "entity_id": "10000:84:1256619295",
    "entity_type": "NSXIPSet"
  }
],
"cluster": {
  "entity_id": "10000:66:850307201",
  "entity_type": "Cluster"
},
"resource_pool": {
  "entity_id": "10000:79:1730744919",
  "entity_type": "ResourcePool"
},
"security_tags": [
  {
    "entity_id": "10000:99:1129991034",
    "entity_type": "SecurityTag"
  }
],
"layer2_networks": [
  {
    "entity_id": "10000:11:604852557",
    "entity_type": "VxlanLayer2Network"
  }
],
"host": {
  "entity_id": "10000:4:2142603919",
  "entity_type": "Host"
},
"vlans": [],
"vendor_id": "vm-174",
"vcenter_manager": {

```

```

    "entity_id": "10000:8:2048038675",
    "entity_type": "VCenterManager"
  },
  "folders": [
    {
      "entity_id": "10000:81:1937753507",
      "entity_type": "Folder"
    }
  ],
  "datastores": [
    {
      "entity_id": "10000:80:2099650948",
      "entity_type": "Datastore"
    }
  ],
  "datacenter": {
    "entity_id": "10000:105:246393763",
    "entity_type": "VCDDatacenter"
  },
  "nsx_manager": {
    "entity_id": "10000:7:935317189",
    "entity_type": "NSXVManager"
  },
  "applied_to_source_rules": [
    {
      "rules": [
        {
          "entity_id": "10000:87:1310503280",
          "entity_type": "NSXFirewallRule"
        },
        {
          "entity_id": "10000:87:1310622351",
          "entity_type": "NSXFirewallRule"
        }
      ]
    },
    {
      "firewall": {
        "entity_id": "10000:39:306795770",
        "entity_type": "NSXDistributedFirewall"
      },
      "rule_set_type": "NSX_STANDARD"
    }
  ],
  "applied_to_destination_rules": [
    {
      "rules": [
        {
          "entity_id": "10000:87:1310621266",
          "entity_type": "NSXFirewallRule"
        },
        {
          "entity_id": "10000:87:1310503280",
          "entity_type": "NSXFirewallRule"
        }
      ]
    }
  ]
}

```

```

    ],
    "firewall": {
      "entity_id": "10000:39:306795770",
      "entity_type": "NSXDistributedFirewall"
    },
    "rule_set_type": "NSX_STANDARD"
  }
],
"source_inversion_rules": [],
"destination_inversion_rules": []
}

```

Retrieve an Entity at Specified Time

VCF Operations for networks stores different versions of an entity and creates an image of the entity with a new timestamp when the data changes. A previous version of an entity can be retrieved by passing the time as query parameter in the retrieve entity request.

Ensure to provide the time in epoch seconds.

Make a GET request.

```

GET https://operations-for-networks.example.com/api/ni/entities/vms/10000:1:875335999?
time=1508745500

```

Bulk Fetch of Entities

Bulk fetch of entities can be fetched using a bulk fetch API. The maximum batch size of bulk fetch is 1000 entities

To fetch bulk entities, make a POST request.

```

POST https://operations-for-networks.example.com/api/ni/entities/fetch

```

Request Body

```

{
  "entity_ids": [
    {
      "entity_id": "10000:4:132029350",
      "entity_type": "Host",
      "time": 1523605300
    },
    {
      "entity_id": "10000:4:132027614",
      "entity_type": "Host",
      "time": 1523605419
    }
  ]
}

```

Response Body

```

{
  "results": [
    {
      "entity_id": "10000:4:132029350",
      "entity_type": "Host",
      "entity": {

```

```

    "entity_id": "10000:4:132029350",
    "name": "10.197.52.176",
    "entity_type": "Host",
    "vmknics": [
      {
        "entity_id": "10000:17:771614149",
        "entity_type": "Vmknics"
      }
    ],
    "vcenter_manager": {
      "entity_id": "10000:8:824493708",
      "entity_type": "VCenterManager"
    },
    "vm_count": 4,
    "datastores": [
      {
        "entity_id": "10000:80:1910974952",
        "entity_type": "Datastore"
      },
      {
        "entity_id": "10000:80:1910975665",
        "entity_type": "Datastore"
      },
      {
        "entity_id": "10000:80:1910975696",
        "entity_type": "Datastore"
      },
      {
        "entity_id": "10000:80:889229135",
        "entity_type": "Datastore"
      },
      {
        "entity_id": "10000:80:1910974983",
        "entity_type": "Datastore"
      }
    ],
    "service_tag": "",
    "vendor_id": "host-127",
    "maintenance_mode": "NOTINMAINTENANCEMODE",
    "connection_state": "CONNECTED"
  },
  "time": 1523605300
},
{
  "entity_id": "10000:4:132027614",
  "entity_type": "Host",
  "entity": {
    "entity_id": "10000:4:132027614",
    "name": "10.197.52.178",
    "entity_type": "Host",
    "vmknics": [
      {
        "entity_id": "10000:17:1090151026",

```



```

        "entity_type": "Vmknics"
    },
    ],
    "vcenter_manager": {
        "entity_id": "10000:8:824493708",
        "entity_type": "VCenterManager"
    },
    "vm_count": 3,
    "datastores": [
        {
            "entity_id": "10000:80:1910974952",
            "entity_type": "Datastore"
        },
        {
            "entity_id": "10000:80:889230871",
            "entity_type": "Datastore"
        },
        {
            "entity_id": "10000:80:1910975665",
            "entity_type": "Datastore"
        },
        {
            "entity_id": "10000:80:1910975696",
            "entity_type": "Datastore"
        },
        {
            "entity_id": "10000:80:1910974983",
            "entity_type": "Datastore"
        }
    ],
    "service_tag": "",
    "vendor_id": "host-141",
    "maintenance_mode": "NOTINMAINTENANCEMODE",
    "connection_state": "CONNECTED"
},
"time": 1523605419
}
]
}

```

Creating Applications and Tiers

Operations for Networks provides APIs to create Applications and Tiers.

A Tier definition is based on membership criteria that can either be search based or IP address/subnet based. These applications and tiers can be used to fetch the recommended rules based on the traffic flow recorded by VCF Operations for networks from the environment.

Create an Application

Ensure you have the application's name. It must be unique and must not conflict with any existing application in the system.

To create the application, make a POST request.

```
Request
POST https://operations-for-networks.example.com/api/ni/groups/applications
Request Body
{
  "name" : "App-1",
}
Response Body
{
  "entity_id": "18230:561:271275765",
  "name": "App-1",
  "entity_type": "Application",
  "create_time": 1509410056733,
  "created_by": "admin@local",
  "last_modified_time": 0,
  "last_modified_by": ""
}
```

Create a Tier Using Search Membership Criteria

Ensure that you have the entity ID of the application. Name of the tier must be unique.

To create the tier, make a POST request.

```
Request
POST https://operations-for-networks.example.com/api/ni/groups/applications/18230:561:271275765/tiers
Request Body
{
  "name": "tier-1",
  "group_membership_criteria" : [
    {
      "membership_type": "SearchMembershipCriteria",
      "search_membership_criteria": {
        "entity_type" : "VirtualMachine",
        "filter": "security_groups.entity_id = '18230:82:604573173'"
      }
    }
  ]
}
Response Body
{
  "entity_id": "18230:562:1266458745",
  "name": "tier-1",
  "entity_type": "Tier",
  "group_membership_criteria": [
    {
      "membership_type": "SearchMembershipCriteria",
      "search_membership_criteria": {
        "entity_type": "VirtualMachine",
        "filter": "security_groups.entity_id = '18230:82:604573173'"
      }
    }
  ]
},
```

```

    "application": {
      "entity_id": "18230:561:271275765",
      "entity_type": "Application"
    }
  }
}

```

Create a Tier Using IP Address Membership Criteria

Ensure the application's entity ID is available. The tier name must be unique

To create the tier, make a POST request.

Request

POST <https://operations-for-networks.example.com/api/ni/groups/applications/18230:561:271275765/tiers>

Request Body

```

{
  "name": "app-1-tier-2",
  "group_membership_criteria": [
    {
      "membership_type": "IPAddressMembershipCriteria",
      "ip_address_membership_criteria": {
        "ip_addresses": [
          "11.122.2.212",
          "11.122.1.0/24"
        ]
      }
    }
  ]
}

```

Response Body

```

{
  "name": "app-1-tier-2",
  "group_membership_criteria": [
    {
      "membership_type": "IPAddressMembershipCriteria",
      "ip_address_membership_criteria": {
        "ip_addresses": [
          "11.122.2.212",
          "11.122.1.0/24"
        ]
      }
    }
  ],
  "application": {
    "entity_id": "18230:561:271275765",
    "entity_type": "Application"
  }
}

```

Working with Streaming Databus

VCF Operations for networks Databus is a framework used to share the data stored in VCF Operations for networks with the other applications. You can share high volumes of data with low latency over the Databus, and the data will

be delivered in near real-time. The data published over the Databus is divided into different message groups where a subscriber can subscribe to the message groups and receive the data over HTTP or HTTPs endpoints. The supported message groups are applications and problems.

Subscribe

A subscribe is an endpoint that registers with the Databus to receive the data from the message group.

To subscribe to a message group, make a POST request.

```
Request:
POST https://operations-for-networks.example.com/api/ni/settings/databus/subscribers
Content-type : application/json
Accept : application/json
Authorization : NetworkInsight {auth-token}
Body:
{
  "id": "9c488c91-692e-4992-a83f-ee33a0f9a6c6",
  "message_group": "problems",
  "url": "https://data-pipeline.vmware.com/operations-for-networks/events"
}

Response:
{
  "id": "9c488c91-692e-4992-a83f-ee33a0f9a6c6",
  "message_group": "problems",
  "url": "https://data-pipeline.vmware.com/operations-for-networks/events"
}
```

Use cases of the data received are:

- Send alerts to: Slack, ServiceNow, PagerDuty, and any other supported platforms.
- Based off alert Allow traffic by default firewall rule: Trigger an NSX firewall rule change that changes the default rule to deny.
- Update CMDB based off application updates in VCF Operations for networks (flow-based app discovery).
- Fix a VLAN trunk mismatch on 2 switch ports.
- Disable alerts for vSphere hosts or NSX-T Transport nodes in maintenance mode.

Generating the Recommended Firewall Rules

VCF Operations for networks provides APIs to generate the recommended firewall rules based on the flow data.

Recommended firewall rules API provides the service to retrieve recommended rules based on flow traffic that is observed between two groups or for a single group based on all the inbound and outbound traffic for that group. If two groups are provided, both groups must be of the same type. Groups that are currently supported include Application, Tier, NSXSecurityGroup, and EC2SecurityGroup. You can provide *time_range* to determine the flow traffic that is considered for the recommended rules computation. If *ime_range* is not provided, flow traffic for the last 24 hours is considered.

Retrieve the Recommended Rules for an Application

Verify that you have the entity ID of the application. Use the search service to determine retrieve recommended the application entity ID from application name.

To determine the recommended rules for the application, make a POST request with the entity ID of the application.

In the following sample request, rules are computed for the application with entity ID 10000:561:1663604768

Request

POST https://operations-for-networks.example.com/api/ni/micro-seg/recommended-rules

Request body:

```
{
  "group_1": {
    "entity": {
      "entity_type": "Application",
      "entity_id": "10000:561:1663604768"
    }
  },
  "time_range": {
    "start_time": 1508993971275,
    "end_time": 1509080371275
  }
}
```

Response body:

```
{
  "recommended_rules": [
    {
      "sources": [
        {
          "entity_id": "10000:562:1904698621",
          "entity_type": "Tier"
        }
      ],
      "destinations": [
        {
          "entity_id": "10000:562:1780351215",
          "entity_type": "Tier"
        }
      ],
      "protocols": [
        "UDP"
      ],
      "port_ranges": [
        {
          "start": 53,
          "end": 53
        },
        {
          "start": 1025,
          "end": 1025
        }
      ],
      "action": "ALLOW"
    },
    ...
  ],
  "time_range": {
    "start_time": 1508993971275,
    "end_time": 1509080371275
  }
}
```

```
}

```

Retrieve the Recommended Rules Between Two Tiers

Ensure that you have the entity ID of Tier1 and Tier2.

To determine the recommended rules, make a POST request with the entityID of the tiers.

In the following sample request, rules are computed for the tier with entity ID 10000:562:190469862 and tier with entity ID 10000:562:178035121.

Request

POST https://operations-for-networks.example.com/api/ni/micro-seg/recommended-rules

Request body:

```
{
  "group_1": {
    "entity": {
      "entity_type": "Tier",
      "entity_id": "10000:562:1904698621"
    }
  },
  "group_2": {
    "entity": {
      "entity_type": "Tier",
      "entity_id": "10000:562:1780351215"
    }
  }
}
```

Response body:

```
{
  "recommended_rules": [
    {
      "sources": [
        {
          "entity_id": "10000:562:1904698621",
          "entity_type": "Tier"
        }
      ],
      "destinations": [
        {
          "entity_id": "10000:562:1780351215",
          "entity_type": "Tier"
        }
      ],
      "protocols": [
        "UDP"
      ],
      "port_ranges": [
        {
          "start": 53,
          "end": 53
        },
        {

```

```

        "start": 1025,
        "end": 1025
    }
],
    "action": "ALLOW"
}
],
    "time_range": {
        "start_time": 1508996919391,
        "end_time": 1509083319391
    }
}

```

Export the Recommended Rules for an Application in the NSX Compatible Format

You can also export the recommended firewall rules and security groups in NSX compatible format as a ZIP file using this API.

Ensure that you have the entity ID of the application. Use the search service to fetch the application ID from the name.

To determine the recommended rules for the application, make a POST request using the application entity ID.

```

Request
POST https://operations-for-networks.example.com/api/ni/micro-seg/recommended-rule/nsx
Request Header
Accept-Type: "application/octet-stream"
Request Body:
{
    "group_1": {
        "entity": {
            "entity_type": "Application",
            "entity_id": "10000:561:1663604768"
        }
    },
    "time_range": {
        "start_time": 1508993971275,
        "end_time": 1509080371275
    }
}

Response:
Zip File containing the NSX artifacts.

```

Exporting Recommended Rules Between Two Tiers in NSX Compatible Format

Ensure that you have the entity ID of Tier1 and Tier2

To determine the recommended rules, make a POST request

```

Request
POST https://operations-for-networks.example.com/api/ni/micro-seg/recommended-rule/nsx
Request Header
Accept-Type: "application/octet-stream"
Request Body:
{

```

```

"group_1": {
  "entity": {
    "entity_type": "Tier",
    "entity_id": "10000:562:1904698621"
  }
},
"group_2": {
  "entity": {
    "entity_type": "Tier",
    "entity_id": "10000:562:1780351215"
  }
}
}
Response:
Zip File containing the NSX artifacts.

```

Fetching Metrics

Metrics can be fetched for an entity based using this API. Metrics schema APIs give the available metrics and their intervals for an entity type.

Retrieve Metrics Schema for an Entity Type

Verify that you have the entity type for which metrics are required.

To determine the metrics info for an entity type, make a GET request with the entity type of the application as a path parameter.

In the following sample request, metrics info for a FLOW entity type is retrieved.

```

Request
GET https://operations-for-networks.example.com/api/ni/schema/Flow/metrics
Response body:
{
  "results": [
    {
      "metric": "flow.dstBytes.delta.summation.bytes",
      "display_name": "Destination Bytes",
      "intervals": [
        1800,
        7200,
        28800
      ],
      "description": "Total bytes sent by the server to client",
      "unit": "NoUnit"
    },
    {
      "metric": "flow.srcBytes.delta.summation.bytes",
      "display_name": "Source Bytes",
      "intervals": [
        1800,
        7200,
        28800
      ],
      "description": "Total bytes sent by the client to server",
    }
  ]
}

```



```

        "unit": "NoUnit"
    },
    {
        "metric": "flow.totalBytes.delta.summation.bytes",
        "display_name": "Bytes",
        "intervals": [
            1800,
            7200,
            28800
        ],
        "description": "Total bytes transferred in both directions",
        "unit": "NoUnit"
    },
    {
        "metric": "flow.totalBytesRate.rate.average.bitsPerSecond",
        "display_name": "Bytes Rate",
        "intervals": [
            1800,
            7200,
            28800
        ],
        "description": "Average Bits/sec rate of total traffic in both directions",
        "unit": "NoUnit"
    },
    {
        "metric": "flow.totalPackets.delta.summation.number",
        "display_name": "Packets",
        "intervals": [
            1800,
            7200,
            28800
        ],
        "description": "Total packets transferred in both directions",
        "unit": "NoUnit"
    },
    {
        "metric": "flow.allowedSessionCount.delta.summation.number",
        "display_name": "Session Count",
        "intervals": [
            1800,
            7200,
            28800
        ],
        "description": "Total number of allowed sessions",
        "unit": "NoUnit"
    }
]
}

```

Retrieve Metrics Points for an Entity

Maximum number of metrics point returned by API is 300 and a 400 response is returned in case the interval and time period combination have more than 300 metrics points. The client can break the time period to multiple batches to get all the metrics points

Verify that you have the entity id, metrics name, interval, start, and end time.

To fetch metrics points for an entity id and metric for a given time interval , make a GET request as shown in the following example.

Request

```
GET https://operations-for-networks.example.com/api/ni/metrics/v2?entity_id=12347:515:1787493997
&metric=flow.srcBytes.delta.summation.bytes&interval=1800&start=1523351589&end=1523437989
```

Response body:

```
{
  "metric": "flow.srcBytes.delta.summation.bytes",
  "display_name": "Source Bytes",
  "interval": 1800,
  "unit": "NoUnit",
  "pointlist": [
    [
      1523352600,
      7141
    ],
    [
      1523354400,
      6898
    ],
    [
      1523356200,
      7370
    ],
    [
      1523358000,
      6898
    ]
  ],
  "start": 1523351589,
  "end": 1523437989
}
```

Get Proxy Node Details

These APIs are used to fetch all proxy nodes details. Proxy node id is required for adding data source.

Get Proxy Nodes List

To fetch a list of proxy nodes, make a GET request as shown below.

```
GET https://operations-for-networks.example.com/api/ni/infra/nodes
{
  "results": [
    {
      "id": "10000:901:1575899868",

```

```

    "entity_type": "NODE"
  }
],
"total_count": 1
}

```

Get Proxy Node Details

To fetch proxy nodes details, make a GET request as shown below.

```

GET https://operations-for-networks.example.com/api/ni/infra/nodes/10000:901:1575899868
Response
{
  "id": "10000:901:1575899868",
  "entity_type": "NODE",
  "node_type": "PROXY_VM",
  "node_id": "IPE97DK",
  "ip_address": "10.153.190.94"
}

```

Get Version Info

This API returns API version.

Get API Version Info

```

Request
GET https://operations-for-networks.example.com/api/ni/info/version
Response
{
  "api_version": "9.0.0"
}

```

VMware Cloud Foundation Automation APIs for VM Apps Programming Guide

As a VMware Cloud Foundation Automation® (formerly VMware Aria Automation) user or customer, you can perform VCF Automation functions programmatically by using REST API service calls.

API Services for VM Apps

VCF Automation includes the following APIs for the VM Apps Org. API documentation is available with the product. To access all Swagger specifications from a single landing page, log in as admin to the VCF Automation for VM Apps Organization tenant. At the top-right corner of the home page, click the user name and open the **API Help Center**. Then click **Explore Automation APIs**.

API documentation is also available from the Broadcom Developer Portal at <https://developer.broadcom.com/xapis/org-management-vm-apps-org/latest/>

Note:

- Automation scripts that authenticate against new VM Apps organizations should use only VCF Automation Provider Management Access Control APIs, and not CSP APIs.

Service Name	Description
ABX	Create or manage actions and their versions. Execute actions and flows.
Access Control	Supports identity operations for VCF Automation, enabling management of groups and users.
Approvals	Enforce policies that control required approvals for a deployment or Day 2 action before the request is provisioned.
Blueprint	Create, validate, and provision blueprints.
Catalog	Access catalog items and catalog sources, including content sharing and the request of catalog items.
Content Gateway	Connect to your infrastructure as code content in external content sources such as Source Code Management providers.
Custom Forms	Define dynamic form rendering in Catalog and VCF Automation services.
Deployment	Access deployment objects and platforms or blueprints that have been deployed into the system.
Deployment metric	Aggregated metric values for the deployment objects.
Event Broker	View audit log, events, event topics, and extensibility subscriptions.
Identity	Authenticate and manage the authorization provider.
Onboarding	Define policies and plans to bring existing VMs from any cloud under management.
Orchestrator Gateway	Run workflows and actions to automate complex IT tasks.
Policies	Interact with policies created in Catalog.
Projects	Provide visibility and isolation of provisioned resources for users with a project role.
Provisioning Service	Perform infrastructure setup tasks, including validation and provisioning of resources in an iterative manner.

API Deprecations and Changes

CSP APIs are deprecated. For a complete list of Identity API changes, log in as admin to the VCF Automation for VM Apps Organization tenant. At the top-right corner of the home page, click the user name and open the **API Help Center**. At the left, click **API Transition** to see all the changes to the Identity Service functionality that may affect your existing workflows.

As of VCF Automation 9.1, the following APIs have changed:

- The CMX API is no longer available because the CMX service has been removed. Formerly CMX service was responsible for Kubernetes integration for deployment and management of Kubernetes clusters and namespaces. In VCF Automation, these capabilities have been moved from VM Apps to All Apps organization types.
- The Pipelines API behind the `/codestream/api/*` endpoint are not available because the feature has been discontinued.
- The Event Broker API behind the `/event-broker/api/*` endpoint is public.
- VCF Automation tenant branding is now controlled from the VCFA Provider Management portal and Provider Management APIs. See the VCF Automation Branding Theme API.
- The Migration Assessment and Migration Service API responsible for orchestrating migrations from vRealize Automation 7.x to Aria Automation 8 has been removed.

VKS integration is available through CCI in the All Apps organization. See [VCF Automation and All Apps API](#).

The following API endpoints available in Aria Automation have been removed in VCF Automation :

- **Catalog Service**
 - /policy/api/policies/limit-usage
 - /catalog/api/items/{id}/data/{dataElementType}/{dataElementId}
 - /price/api/cloudhealth/perspective-sync
 - /catalog/api/about /catalog/api/admin/entitlements
 - /catalog/api/admin/sources/limit-usage
 - /catalog/api/admin/entitlements/{id}
- **Infrastructure as a Service**
 - /iaas/api/cloud-accounts-nsx-v
 - /iaas/api/cloud-accounts-avilb/{id}
 - /iaas/api/cloud-accounts-nsx-v/{id}
 - /iaas/api/cloud-accounts-avilb
- **Approval Service**
 - /core/authn/logout /access-token /admin/api/upgrade

How developers use the VCF Automation APIs

To make API service calls, you use a browser application or an HTTP client application to send requests and review responses. The following open-source applications are commonly used:

- cURL. <https://curl.se/>
- jq parser. <https://jqlang.org/>
- Postman application. <https://www.postman.com/>
- API Client Bruno. <https://www.usebruno.com/>

To learn how to use the API, you start by getting an authentication token. Then you can perform steps outlined in the use cases in this guide. The use cases include `curl` commands in request examples. To use the commands, ensure that the `jq` command-line JSON processor is installed with `curl`. The `jq` parser ensures that responses are formatted for optimum readability. For information about `jq` installation, see <https://jqlang.org/download/>.

Getting Your Authentication Token

In the REST API, VCF Automation requires an HTTP authentication token in the Authorization request header. Getting your authentication token is a prerequisite for any use case.

You use an API bearer token to authenticate all API calls. The API bearer token is known as the access token in the REST API service call examples. To acquire an authentication token, you must:

- Log in to the VCF Automation Provider Management Portal to find the name of the VM Apps organization.
- Log in to the VM Apps tenant to get the API token for the organization. The API token for the organization is the refresh token that is used to get the access token.
- Use the API token to get an access token either by:
 - Using an API client such as Postman to create an environment for the VM Apps tenant and get an access token that expires in one hour by default.
 - Registering an OAuth Client, then fetching the API token of the registered client to set an expiration on the access token.

The default lifespan of an API token is 129600 minutes or 90 days. The default lifespan of an access token is one hour. If you want to change the default lifespan of the API tokens you create, review the information in [Changing the Lifetime Setting of an API Token](#) before obtaining your refresh token.

Get a Refresh Token for the VM Apps Tenant

To get an API token for the VCF Automation for VM Apps API, you must authenticate against the VM Apps tenant. To get the name of the organization for the VM Apps tenant, you use the VCF Automation Provider Management Portal. Then you log in to the VM Apps tenant to get the API token to be used as a refresh token.

Important:

As of VCF 9.0, the Identity Service API functionality has changed and is no longer used to obtain a refresh token programmatically. To get an API token for the VM Apps organization, you use the VCF Automation UI to authenticate against the VM Apps tenant.

Prerequisites

- Secure a channel between the web browser and the VCF Automation server. Open a browser and enter the URL such as: `https://appliance.domain.com`.
- Verify that you are an authenticated user with the Assembler Administrator or Service Broker Administrator role in the VM Apps organization. See [What are the VCF Automation for VM Apps user roles](#).

Get the Name of the Organization for the VM Apps Tenant

Log in to the Provider Management Portal and perform the following steps to get the name of the organization for the VM Apps tenant.

1. Log in to the Provider Management Portal with the username: `admin` and the password that you used to upgrade from VMware Aria Automation 8.18.1.

- Click **Organizations** and choose the name of your VM Apps classic tenant.

VMware Cloud Foundation
Automation | Provider Management

Organizations

CREATE ORGANIZATION

Display Name	State
e2eClassicOrg VM Apps	Enabled
e2eClassicOrg	Enabled
e2eClassicOrg	Enabled
Kevin-Classic	Enabled
Kevin-Classic	Enabled
Kevin-Classic	Enabled
vcf-e2e-vidm Classic Ten. VM Apps	Enabled

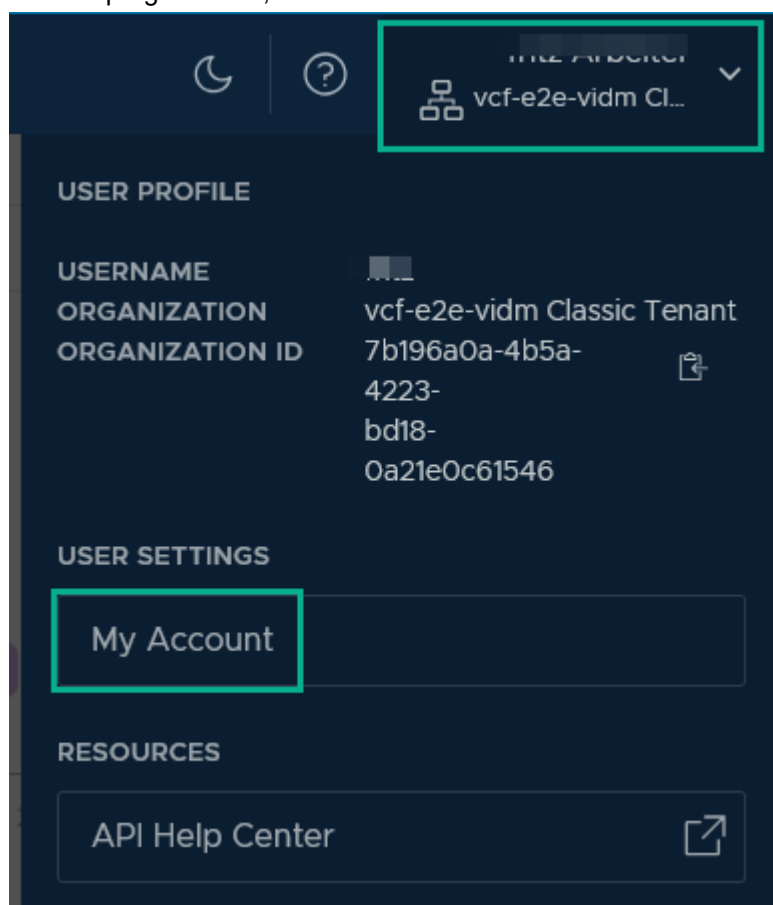
- When the **Overview** window opens, copy and save:
 - VM Apps tenant name that is displayed as the Name, such as `vcf-e2e-vidm`.
 - VCFA hostname or `VCFA_FQDN` that is displayed in the Default organization URL.

Get the API Token for the Organization

Perform the following steps to get the API token for the organization, also known as the refresh token.

- Open the VCF Automation UI, and change the organization to the name of the VM Apps tenant, such as `vcf-e2e-vidm`.
- Log in with your tenant administrator username and password.

3. In the top-right corner, click the name of the tenant and then click the **My Account** link under User Settings.

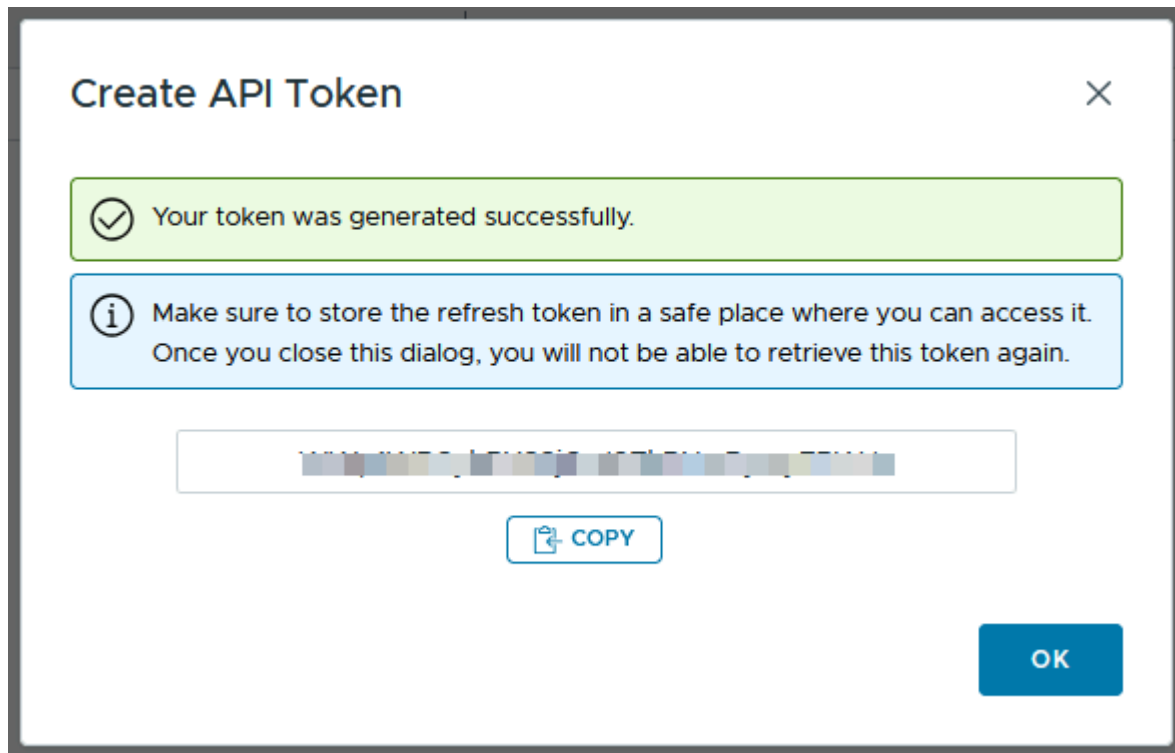


4. On the My Account page, click the API Tokens tab, then click **New**.
5. On the Create API Token page that appears, enter a name for the token and click **Create**.

- Click **Copy** and save the API token.

Important:

You need this token to obtain your access token.



Get Your Access Token for the VCF Automation API

You use an access token to authenticate your session for a VMware Aria Automation instance that has been upgraded to VCF Automation. To get the access token, you use the API token that you created for the VM Apps tenant organization.

Verify that you have your:

- VCF Automation hostname from the VM Apps organization in the Provider Management Portal.
- VCF Automation tenant name from the VM Apps organization in the Provider Management Portal, such as `vcf-e2e-vidm`.
- API token for the VM Apps tenant organization, also known as the refresh token.

See [Get a Refresh Token for the VM Apps Tenant](#).

Note:

To get a token for a newly deployed instance of VCF Automation that does not include a VM Apps tenant, use the same method that is used for VCF Automation for All Apps. See [Generating API Tokens for Service Provider Account](#).

The following procedure shows how to obtain the access token using the Bruno API Client. The access token expires after one hour. To obtain an access token rather than through an API client, see [Getting Your Access Token for the VCF Automation VM Apps API Programmatically](#).

- Create and save Global Environment for your VCF Automation VM Apps tenant with the following parameters:

```
vcfaHostname: {VCFA_FQDN}
vcfaTenant: {VM_app_tenant_name}
```


Prerequisites

Verify that you have your:

- VCF Automation tenant name from the VM Apps organization in Provider Management Portal, such as `vcf-e2e-vidm`.
- API token for the VM Apps tenant organization, also known as the refresh token.

See [Get a Refresh Token for the VM Apps Tenant](#).

Register a New Client API Token

Use the refresh token to obtain the ID of the registered client, where:

```
• org_name: {VM_app_tenant_name}
• access_token: {API_token_for_VMApps_org}

POST /oauth/tenant/{org_name}/register
Authorization: Bearer {{access_token}}
Content-Type: application/json

# Body
{"client_name":"my_client"}

# Response
{
  "client_name": "my_client",
  "client_id": "0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a",
  "grant_types": [
    "urn:ietf:params:oauth:grant-type:jwt-bearer",
    "refresh_token"
  ],
  "token_endpoint_auth_method": "none",
  "client_uri": null,
  "software_id": null,
  "software_version": null,
  "scope": null
}
```

Fetch the API Token of a Registered Client

Use the ID of the registered client to fetch the API bearer token. The new API token in the "refresh_token" field is the API bearer token that is required to access tenant-specific APIs.

```
POST /oauth/tenant/org/token
Authorization: Bearer {{access_token}}
Content-Type: application/x-www-form-urlencoded

# Body
# take the client id from the previous response
grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
&client_id=0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a
&assertion={{access_token}}

# Response
# The new API token will be contained in the refresh_token field
```

```
{
  "access_token": "new_access_token",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "api_token"
}
```

Changing the Lifetime Setting of an API Token

The lifespan of an API token is defined by the API token lifetime setting when you create the token and cannot be changed retroactively. As a best practice, you should change the API token lifetime before creating your token so that you control its expiration period. Or if you created tokens before changing the API token lifetime, you must revoke the old tokens before creating tokens again with the changed lifetime setting.

Change the Lifetime Setting

To change the lifespan of the API Token for your organization, log in to the Provider Management Portal, navigate to **General Settings > Timeouts** and enter a value for the API token lifetime. Change this setting before getting your refresh token. See [Get a Refresh Token for the VM Apps Tenant](#).

The default lifespan of the API token is 129600 minutes or 90 days. To change the API token lifetime:

- Enter 0 for an unlimited lifespan.
- Enter an integer value for the lifetime number of minutes.


VMware Cloud Foundation
Automation | Provider Management

<<

Infrastructure

Organizations

Networking

External IP Blocks

External Connections

Edge/VNA Clusters

Subnets

Regions

Supervisors

Infrastructure Policies

Content Libraries

Service Management

VCF Services

Data Services

Migration Tool

Encryption

GitOps

Administration

General Settings

Connections

VCF Instances

Access Control

General Settings

Configure global system parameters, including activity logging retention, networking IP blocks, and timeouts.

Activity Log

EDIT

Log history to keep	90 Days
Log history shown	30 Days
Display debug information	Disabled

Networking

EDIT

Default Private TGW IP Block	172.31.0.0/16
Default Private VPC IP Block	172.30.0.0/16

Timeouts

EDIT

API token lifetime	0 Minutes
Host refresh frequency	300 Seconds
Host hung timeout	30 Seconds
Transfer session timeout	10 Minutes

Organization Limits

EDIT

Maximum number of Region Quotas	Unlimited
---------------------------------	-----------

Revoke an Existing Token

To change the lifespan of an API token that was previously created, you must revoke it and create it again with the changed lifetime setting. If you have multiple tokens, you revoke each token separately. You can revoke tokens either programmatically using the API or from the UI by logging in to the VM Apps or All Apps tenant.

In the following example, the API request deletes the refresh token `0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a`. If you have multiple refresh tokens, insert each token in the command, and run the delete API request again.

```
DELETE /cloudapi/1.0.0/tokens/urn:vcloud:token:0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a
```

Verify User Roles

To use the API, aVCF Automation user must be an organization member with at least a user service role. You use the access token to verify user roles.

Verify that you have an access token. See [Get Your Access Token for the VCF Automation API](#).

1. Assign values to the variables for the hostname of your VCF Automation instance, your user name, and your password.

```
url='http://<FQDN>'
username='<your_username>'
password='<your_password>'
```

2. Get your organization ID.

```
curl -X GET \
"$url/csp/gateway/am/api/loggedin/user/orgs" \
-H "csp-auth-token: $access_token"
```

3. Examine the response and assign the organization ID variable.

```
org_id='<your_org_id>'
```

4. Get your organization role.

```
curl -X GET \
"$url/csp/gateway/am/api/loggedin/user/orgs/$org_id/roles" \
-H "csp-auth-token: $access_token" | jq "."
```

The name field displays the organization role and must be either `org_owner` or `org_member`.

5. Get your service role.

```
curl -X GET \
"$url/csp/gateway/am/api/loggedin/user/orgs/$org_id/service-roles" \
-H "csp-auth-token: $access_token" | jq "."
```

The `serviceRolesNames` field displays the service role for each service and must be at least `user`.

Verify User Roles

Using the access token previously obtained and assigned, verify user roles. See [Get Your Access Token for the VCF Automation API](#).

Assign variables.

```
# url='https://appliance.company.com'
# username='user@example.local'
# password='example_password'
```

Get your organization ID.

```
# curl -X GET \
"$url/csp/gateway/am/api/loggedin/user/orgs" \
-H "csp-auth-token: $access_token"
```

The response shows the organization ID.

```
{
  "refLinks": [
    "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30"
  ],
}
```

```

"items": [
  {
    "name": "DEFAULT-ORG",
    "displayName": "DEFAULT-ORG",
    "refLink": "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30",
    "id": "7f8c518a-65f5-494b-b714-f7e349957a30",
    "metadata": null,
    "parentRefLink": null
  }
]
}

```

Assign the organization ID variable.

```
# org_id='7f8c518a-65f5-494b-b714-f7e349957a30'
```

Verify the organization role.

```
# curl -X GET \
  $url/csp/gateway/am/api/loggedin/user/orgs/$org_id/roles \
  -H "csp-auth-token: $access_token" | jq "."
```

The response shows that the organization role is `org_owner`.

```

{
  "refLink": "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30/
roles/52a6a411-2339-4bc3-91bc-62418977df11",
  "name": "org_owner",
  "displayName": "Organization Owner",
  "organizationLink": "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30"
}

```

Verify the service role.

```
# curl -X GET \
  $url/csp/gateway/am/api/loggedin/user/orgs/$org_id/service-roles \
  -H "csp-auth-token: $access_token" | jq "."
```

A snippet of the response shows the Service Role Names for the VCF Automation service. `cloud_admin` satisfies the minimum service role.

```

...
{
  "serviceDefinitionLink": "/csp/gateway/slc/api/definitions/external/<service_id>",
  "serviceRoleNames": [
    "automationService:cloud_admin"
  ]
}
...

```

Prerequisites for API Use Case Examples

Before performing any task using API services, you must review the prerequisites. General prerequisites apply to all services. Prerequisites that are service-specific are common to all endpoints for the service, but may vary depending upon the service role for example, administrator or user.

General prerequisites for all services

Before performing any task for any service, the following prerequisites must be satisfied:

- Verify that you have an active access token. See [Getting Your Authentication Token](#).
- Verify that the URL variable is assigned.

```
url='https://appliance.domain.com'
```

Prerequisites specific to API services

The following table lists prerequisites that are specific to the services with use cases in this guide.

Note: Every service includes an API version variable. If you choose not to assign a value to the *apiVersion* or to assign it to a different value, review the information in <https://techdocs.broadcom.com/us/en/vmware-cis/aria/aria-automation/8-18/aria-automation-api-programming-guide-on-prem-8-18.html>.

Table 1903: Prerequisites for VCF Automation Service Use Cases

Product: Service	Service prerequisites	Role-specific prerequisites
VCF Automation: Blueprint service To access the Swagger documentation for the Blueprint API, see <a href="https://<your_FQDN>/blueprint/api/swagger/ui/">https://<your_FQDN>/blueprint/api/swagger/ui/ .	Verify that the Blueprint API version variable is assigned as in the following example. <code>api_version='2019-09-12 '</code>	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
VCF Automation: Catalog service To access the Swagger documentation for the Catalog API, see <a href="https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=catalog">https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=catalog .	Verify that the Catalog API version variable is assigned as in the following example. <code>api_version='2020-08-25 '</code>	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
VCF Automation: Deployment service To access the Swagger documentation for the Deployment API, see <a href="https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=deployments">https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=deployments . Note: For the VCF Automation: Deployment service, use the prerequisites for the VCF Automation: Infrastructure as a Service (IaaS) service.	Verify that the Deployment API version variable is assigned as in the following example. <code>api_version='2020-08-25 '</code>	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
VCF Automation: Infrastructure as a Service (IaaS) service To access the Swagger documentation for the IaaS API, see <a href="https://<your_FQDN>/iaas/api/swagger/ui/">https://<your_FQDN>/iaas/api/swagger/ui/ .	Verify that the IaaS API version variable is assigned as in the following example. <code>api_version=' 2021-07-15'</code> If you do not assign a value to the <i>apiVersion</i> variable or you assign it to a value other than 2021-07-15, the IaaS API behavior will default to value of the previous version or 2019-01-15.	• Verify that you are an organization owner in VCF Automation with a VCF Automation administrator service role. • If working with cloud accounts, verify that you have cloud administrator credentials. .
All products: Onboarding service To access the Swagger documentation for the Onboarding API, see <a href="https://<your_FQDN>/relocation/api/swagger/swagger-ui.html">https://<your_FQDN>/relocation/api/swagger/swagger-ui.html	No version variable needed	Verify that you are at least an organization member in VCF Automation with an VCF Automation administrator service role.

Product: Service	Service prerequisites	Role-specific prerequisites
VCF Automation: Pipelines service To access the Swagger documentation for the Pipelines API, see <a href="https://<your_FQDN>/pipeline/api/swagger/swagger-ui.html">https://<your_FQDN>/pipeline/api/swagger/swagger-ui.html .	Verify that the Pipelines API version variable is assigned as in the following example. <code>api_version='2019-10-17 '</code>	Verify that you are at least an organization member in VCF Automation with an VCF Automation administrator service role.
VCF Automation: Policies service To access the Swagger documentation for the Policies API, see <a href="https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=policies">https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=policies .	Verify that the Policies API version is assigned as in the following example. <code>api_version='2020-08-25 '</code>	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
All products: Project service To access the Swagger documentation for the Projects API, see <a href="https://<your_FQDN>/project/api/swagger/swagger-ui.html">https://<your_FQDN>/project/api/swagger/swagger-ui.html .	Verify that the Projects API version variable is assigned as in the following example. <code>api_version='2019-01-15 '</code>	Verify that you are at least an organization member in VCF Automation with an administrator service role.

VCF Automation Tutorials

As a VCF Automation administrator, you can use these tutorials to learn how to perform tasks programmatically using the IaaS APIs.

In addition to the steps in the tutorials, there is additional information in the guide. Links are provided to relevant topics.

Working with tags

As an Organization administrator with the Assembler admin service role, you can use the Tags endpoints to create, manage, or delete tags in VCF Automation.

Note:

- Tag filters are case-insensitive.
- There is no way to query for unused tags using the IaaS API. However, you can submit a request to check for tag usage of specific tag IDs.

Prerequisites for working with tags

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

Filtering for tags

You can filter for tags based on a key, value, or origin and the tag filters are case-insensitive.

The following example shows how to filter for a tag with `location` in the key name.

```
curl -X GET \
  "$url/iaas/api/tags?filter=key%20eq%20'location'&apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" | jq "
```

A sample response shows `"key": "location"`.

```
{
  "content": [
    {
```

```

    "key": "location",
    "value": "MUM",
    "id": "9aabbbe23-d4bf-3d55-89f7-dc7e67317442"
  }
],
"totalElements": 1,
"numberOfElements": 1
}

```

To filter for the origin of a tag, you specify `USER_DEFINED` or `DISCOVERED` for the origin value. The following example filters for tags with the `DISCOVERED` origin.

```

curl -X GET \
"$url/iaas/api/tags?$filter=origins.item%20eq%20%27DISCOVERED%27&apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."

```

Tag origins can also be both `USER_DEFINED` and `DISCOVERED` as in the following cases:

- If a tag's origin is `USER_DEFINED` and another tag with the same case-sensitive key/value pair is assigned to an endpoint resource from which VCF Automation collects data, VCF Automation adds the `DISCOVERED` origin.
- If a user assigns a tag with the `DISCOVERED` origin to any VCF Automation resource such as a network profile, VCF Automation also assigns the `USER_DEFINED` origin to the tag.

Creating tags

You create a tag with a name and an optional value. The following example shows how to create a tag named `environment` with the value `prod` that you could add to resources in your production environment.

```

curl -X POST \
"$url/iaas/api/tags?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
  "key" : "environment",
  "value" : "prod"
}' | jq "."

```

The response provides the tag ID.

```

{
  "key": "environment",
  "value": "prod",
  "id": "ca0ed5ef-72b5-3d9f-bc6d-0707defff540"
}

```

Listing and deleting tags

To delete a tag, you specify the tag ID. To get the tag ID, you list all tags in your infrastructure.

```

curl -X GET \
"$url/iaas/api/tags?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
}' | jq "."

```

The response lists all the tag names, values, and IDs as in the following example.

```

{

```

```

"content": [
  {
    "key": "location",
    "value": "MUM",
    "id": "9aabb23-d4bf-3d55-89f7-dc7e67317442"
  }
  {
    "key": "Application",
    "value": "testing",
    "id": "57ead12-523a-3cd2-9447-06e757dcf382"
  }
],
"totalElements": 2,
"numberOfElements": 2
}

```

Before deleting a tag, a best practice is to check the tag usage of the ID that you plan to delete. The following example checks for usage of the tag with ID 9aabb23-d4bf-3d55-89f7-dc7e67317442.

```

curl -X POST \
"$url/iaas/api/tags/tags-usage?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "tagIds" : ["9aabb23-d4bf-3d55-89f7-dc7e67317442"]
}' | jq "."

```

- If the tag is being used, the response shows where it is being used. This sample response shows that the tag is being used in a network profile.

```

{
  "documentLinks": [
    "/provisioning/resources/network-profiles/8615bbfe-ea90-420a-a1cf-8048ba4271d4"
  ],
  "documents": {
    "/provisioning/resources/network-profiles/8615bbfe-ea90-420a-a1cf-8048ba4271d4": {
      "id": "a30929ec-3b06-484e-8785-cc43619b5b01",
      "name": "23360",
      "customProperties": {
        "edgeClusterRouterStateLink": "/resources/routers/2d57f1c4-fbe0-46ce-a061-3a5f1396f37b",
        "tier0LogicalRouterStateLink": "/resources/routers/9c965abe-9d69-40c0-99e0-be5aa7e52097",
        "onDemandNetworkIPAssignmentType": "mixed"
      },
      "tagLinks": [
        "/resources/tags/ef254637-594d-3960-a642-f157f859a830",
        "/resources/tags/9aabb23-d4bf-3d55-89f7-dc7e67317442"
      ],
      "expandedTags": [
        {
          "tag": "23360\n"
        },
        {
          "tag": "location\nMUM"
        }
      ]
    }
  }
}

```

```

    "documentVersion": 11,
    "documentKind": "com:vmware:admiral:compute:profile:NetworkProfileService:NetworkProfile",
    "documentSelfLink": "/provisioning/resources/network-profiles/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
    "documentUpdateTimeMicros": 1696947968305000,
    "documentExpirationTimeMicros": 0
  },
  "documentCount": 1,
  "documentVersion": 0,
  "documentUpdateTimeMicros": 0,
  "documentExpirationTimeMicros": 0}

```

- If the tag is not being used, the response shows no resource links.

```

{
  "documentLinks": [],
  "documents": {},
  "documentCount": 0,
  "documentVersion": 0,
  "documentUpdateTimeMicros": 0,
  "documentExpirationTimeMicros": 0
}

```

To delete the unused tag, specify the tag ID. The delete action first checks to ensure that the tag is not associated with a resource before deleting it. The following example deletes the tag with ID 9aabb23-d4bf-3d55-89f7-dc7e67317442.

```

curl -X DELETE \
"$url/iaas/api/tags/9aabb23-d4bf-3d55-89f7-dc7e67317442?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
}' | jq "."

```

Note: To delete a tag that is associated with a resource, you can use the `ignoreUsage` query parameter as in the following example that forces the deletion of a tag with ID 9aabb23-d4bf-3d55-89f7-dc7e67317442. However, this removes all tag assignments from all the resources associated with the tag.

```

curl -X DELETE \
"$url/iaas/api/tags/9aabb23-d4bf-3d55-89f7-dc7e67317442?ignoreUsage=true&apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."

```

Updating tags

You can use the IaaS API to add, remove, or update the tags associated with a resource. Using the ID of the resource, you submit a request get the details about the resource including the tag definitions. Then you submit a request to update the resource with new tag key/value pairs.

The following example shows how to update tags for a vSphere datastore

List all vSphere datastores.

```

curl -X GET \
"$url/iaas/api/fabric-vsphere-datastores?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."

```

Examine the response to find the ID of the datastore that you want to use and use the ID to get details about the datastore. The following request uses the datastore ID 002e0a62-846d-4fb2-a153-3dcd80e57ba9.

```
curl -X GET \
"$url/iaas/api/fabric-vsphere-datastores/002e0a62-846d-4fb2-a153-3dcd80e57ba9/?apiVersion=$api_version"
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response lists the tags on the datastore.

```
...
"tags": [
  {
    "key": "Team",
    "value": "E2E"
  },
  {
    "key": "Owner/Requestor",
    "value": "User1"
  },
  {
    "key": "Product",
    "value": "vRA"
  }
],
...
```

Update the tags for the datastore. The following example payload:

- Removes the Team and Owner/Requestor keys.
- Adds the environment key.
- Updates the Product key.

```
curl -X PATCH \
"$url/iaas/api/fabric-vsphere-datastore/002e0a62-846d-4fb2-a153-3dcd80e57ba9?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d {
"tags": [
  {
    "key": "environment",
    "value": "dev"
  },
  {
    "key": "Product",
    "value": "saltstack"
  }
],
}' | jq "."
```

Create a Kubernetes Zone with a Tag

As a VCF Automation admin, you can use APIs to create a new tag and add it to a Kubernetes zone.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

- Verify that you have the cloud account ID for a vSphere cloud account. See [Add avSphere Cloud Account](#).

In the following procedure, you use the IaaS API to create a new tag. Then using the CMX API, you create a Kubernetes zone with a supervisor namespace on vSphere and assign the tag to the zone.

Note:

To create a Kubernetes zone with a vSphere cloud account, the account must be Tanzu-enabled. When you open a Tanzu-enabled vSphere cloud account in the UI, the cloud account appears with the label **Available for Kubernetes deployment**.

1. Create a new tag with a key/value pair.

```
curl -X POST \
  $url/iaas/api/tags?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "key": "<your_tag_key>",
    "value": "<your_tag_value>"
  }' | jq "
```

The response includes a tag ID.

2. Assign the tag ID variable.

```
tag_id='<example-tagID-alphanumeric-string>'
```

3. Assign your vSphere cloud account ID to the cloud account ID variable.

```
cloud_account_id='<vsphere_cloud_account_ID>'
```

4. Use the cloud account ID to create the Kubernetes zone with the tag.

To associate the Kubernetes zone with a vSphere cloud account you specify "providerType": "VSPHERE_NAMESPACES".

```
curl -X POST \
  "$url/cmx/api/resources/k8s-zones" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your_k8s_zone_name>",
    "providerId": "$cloud_account_id",
    "providerType": "VSPHERE_NAMESPACES",
    "tagIds": [
      "$tag_id"
    ]
  }' | jq "
```

Create a Kubernetes Zone with a Tag

Create a new tag with key `example_tag_key` and value `example_tag_value`. Create a Kubernetes zone named `K8s-test-zone` with the tag. To create a Kubernetes zone, you must associate it with a cloud account configured for VCF Automation such as a vSphere cloud account with ID `8d4646bd-d629-4526-9009-10e3d4b66e44`.

Assign variables.

```
url='https://appliance.domain.com'
$ api_version=' '
```

Create a new tag with the key/value.

```
$ curl -X POST \
  "$url/iaas/api/tags?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "key": "example_tag_key",
    "value": "example_tag_value"
  }' | jq "
```

Examine the response for the tag ID.

```
{
  "key": "example_tag_key",
  "value": "example_tag_value",
  "id": "a6f324e4-101c-33f6-ac51-d9aaaa020123"
}
```

Assign a value to the tag ID variable.

```
$ tag_id1= 'a6f324e4-101c-33f6-ac51-d9aaaa020123'
```

Assign your vSphere cloud account ID to the cloud account ID variable.

```
$ cloud_account_id='8d4646bd-d629-4526-9009-10e3d4b66e44'
```

Create the Kubernetes zone with the tag and the cloud account ID.

```
$ curl -X POST \
  "$url/cmx/api/resources/k8s-zones" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "k8s-test-zone",
    "providerId": "'$cloud_account_id'",
    "providerType": "VSPHERE_NAMESPACES",
    "tagIds": [
      "'$tag_id1'"
    ]
  }' | jq "
```

The response shows the Kubernetes zone.

```
{
  "id": "220a81ba-c8ca-4f01-97bc-6f497576d564",
  "createdMillis": 1674847375758,
  "updatedMillis": 1674847375758,
  "orgId": "20451685-2bf6-4ba4-9ea4-87ee61bd32f8",
  "name": "k8s-test-zone",
  "providerId": "8d4646bd-d629-4526-9009-10e3d4b66e44",
  "providerType": "VSPHERE_NAMESPACES",
  "tagIds": [
    "a6f324e4-101c-33f6-ac51-d9aaaa020123"
  ]
}
```

If you want to update the Kubernetes zone, assign the Kubernetes zone ID variable.

```
$ k8s_zone_id='220a81ba-c8ca-4f01-97bc-6f497576d564'
```

If you want to add another tag, assign another tag ID variable.

```
$ tag_id2='21ccbdd6-c849-37f3-a784-1f2452cf802d'
```

Update the Kubernetes zone to add the tag.

Note: When updating the zone, the update deletes any tags that are not included in the request payload. So when making the request, you must include all the tags that will remain assigned to the zone after the update.

```
$ curl -X PUT \
  "$url/cm/api/resources/k8s-zones/$k8s_zone_id" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "id": "220a81ba-c8ca-4f01-97bc-6f497576d564",
    "createdMillis": 1674847375758,
    "updatedMillis": 1674847375758,
    "orgId": "20451685-2bf6-4ba4-9ea4-87ee61bd32f8",
    "name": "k8s-test-zone",
    "providerId": "'$cloud_account_id'",
    "providerType": "VSPHERE_NAMESPACES",
    "tagIds": [
      "'$tag_id1'",
      "'$tag_id2'"
    ]
  }' | jq "."
```

How do I retrieve provisioning request details

To validate placement scenarios before deploying a blueprint, you use the IaaS API to retrieve the request graph for a simulated provisioning request.

Prerequisites for retrieving provisioning request details

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Assign an API version variable for the Blueprint API.

```
api_version_blueprint='2019-09-12'
```

Note: The VCF Automation Infrastructure as a Service (IaaS) service and the VCF Automation Blueprint service have different API version values. You set the API version value for the VCF Automation Infrastructure as a Service (IaaS) service when you satisfied the general prerequisites.

- Verify that you have the ID for the blueprint that you plan to deploy. You can also get the blueprint ID from the UI, by opening the blueprint listed on **Design > Templates** and inspecting the ID at the end of the URL.

Retrieving provisioning request details

This example shows how to get request details for a blueprint with the ID `c8197446-d636-4ed9-aa2b-796da98ad10c`. In the following procedure, you use the blueprint API to get the deployment ID and flow ID and provide those values as input for the IaaS API request graph endpoint.

Use the template ID to get the deployment ID and flow ID.

```
curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version_blueprint \
  -H 'Accept: application/json' \
```



```
-H "Authorization: Bearer $access_token" \
-d '{
  "deploymentId": null,
  "deploymentName": null,
  "description": null,
  "plan": false,
  "blueprintId": "c8197446-d636-4ed9-aa2b-796da98ad10c",
  "content": null,
  "simulate": true
} | jq "
```

Examine the response to get the deployment ID and the flow ID.

```
{
  "id": "8b918325-21a7-4902-b464-f39f5fb1bb64",
  "createdAt": "2023-11-22T07:12:54.585430Z",
  "createdBy": "user@mycompany.com",
  "updatedAt": "2023-11-22T07:12:54.585430Z",
  "updatedBy": "user@mycompany.com",
  "orgId": "434f6917-4e34-4537-b6c0-3bf3638a71bc",
  "projectId": "267f8448-d26f-4b65-b310-9212adb3c455",
  "projectName": "testing",
  "deploymentId": "924fcbc3-2076-48c9-a5f5-64de7f1cbe3a",
  "requestTrackerId": "8b918325-21a7-4902-b464-f39f5fb1bb64",
  "deploymentName": "deployment_924fcbc3-2076-48c9-a5f5-64de7f1cbe3a",
  "reason": "Simulate",
  "plan": false,
  "destroy": false,
  "ignoreDeleteFailures": false,
  "simulate": true,
  "blueprintId": "c8197446-d636-4ed9-aa2b-796da98ad10c",
  "inputs": {},
  "status": "STARTED",
  "flowId": "8b918325-21a7-4902-b464-f39f5fb1bb64"
}
```

Use the deployment ID and flow ID to get the provisioning request details.

```
curl -X GET \
$curl/iaas/api/request-graph?apiVersion=$api_version \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "deploymentId": "924fcbc3-2076-48c9-a5f5-64de7f1cbe3a",
  "flowId": "8b918325-21a7-4902-b464-f39f5fb1bb64",
} | jq "
```

Using the response to validate placement scenarios

The response provides task and project information for the simulated provisioning request.

The tasks section of the response lists the task ID and stages. To validate a placement scenario, the request must successfully transition through all stages and substages. Any stage that shows an error includes a failure message that you can use to troubleshoot the problem with the placement scenario.

In the following sample response, a snippet shows an error during the reserving stage. To validate the provisioning request, correct errors and rerun the API until all tasks finish successfully.

```
"tasks": [
  {
    "id": "46414006-8f71-46f8-b965-eea0bacbba7e",
    "stages": [
      {
        "transitionSource": {
          "timestampMicros": 0
        },
        "taskSubStage": "CREATED",
        "taskInfo": {
          "stage": "STARTED",
          "isDirect": false
        },
        "resourceLinks": [
          "/iaas/api/machines/4323fd90-ad4e-4342-b9ca-62a9bb7193c9"
        ],
        "timestampMicros": 1702877437358001
      },
      ...
    ],
    "taskSubStage": "ENHANCED",
    "taskInfo": {
      "stage": "STARTED",
      "isDirect": false
    },
    ...
  },
  {
    "taskSubStage": "RESOURCE_COUNTED",
    "taskInfo": {
      "stage": "STARTED",
      "isDirect": false
    },
    ...
  },
  {
    "taskSubStage": "RESERVING",
    "taskInfo": {
      "stage": "STARTED",
      "isDirect": false
    },
    ...
  },
  {
    "taskSubStage": "ERROR",
    "taskInfo": {
      "stage": "STARTED",
      "isDirect": false,
      "failure": {
        "message": "Cannot find matching image mappings for image: im1",
        "statusCode": 0,
        "errorCode": 0,
        "serverErrorId": "ad4d91ea-4319-4f11-abcf-3601c08f3ecd",
        "documentKind": "com:vmware:xenon:common:ServiceErrorResponse"
      }
    }
  }
]
```

```

    },
    ...
  }
]

```

The project section of the response provides reference information for the project associated with the template.

```

"project": {
  "administrators": [],
  "members": [],
  "viewers": [],
  "supervisors": [],
  "zones": [
    {
      "zoneId": "8830680d-71d1-4dcb-8234-f22de4254c6b",
      "priority": 0,
      "maxNumberInstances": 0,
      "allocatedInstancesCount": 0,
      "memoryLimitMB": 0,
      "allocatedMemoryMB": 0,
      "cpuLimit": 0,
      "allocatedCpu": 0,
      "storageLimitGB": 0,
      "allocatedStorageGB": 0.0,
      "id": "8a342007-8e8a-432b-9a12-14ebb51e3c1a-8830680d-71d1-4dcb-8234-f22de4254c6b"
    }
  ],
  "constraints": {},
  "operationTimeout": 0,
  "sharedResources": true,
  "placementPolicy": "DEFAULT",
  "customProperties": {},
  "name": "p",
  "description": "",
  "id": "8a342007-8e8a-432b-9a12-14ebb51e3c1a",
  "orgId": "celfb992-7495-48fd-8988-412658094f6b",
  "_links": {
    "self": {
      "href": "/iaas/api/request-graph/8a342007-8e8a-432b-9a12-14ebb51e3c1a"
    }
  }
}

```

How do I list and edit zones associated with a project

To query and edit zones associated with a project, you use Project endpoints in the IaaS API.

Prerequisites for extracting zones associated with a project

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

- Verify that you have your project ID. If you do not have the ID, list all projects to find the name and ID of the project with the associated zones that you want to query or edit.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects?apiVersion=$api_version" | jq "."
```

Examine the response to find your project name and ID as in the following example snippet.

```
...
],
  "name": "project1",
  "description": "",
  "id": "6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9",
  "orgId": "f098d692-e980-41a5-b349-83084fcea0",
...

```

Querying zones associated with a project

You can retrieve the first 100 cloud zones associated with a project without including a query option. To retrieve information about cloud zones that are not among the first 100 listed, you add query options to your request.

For a complete list of query options, see [Querying with the VCF Automation APIs](#).

This example assumes that you have more than 100 zones associated with the project ID `6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9`. The following procedure shows how to use paging to get the second page of zones associated with the project.

For more information about pagination parameters, see [Using Pagination and Count](#).

Assign the project ID variable.

```
project_id='6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9'
```

Append query options `top=100` and `skip=100` to the request to retrieve cloud zones.

```
curl -X GET \
"$url/iaas/api/projects/$project_id/zones?apiVersion=$api_version&$top=100&skip=100" \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
| jq "."
```

The following response lists the second page of zones. Since there are 102 zones associated with the project, the second page lists two zones.

```
"content": [
  {
    "zoneId": "3cf514d6-0dfc-4941-95de-40b01d60e8d3",
    "priority": 0,
    "maxNumberInstances": 0,
    "allocatedInstancesCount": 2,
    "memoryLimitMB": 0,
    "allocatedMemoryMB": 1254,
    "cpuLimit": 0,
    "allocatedCpu": 2,
    "gpuLimit": 0,
    "allocatedGpu": 0,
    "storageLimitGB": 0,
    "allocatedStorageGB": 0.0,
    "id": "0966203d-63f5-41c7-8dcd-7c1833932ec4-3cf514d6-0dfc-4941-95de-40b01d60e8d3",
    "updatedAt": "2021-10-28",
  }
]
```

```

    "orgId": "ce811934-ea1a-4f53-b6ec-465e6ca7d126",
    "_links": {
      "project": { "href": "/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9" }
    }
  },
  {
    "zoneId": "e4c56d64-a5bc-4656-bfc6-9f8009af66d3",
    "priority": 0,
    "maxNumberInstances": 0,
    "allocatedInstancesCount": 0,
    "memoryLimitMB": 0,
    "allocatedMemoryMB": 0,
    "cpuLimit": 0,
    "allocatedCpu": 0,
    "gpuLimit": 0,
    "allocatedGpu": 0,
    "storageLimitGB": 0,
    "allocatedStorageGB": 0.0,
    "id": "0966203d-63f5-41c7-8dcd-7c1833932ec4-e4c56d64-a5bc-4656-bfc6-9f8009af66d3",
    "updatedAt": "2022-01-07",
    "orgId": "ce811934-ea1a-4f53-b6ec-465e6ca7d126",
    "_links": {
      "project": { "href": "/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9" }
    }
  }
],
"totalElements": 2,
"numberOfElements": 2

```

Editing cloud zone assignments

You can edit the following cloud zone assignments in your project:

- Storage limit (GB)
- CPU limit
- Memory limit (MB)
- Maximum number of instances
- Provisioning priority

This example assumes that you have two zones associated with the project ID

094a2fab-7715-4844-94f9-71b45452da27 , one with provisioning priority 0 and one with provisioning priority 1. The following procedure shows how to edit the zone assignments to add a new zone with priority 1 and change an existing zone to priority 2.

Assign the project ID variable.

```
project_id='094a2fab-7715-4844-94f9-71b45452da27'
```

List the zones provisioned in your project.

```

curl -X GET \
"$url/iaas/api/projects/$project_id/zones?apiVersion=$api_version&$top=100&skip=100" \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
| jq "."

```

A snippet of the response shows two zone IDs:

- 3c2bbe36-bf8e-4484-9c31-ce552422aaf1
- 8992bdf0-136f-401c-822a-e22dae67259b

```
...
  }
  "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
  "priority": 0
  "maxNumberInstances": 0,
  "allocatedInstancesCount": 0,
  ...
},
{
  "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
  "priority": 1
  "maxNumberInstances": 0,
  "allocatedInstancesCount": 0,
  ...
}
}
...
```

Note: When updating project zone assignments, the update deletes any cloud zones that are not included in the request payload. So when making the request, you must include zone IDs for all cloud zones that will remain in the project after the update, even if there is no change to the zone assignment specification.

The zone assignment update includes information for three zones in the project:

- For zone ID 3c2bbe36-bf8e-4484-9c31-ce552422aaf1 , maintain the zone with "priority": 0.
- For zone ID 8992bdf0-136f-401c-822a-e22dae67259b , maintain the zone but change to "priority": 2.
- For zone ID 66067958-7e43-47f8-9bc9-0d32594c47e9 , add a new zone with "priority": 1.

```
curl -X PUT \
  "$url/iaas/api/projects/$project_id/zones?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
  "zoneAssignmentSpecifications": [
    {
      "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
      "priority": 0
    },
    {
      "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
      "priority": 2
    },
    {
      "zoneId": "66067958-7e43-47f8-9bc9-0d32594c47e9",
      "priority": 1
    }
  ]
}' | jq "
```

Examine the response.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Project Zones Assignment Task",
  "id": "a6241aeb-909e-4689-af5a-940b52d216ff",
  "selfLink": "/iaas/api/request-tracker/a6241aeb-909e-4689-af5a-940b52d216ff"
}
```

Assign the selflink variable.

```
selfLink_id='a6241aeb-909e-4689-af5a-940b52d216ff'
```

Use the selfLink for tracking.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The zone assignments are complete when the response includes "status": "FINISHED" .

```
{
  "progress": 100,
  "status": "FINISHED",
  "name": "Project Zones Assignment Task",
  "id": "a6241aeb-909e-4689-af5a-940b52d216ff",
  "selfLink": "/iaas/api/request-tracker/a6241aeb-909e-4689-af5a-940b52d216ff"
}
```

If you list the zones provisioned in your project again, a snippet of the response shows the newly added cloud zone and updated provisioning priorities.

```
...
}
  "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
  "priority": 0
  "maxNumberInstances": 0,
  "allocatedInstancesCount":0,
  ...
},
{
  "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
  "priority": 2
  "maxNumberInstances": 0,
  "allocatedInstancesCount":0,
  ...
}
{
  "zoneId": "66067958-7e43-47f8-9bc9-0d32594c47e9",
  "priority": 1
  "maxNumberInstances": 0,
  "allocatedInstancesCount":0,
  ...
}
}
...
```

Deploying and Managing Resources

As a VCF Automation administrator, you can use the IaaS APIs to deploy and manage cloud resources.

Create and Deploy a Machine Resource

To create a new resource such as a VM, you can use the resources API to make a POST request with a project ID. The deployment creates a new resource without using a blueprint.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Assign an API version variable for the Deployment API.

```
api_version_deployment='2020-08-25'
```

Note: The VCF Automation Infrastructure as a Service (IaaS) service and the VCF Automation Deployment service have different API version values. You set the API version value for the VCF Automation Infrastructure as a Service (IaaS) service when you satisfied the general prerequisites.

- Verify that you have the ID of the cloud account where you want to deploy the VM. See [Adding Cloud Accounts](#).
- Verify that you know the cloud zone in your cloud account where you want the new VM to be deployed. See [Create a Cloud Zone](#).
- Verify that you have the ID for a project that includes the cloud zone in your cloud account. See [Add a Cloud Zone to Your Project](#).
- Verify that the flavor and image for the new VM exist in your cloud account.
- Verify that you know the resource type for the AWS machine, GCP machine, vSphere machine, or Azure machine that you plan to create. For a list of all resource types and request schema, see the link to the schema that's available from [VCF Automation Resource Schema Documentation](#).

This procedure shows how to provision a VM with a project and a cloud zone assigned to the project. The flavor and image for the VM must exist in your cloud account.

1. Assign the cloud account ID variable.

```
cloud_account_id='<your_cloud_account_id>'
```

2. Assign the project ID variable.

```
project_id='<your_project_id>'
```

3. List all cloud zones.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/zones?apiVersion=$api_version" | jq "."
```

4. Examine the response for the zone where you want to place your VM.

You use the `externalRegionId` to filter for fabric flavor and fabric image. You use the cloud zone ID for VM placement.

5. Assign the external region ID variable.

```
external_region_id='<your_external_region_id>'
```

6. Assign the cloud zone placement variable.

```
placement_name='/iaas/api/zone/<your_cloud_zone_id>'
```

7. To list the fabric images in your cloud account and zone, specify the cloud account ID and external region ID in the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
"$url/iaas/api/fabric-images/?apiVersion=$api_version&"
```



```
'$filter'="(((externalRegionId eq '') or (externalRegionId eq '$external_region_id')) \
and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id'))" | jq "."
```

8. Examine the response to select a fabric image.

9. Assign the fabric image variable.

```
image_name='<your_image_name>'
```

10. To list the fabric flavors in your cloud account and zone, specify the cloud account ID and external region ID in the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
"$url/iaas/api/fabric-flavors/?apiVersion=$api_version"& \
'$filter'="(((externalRegionId eq '') or (externalRegionId eq '$external_region_id')) \
and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id'))" | jq "."
```

11. Examine the response to select a fabric flavor.

12. Assign the fabric flavor variable.

```
flavor_name='<your_flavor_name>'
```

13. Assign the resource type variable.

```
resource_type='<your_resource_type>'
```

14. Create and deploy the VM.

```
curl -X POST \
"$url/deployment/api/resources?apiVersion=$api_version_deployment" \
-H "Authorization: Bearer $access_token" \
-H "Content-Type: application/json" \
-d '{
  "name": "<your_resource_name>",
  "projectId": "'$project_id'",
  "type": "'$resource_type'",
  "properties": {
    "imageRef": "'$image_name'",
    "flavor": "'$flavor_name'",
    "placement": "'$placement_name'"
  }
},
}' | jq "."
```

15. Examine the response for the deployment ID.

16. Assign the deployment ID.

17. Get the status of the deployment.

```
curl -X GET \
"$url/deployment/api/deployments/$deployment_id?apiVersion=$api_version_deployment" \
-H "Authorization: Bearer $access_token" | jq "."
```

When the status shows `CREATE_SUCCESSFUL` the VM is deployed.

Create and deploy a VM

With a cloud account ID and cloud zone in the cloud account, create and deploy an AWS machine named `cloud_machine_1` using the `Cloud.AWS.EC2.Instance` resource type.

Assign variables.

```
$ url='https://appliance.domain.com'
```

```
$ api_version=' '
$ api_version_deployment=' '
$ cloud_account_id='14e6b70c-0e76-4c5e-bb61-e6d70a5b43ef'
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
```

List all cloud zones.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/zones?apiVersion=$api_version" | jq "
```

A snippet of the response shows the cloud account ID with the external region ID and cloud zone ID.

```
...
    "externalRegionId": "eu-west-1",
    "cloudAccountId": "14e6b70c-0e76-4c5e-bb61-e6d70a5b43ef",
    "name": "AWS / eu-west-1-changed",
    "description": "test description",
    "id": "f32a30fd-67ac-43e3-9512-60cf6ef7bee8"
...

```

Assign the external region ID variable.

```
$ external_region_id='eu-west-1'
```

Assign the cloud zone placement variable.

```
$ placement_name='/iaas/api/zone/f32a30fd-67ac-43e3-9512-60cf6ef7bee8'
```

To list the fabric images in your cloud account and zone, specify the cloud account ID and external region ID in the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
"$url/iaas/api/fabric-images/?apiVersion=$api_version&" \
'$filter'="(((externalRegionId eq '')) or (externalRegionId eq '$external_region_id')) \
and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id'))" | jq "
```

Examine the response to find the image ID that you want.

```
...
    "externalRegionId": "eu-west-1",
    "isPrivate": false,
    "externalId": "ami-9a9012e9",
...

```

To list the fabric flavors in your cloud account and zone, specify the cloud account ID and external region ID in the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
"$url/iaas/api/fabric-flavors/?apiVersion=$api_version&" \
'$filter'="(((externalRegionId eq '')) or (externalRegionId eq '$external_region_id')) \
and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id'))" | jq "
```

Examine the response to find the flavor ID that you want.

```
...
    {
        "id": "x1e.xlarge",
        "name": "x1e.xlarge",
        "cpuCount": 4,
        "memoryInMB": 124928,
    }

```

```

        "storageType": "ssd",
        "dataDiskSizeInMB": 122880,
        "dataDiskMaxCount": 1,
        "networkType": "Up to 10 Gigabit"
    },
    ...

```

Assign the resource type for the VM.

```
$ resource_type = 'Cloud.AWS.EC2.Instance'
```

To deploy the VM, assign variables for image and flavor.

```
$ image_name='ami-9a9012e9'
$ flavor_name='x1e.xlarge'
```

Create and deploy the VM.

```

curl -X POST \
  $url/deployment/api/resources?apiVersion=$api_version_deployment \
  -H "Authorization: Bearer $access_token" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "cloud_machine_1",
    "projectId": "'$project_id'",
    "type": "'$resource_type'",
    "properties": {
      "imageRef": "'$image_name'",
      "flavor": "'$flavor_name'",
      "placement": "'$placement_name'"
    }
  },
  | jq "."

```

The response shows the deployment ID.

```

{
  "deploymentId": "fccd2081-fd44-44c8-878c-f962ef71969a",
  "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbefb51",
  "requestId": "1f8f2e4f-0b2e-448d-8439-f1a05b1e90c1",
  "resourceId": "9a510ccb-0543-47e8-a2f2-f1f65fcd0b0a"
}

```

Assign the deployment ID variable.

```
$ deployment_id='fccd2081-fd44-44c8-878c-f962ef71969a'
```

Get the status of the deployment.

```

$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id?apiVersion=$api_version"
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows the deployment status.

```

...
},
"status": "CREATE_SUCCESSFUL"
}

```

Managing IP Addresses

As an VCF Automation administrator, you can reserve IP addresses so that VCF Automation does not use them for deployment. If you no longer need them, you can release them.

Before reserving IP addresses, you must verify that available IP addresses exist within a network IP range.

Query for IP Addresses

When you use VCF Automation to create a deployment, VCF Automation allocates IP addresses to manage resources. To identify the IP addresses that have been allocated, you query the network IP range. You can only reserve IP addresses that have not been allocated.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

You create network IP ranges with a range of IP addresses. The following procedure shows how to determine the IP addresses in a range that are available.

- To get the ID of the network range that you want to use, list the internal network IP ranges in your deployment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/network-ip-ranges?apiVersion=$api_version" | jq "."
```

Examine the response to find the ID of the network IP range that you want to use.

- Assign a variable for the network IP range ID.

```
ip_range_id='<your_network_ip_range_id>'
```

- To query IP ranges associated with a particular network and get information such as IP availability, you can use one of the following commands:

- If you have the network IP range ID, use:

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/network-ip-ranges/$ip_range_id?apiVersion=$api_version" | jq "."
```

- If you have the network ID, use:

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/networks/<your_network_id>/network-ip-ranges?apiVersion=$api_version" | jq "."
```

- If you have the networking fabric ID, use:

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-networks/<your_fabric_network_id>/network-ip-ranges?apiVersion=$api_version" | jq "."
```

- If you have the networking fabric for vSphere ID, use:

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-networks-vsphere/<your_fabric_network_vsphere_id>/network-ip-ranges?apiVersion=$api_version" | jq "."
```

Examine the response to see the number of allocated and available IP addresses. If the number of available IP addresses is too low, check another network IP range until you find one with enough IP addresses to fit your needs.

- Use the network IP range ID to query the status of IP addresses within the range.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses?apiVersion=$api_version" | jq "."
```

Examine the result. All IP addresses appear with "ipAddressStatus": "ALLOCATED" or "ipAddressStatus": "RELEASED" and are not available.

Query for IP addresses within an IP range

The example query requests and responses show how to find allocated IP addresses.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=''
```

List the internal network IP ranges in your deployment.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-ip-ranges?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the network IP ranges with their IDs.

```
...
  "startIPAddress": "fe45::2",
  "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
  "endIPAddress": "fe45::ffff",
  "createdAt": "2023-02-28",
  "ipVersion": "IPv6",
  "name": "rangeIPv6",
  "id": "703a02df-3b6d-4ad7-b146-582b398055f2",
  ...
```

Assign the variable for the network IP range ID.

```
$ ip_range_id='703a02df-3b6d-4ad7-b146-582b398055f2'
```

Query the IP range associated with the network IP range ID.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-ip-ranges/$ip_range_id?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the number of available and allocated IPv6 addresses. It also shows the IP version and the start and end of the IP address sequence.

```
{
  "numberOfAllocatedIPs": 2,
  "numberOfAvailableIPs": 65531,
  "numberOfReleasedIPs": 0,
  "totalNumberOfIPs": 65533,
  "numberOfUserAllocatedIPs": 0,
  "numberOfSystemAllocatedIPs": 2,
  "startIPAddress": "fe45::2",
  "endIPAddress": "fe45::ffff",
  "ipVersion": "IPv6",
  "name": "rangeIPv6",
  "id": "703a02df-3b6d-4ad7-b146-582b398055f2",
  "createdAt": "2023-02-28",
  "updatedAt": "2023-02-28",
  "organizationId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
  "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
  "_links": {
    "self": {
      "href": "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2"
    }
  }
}
```

The following example shows how to query of IP ranges associated with fabric network ID 33c2bbb5-5b26-4a5a-87c6-9e96db72451d.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d/network-ip-ranges?apiVersion=$api_version" | jq "."
```

The response shows two IP ranges with details about associated IPv4 addresses.

```
"content": [
  {
    "numberOfAllocatedIPs": 0,
    "numberOfAvailableIPs": 10,
    "numberOfReleasedIPs": 0,
    "totalNumberOfIPs": 10,
    "numberOfUserAllocatedIPs": 0,
    "numberOfSystemAllocatedIPs": 0,
    "startIPAddress": "10.10.10.1",
    "endIPAddress": "10.10.10.10",
    "ipVersion": "IPv4",
    "name": "range-1",
    "id": "578ec19a-2b52-497e-8777-029b57d685ab",
    "createdAt": "2023-03-15",
    "updatedAt": "2023-03-29",
    "orgId": "89db05f7-1b93-4bd8-b395-1772d50813a4",
    "_links": {
      "fabric-networks": {
        "hrefs": [
          "/iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d",
          "/iaas/api/fabric-networks/51f1ed6c-3db3-4877-857d-2bcc84f81897"
        ]
      },
      "self": {
        "href": "/iaas/api/network-ip-ranges/578ec19a-2b52-497e-8777-029b57d685ab"
      }
    }
  },
  {
    "numberOfAllocatedIPs": 0,
    "numberOfAvailableIPs": 10,
    "numberOfReleasedIPs": 0,
    "totalNumberOfIPs": 10,
    "numberOfUserAllocatedIPs": 0,
    "numberOfSystemAllocatedIPs": 0,
    "startIPAddress": "10.10.10.11",
    "endIPAddress": "10.10.10.20",
    "ipVersion": "IPv4",
    "name": "range-2",
    "id": "95a0053c-b57f-4500-a59b-970042f4ce8c",
    "createdAt": "2023-03-29",
    "updatedAt": "2023-03-29",
    "orgId": "89db05f7-1b93-4bd8-b395-1772d50813a4",
    "_links": {
      "fabric-networks": {
```

```

        "href": [
            "/iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d"
        ]
    },
    "self": {
        "href": "/iaas/api/network-ip-ranges/95a0053c-b57f-4500-a59b-970042f4ce8c"
    },
    "fabric-network": {
        "href": "/iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d"
    }
}
}
},
"totalElements": 2,
"numberOfElements": 2
}

```

If the IP range has a sufficient number of available IP addresses, get more information about the IP addresses in that range.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses?apiVersion=$api_version" | jq "
```

The response provides details about the unavailable IP addresses including their IP version and how they were allocated.

- "ipAllocationType": "SYSTEM" indicates that an IP address was automatically allocated for use in a deployment.
- "ipAllocationType": "USER" indicates that an IP address was manually allocated for use by a user.

```

{
  "content": [
    {
      "ipAddress": "fe4500:10:118:136:fcd8:d68d:9701:4440",
      "ipAddressDecimalValue": 168561938,
      "ipVersion": "IPv6",
      "ipAddressStatus": "ALLOCATED",
      "ipAllocationType": "SYSTEM",
      "id": "e83d9a43-ea2d-428c-ae75-da396c1bb205",
      "createdAt": "2023-02-21",
      "updatedAt": "2023-02-21",
      "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
      "_links": {
        "network-ip-range": {
          "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a"
        },
        "self": {
          "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-addresses/e83d9a43-ea2d-428c-ae75-da396c1bb205"
        },
        "connected-resource": {
          "href": "/iaas/api/machines/83ede26c-656a-4a08-8716-bea29c21d3f4/network-interfaces/051d92e6-5729-495d-ad66-9e443e5747c8"
        }
      }
    }
  ]
}

```

```

    }
  },
  {
    "ipAddress": "fe45:10:118:136:fcd8:d68d:9701:4450",
    "ipAddressDecimalValue": 168561941,
    "ipVersion": "IPv6",
    "ipAddressStatus": "ALLOCATED",
    "ipAllocationType": "SYSTEM",
    "id": "439f9c9b-2b2f-484d-8867-2d7b541ddeec",
    "createdAt": "2023-02-22",
    "updatedAt": "2023-02-22",
    "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
    "_links": {
      "network-ip-range": {
        "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a"
      },
      "self": {
        "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-addresses/439f9c9b-2b2f-484d-8867-2d7b541ddeec"
      },
      "connected-resource": {
        "href": "/iaas/api/machines/81633fba-e86b-4bfa-a06f-3c4f7a754568/network-interfaces/1c26b194-2862-4d7e-be9c-9881d6cdb871"
      }
    }
  }
],
"totalElements": 2,
"numberOfElements": 2
}

```

You can allocate any of the IP addresses in the network range except

fe4500:10:118:136:fcd8:d68d:9701:4440 and fe45:10:118:136:fcd8:d68d:9701:4450 .

IP addresses with the status "ipAddressStatus": "ALLOCATED" or "ipAddressStatus": "RELEASED" are not available for allocation. You can allocate any of the remaining the IP addresses in the range. See [Allocate IP Addresses](#).

Allocate IP Addresses

As a cloud administrator, you can use the network IP range API to allocate the IP addresses that you want to reserve and make them unavailable for deployment by VCF Automation.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the network IP range that includes the IP addresses that you want to reserve. See [Query for IP Addresses](#).
- If you plan to specify the IP addresses from the network IP range, verify that you know the IPv4 or IPv6 addresses. See [Query for IP Addresses](#).
- If you are a cloud administrator and you plan to specify the number of IP addresses from the network IP range, verify that the number is less than or equal to the number of available IP addresses in the range.

To allocate IP addresses, you submit a request with either:

- A list of specific IP addresses.
- The number of IP addresses.

You can also provide an optional description.

Note: The following restrictions apply to the request:

- No more than 100 IP addresses can be allocated in a single call.
- If listing specific IP addresses, the addresses must all be of the same type. Mixing IPv4 and IPv6 addresses is not supported.

1. Assign the variable for the network IP range ID.

```
ip_range_id='<your_network_ip_range_id>'
```

2. Allocate the IP addresses by providing a list of addresses.

The following request example allocates two IP addresses.

- Specify either IPv4 or IPv6 addresses.
- To release additional IP addresses, provide additional addresses in the payload.

```
curl --location --request POST \
  $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "<your_optional_description>",
    "ipAddresses": ["<ipv_address_1>", "<ipv_address_2>"]
  }'
```

Examine the response for the self link to track the request.

3. Optional: If you do not list IP addresses, provide the number of IP addresses to allocate.

```
curl --location --request POST \
  $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "<your_optional_description>",
    "numberOfIps": "<number_of_IPs>"
  }'
```

Examine the response for the self link to track the request.

4. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

5. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The IPs are allocated when the response includes "status": "FINISHED" and provides a list of allocated resources.

Note: If you specified the number of IP addresses to allocate, you query for the details of each resource in the response to get the allocated IP address associated with the resource.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/<ip_address_id>?apiVersion=$api_version" | jq "."
```

Allocate IP addresses within an IP range

Using the network IP range with the IP addresses that you want to reserve, specify the IP addresses to allocate.

- Two IPv4 IP addresses used in this example are: ["196.192.2.19", "196.192.2.20"]

- Two IPv6 IP addresses used in this example are:

```
["fe45:10:118:136:fcd8:d68d:9701:8975", "fe45:10:118:136:fcd8:d68d:9701:8985"]
```

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ ip_range_id='703a02df-3b6d-4ad7-b146-582b398055f2'
```

Allocate IPv4 addresses by providing a list.

```
curl --location --request POST \
  $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "Allocate ipv4 addresses for QA test machines",
    "ipAddresses": ["196.192.2.19", "196.192.2.20"]
  }'
```

Or allocate IPv6 addresses by providing a list.

```
curl --location --request POST \
  $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "Allocate ipv6 addresses for QA test machines",
    "ipAddresses": ["fe45:10:118:136:fcd8:d68d:9701:8975", "fe45:10:118:136:fcd8:d68d:9701:8985"]
  }'
```

Or specify the number of IP addresses to allocate, such as four.

```
curl --location --request POST \
  $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "Allocate four IP addresses for QA test machines",
    "numberOfIps": "4"
  }'
```

The response provides the selfLink variable for tracking.

```
{
  "progress": 1,
  "status": "INPROGRESS",
  "name": "IP Address Allocation Task",
  "id": "ab726e31-ed8bfdfb-83d9-4964-b719-8b395c87d2c9",
  "selfLink": "/iaas/api/request-tracker/ab726e31-ed8bfdfb-83d9-4964-b719-8b395c87d2c9"
}
```

Assign the selfLink variable.

```
selfLink_id= 'ab726e31-e909-4bcc-87ef-5139294eb26a'
```

Track the request

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "
```

When the request is complete, the response includes "status": "FINISHED".

- This is an example of the response when two IPv4 or IPv6 addresses are specified and allocated. You know the IP addresses because you provided them in the request.

```
{
  "progress": 100,
  "status": "FINISHED",
  "resources": [
    "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-address-
es/6212ac8b-a885-4416-bf0d-ebb88628bef1",
    "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-address-
es/9b65104d-debd-4830-8296-3cb87509d293"
  ],
  "name": "IP Address Allocation Task",
  "id": "ab726e31-e909-4bcc-87ef-5139294eb26a",
  "selfLink": "/iaas/api/request-tracker/ab726e31-e909-4bcc-87ef-5139294eb26a"
}
```

- This is an example of the response when four IP addresses are requested and allocated.

```
{
  "progress": 100,
  "status": "FINISHED",
  "resources": [
    "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-addresses/a4c-
ceea7-3a94-439b-99fa-420774e61eba",
    "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-address-
es/4c492669-5ec2-4465-992b-c9c252edf052",
    "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-address-
es/0ce9ee20-1a40-460d-80df-fb1791fbd42a",
    "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-address-
es/d7d876b6-92b1-4530-a305-d586c8474f11"
  ],
  "name": "IP Address Allocation Task",
  "id": "ab726e31-e909-4bcc-87ef-5139294eb26a",
  "selfLink": "/iaas/api/request-tracker/ab726e31-e909-4bcc-87ef-5139294eb26a"
}
```

To get an IP address, query for the details of an allocated resource. The following example queries for the details of the resource with IP address ID a4ccee7-3a94-439b-99fa-420774e61eba.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/a4ccee7-3a94-439b-99fa-420774e61e-
ba?apiVersion=$api_version" | jq "
```

The response provides details about the resource including the allocated IP address and IP version. "ipAllocationType": "USER" indicates that an VCF Automation administrator manually reserved the IP address.

```
{
  "ipAddress": "196.192.2.20",
  "ipAddressDecimalValue": 168561941,
  "ipVersion": "IPv4",
  "ipAddressStatus": "ALLOCATED",
  "ipAllocationType": "USER",
  "id": "439f9c9b-2b2f-484d-8867-2d7b541ddeec",
```

```

    "createdAt": "2023-02-22",
    "updatedAt": "2023-02-22",
    "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
    "_links": {
      "network-ip-range": {
        "href": "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2"
      },
      "self": {
        "href": "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-
addresses/439f9c9b-2b2f-484d-8867-2d7b541ddeec"
      },
      "connected-resource": {
        "href": "/iaas/api/machines/81633fba-e86b-4bfa-a06f-3c4f7a754568/network-in-
terfaces/1c26b194-2862-4d7e-be9c-9881d6cdb871"
      }
    }
  }
]
}

```

If you no longer need allocated IP addresses, you can release them. See [Release IP Addresses](#).

Release IP Addresses

If you no longer need reserved IP addresses, you can release them so that VCF Automation can use them for deployment.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the network IP range that includes the IP addresses that you want to release. See [Query for IP Addresses](#).
- Verify that you know the IPv4 or IPv6 addresses that you want to release. The IP addresses must all be the same type. Releasing a mix of IPv4 and IPv6 addresses in the same request is not supported. See [Allocate IP Addresses](#).

This procedure shows how to request specific IP addresses to release.

1. Assign the variable for the network IP range ID.

```
ip_range_id='<your_network_ip_range_id>'
```

2. Release the IP addresses.

The following request releases two IP addresses.

- Specify either IPv4 or IPv6 addresses.
 - To release more IP addresses, provide the IP addresses in the payload.
- Note:** No more than 100 IP addresses can be released in a single API call.

```

curl --location --request POST \
  $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/release?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "ipAddresses":["<ipv_address_1>", "<ipv_address_2>"]
  }'

```

Examine the response for the self link to track the request.

3. Assign the self link variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

4. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The IP addresses are released, when the response includes "status": "FINISHED".

Release IP addresses within an IP range

To release IP addresses, make a request with the network IP range and specify the IP addresses to release.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ ip_range_id='4e6b700c-5d07-4247-b0c7-ae13d1d7188a'
```

Release two IPv4 addresses.

```
curl --location --request POST \
  $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/release?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "ipAddresses": ["196.192.2.19", "196.192.2.20"]
  }'
```

The response provides a selfLink for tracking.

```
{
  "progress": 1,
  "status": "INPROGRESS",
  "name": "IP Address Release Task",
  "id": "452bb595-ca0f-4685-8fcc-8bd6321bf7c6",
  "selfLink": "/iaas/api/request-tracker/452bb595-ca0f-4685-8fcc-8bd6321bf7c6"
}
```

Assign the selfLink variable.

```
selfLink_id= '452bb595-ca0f-4685-8fcc-8bd6321bf7c6'
```

Track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The IP addresses are released, when the response includes "status": "FINISHED".

```
{
  "progress": 100,
  "status": "FINISHED",
  "resources": [
    "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-addresses/3f08cabe-fed-f-4e9b-96f6-d3bf69016cc4",
    "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-addresses/ca287794-c06b-401c-9626-0b8f5f75d1c2"
  ],
  "name": "IP Address Release Task",
  "id": "452bb595-ca0f-4685-8fcc-8bd6321bf7c6",
  "selfLink": "/iaas/api/request-tracker/452bb595-ca0f-4685-8fcc-8bd6321bf7c6"
}
```

}

Creating and Using a First Class Disk

Using a vSphere Storage Profile that supports First Class Disk (FCD) storage, you can create block device-based storage that is independent of a VM.

First Class Disks offer many advantages. For example, you can attach an FCD to a VM and take multiple snapshots of the disk over time. If you find that you do not need certain snapshots, you can delete them. You can also revert an FCD to an earlier snapshot.

Note: If you attach an FCD to a VM and then delete the VM, both the FCD and its snapshots are deleted. And if the FCD is detached from a VM, you cannot delete the FCD without deleting its snapshots first.

Create a First Class Disk

To create a First Class Disk (FCD), you make a POST request using the block device specification. The request body includes a project ID, disk capacity, persistence setting, and constraints from the vSphere Storage Profile for an FCD creation.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have added a project and you have the project ID. See [Create a Project to use in VCF Automation](#).
- Know the capacity of the disk and the persistence of the disk that you are creating.
- Verify that you have created a storage profile for an FCD and that you have the `defaultItem` and the tags from the response. See [Create a vSphere Storage Profile for a First Class Disk](#).

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. Set the capacity and persistence for the disk.

```
capacity_in_gb=<integer>
persistence=<true|false>
```

3. Deploy the FCD.

- With `mandatory` set to `true`, all tags in the `expression` must match a storage profile, otherwise provisioning fails.
- The `expression` is the key:value tag pair used to create the storage profile. See [Create a vSphere Storage Profile for a First Class Disk](#).

```
curl -X POST \
  $url/iaas/api/block-devices?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "projectId": "'$project_id'",
    "name": "FCD-example",
    "capacityInGB": "'$capacity_in_gb'",
    "persistent" : "'$persistent'",
    "constraints": [
      {
        "mandatory": "true",
        "expression": "type:fcd"
      }
    ]
  }
```

```
} ' | jq '"
```

The response includes a selfLink value.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

4. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

5. Use the selfLink variable to track the progress of the FCD creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq '"
```

In the list of resources, the response includes block devices with the block device ID in the path.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/block-devices/example-blockdevice-alphanumeric-string"
  ],
  ...
}
```

6. Assign the block device ID variable.

```
block_device_id='example-blockdevice-alphanumeric-string'
```

7. Optional: Retrieve the created block device object.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id?apiVersion=$api_version | jq '"
```

8. Optional: Retrieve all the FCD block device types.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/block-devices?%24filter=customProperties.diskType%20eq%20firstClass&apiVersion=$api_version" | jq '"
```

9. Optional: Delete the FCD block device.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id?apiVersion=$api_version | jq '"
```

Create a First Class Disk

With constraints from a vSphere Storage Profile for FCD storage, use the block device specification to deploy a First Class Disk for a project ID with a two GB capacity and persistence set to false.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ project_id='f5357a28-df59-47e0-b983-8a562910d0be'
```

```
$ capacity_in_gb=2
$ persistent=false
```

Deploy the FCD.

```
$ curl -X POST \
  $url/iaas/api/block-devices?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "projectId": "'$project_id'",
    "name": "FCD-example",
    "capacityInGB": "'$capacity_in_gb'",
    "persistent" : "'$persistent'",
    "constraints": [
      {
        "mandatory": "true",
        "expression": "type:fcd"
      }
    ]
  }' | jq "."
```

The response provides a selfLink to the request.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "86707da6-d5d6-4ebc-94a2-0a22f3fcb794",
  "selfLink": "/iaas/api/request-tracker/86707da6-d5d6-4ebc-94a2-0a22f3fcb794"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='86707da6-d5d6-4ebc-94a2-0a22f3fcb794'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

After the request completes successfully, the response provides the block device ID.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2"
  ],
  "name": "Provisioning",
  "id": "86707da6-d5d6-4ebc-94a2-0a22f3fcb794",
  "selfLink": "/iaas/api/request-tracker/86707da6-d5d6-4ebc-94a2-0a22f3fcb794"
}
```


Assign the block device ID variable.

```
$ block_device_id='elcbc8e1-76bb-4bef-8e51-a582437266c2'
```

Use the block device ID to attach your FCD to a VM and manage your FCD snapshots. See [Attach a First Class Disk](#) and [Manage First Class Disk Snapshots](#).

Attach a First Class Disk

To attach a First Class Disk (FCD) to a VM, you make a POST request with the machine ID of the VM. The request body includes the block device ID that you obtained from creating the FCD.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have created an FCD and you have a block device ID. See [Create a First Class Disk](#).
- Verify that the hardware version of the machine to which you plan to attach the FCD is vmx-13 or later.

1. Assign the block device ID variable.

```
block_device_id='<your_block_device_id>'
```

2. Get a list of machines.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/machines?apiVersion=$api_version | jq "."
```

3. Examine the response to find the machine that you want to attach the FCD to.

4. Assign a machine ID.

```
machine_id='<your_machine_id>'
```

5. Attach the FCD to the machine.

```
curl -X POST \
  $url/iaas/api/machines/$machine_id/disks?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "blockDeviceId": "'$block_device_id'"
  }' | jq "."
```

The response includes a selfLink value.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

6. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

7. Use the selfLink to track the progress of the FCD attachment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

Once complete, the response includes a list of resources with a machine that has your machine ID in the path.

```
{
  "progress": 100,
  "message": "success",
}
```

```

    "status": "FINISHED",
    "resources": [
      "/iaas/api/machines/your-machine-id"
    ],
    ...
  }

```

8. Optional: Detach the FCD.

```

curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
machines/$machine_id/disks/$block_device_id?apiVersion=$api_version | jq "."

```

Attach a First Class Disk

With the block device ID from the FCD, attach the FCD to a VM.

```

$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ block_device_id='elcbc8e1-76bb-4bef-8e51-a582437266c2'

```

Get a list of machines.

```

$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/machines?apiVersion=$api_version | jq "."

```

Examine the response. Identify the machine that you want to attach the FCD to.

```

...
{
  "powerState": "ON",
  "externalRegionId": "Datacenter:datacenter-3",
  "cloudAccountIds": [
    "683c647b-413d-4673-a236-08b3694cd652"
  ],
  "provisioningStatus": "READY",
  "customProperties": {
    "osType": "LINUX",
    "vcUuid": "8d6dabbb-46b4-41b2-b76e-7745330f8f7d",
    "memoryGB": "0",
    "datacenter": "Datacenter:datacenter-3",
    "instanceUUID": "502a55ea-580c-9ad0-4275-82f96d3a4683",
    "softwareName": "Red Hat Enterprise Linux 7 (64-bit)",
    "cpuCount": "1",
    "memoryInMB": "256"
  },
  "externalId": "502a55ea-580c-9ad0-4275-82f96d3a4683",
  "name": "Cloud_vSphere_Machine_1-mcm100156-139639218287",
  "id": "fcaad107-48c3-320f-989f-31b0c8d4a6a0",
  "createdAt": " 2022-04-02",
  "updatedAt": " 2022-04-02",
  ...

```

Assign the machine ID variable.

```

$ machine_id='fcaad107-48c3-320f-989f-31b0c8d4a6a0'

```

Attach the FCD to the machine.

```
$ curl -X POST \
  $url/iaas/api/machines/$machine_id/disks?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "blockDeviceId": "'$block_device_id'"
  }' | jq "
```

The response provides a selfLink to the request.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "18050d7d-e3b2-4dd0-b0a0-5883ec766999",
  "selfLink": "/iaas/api/request-tracker/18050d7d-e3b2-4dd0-b0a0-5883ec766999"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='18050d7d-e3b2-4dd0-b0a0-5883ec766999'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "
```

After the request completes successfully, the response includes your machine ID.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/machines/fcaad107-48c3-320f-989f-31b0c8d4a6a0"
  ],
  ...
}
```

Manage First Class Disk Snapshots

To create a snapshot of a First Class Disk (FCD), you make a POST request with the block device ID of the FCD. Using the snapshot ID created, you can revert an FCD to a snapshot or delete a snapshot of an FCD.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have created an FCD and you have a block device ID. See [Create a First Class Disk](#).

1. Assign the block device ID variable.

```
block_device_id='<your_block_device_id>'
```

2. Create a snapshot of the FCD.

```
curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
```

```
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "description": "example description"
}' | jq "
```

The response includes a selfLink value.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

3. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

4. Use the selfLink to track the progress of the FCD snapshot creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "
```

The response indicates if the snapshot is successful.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/block-devices/your-block-device-id"
  ],
  ...
}
```

5. Optional: To create additional FCD snapshots, repeat Step 2 to Step 4.

6. To get a snapshot ID, list all FCD snapshots.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq "
```

If you created multiple snapshots, the response lists multiple snapshot IDs.

7. Examine the response and select a snapshot ID to assign as a variable.

```
snapshot_id=<your_snapshot_id_1>
```

8. Optional: You can list an individual snapshot.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "
```

9. Optional: You can revert an FCD to a snapshot.

```
curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/revert?id=$snapshot_id&apiVersion=$api_version \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
```

```
-d '' | jq ""
```

To track the progress of the reversion, perform Step 3 and Step 4.

10. Optional: You can delete a snapshot.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq ""
```

Create snapshots of a First Class Disk and revert an FCD to a snapshot

With the block device ID from the created FCD, create multiple snapshots of an FCD.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ block_device_id='elcbc8e1-76bb-4bef-8e51-a582437266c2'
```

Create a snapshot of the FCD.

```
$ curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "description": "Example description 1"
  }' | jq ""
```

The response provides a selfLink to the request.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "d08bb46c-cf7e-40b6-bdf8-893390ba4d51",
  "selfLink": "/iaas/api/request-tracker/d08bb46c-cf7e-40b6-bdf8-893390ba4d51"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='d08bb46c-cf7e-40b6-bdf8-893390ba4d51'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq ""
```

Create a second snapshot of the FCD.

```
$ curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "description": "Example description 2"
  }' | jq ""
```

List all the snapshots of the FCD.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/
api/block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq "."
```

Examine the response to see all snapshot IDs.

```
[
  {
    "name": "357ed3e5-8b2e-4533-b6fe-3ea6e15b8de5",
    "desc": "Example description 1",
    "isCurrent": false,
    "id": "16cfddb8-559c-49ff-8162-0a4c57079c81",
    "createdAt": " 2022-04-02",
    "updatedAt": " 2022-04-02",
    "owner": "user@mycompany.com",
    "organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
    "orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
    "_links": {
      "self": {
        "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/snapshots/16cfdb-
b8-559c-49ff-8162-0a4c57079c81"
      }
    }
  },
  {
    "name": "b04f7513-c695-4662-b5e8-a023a7b1bfe7",
    "desc": "Example description 2",
    "isCurrent": true,
    "id": "ed1b09ff-1175-4cdd-b07e-7bb906a9ddc4",
    "createdAt": " 2022-04-22",
    "updatedAt": " 2022-04-22",
    "owner": "user@mycompany.com",
    "organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
    "orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
    "_links": {
      "self": {
        "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/snapshots/ed1b09f-
f-1175-4cdd-b07e-7bb906a9ddc4"
      }
    }
  }
]
```

Assign a snapshot ID variable.

```
snapshot_id='16cfddb8-559c-49ff-8162-0a4c57079c81'
```

List information about the snapshot.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

The response shows information about the single snapshot.

```
{
```

```

"name": "357ed3e5-8b2e-4533-b6fe-3ea6e15b8de5",
"desc": "Example description 1",
"isCurrent": false,
"id": "16cfdbb8-559c-49ff-8162-0a4c57079c81",
"createdAt": " 2022-04-02",
"updatedAt": " 2022-04-02",
"owner": "user@mycompany.com",
"organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
"orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
"_links": {
  "self": {
    "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/snapshots/16cfdb-
b8-559c-49ff-8162-0a4c57079c81"
  }
}

```

Revert the FCD to the snapshot.

```

$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."

```

To validate the reversion, list information about the snapshot again.

```

$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."

```

In the response, `"isCurrent":true` shows that the FCD has reverted to the snapshot.

```

{
  "name": "357ed3e5-8b2e-4533-b6fe-3ea6e15b8de5",
  "desc": "Example description 1",
  "isCurrent": true,
  "id": "16cfdbb8-559c-49ff-8162-0a4c57079c81",
  "createdAt": " 2022-04-02",
  "updatedAt": " 2022-04-02",
  "owner": "user@mycompany.com",
  "organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
  "orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
  "_links": {
    "self": {
      "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/snapshots/16cfdb-
b8-559c-49ff-8162-0a4c57079c81"
    }
  }
}

```

Working with Azure Disk Snapshots

You can use the VCF Automation IaaS API to create or delete snapshots of Azure managed disks. The snapshot provides a backup of your block device.

Note: If you create a snapshot of an independent disk and then delete the disk, the snapshot is left behind. So before deleting a disk from the Azure cloud account, check for snapshots. If snapshots exist, delete them first before deleting the disk.

Create a Block Device

If you do not already have an Azure managed disk, create a block device to use for your snapshot. To create a block device, you make a POST request using the block device specification. The request body includes a project ID, disk capacity, persistence setting, and constraints from the Azure Storage Profile for a managed disk.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have added a project and you have the project ID. See [Create a Project to use in VCF Automation](#).
- Know the capacity of the disk and the persistence of the disk that you are creating.
- Verify that you have created a storage profile for a managed disk and that you have the `defaultItem` and the tags from the response. See [Create a Microsoft Azure Storage Profile for a Managed Disk](#).

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. Set the capacity and persistence for the disk.

```
capacity_in_gb=<integer>
persistence=<true|false>
```

3. Deploy the block device.

- With `mandatory` set to `false`, tags in the `expression` are not required to match tags in an existing storage profile for the deployment to succeed. However if tags are provided, VCF Automation will try to match them when deploying the block device.
- The `expression` is the key:value tag pair used to create the storage profile. See [Create a vSphere Storage Profile for a First Class Disk](#).

```
curl -X POST \
  $url/iaas/api/block-devices?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "projectId": "'$project_id'",
    "name": "block-device-example",
    "capacityInGB": "'$capacity_in_gb'",
    "persistent" : "'$persistent'",
    "constraints": [
      {
        "mandatory": "false",
        "expression": "type:managed"
      }
    ]
  }' | jq "."
```

The response includes a `selfLink` value.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```


4. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

5. Use the selfLink variable to track the progress of the block device creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

In the list of resources, the response includes block devices with the block device ID in the path.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/block-devices/example-blockdevice-alphanumeric-string"
  ],
  ...
}
```

6. Optional: If you want to retrieve the ID of an existing block device, use an OData filter with the block device name in the request.

```
block_device_name='<your_block_device_name>'
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/block-devices?apiVersion=$api_version&"'$filter'="'name%20eq%20$block_device_name" | jq "."
```

7. Assign the block device ID variable.

```
block_device_id='example-blockdevice-alphanumeric-string'
```

Create a Block Device

With constraints from an Azure Storage Profile for a managed disk, use the block device specification to deploy a managed disk for a project ID with a two GB capacity and persistence set to false.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ project_id='f5357a28-df59-47e0-b983-8a562910d0be'
$ capacity_in_gb=2
$ persistent=false
```

Deploy the block device.

```
$ curl -X POST \
  $url/iaas/api/block-devices?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "projectId": "'$project_id'",
    "name": "block-device-example",
    "capacityInGB": "'$capacity_in_gb'",
    "persistent" : "'$persistent'",
    "constraints": [
      {
        "mandatory": "false",
        "expression": "type:managed"
      }
    ]
  }
```

```
]
}' | jq ". "
```

The response provides a selfLink to the request.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
  "selfLink": "/iaas/api/request-tracker/22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
  "deploymentId": "cf33c90e-6f6d-48ed-82dd-a6a9f0e6f700"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='22bdaf20-ce48-4a9f-8c1f-f4e74263645f'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq ". "
```

After the request completes successfully, the response provides the block device ID.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101"
  ],
  "name": "Provisioning",
  "id": "22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
  "selfLink": "/iaas/api/request-tracker/22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
  "deploymentId": "cf33c90e-6f6d-48ed-82dd-a6a9f0e6f700"
}
```

Assign the block device ID variable.

```
$ block_device_id='41d600c2-429e-4c90-98d4-638e77724101'
```

Use the block device ID to create a snapshot. See [Create and Manage Azure Disk Snapshots](#).

Create and Manage Azure Disk Snapshots

To create a snapshot of a Azure managed disk, you make a POST request with the block device ID of the managed disk. Using the snapshot ID created, you can list a snapshot or delete a snapshot of a managed disk.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have an Azure managed disk and a block device ID. See [Create a Block Device](#).

1. Assign the block device ID variable.

```
block_device_id='<your_block_device_id>'
```

2. Create a snapshot of the managed disk.

```
curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example_name"
  }' | jq "
```

In addition to the required snapshot name, you can include optional snapshot properties that can be used independently or combined as in the following example.

```
curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example_name"
    "snapshotProperties": {
      "incremental": "true",
      "resourceGroupName": "newRG",
      "encryptionSetId" : ""$encryption_key""
    },
  }' | jq "
```

- To create a snapshot that only consists of changes since the last snapshot, set `incremental` to `true`.
- To specify a target resource group for the snapshot, include the `resourceGroupName` property. You can look up existing resource group names in the Azure portal, or specify a name for a new resource group. If not specified, VCF Automation creates the snapshot in the same resource group as the block device by default.
- To encrypt the snapshot with a customer-managed key, include the `encryptionSetId` property.

To get the `encryptionSetId` property for a particular region ID, use the following API request:

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/fabric-azure-disk-encryption-sets?regionId=$region_id&apiVersion=$api_version | jq "
```

Examine the response for the `id` as in the following example.

```
{
  "name": "test-encryption-1.1",
  "id": "/subscriptions/b8ef63a7-a5e3-44fa-8745-1ead33falf25/resourceGroups/DISKRG67970/providers/Microsoft.Compute/diskEncryptionSets/test-encryption-1.1",
  "regionId": "eastus",
  "key": "key1234",
  "vault": "KeyVault1234"
},
```

Assign the encryption key variable.

```
$encryption_key='/subscriptions/b8ef63a7-a5e3-44fa-8745-1ead33falf25/resourceGroups/DISKRG67970/providers/Microsoft.Compute/diskEncryptionSets/test-encryption-1.1'
```

3. Examine the response for the selfLink value.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
```

```
}
```

4. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

5. Use the selfLink to track the progress of the snapshot creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

The response indicates if the snapshot is successful.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/block-devices/your-block-device-id"
  ],
  ...
}
```

6. To get a snapshot ID, list all snapshots.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq "."
```

If you created multiple snapshots, the response lists multiple snapshot IDs.

7. Examine the response and select a snapshot ID to assign as a variable.

```
snapshot_id=<your_snapshot_id_1>
```

8. Optional: You can list an individual snapshot.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

9. Optional: You can delete a snapshot.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

Create snapshots of a managed disk

With the block device ID, create multiple snapshots of a managed disk.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ block_device_id='41d600c2-429e-4c90-98d4-638e77724101'
```

Create a snapshot of the managed disk.

```
$ curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "demo-snapshot-1"
```

```
}' | jq ". "
```

The response provides a selfLink to the request.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "66123d15-8e5a-42b0-a0b4-e9ed8e21180a",
  "selfLink": "/iaas/api/request-tracker/66123d15-8e5a-42b0-a0b4-e9ed8e21180a"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='66123d15-8e5a-42b0-a0b4-e9ed8e21180a'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq ". "
```

Create a second snapshot of the managed disk.

```
$ curl -X POST \
  $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "demo-snapshot-2"
  }' | jq ". "
```

List all the snapshots of the managed disk.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq ". "
```

Examine the response to see all snapshot IDs.

```
[
  {
    "name": "demo-snapshot-1",
    "snapshotProperties": {
      "incremental": "false"
    },
    "id": "38348991-00a1-48be-80d6-00d62afcd280",
    "createdAt": " 2022-04-02",
    "updatedAt": " 2022-04-02",
    "owner": "user@mycompany.com",
    "organizationId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "orgId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "_links": {
      "self": {
        "href": "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101/snapshot-
s/38348991-00a1-48be-80d6-00d62afcd280"
      }
    }
  }
]
```

```

    }
  },
  {
    "name": "demo-snapshot-2",
    "snapshotProperties": {
      "incremental": "false"
    },
    "id": "80407f78-7f90-4d9f-83f4-10d3a1e982ac",
    "createdAt": " 2022-04-02",
    "updatedAt": " 2022-04-02",
    "owner": "user@mycompany.com",
    "organizationId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "orgId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "_links": {
      "self": {
        "href": "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101/snapshot-
s/80407f78-7f90-4d9f-83f4-10d3a1e982ac"
      }
    }
  }
}
]

```

Assign a snapshot ID variable.

```
$ snapshot_id='80407f78-7f90-4d9f-83f4-10d3a1e982ac'
```

List information about the snapshot.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq ""
```

The response shows information about the single snapshot.

```

{
  "name": "demo-snapshot-2",
  "snapshotProperties": {
    "incremental": "false"
  },
  "id": "80407f78-7f90-4d9f-83f4-10d3a1e982ac",
  "createdAt": " 2022-04-02",
  "updatedAt": " 2022-04-02",
  "owner": "user@mycompany.com",
  "organizationId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
  "orgId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
  "_links": {
    "self": {
      "href": "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101/snapshot-
s/80407f78-7f90-4d9f-83f4-10d3a1e982ac"
    }
  }
}

```

Update the Custom Properties of a Machine

After deploying a machine, you can use the IaaS APIs to update the machine with custom properties. Custom properties provide you with the flexibility to add any information about the machine that you want.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the virtual machine in your deployment. See [Get Deployment Resource IDs](#).

For example, machine IDs are typically autogenerated. By updating the custom properties, you can identify the machine owner and include their contact email or phone information.

1. Assign your virtual machine ID variable.

Assigning this variable is useful if you plan to update the machine again.

```
virtual_machine_id='<your_virtual_machine_id>'
```

2. Update the machine with custom property names and values that you choose.

```
curl -X PATCH \
  $url/iaas/api/machines/$virtual_machine_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "customProperties": {
      "additionalPropName1": "<custom_prop_value_1>",
      "additionalPropName2": "<custom_prop_value_2>",
      "additionalPropName3": "<custom_prop_value_3>"
    },
    "description": "string",
    "tags": "[ { \"key\" : \"ownedBy\", \"value\": \"Rainpole\" } ]"
  }' | jq "
```

3. A snippet of the response lists the added custom properties.

Add a Custom Properties to Your Virtual Machine

Update the virtual machine with resource ID 42f49781-1490-4a08-ae21-8baf383a72ac by adding custom properties.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
```

Assign the virtual machine ID.

```
$ virtual_machine_id='42f49781-1490-4a08-ae21-8baf383a72ac'
```

Update the machine with custom properties.

```
$ curl -X PATCH \
  $url/iaas/api/machines/$virtual_machine_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "customProperties": {
      "ownerName": "VMuser_Example",
      "ownerEmail": "VMuser_Example@mycompany.com",
      "ownerCell": "123.456.7890"
    }
  }'
```

```

},
"description": "string",
"tags": "[ { \"key\" : \"my.enumeration.type\", \"value\": \"ec2_instance\" } ]"
}' | jq "."

```

A snippet of the response shows that the request was successful.

```

...
"customProperties": {
  "ownerName": "VMuser_Example",
  "ownerEmail": "VMuser_Example@mycompany.com",
  "ownerCell": "123.456.7890"
  "image": "ubuntu",
  "OSType": "LINUX",
  "imageId": "ami-b1234cc5",
  ...
},
...

```

Provision a VLAN Private Network

After creating a network profile with a VLAN transport zone, you can provision a VLAN private network.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the project ID. See [Create a Project to use in VCF Automation](#).
- Verify that you know the VLAN IDs that you want to use. Comma separated values can be 0 - 4094.

With VLAN IDs in your private network definition, VCF Automation allocates the blueprint to a network profile with a VLAN transport zone specified. If no VLAN ID is specified, the blueprint is allocated to a network profile with an overlay transport zone.

1. Assign your project ID variable.

```
project_id='<your_project_id>'
```

2. Provision the network with VLAN IDs that you choose.

```

curl -X POST \
  $url/iaas/api/networks?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "<your_network_name>",
    "description": "<your_description>",
    "projectId": "'$project_id'",
    "customProperties": {
      "networkType": "PRIVATE",
      "vlanIds": "<integer_values_0_to_4094>"
    }
  }' | jq "."

```

The response includes a selfLink value.

```

{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",

```



```
    "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
  }
}
```

3. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

4. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id | jq ". "
```

Provision a VLAN Private Network

Provision a VLAN private network for a project ID

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
```

Assign the project ID.

```
$ project_id='5944aacb-91de-4541-bb9e-ef2a5403f81b'
```

Provision the VLAN private network.

```
curl -X POST \
  $url/iaas/api/networks?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "example-vlan-network",
    "description": "Example VLAN Network created using API",
    "projectId": "'$project_id'",
    "customProperties": {
      "networkType": "PRIVATE",
      "vlanIds": "1004, 1005"
    }
  }' | jq ". "
```

The response includes a selfLink.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "dab1fe2f-4104-4fce-b534-e7ab5c172788",
  "selfLink": "/iaas/api/request-tracker/dab1fe2f-4104-4fce-b534-e7ab5c172788",
  "deploymentId": "8964a3f6-e829-40ad-b07c-0177abb7b4f4"
}
```

Assign the selfLink variable.

```
selfLink_id='dab1fe2f-4104-4fce-b534-e7ab5c172788'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id | jq ". "
```

When the request completes successfully, the response shows the network has been created.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/networks/3f3b4f87-3e0d-48e8-a484-a8f2890977df"
  ],
  "name": "example-vlan-network Allocation",
  "id": "1cca9d9c-1e93-40d6-8bab-fa92dd89566a",
  "selfLink": "/iaas/api/request-tracker/1cca9d9c-1e93-40d6-8bab-fa92dd89566a",
  "deploymentId": "8b8eef00-e159-4ba9-9e78-09089a3f5786"
}
```

How do I use a placement policy to spread VMs by memory

If you want to balance your deployed resources across multiple cloud zones, you can use the IaaS API to define the placement policy in a project and its cloud zones. When you deploy a blueprint that uses the project, VCF Automation allocates new VMs to zones and clusters with the most free memory, effectively spreading them for better memory usage.

Prerequisites for defining a placement policy

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

How to specify spread by memory in your project

This example shows how to find the project with cloud zones where you want to place your VMs. Then using the project ID, you update the project with "placementPolicy": "SPREAD_MEMORY" to spread VM memory use across the cloud zones.

Get all projects.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects?apiVersion=$api_version" | jq "."
```

A snippet of the response shows a project with its assigned cloud zones. The project ID 6c2f2d0d-ecce-42e3-90be-7bb66d6da2f9 has two zones with zone IDs:

- 3c2bbe36-bf8e-4484-9c31-ce552422aaf1
- 8992bdf0-136f-401c-822a-e22dae67259b

```
{
  "administrators": [],
  "members": [],
  "viewers": [],
  "zones": [
    {
      "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
      "priority": 0,
      "maxNumberInstances": 0,
      "allocatedInstancesCount": 0,
      "memoryLimitMB": 0,
      "allocatedMemoryMB": 0,
    }
  ]
}
```

```

        "cpuLimit": 0,
        "allocatedCpu": 0,
        "storageLimitGB": 0,
        "allocatedStorageGB": 0.0
    },
    {
        "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
        "priority": 0,
        "maxNumberInstances": 0,
        "allocatedInstancesCount": 0,
        "memoryLimitMB": 0,
        "allocatedMemoryMB": 0,
        "cpuLimit": 0,
        "allocatedCpu": 0,
        "storageLimitGB": 0,
        "allocatedStorageGB": 0.0
    }
],
"constraints": {},
"operationTimeout": 0,
"sharedResources": true,
"placementPolicy": "DEFAULT",
"customProperties": {},
"name": "project1",
"description": "",
"id": "6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9",
"orgId": "f098d692-e980-41a5-b349-83084fcea0",
"_links": {
    "self": {
        "href": "/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9"
    }
}
}
}

```

Use the project ID to update the placement policy and spread memory over the two cloud zones.

```

curl -X PATCH \
  "$url/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
  "placementPolicy": "SPREAD_MEMORY"
}' | jq "."

```

How to specify spread by memory in cloud zones

This example shows how to use the cloud zone IDs in the project ID 6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9 to check free memory in compute resources. Provided that free memory is available, you update the cloud zones with "placementPolicy": "SPREAD_MEMORY".

To check available memory in zone ID 3c2bbe36-bf8e-4484-9c31-ce552422aaf1, list compute resources.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/zones/3c2bbe36-bf8e-4484-9c31-ce552422aaf1/computes?apiVersion=$api_version" | jq "."

```

The response shows two clusters with available memory:

- ESO_PKS_VC01_Cluster03 has 747 Gbytes available memory.
- ESO_PKS_VC01_Cluster04 has 397 Gbytes available memory.

```
{
  "content": [
    {
      "externalRegionId": "Datacenter:datacenter-3",
      "tags": [],
      "type": "Cluster",
      "lifecycleState": "READY",
      "powerState": "ON",
      "customProperties": {
        "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
        "hostCount": "2",
        "datacenter": "Datacenter:datacenter-3",
        "cpuPkgCount": "4",
        "cpuCoreCount": "56",
        "vsanConfigId": "52a60c7a-ef2e-af08-7cb2-36b06f686ebb",
        "isVsanEnabled": "true",
        "MaxVCPUPerInstance": "56",
        "MemoryAvailableBytes": "747407147008"
      },
      "externalId": "domain-c21",
      "name": "ESO_PKS_VC01_Cluster03",
      "id": "e03f62e1-9a48-4d2c-8aa7-7bdb97293749",
      "createdAt": "2022-07-25",
      "updatedAt": "2022-07-26",
      "orgId": "f098d692-e980-41a5-b349-83084fcea0"
    },
    {
      "externalRegionId": "Datacenter:datacenter-3",
      "tags": [],
      "type": "Cluster",
      "lifecycleState": "READY",
      "powerState": "ON",
      "customProperties": {
        "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
        "hostCount": "1",
        "datacenter": "Datacenter:datacenter-3",
        "cpuPkgCount": "2",
        "cpuCoreCount": "28",
        "vsanConfigId": "",
        "isVsanEnabled": "false",
        "MaxVCPUPerInstance": "56",
        "MemoryAvailableBytes": "397091536896"
      },
      "externalId": "domain-c24",
      "name": "ESO_PKS_VC01_Cluster04",
      "id": "d2d42957-b6df-4e45-879a-93dfbec9a528",
      "createdAt": "2022-07-25",
      "updatedAt": "2022-07-26",
      "orgId": "f098d692-e980-41a5-b349-83084fcea0"
    }
  ]
}
```

```

    }
  ],
  "totalElements": 2,
  "numberOfElements": 2
}

```

Use the zone ID `3c2bbe36-bf8e-4484-9c31-ce552422aaf1` to spread memory over the clusters.

```

curl -X PATCH \
  "$url/iaas/api/zones/3c2bbe36-bf8e-4484-9c31-ce552422aaf1?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
  "placementPolicy": "SPREAD_MEMORY"
}' | jq "."

```

To check available memory in zone ID `8992bdf0-136f-401c-822a-e22dae67259b`, list compute resources.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/zones/8992bdf0-136f-401c-822a-e22dae67259b/computes?apiVersion=$api_version" | jq "."

```

The response shows two clusters with available memory:

- ESO_PKS_VC01_Cluster01 has 830 Gbytes available memory.
- ESO_PKS_VC01_Cluster05 has 397 Gbytes available memory.

```

{
  "content": [
    {
      "externalRegionId": "Datacenter:datacenter-3",
      "tags": [],
      "type": "Cluster",
      "lifecycleState": "READY",
      "powerState": "ON",
      "customProperties": {
        "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
        "hostCount": "5",
        "datacenter": "Datacenter:datacenter-3",
        "cpuPkgCount": "10",
        "cpuCoreCount": "140",
        "vsanConfigId": "52de077e-2f21-b24c-536b-79ef9a412968",
        "isVsanEnabled": "true",
        "MaxVCPUperInstance": "56",
        "MemoryAvailableBytes": "830677712896"
      },
      "externalId": "domain-c18",
      "name": "ESO_PKS_VC01_Cluster01",
      "id": "98691ba2-2f84-4d31-a9f1-690d254c5305",
      "createdAt": "2022-07-25",
      "updatedAt": "2022-07-26",
      "orgId": "f098d692-e980-41a5-b349-83084fcea0"
    },
    {
      "externalRegionId": "Datacenter:datacenter-3",
      "tags": [],
      "type": "Cluster",

```

```

    "lifecycleState": "READY",
    "powerState": "ON",
    "customProperties": {
      "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
      "hostCount": "1",
      "datacenter": "Datacenter:datacenter-3",
      "cpuPkgCount": "2",
      "cpuCoreCount": "28",
      "vsanConfigId": "",
      "isVsanEnabled": "false",
      "MaxVCPUPerInstance": "56",
      "MemoryAvailableBytes": "397045399552"
    },
    "externalId": "domain-c26",
    "name": "ESO_PKS_VC01_Cluster05",
    "id": "f576f346-ed3b-4180-acdc-3c217e9fa0fd",
    "createdAt": "2022-07-25",
    "updatedAt": "2022-07-26",
    "orgId": "f098d692-e980-41a5-b349-83084fcea0"
  }
],
"totalElements": 2,
"numberOfElements": 2
}

```

Use the zone ID 8992bdf0-136f-401c-822a-e22dae67259b to spread memory over the clusters.

```

curl -X PATCH \
  "$url/iaas/api/zones/8992bdf0-136f-401c-822a-e22dae67259b?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
  "placementPolicy": "SPREAD_MEMORY"
}' | jq "."

```

Protecting Sensitive Data

By using the VCF Automation IaaS API to mark certain data as sensitive in a request body, you can store the data in encrypted form, and ensure that only the encrypted form of data is visible in the response. VCF Automation decrypts the data only when the actual value is needed, for example before sending a request to the cloud.

Data encryption works for certain types of data and is limited to the following use cases:

- When provisioning resources such as machines, load balancers, disks, or networks, the following types of data support encryption:
 - Custom property values for all types of resources.
 - Remote access passwords for machines.
 - Sensitive parts of the cloud config for machines.
- When creating or updating projects, custom properties support encryption.
- When updating a deployed machine, custom properties support encryption.

Note: Data encryption is only supported for deployed machines. It is not supported for discovered machines.

- When creating or updating image profiles, cloud config supports encryption. This means that you can mark parts of the cloud config script as sensitive. For example if the script includes passwords, you can mark the passwords as sensitive.

How to provision a machine with sensitive data

To mark data as sensitive, you add sensitive values with a prefix and suffix. The following example shows how to provision a new machine with sensitive values such as custom properties and a remote access password. This machine is also provisioned with a project that includes an encrypted custom property, so that the custom property is added to the machine.

1. In VCF Automation, create a cloud account. Add a cloud zone to the cloud account and add a flavor mapping and image mapping to the cloud zone.
2. In your browser or HTTP client application, verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
3. Create a project with the cloud zone that you created using the VCF Automation UI. Include a sensitive custom property for the Active Directory (AD) password. In this way, when users related to the project provision resources with the project, they have the same AD password.

The following example shows the AD password enclosed with the `((sensitive: prefix and the ))` suffix to mark it as sensitive.

```
curl -X POST \
  "$url/iaas/api/projects?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
  "name" : "example-project",
  "customProperties": {
    "activeDirectoryPassword": ((sensitive:My-password123!)) "
  }
}' | jq "
```

A snippet of the response lists the project ID.

```
...
"name": "example-project",
"description": "This is an example project",
"id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

4. Provision a virtual machine with sensitive data.

The following example includes the custom property `costCenterPassword` and a password for remote access, with values that are both marked as sensitive using the `((sensitive: prefix and the ))` suffix. The request body also includes the ID of the project with the encrypted AD password.

```
curl -X POST \
  "$url/iaas/api/machines?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
  "name" : "example-vm",
  "image" : "ubuntu",
  "flavor" : "small",
  "projectId" : "5944aacb-91de-4541-bb9e-ef2a5403f81b",
```

```

    "customProperties": {
      "costCenterPassword": "((sensitive:Pass4costCtr$$$))"
    }
    "remoteAccess": {
      "authentication": "usernamePassword",
      "username": "example-user",
      "password": "((sensitive:example-sensitive-pass!123))"
    }
  } | jq ".

```

Note: The password for remote access is marked sensitive as an example. If left unmarked, the remote access password is encrypted because it is sensitive by default.

5. After successfully provisioning the machine, issue a `GET /iaas/api/machines` request to obtain information about the machine.

In a snippet of the response, values for the custom property `costCenterPassword` and remote access password are encrypted and appear in their encrypted form with the `((secret:v1: prefix` as in the following example.

```

... "customProperties": {
  ...
  "costCenterPassword": "((secret:v1:AAHeSZhRynh8+NSdswAdsfdsgSDffhbfh))",
  ...
},
...
"bootConfig": {
  "content": "#cloud-config\nusers:\n- default\n- name: example-user\n...\n passwd: ((secret:v1:AAF-PdqFQBiJbGKdklseihSN28ckjSghjngj))\n...\n"
}
...

```

VCF Automation converts the remote access information in the request into a cloud config script in the response. The encrypted password appears as a content value in the `bootConfig`.

Verify that the remote access password works

Even though the password is encrypted in the VCF Automation database, you can use the user name and plain text password from the request to log in to the machine because the password is decrypted before it is sent to the cloud.

Note: You can choose to verify that your remote access password works only if the cloud provider allows remote access. For example, Azure might allow remote access while GCP or AWS might not.

To test your password, use the IP address of the newly provisioned machine such as `192.168.12.1234` and the user name such as `example-user`. Log in to the remote machine with:

```
$ ssh example-user@192.168.12.1234
```

When prompted for the password, copy and paste the plain text password from the request or `example-sensitive-pass!123`. A successful login verifies that the machine was provisioned with the remote access password provided in the request.

Properties that Support Encryption

Image profiles, projects and all types of provisioned entities can include sensitive information. The table below lists all endpoints that support encryption and the parts of the request body that can contain sensitive data.

Endpoint	You can apply encryption to:	Example Input
Create/update/list machines	- customProperties: value - nics: customProperties: value	<pre>{ "name": "machine1",</pre>

Endpoint	You can apply encryption to:	Example Input
Note: Data encryption is only supported for deployed machines. It is not supported for discovered machines.	- Sensitive parts of bootConfig: content - remoteAccess: password	<pre>"customProperties": { "username": "guest", "password": "((sensitive:my- pass))" }</pre>
Create/update/list image profiles	Sensitive parts of imageMapping: value: cloudConfig	<pre>{ "imageMapping" : { "ubuntu": { "id": "{{awsUbuntuId}}", "cloudConfig": "#cloud-con- fig\nchpasswd:\n list:\nuser1:((sensitive:pass- word1))" } }, "name" : "aws-image-profile", "regionId": "{{awsRegionId}}" }</pre>
Create/update/list projects	customProperties: value	<pre>{ "name": "project1", "customProperties": { "vidm-password": "((sensi- tive:mypass))" } }</pre>
Create/list load balancers	- customProperties: value - nics: customProperties: value	<pre>{ "name": "load-balancer1", "nics": { "customProperties": { "dhcp-server-password": "((sensitive:mypass))" } } }</pre>
Create/list networks	customProperties: value	<pre>{ "name": "network1", "customProperties": { "dhcp-server-password": "((sensitive:mypass))" } }</pre>
Create/list security groups	customProperties: value	<pre>{ "name": "security-group1", "customProperties": { "some-password": "((sensi- tive:mypass))" } }</pre>

Endpoint	You can apply encryption to:	Example Input
		<pre> } } </pre>
Create/list block device	customProperties: value	<pre> { "name": "device1", "customProperties": { "some-password": "((sensitive:mypass))" } } </pre>
Create/list compute gateway	customProperties: value	<pre> { "name": "gateway1", "customProperties": { "some-password": "((sensitive:mypass))" } } </pre>

Querying with the VCF Automation APIs

By adding query options to an API request, you control the amount of output returned by the server and make the response easier to interpret. The API service uses the options specified to transform the data by filtering or paginating before returning the results.

You can use the following query options in your API requests. The options do not apply to all endpoints.

\$top	Number of records to get. For more information, see Using Pagination and Count .
\$skip	Number of records to skip. For more information, see Using Pagination and Count .
\$count	If set to <code>true</code> , shows the total number of records. If used with a filter, shows the number of records matching the filter. For more information, see Using Pagination and Count .
\$select	Names the subset of properties to list in the response.
\$filter	Filters results by a predicate expression with operators such as, <code>eq</code> , <code>ne</code> , <code>and</code> , and <code>or</code> . For specialized filtering examples, see: Filtering Resources by Region ID Filtering Operations for Projects Filtering for Machine Status

Endpoints that support all query options

To query for any of the following endpoints, you can use all options. Examples show how to construct a request using the `$filter` option with a logical `or` operation.

Note: VCF Automation APIs do not support filtering for nested properties with a "." in the property name.

For example, you can filter for a property with the name `createdByEmail` as in the following example:

```
$filter=customProperties.createdByEmail%20eq%20'user@mycompany.com'
```

However, API filtering does not support a property with the name `my.createdByEmail` as in the following example:

```
$filter=customProperties.my.createdByEmail%20eq%20'user@mycompany.com'
```

Endpoint	Example
Machine	<code>\$url/iaas/api/machines?\$filter=name%20ne%20'example-name'%20or%20customProperties.osType%20eq%20'example-os'</code>
Cloud Account	<code>\$url/iaas/api/cloud-accounts?\$filter=name%20ne%20'example-cloud-account'%20or%20customProperties.isExternal%20eq%20'false'</code>
Fabric Azure Storage Account	<code>\$url/iaas/api/fabric-azure-storage-accounts/?\$filter=name%20ne%20'example-name'%20or%20type%20eq%20'example-type'</code>
Fabric Compute	<code>\$url/iaas/api/fabric-computes?\$filter=name%20ne%20'example-name'%20or%20customProperties.isExternal%20eq%20'false'</code>
Fabric Image	<code>\$url/iaas/api/fabric-images?\$filter=name%20ne%20'example-name'%20or%20osFamily%20eq%20'example-os'</code>
Fabric Network	<code>\$url/iaas/api/fabric-networks?\$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'</code>
Fabric Network (vSphere)	<code>\$url/iaas/api/fabric-networks-vsphere?\$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'</code>
Fabric vSphere Datastores	<code>\$url/iaas/api/fabric-vsphere-datastores?\$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'</code>
Fabric vSphere Storage Policies	<code>\$url/iaas/api/fabric-vsphere-storage-policies?\$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'</code>

Querying for endpoints with a specified ID

To query for an endpoint with specified ID, you can only use the `$select` option. Examples show how to construct a request.

Endpoint	Example
Cloud Account by ID	<code>\$url/iaas/api/cloud-account/{id}?\$select=name</code>
Machine by ID	<code>\$url/iaas/api/machines/{id}?\$select=name</code>
Fabric Azure Storage Account by ID	<code>\$url/iaas/api/fabric-azure-storage-accounts/{id}?\$select=name</code>
Fabric Image by ID	<code>\$url/iaas/api/fabric-images/{id}?\$select=name</code>
Fabric Network by ID	<code>\$url/iaas/api/fabric-networks/{id}?\$select=name</code>
Fabric Network (vSphere) by ID	<code>\$url/iaas/api/fabric-networks-vsphere/{id}?\$select=name</code>

Endpoint	Example
Fabric vSphere Datastores by ID	<code>\$url/iaas/api/fabric-vsphere-datastores/{id}?select=name</code>
Fabric vSphere Storage Policies by ID	<code>\$url/iaas/api/fabric-vsphere-storage-policies/{id}?select=name</code>

Querying for a partial match

To query for the partial match of a name that starts with, ends with, or is contained within another name, the `$filter` options are the same for most IaaS endpoints but are different for `iaas/api/projects` and `iaas/api/deployments` endpoints. Examples show how to construct the partial match filters for the different endpoint types.

Filter Operation	Query with <code>iaas/api/projects</code> or <code>iaas/api/deployments</code>	Query with most IaaS endpoints
Name starts with <code>foo</code>	<code>?\$filter=startswith(name, 'foo')</code>	<code>\$filter=name%20eq%20'foo*'</code>
Name ends with <code>foo</code>	<code>?\$filter=endswith(name, 'foo')</code>	<code>\$filter=name%20eq%20'*foo'</code>
<code>foo</code> contained within the name	<code>?\$filter=substringof('foo', name)</code>	<code>\$filter=name%20eq%20'*foo*'</code>

Querying for deployments

To query for deployments, you can use all options except `$select`. The following example shows how to use the `$filter` option to list deployments that are not named `example-name` or have `projectId='example-id'`.

```
GET $url/iaas/api/deployments?$filter=name%20ne%20'example-name'%20or%20projectId%20eq%20'example-id'
```

Using Pagination and Count

Pagination controls the number of elements or the range of elements returned in an API response. When the count flag is specified, the response shows the total number of records. Use parameters in combination to traverse all elements in a result.

To paginate a response, you use the following parameters.

<code>\$top=N</code>	Selects only the first N elements of the set. N must be a positive integer. Specifying N limits the maximum number of elements that the server returns in the response. The default IaaS API page size is 100. If <code>\$top</code> is left unspecified, pagination selects the first 100 elements. If you want the output to include elements beyond the first 100 elements, specify a top value greater than 100.
<code>\$skip=N</code>	Skips N elements and selects only the remaining elements starting with element N+1.

Pagination examples

The following examples show how to combine parameter values to control the elements returned in a vSphere deployment with 45 cloud accounts.

If you want to...	Use this request
List the first 20 cloud accounts, or 1–20	<pre>curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" "\$url/iaas/ api/cloud-accounts?apiVersion=\$api_version&\$fil- ter=cloudAccountType%20eq%20%27vsphere%27&\$top=20& \$skip=0" jq "."</pre>
List the second set of 20 cloud accounts, or 21–40	<pre>curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" "\$url/iaas/ api/cloud-accounts?apiVersion=\$api_version&\$fil- ter=cloudAccountType%20eq%20%27vsphere%27&\$top=20& \$skip=20" jq "."</pre>
List the third set of 20 cloud accounts, or 41–45	<pre>curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" "\$url/iaas/ api/cloud-accounts?apiVersion=\$api_version&\$fil- ter=cloudAccountType%20eq%20%27vsphere%27&\$top=20& \$skip=40" jq "."</pre>

Count example

The following example shows how to count the AWS cloud accounts.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-
accounts?$filter=cloudAccountType%20eq%20%27aws%27&$count=true" | jq "."
```

Filtering Resources by Region ID

You can use a filter in the IaaS APIs to identify resources provisioned in a cloud account of a particular region. By specifying the resources for which you want information such as network and security, compute, storage, and tags content, you limit the API response to provide only the information that you want.

You can also use filtering to identify and automatically update resources. For example, if you have automated builds on an vSphere private cloud account, you can use filtering to identify the image profiles associated with a region of the cloud account and automatically update those profiles in VCF Automation.

Obtaining the externalRegionID and cloudAccountID

To get information about a resource that is in a region of a cloud account, you filter by its externalRegionID and its cloudAccountID. The following example shows how to obtain the IDs for a vSphere cloud account and use them to construct a generic query. It assumes that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

1. List vSphere cloud accounts.

```
curl -X GET "$url/iaas/api/cloud-accounts?apiVersion=$api_version&""$filter=""cloudAccountType%20eq
%20'vsphere'" -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."
```

2. Examine the response to obtain the cloud account ID and region ID for the vSphere account that you want.

```
...
},
"name": "vc60",
"description": "Created by User",
"id": "e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bd-
f1c802400819ad45a341cba1f3-c39814fe67b8247557cab2652647d",
```

```

    "updatedAt": " ",
    "organizationId": "f670fd6c-66d6-4689-9793-d524e7066d1e",
    "orgId": "f670fd6c-66d6-4689-9793-d524e7066d1e",
    "_links": {
      "regions": {
        "hrefs": [
          "/iaas/api/regions/277e3cd9fe87527557cab268cae5a"
        ]
      }
    },
  },
  ...

```

3. Assign the cloud account ID and region ID variables.

```

cloud_account_id='e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bd-
f1c802400819ad45a341cba1f3-c39814fe67b8247557cab2652647d'
region_id='277e3cd9fe87527557cab268cae5a'

```

4. List regions with the region ID and associated with the cloud account ID.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/regions/?apiVersion=$api_version&"'$filter'="'id%20eq%20"$region_id"%20and%20cloudAccountId%20eq
%20"$cloud_account_id"'" | jq "."

```

5. Examine the response to obtain the external region ID.

```

...
{
  "externalRegionId": "Datacenter:datacenter-2",
  "name": "VC60-Datacenter",
  "cloudAccountId": "e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bd-
f1c802400819ad45a341cba1f3-c39814fe67b8247557cab2652647d",
  "id": "277e3cd9fe87527557cab268cae5a",
  "updatedAt": " 2022-04-02",
  ...
}

```

6. Assign the external region ID variable.

```

external_region_id='Datacenter:datacenter-2'

```

Using the externalRegionId and cloudAccountId, you can construct a generic filter to use in any IaaS API that queries for resources.

```

$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"

```

Constructing a query for a VMC cloud account

The VMC cloud account consists of an AWS cloud account facade and associated internal vSphere and NSX cloud accounts. To construct the query for a resource in the VMC cloud account, you use the cloudAccountId of the associated vSphere cloud account and not the cloudAccountId of the VMC cloud account.

The following example shows how to obtain the cloudAccountId and externalRegionId and use them to construct a generic query. It assumes that you know the name of the VMC cloud account and that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

1. Assign the cloud account name variable.

```

cloud_account_name='<my_vmc_cloud_account>'

```

2. List VMC cloud accounts with your VMC cloud account name.

```

curl -X GET "$url/iaas/api/cloud-accounts?apiVersion=$api_version&"'$filter="'cloudAccountType%20eq%20'vm-
c'%20and%20name%20eq%20'$cloud_account_name'" -H 'Content-Type: application/json' -H "Authorization:
Bearer $access_token" | jq "."

```

3. Examine the response to obtain the cloud account ID for the associated vSphere cloud account.

Under `_links`, two associated cloud accounts are listed. The first cloud account is the vSphere cloud account. The second is the NSX cloud account.

```
...
  "_links": {
    "regions": {
      "hrefs": [
        "/iaas/api/regions/5a8da7fe-63d9-4f82-84e8-f47bdfdef43c"
      ]
    },
    "associated-cloud-accounts": {
      "hrefs": [
        "/iaas/api/cloud-accounts/df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05",
        "/iaas/api/cloud-accounts/6993943b-82bc-4ec7-bad6-65f2f595da7e"
      ]
    },
    "self": {
      "href": "/iaas/api/cloud-accounts/95be1cc2-c70a-4311-9f18-6218786ac51b"
    }
  }
}

...
```

4. To assign the cloud account ID variable, use the associated vSphere cloud account.

```
assoc_cloud_account_id='df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05'
```

5. List regions in the associated vSphere cloud account.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'="'cloudAccountId%20eq%20'$assoc_cloud_account_id'" | jq "."
```

6. Examine the response to obtain the external region ID.

```
...
{
  "externalRegionId": "Datacenter:datacenter-3",
  "cloudAccountId": "df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05",
  "id": "e572c96b-9084-4c37-b74d-bf61f0b08e79",
  "updatedAt": " 2022-04-02",
  "organizationId": "33a92056-18f0-469c-a088-9c8eae4e4888",
  "orgId": "33a92056-18f0-469c-a088-9c8eae4e4888",
  "_links": {
    "self": {
      "href": "/iaas/api/regions/e572c96b-9084-4c37-b74d-bf61f0b08e79"
    },
    "cloud-account": {
      "href": "/iaas/api/cloud-accounts/df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05"
    }
  }
}

...
```

7. Assign the external region ID variable.

```
external_region_id='Datacenter:datacenter-3'
```

Using the `externalRegionId` and `cloudAccountId` for the associated vSphere cloud account, you can construct a generic filter to use in any IaaS API that queries for resources in the VMC cloud account.

```
$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$assoc_cloud_account_id'"
```

Resource Query Examples

The following examples show how to incorporate the filter into resource queries.

Resource	Query
Security Group	<code>\$url/iaas/api/security-groups?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Security Group (NSX only)	<code>\$url/iaas/api/security-groups?\$filter="externalRegionId%20eq%20'global'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Region	<code>\$url/iaas/api/regions?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Network	<code>\$url/iaas/api/networks?\$filter="externalRegionId%20eq%20'\$region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Network Domains	<code>\$url/iaas/api/network-domains?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Machine	<code>\$url/iaas/api/machines?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Load Balancer	<code>\$url/iaas/api/load-balancers?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Fabric vSphere Datastore	<code>\$url/iaas/api/fabric-vsphere-datastores?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Fabric Network vSphere	<code>\$url/iaas/api/fabric-networks-vsphere?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Fabric Network	<code>\$url/iaas/api/fabric-networks?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Fabric Compute	<code>\$url/iaas/api/fabric-computes?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Block Device	<code>\$url/iaas/api/block-devices?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>

Resource	Query
Network Profile	<code>\$url/iaas/api/network-profiles?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Flavor Profile	<code>\$url/iaas/api/flavor-profiles?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Image Profiles	<code>\$url/iaas/api/image-profiles?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Storage Profiles	<code>\$url/iaas/api/storage-profiles?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Storage Profiles Azure	<code>\$url/iaas/api/storage-profiles-azure?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Storage Profiles AWS	<code>\$url/iaas/api/storage-profiles-aws?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>
Storage Profiles vSphere	<code>\$url/iaas/api/storage-profiles-vsphere?\$filter="externalRegionId%20eq%20'\$external_region_id'%20and%20cloudAccountId%20eq%20'\$cloud_account_id'"</code>

Filtering for Machine Status

To filter for machines with deployed or discovered status, you can use the IaaS APIs. A machine with the discovered status is in the cloud and has not yet been onboarded. A machine with the deployed status has been onboarded or provisioned from VCF Automation.

After onboarding, the machine has a deployed status and is associated with the deployment. Then you can manage it in the same way as any other provisioned machine.

To query for deployed or discovered status, you use the following parameters.

<code>\$filter=deploymentId eq '*'</code>	Returns all machines that are deployed and under VCF Automation management. Deployed machines have a deployment ID.
<code>\$filter=deploymentId ne '*'</code>	Returns all machines that are discovered in the cloud. Machines without the deployment ID property are not deployed.

Machine filter examples

The following examples show how to list deployed or discovered machines.

If you want to...	Use this request
List all deployed machines	<code>curl -X GET "\$url/iaas/api/machines?apiVersion=\$api_version&"'\$filter="deploymentId%20eq%20*' -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" jq ""</code>
List all discovered machines	<code>curl -X GET "\$url/iaas/api/cloud-accounts?apiVersion=\$api_version&"'\$filter="deploymentId%20ne%20*' -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" jq ""</code>

Filtering Operations for Projects

To restrict the output from a project endpoint, you include a filter in the GET `/project-service/api/projects` request. The API service uses the operation to filter for a collection of projects to paginate, order, or count your results.

Filter operation examples

The following examples show how to incorporate filter operations into project queries.

Operation	Query
equal	<code>\$url/project-service/api/projects?\$filter=name%20eq%20'00-TestProject'</code>
not equal	<code>\$url/project-service/api/projects?\$filter=name%20ne%20'00-TestProject'</code>
logical and	<code>\$url/project-service/api/projects?\$filter=name%20eq%20'00-TestProject'%20and%20sharedResources%20eq%20true</code>
logical or	<code>\$url/project-service/api/projects?\$filter=name%20eq%20'00-TestProject'%20or%20sharedResources%20eq%20false</code>
logical negation	<code>\$url/project-service/api/projects?\$filter=not%20startswith(name, '00')</code>
starts with	<code>\$url/project-service/api/projects?\$filter=startswith(name, '00')</code>
substring of	<code>\$url/project-service/api/projects?\$filter=substringof('00', name)</code>
ends with	<code>\$url/project-service/api/projects?\$filter=endswith(name, '00')</code>
length	<code>\$url/project-service/api/projects?\$filter=length(name)%20eq%205</code>
index of	<code>\$url/project-service/api/projects?\$filter=indexof(name, '00')%20eq%200</code>
substring from	<code>\$url/project-service/api/projects?\$filter=substring(name, 1)%20eq%20'0-Te'</code>

Operation	Query
substring from to	<code>\$url/project-service/api/projects?\$filter=substring(name, 1, 2)%20eq%20'0-'</code>
to lower	<code>\$url/project-service/api/projects?\$filter=tolower(name)%20eq%20'00-testproject'</code>
to upper	<code>\$url/project-service/api/projects?\$filter=toupper(name)%20eq%20'00-TESTPROJECT'</code>
concatenate	<code>\$url/project-service/api/projects?\$filter=concat(concat(name, ','), description)%20eq%20'test project, test project description'</code>
trim	<code>\$url/project-service/api/projects?\$filter=trim(name)%20eq%20'00-TestProject'</code>

Related query examples

The following examples show how to use related query options to paginate, order, or count your results.

If you want to...	Use this request
List the first twenty items	<code>curl -X GET \$url/project-service/api/projects?\$top=20 -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" jq "."</code>
Skip the first ten items in the collection	<code>curl -X GET \$url/project-service/api/projects?\$skip=10 -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" jq "."</code>
List the items in ascending order with <code>asc</code> or descending order with <code>desc</code> . If the type of order is not specified, the list defaults to ascending order.	For ascending order: <pre>curl -X GET \$url/project-service/api/projects?\$skip=0\$orderBy=name%20asc -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" jq "."</pre> For descending order: <pre>curl -X GET \$url/project-service/api/projects?\$skip=0\$orderBy=name%20desc -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" jq "."</pre>
List the total number of items in the collection.	<code>curl -X GET \$url/project-service/api/projects?\$count=true -H 'Content-Type: application/json' -H "Authorization: Bearer \$access_token" jq "."</code>

Setting up VCF Automation using APIs

As a VCF Automation administrator, you can use the Infrastructure as a Service (IaaS) APIs to set up connections with your cloud account vendor and integration applications in VCF Automation.

Adding Cloud Accounts

You use the IaaS APIs to create cloud accounts for various cloud platforms. VCF Automation uses permissions configured in the cloud accounts to collect data from regions or data centers, and to deploy blueprints to those regions.

After adding a cloud account, you use the cloud account ID to create a cloud zone, flavor mappings, image mappings, network, profiles, or storage profiles.

Add an Amazon Web Services Cloud Account

To create an Amazon Web Services cloud account with or without cloud zones, you make a POST request. The request body includes the parameters specific to Amazon Web Services that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the Amazon Web Services access key ID and the Amazon Web Services secret access key for the new cloud account.

1. Assign the Amazon Web Services account variables.

```
aws_access_key_id='<your_aws_access_key_id>'
aws_secret_access_key='<your_aws_secret_access_key>'
```

2. Submit a request to create an Amazon Web Services cloud account without default cloud zones.

When the parameter `createDefaultZones` is left unspecified, the cloud account is created without default cloud zones by default.

```
curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "privateKeyId": "'$aws_access_key_id'",
  "cloudAccountType": "aws",
  "name": "<your_aws_cloud_account>",
  "description": "This is a demo AWS cloud account without cloud zones",
  "regionIds": [ "<your_regions_id1>", "<your_region_id2>" ],
  "privateKey": "'$aws_secret_access_key'"
}' | jq "
```

3. Optional: Submit a request to create an Amazon Web Services cloud account with default cloud zones.

```
curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "privateKeyId": "'$aws_access_key_id'",
  "cloudAccountType": "aws",
  "name": "<your_aws_cloud_account>",
  "description": "This is a demo AWS cloud account with default cloud zones",
  "regionIds": [ "<your_region_id1>", "<your_region_id2>" ],
  "createDefaultZones": true,
  "privateKey": "'$aws_secret_access_key'"
}' | jq "
```

4. To obtain your cloud account ID, examine the response.

5. Assign the cloud account ID variable to your cloud account ID.

```
cloud_account_id='<your_cloud_account_id>'
```

6. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."
```

7. Optional: To look up the cloud account you created, list the cloud account with your cloud account ID.

```
curl -X GET $url/iaas/api/cloud-accounts/$cloud_account_id?apiVersion=$api_version -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."
```

The response shows the cloud account name and ID for the Amazon Web Services cloud account you created.

Create an Amazon Web Services Cloud Account

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ aws_access_key_id='FEDCBA5JJ2F43TUVW7XA'
$ aws_secret_access_key='XYZfGo0/Vb5XPezeC58QRSvLMNOwsHhuB2IbEJxL'
```

Create a cloud account named `demo-aws-account` without default cloud zones.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "privateKeyId": "'$aws_access_key_id'",
  "cloudAccountType": "aws",
  "name": "demo-aws-account",
  "description": "This is a demo AWS cloud account without cloud zones",
  "regionIds": [ "us-east-1", "us-west-1" ],
  "privateKey": "'$aws_secret_access_key'"
}' | jq "."
```

A snippet of the response from your request shows the account ID.

```
...
"tags": [],
"name": "demo-aws-account",
"id": "c8c3c9bfdb449475-7f703c5265a63d87-f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31e-b8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"updatedAt": "2022-04-",
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Add avSphere Cloud Account

To create avSphere cloud account, you make a POST request. The request body includes the parameters specific to avSphere that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

- Verify that you have the following parameters for the new cloud account:
 - vSphere host name
 - vSphere user name
 - vSphere password
- Verify that you have an existing vSphere, NSX, NSX-V, or VMC cloud account that you want to associate with the new cloud account and obtain the cloud account ID.

a. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
  "Authorization: Bearer $access_token" | jq "."
```

b. Examine the response to obtain the cloud account ID such as the `id` value in this example.

```
...
  "name": "vsphere-account-example",
  "id": "b9fa1b42c767de7558ceff3b78004",
  "updatedAt": "2022-04-02",
  "orgId": "f670fd6c-66d6-4689-9793-d524e7066d1e",
...
```

The following procedure shows how to create a vSphere cloud account that supports a trusted certificate. To obtain a trusted certificate, you submit a request to validate asynchronously with the vSphere cloud account specification. When the validation request completes successfully, you use the certificate ID from the response to obtain the trusted certificate that you submit when you create the vSphere cloud account.

1. List all cloud proxies.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
data-collectors?apiVersion=$api_version" | jq "."
```

2. To obtain the data collector ID, examine the response.

3. Assign the data collector ID variable.

```
data_collector_id='<your_datacollector_id>'
```

4. Assign the vSphere account variables.

```
vsphere_host_name='<your_vsphere_host_name>'
vsphere_user='<your_vsphere_user_name>'
vsphere_password='<your_vsphere_password>'
```

5. List external region IDs from a vSphere cloud account.

```
curl -X POST \
"$url/iaas/api/cloud-accounts-vsphere/region-enumeration?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "cloudAccountType": "vsphere",
  "username": "'$vsphere_user'",
  "password": "'$vsphere_password'",
  "hostName": "'$vsphere_host_name'",
  "dcid": "'$data_collector_id'",
  "acceptSelfSignedCertificate": "false"
}' | jq "."
```

6. To obtain the external region ID, examine the response and assign the region ID variable.

```
vsphere_region_id='<your_vsphere_region_id>'
```

7. Submit a request to validate asynchronously with the vSphere cloud account specification.

```
curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts-vsphere?apiVersion=$api_version&validateOnly" | jq "
```

The response includes a selfLink.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Cloud account specification validation",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

8. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

9. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "
```

When the validation request completes successfully, the response includes a resource with the certificate ID.

```
{
  "progress": 0,
  "message": "valid certificate found",
  "status": "SUCCEEDED",
  "resources": [
    "/iaas/api/certificates/example-certificate-id-string"
  ],
  "name": "Cloud account specification validation",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

10. Assign the certificate ID variable.

```
certificate_id='example-certificate-id-string'
```

11. Submit a request to get the certificate information.

```
PEM_for_X509Certificate='curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/certificates/$certificate_id?apiVersion=$api_version" | jq ""'
```

12. Assign the ID of the existing cloud account to associate with the new cloud account.

```
existing_cloud_account_ID='<your_existing_cloud_account_ID>'
```

13. Include the certificate in the request to create avSphere cloud account with default cloud zones.

To create avSphere cloud account without default cloud zones, use "createDefaultZones":false .

```
curl -X POST \
"$url/iaas/api/cloud-accounts-vsphere?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name": "demo-vsphere-account",
  "description": "This is a demo vSphere account with default cloud zones",
```

```

"username": "'$vsphere_user'",
"password": "'$vsphere_password'",
"hostName": "'$vsphere_host_name'",
"acceptSelfSignedCertificate":false,
"associatedCloudAccountIds": "'$existing_cloud_account_ID'",
"createDefaultZones":true,
"dcId": "'$data_collector_id'",
"regions":[
  {
    "name": "'$vsphere_region_id'",
    "ExternalRegionId":"'$vsphere_region_id'"
  }
],
"tags": [
  {
    "key": "env",
    "value": "dev"
  }
],
"certificateInfo":{
  "certificate": "'$PEM_for_X509Certificate'"
}
}' | jq "."

```

Note: The following example shows how to create avSphere cloud account with multiple cloud account IDs. However, the payload can only include a single NSX-P-Cloud endpoint and a single VMC endpoint.

```

"associatedCloudAccountIds": "[\"'$existing_NSXT_cloud_account_ID'\", \"'$existing_VMC_cloud_account_ID'\", \"'$existing_vSphere_cloud_account_ID'\"]",

```

14. List all cloud accounts.

```

curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
"Authorization: Bearer $access_token" | jq "."

```

15. Examine the response and verify that the name and ID of the vSphere cloud account you created is listed.

Create avSphere Cloud Account

This example creates a cloud account with default cloud zones.

Assign the required variables.

```

$ url='https://appliance.domain.com'
$ api_version='2021-07-15'

```

List all cloud proxies.

```

$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/data-collectors?apiVersion=$api_version" | jq "."

```

A snippet of the response from your request shows the data collector IDs.

```

...
{
  "dcId": "60740040-f3cd-4694-96da-15e547242bf7",
  "ipAddress": "10.108.78.154",
  "name": "example-prod-corp-rdc",
  "hostName": "corp-v783-dhcp-79-85.eng.mycompany.com",

```



```
    "status": "ACTIVE"
  },
  ...

```

Assign the data collector ID variable.

```
$ data_collector_id='60740040-f3cd-4694-96da-15e547242bf7'
```

Assign thevSphere account variables.

```
$ vsphere_host_name='corp-v783-dhcp-79-85.eng.mycompany.com'
$ vsphere_user='admin@mycompany.com'
$ vsphere_password='my_vsphere_password'
```

List external region IDs from yourvSphere cloud account.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts-vsphere/region-enumeration?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "cloudAccountType": "vsphere",
  "username": "'$vsphere_user'",
  "password": "'$vsphere_password'",
  "hostName": "'$vsphere_host_name'",
  "dcid": "'$data_collector_id'",
  "acceptSelfSignedCertificate": "false"
}' | jq "
```

A snippet of the response shows the region ID to use.

```
...
{
  "externalRegionIds": [
    "Datacenter:datacenter-2"
  ]
}
...
```

Assign the region ID variable.

```
$ vsphere_region_id='Datacenter:datacenter-2'
```

Submit request to validate asynchronously with thevSphere cloud account specification.

```
$ curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts-vsphere?apiVersion=$api_version&validateOnly" | jq "
```

A snippet of the response shows the selfLink.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Cloud account specification validation",
  "id": "bcdee18-a77d-46f8-b068-4013e80e2b55",
  "selfLink": ".../request-tracker/bcdee18-a77d-46f8-b068-4013e80e2b55"
}
```

Assign the selfLink variable.

```
$ selfLink_id='bbcdee18-a77d-46f8-b068-4013e80e2b55'
```

Submit a request to track the request with the selfLink.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

When the request succeeds, the response shows the resource with the certificate ID.

```
{
  "progress": 0,
  "message": "valid certification path to requested target found",
  "status": "SUCCEEDED",
  "resources": [
    "/iaas/api/certificates/7fe4c108-64ff-4347-92de-b0790bdala3c?apiversion=2021-07-15"
  ],
  "name": "Cloud account specification validation",
  "id": "bbcdee18-a77d-46f8-b068-4013e80e2b55",
  "selfLink": "/iaas/api/request-tracker/bbcdee18-a77d-46f8-b068-4013e80e2b55"
}
```

Assign the certificate ID variable.

```
$ certificate_id='7fe4c108-64ff-4347-92de-b0790bdala3c'
```

To get certificate information, submit a request with the certificate ID.

```
$ PEM_for_X509Certificate='curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/certificates/$certificate_id?apiVersion=$api_version" | jq "."'
```

Assign the ID of the existing cloud account to associate with the new cloud account.

```
$ existing_cloud_account_id='b9falb42c767de7558ceff3b78004'
```

Create a cloud account named demo-vsphere-account with default cloud zones.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts-vmware?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name": "demo-vsphere-account",
  "description": "This is a demo vSphere account with default cloud zones",
  "username": "'$vsphere_user'",
  "password": "'$vsphere_password'",
  "hostName": "'$vsphere_host_name'",
  "acceptSelfSignedCertificate": false,
  "associatedCloudAccountIds": "'$existing_cloud_account_id'",
  "createDefaultZones": true,
  "dcId": "'$data_collector_id'",
  "regions": [
    {
      "name": "'$vsphere_region_id'",
      "ExternalRegionId": "'$vsphere_region_id'"
    }
  ],
  "tags": [
    {
```

```

    "key": "env",
    "value": "dev"
  }
],
"certificateInfo":{
  "certificate": "'$PEM_for_X509Certificate'"
}
}' | jq "."

```

A snippet of the response from your request shows the account ID.

```

...
  "tags": [],
  "name": "demo-vsphere-account",
  "id": "515684cce-
bafde75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a
  "updatedAt": "2022-04-02",
  "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
  "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...

```

Add an NSX Cloud Account

To create an NSX Manager cloud account, you make a POST request. The request body includes the NSX-specific parameters required to create the cloud account. NSX Manager cloud accounts use policy APIs to communicate with NSX.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the following parameters for the new cloud account:
 - NSX host name
 - NSX user name
 - NSX password

As an alternative to using the `cloud-accounts` API call, you can use a `cloud-accounts-nsx-t` API call to list NSX Manager cloud accounts respectively.

1. Assign the NSX account variables.

```

nsx_host_name='<your_nsx_host_name>'
nsx_username='<your_nsx_user_name>'
nsx_password='<your_nsx_password>'

```

- Optional: If you have a vSphere cloud account that you want to associate with the NSX cloud account, assign the ID of the vSphere cloud account to the `vsphere_cloud_account_id` variable. See [Add a vSphere Cloud Account](#).

```

vsphere_cloud_account_id='<your_vsphere_cloud_account_id>'

```

- Submit a request to create an NSX Manager cloud account, use `"cloudAccountType": "nsxt"`. This example includes the `vsphere_cloud_account_id` variable.

```

curl -X POST \
  "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "cloudAccountType": "nsxt",
    "privateKeyId": "'$nsx_username'",

```

```

    "privateKey": "'$nsx_password'",
    "associatedCloudAccountIds": [
      "'$vsphere_cloud_account_id'"
    ],
    "cloudAccountProperties": {
      "hostName": "'$nsx_host_name'",
      "acceptSelfSignedCertificate": "true",
      "privateKeyId": "'$nsx_username'",
      "privateKey": "'$nsx_password'"
    },
    "tags": [
      {
        "key": "env",
        "value": "prod"
      }
    ],
    "name": "<your_nsx_cloud_account>",
    "description": "Example NSX cloud account description"
  } | jq "."

```

4. To obtain the NSX cloud account ID, examine the response.

5. Assign the NSX cloud account ID variable.

```
nsx_cloud_id='<your_nsx_cloud_id>'
```

6. List all cloud accounts.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts?apiVersion=$api_version" | jq "."
```

7. Optional: List all NSX Manager cloud accounts using the `cloud-accounts-nsx-t` API call.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts-nsx-t?apiVersion=$api_version" | jq "."
```

8. Examine the response and verify that the name and ID of the NSX cloud account you created is listed.

Note: The ID of the associated vSphere cloud account appears under `_links` and `hrefs` as in the following code snippet example.

```

...
    "name": "nsx manager",
    "id": "515684cce-
bafde75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-d5a5e16bd-
c3eec75572358fd24ab6",
    "updatedAt": " ",
    "organizationId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
    "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
    "_links": {
      "associated-cloud-accounts": {
        "hrefs": [
          "/iaas/api/cloud-accounts/515684cce-
bafde75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a
        ]
      },
      "self": {

```

```

    "href": "/iaas/api/cloud-accounts/515684cce-
bafde75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-d5a5e16bd-
c3eec75572358fd24ab6"
...

```

Create an NSX Manager Cloud Account

This example creates an NSX Manager cloud account that includes an existing vSphere cloud account.

Assign the required variables.

```

$ url='https://appliance.domain.com'
$ api_version=' '

```

Assign the NSX account variables.

```

$ nsx_host_name='nsx-manager.mycompany.local'
$ nsx_username='admin'
$ nsx_password='my_nsx_password'

```

Assign the account variables for your existing vSphere cloud account.

```

$ vsphere_cloud_account_id='515684cce-
bafde75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a

```

Create an NSX Manager cloud account named `demo-nsx-account`.

```

$ curl -X POST \
  "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "cloudAccountType": "nsxt",
    "privateKeyId": "'$nsx_username'",
    "privateKey": "'$nsx_password'",
    "associatedCloudAccountIds": [
      "'$vsphere_cloud_account_id'"
    ],
    "cloudAccountProperties": {
      "hostName": "'$nsx_host_name'",
      "acceptSelfSignedCertificate": "true",
      "privateKeyId": "'$nsx_username'",
      "privateKey": "'$nsx_password'"
    },
    "tags": [
      {
        "key": "env",
        "value": "prod"
      }
    ],
    "name": "demo-nsx-account",
    "description": "Example NSX cloud account description"
  }' | jq "."

```

A snippet of the response from your request shows the account ID.

```
...
```

```

    "tags": [],
    "name": "demo-nsx-account",
    "id": "7b2c48362c94567559080d8f575a2",
    "updatedAt": " ",
    "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
    "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
    ...

```

Add aVMware Cloud on AWS Cloud Account with a Proxy

Adding aVMware Cloud on AWS cloud account with a proxy requires manual deployment of a cloud proxy VM. Then you make a POST request with a request body that includes the data collector ID along with parameters specific toVMware Cloud on AWS.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that a cloud proxy VM has been manually deployed.
- Verify that you have the following parameters for the new cloud account:
 - VMC API token.
 - SDDC name.
 - vCenter private IP.
 - NSX Manager IP.
 - vCenter user name.
 - vCenter password.
 - vCenter data center ID.

1. List all cloud proxies.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/data-collectors?api_version=$api_version" | jq "."
```

2. Examine the response and assign the data collector variable.

```
data_collector_id='<your_datacollector>'
```

3. Assign the account variables for the VMC cloud account.

```

vmc_api_token='<your_vmc_api_token>'
vmc_sddc_name='<your_vmc_sddc_name>'
vmc_vcenter_private_ip='<your_vcenter_private_ip>'
vmc_nsx_manager_ip='<your_nsx_manager_ip>'
vmc_vcenter_username='<your_vcenter_username>'
vmc_vcenter_password='<your_vcenter_password>'
vmc_vcenter_datacenter_id='<your_datacenter_id>'

```

4. Submit a request to create a VMC cloud account.

```

curl -X POST \
  "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "vmc-endpoint",
    "description": "VMC cloud account",
    "cloudAccountType": "vmc",
    "privateKeyId": "'"$vmc_vcenter_username"'",
    "privateKey": "'"$vmc_vcenter_password"'",

```

```

"cloudAccountProperties": {
  "sddcId": "'$vmc_sddc_name'",
  "apiKey": "'$vmc_api_token'",
  "hostName": "'$vmc_vcenter_private_ip'",
  "nsxHostName": "'$vmc_nsx_manager_ip'",
  "dcId": "'$vmc_data_collector_id'",
  "acceptSelfSignedCertificate": "false"
},
"regionIds": [
  "'$vmc_vcenter_datacenter_id'"
]
}' | jq "."

```

The response includes a selfLink value.

5. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

6. Use the selfLink variable to track the request.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
request-tracker/$selfLink_id | jq "."

```

The VMware Cloud on AWS cloud account is created when the response shows "status": "FINISHED".

Create a VMC Cloud Account with a Proxy

This example creates a VMware Cloud on AWS cloud account with a cloud proxy VM that has been manually deployed.

Assign the required variables.

```

$ url='https://appliance.domain.com'
$ api_version='2021-07-15'

```

List all cloud proxies.

```

$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/data-collectors?api_version=$api_version" | jq "."

```

A snippet of the response from your request shows the data collector IDs.

```

...
{
  "dcId": "cd7d1eb4-573f-4150-8206-de3d536490ca",
  "ipAddress": "10.139.116.60",
  "name": "localhost.localdom",
  "hostName": "localhost.localdom",
  "status": "ACTIVE"
},,
...

```

Assign the data collector ID variable.

```
$ data_collector_id='cd7d1eb4-573f-4150-8206-de3d536490ca'
```

Assign the variables for the VMC cloud account.

```

$ vmc_data_collector_id=a1235a7f-d49f-4365-8ed9-2d7d0805e4bc
$ vmc_api_token=ab392fba-32a8-49a5-a084-d422fa32c5b8
$ vmc_sddc_name=MYCOM-PRD-NSXT-M7GA-052019

```

```
$ vmc_vcenter_private_ip=10.70.57.196
$ vmc_nsx_manager_ip=10.70.57.131
$ vmc_vcenter_username=cloudadmin@vmc.local
$ vmc_vcenter_password=aBcqCW+m4+XEQg7
$ vmc_vcenter_datacenter_id=Datacenter:datacenter-1
```

Create a VMC cloud account named demo-vmc-account.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
-d '{
  "name": "demo-vmc-account",
  "description": "VMC cloud account",
  "cloudAccountType": "vmc",
  "privateKeyId": "'"$vmc_vcenter_username"'",
  "privateKey": "'"$vmc_vcenter_password"'",
  "cloudAccountProperties": {
    "sddcId": "'"$vmc_sddc_name"'",
    "apiKey": "'"$vmc_api_token"'",
    "hostName": "'"$vmc_vcenter_private_ip"'",
    "nsxHostName": "'"$vmc_nsx_manager_ip"'",
    "dcId": "'"$vmc_data_collector_id"'",
    "acceptSelfSignedCertificate": "false"
  },
  "regionIds": [
    "'"$vmc_vcenter_datacenter_id"'
  ]
}' | jq "
```

The response includes a selfLink variable.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Cloud account creation/update",
  "id": "0dc374ba-08ec-4422-8615-24f4f94ef5aa",
  "selfLink": "/iaas/api/request-tracker/0dc374ba-08ec-4422-8615-24f4f94ef5aa"
}
```

Assign the selfLink variable.

```
selfLink_id='0dc374ba-08ec-4422-8615-24f4f94ef5aa'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id | jq "
```

When the request completes successfully, the response shows the cloud account ID at the end of the resources path.

```
{
  "progress": 100,
  "status": "FINISHED",
  "resources": [
```



```

    "/iaas/api/cloud-accounts/e6af1aa6-dc7a-4847-8adc-c4c73727e5b3"
  ],
  "name": "Cloud account creation/update",
  "id": "a85d1476-1b11-45b0-8d14-91951385c95d",
  "selfLink": "/iaas/api/request-tracker/a85d1476-1b11-45b0-8d14-91951385c95d"
}

```

Add aMicrosoft Azure Cloud Account

To create aMicrosoft Azure cloud account, you make a POST request. The request body includes parameters specific toMicrosoft Azure that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the following parameters for the new cloud account:
 - Microsoft Azure subscription ID
 - Microsoft Azure tenant ID
 - Microsoft Azure client application ID
 - Microsoft Azure client application secret key

As an alternative to using the `cloud-accounts` API call, you can use a `cloud-accounts-azure` API call that creates aMicrosoft Azure cloud account with fewer input parameters.

1. Assign theMicrosoft Azure account variables.

```

azure_subscription_id='<your_azure_subscription_id>'
azure_tenant_id='<your_azure_tenant_id>'
azure_client_application_id='<your_azure_client_application_id>'
azure_client_application_secret_key='<your_azure_client_application_secret_key>'

```

2. Submit a request to create aMicrosoft Azure cloud account with default cloud zones.

```

curl -X POST \
  "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "cloudAccountType": "azure",
    "privateKeyId": "'$azure_client_application_id'",
    "privateKey": "'$azure_client_application_secret_key'",
    "cloudAccountProperties": {
      "userLink": "'$azure_subscription_id'",
      "azureTenantId": "'$azure_tenant_id'"
    },
    "regionIds": ["<your_region_id>"],
    "createDefaultZones": true,
    "name": "<your_azure_cloud_account>",
    "description": "This is a demo Azure cloud account",
  }' | jq "."

```

3. Optional: Submit a request to create aMicrosoft Azure cloud account with the `cloud-accounts-azure` API call.

```

curl -X POST \
  "$url/iaas/api/cloud-accounts-azure?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{

```

```

"name": "<your_azure_cloud_account>",
"description": "This is a demo Azure cloud account",
"subscriptionId": "'$azure_subscription_id'",
"tenantId": "'$azure_tenant_id'",
"clientApplicationId": "'$azure_client_application_id'",
"clientApplicationSecretKey": "'$azure_client_application_secret_key'",
"regionIds": [ "<your_region_id1>", "<your_region_id2>" ],
"createDefaultZones": true,
"tags": [ { "key": "env", "value": "dev" } ]
}' | jq "."

```

4. List all cloud accounts.

```

curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
"Authorization: Bearer $access_token" | jq "."

```

5. Examine the response and verify that the name and ID of the Microsoft Azure cloud account you created is listed.

Create a Microsoft Azure Cloud Account

Assign the required variables.

```

$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ azure_subscription_id='r1e31415-4a08-4072-be4a-19de37d12345'
$ azure_tenant_id='s39138ca-3abc-4b4a-a4d6-cd92d9dd62f0'
$ azure_client_application_id='te21wxyz-b183-42ac-cd84-3c4a2459b9a9'
$ azure_client_application_secret_key='udv6lY8MwpP5ABCDfSztP3ABCDEaLMNOPQRmDEUeiI0='

```

Create a cloud account named demo-azure-account.

```

$ curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "cloudAccountType": "azure",
  "privateKeyId": "'$azure_client_application_id'",
  "privateKey": "'$azure_client_application_secret_key'",
  "cloudAccountProperties": {
    "userLink": "'$azure_subscription_id'",
    "azureTenantId": "'$azure_tenant_id'"
  },
  "regionIds": ["eastus"],
  "createDefaultZones": true,
  "name": "demo-azure-account",
  "description": "This is a demo Azure cloud account",
}' | jq "."

```

A snippet of the response from your request shows the account ID.

```

...
"tags": [],
"name": "demo-azure-account",
"id": "c8c3c9bfdb449475-7f703c5265a63d87-f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31e-
b8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"updatedAt": "2022-04-02",

```

```
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Add a Google Cloud Platform Cloud Account

To create a Google Cloud Platform (GCP) cloud account, you make a POST request. The request body includes parameters specific to Google Cloud Platform that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the following parameters for the new cloud account:
 - GCP project ID
 - GCP private key ID
 - GCP private key
 - GCP client email

1. Assign the GCP account variables.

```
gcp_project_id='<your_gcp_project_id>'
gcp_private_key_id='<your_gcp_private_key_id>'
gcp_private_key='<your_gcp_private_key>'
gcp_client_email='<your_gcp_client_email>'
```

2. Submit a request to create a GCP cloud account with default cloud zones.

```
curl -X POST \
  "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "cloudAccountType": "gcp",
    "privateKeyId": "'$gcp_private_key_id'",
    "privateKey": "'$gcp_private_key'",
    "cloudAccountProperties": {
      "projectId": "'$gcp_project_id'",
      "clientEmail": "'$gcp_client_email'"
    },
    "regionIds": ["<your_region_id>"],
    "createDefaultZones": true,
    "name": "<your_gcp_cloud_account>",
    "description": "This is a demo GCP cloud account",
  }' | jq "
```

3. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
  "Authorization: Bearer $access_token" | jq "
```

4. Examine the response and verify that the name and ID of the GCP cloud account you created is listed.

Create a Google Cloud Platform Cloud Account

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ gcp_project_id='Example-e2e'
$ gcp_private_key_id='defg3c20c85abcde6a95b44222c4c1d68554b87e'
$ gcp_private_key='-----BEGIN PRIVATE KEY-----\nMIIEvQIBADANBgkqhkiG9w0BAQEFAA'
```

```
...
3izE4KDeebLh7SkWfBt7lFW25UL20\\nKAY7FRTKpVbO+6Z/BnVePVI=\\n-----END PRIVATE KEY-----\\n'
$ gcp_client_email='123456789123-example@developer.gserviceaccount.com'
```

Create the cloud account.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "cloudAccountType": "gcp",
  "privateKeyId": "'$gcp_private_key_id'",
  "privateKey": "'$gcp_private_key'",
  "cloudAccountProperties": {
    "projectId": "'$gcp_project_id'",
    "clientEmail": "'$gcp_client_email'"
  },
  "regionIds": ["us-west2"],
  "createDefaultZones": true,
  "name": "demo-gcp-account",
  "description": "This is a demo GCP cloud account",
}' | jq "
```

A snippet of the response from your request shows the account ID.

```
...
"tags": [],
"name": "demo-GCP-account",
"id": "c8c3c9b9fdb449475-7f703c5265a63d87-f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31e-
b8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"updatedAt": "2022-04-02",
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Integrating with other applications

To fully automate your organization setup, you can use the IaaS APIs to list, create, update, and delete VCF Automation integrations with other applications programmatically.

Using different input variables, you can create integrations with external systems such as GitHub, Ansible, and external IPAM providers. The following table lists all supported integrations.

Integration	Type	What do I need to create an integration?
Active Directory	activedirectory	- LDAP connection to Active Directory server. - Project configured with appropriate cloud zones, and image and flavor mappings to use with the Active Directory integration.
Ansible	ansible	Ansible control machine running Ansible version 2.6.0 or later.

Integration	Type	What do I need to create an integration?
		- Read/write access to the directory where the Ansible inventory file is located. - Deactivated host key. - Vault password set.
Ansible Tower	<code>ansible.tower</code>	Credentials and templates in Ansible Tower configured to use with deployments.
Bitbucket	<code>org.bitbucket</code>	Personal access API token for Bitbucket.
GitHub	<code>com.github.saas</code>	Personal access API token for GitHub. See https://docs.gitlab.com/ee/user/profile/personal_access_tokens.html
GitLab	<code>com.gitlab.saas</code>	Personal access API token for GitLab.
IPAM	<code>ipam</code>	- IPAM provider package. - Administrator credentials for the account with the external IPAM provider.
Puppet	<code>puppet</code>	Puppet master name and hostname or IP address of the master.
Red Hat OpenShift	<code>cmx.openshift-endpoint</code>	Red Hat OpenShift server configured. Note: If needed, you can create an OpenShift cluster with a blueprint that VMware provides. See https://flings.vmware.com/red-hat-openshift-container-platform-as-a-service-on-vrealize-automation-cloud.
SaltStack	<code>saltstack</code>	- Salt master used in the VCF Automation integration that contains the Master Plugin. - VCF Automation service administrator role in VCF Automation. - VCF Automation service administrator role in VCF Automation.
SDDC Manager	<code>sddc</code>	SDDC manager 4.1 or later installed.
Terraform Runtime	<code>terraform.runtime</code>	Terraform runtime environment.
VMware Enterprise PKS	<code>cmx.pks-endpoint</code>	- Pivotal Container Service (PKS) server configured with UAA authentication. - Cloud administrator credentials.
VCF Operations orchestrator	<code>vro</code>	- VCF Operations orchestrator configured. - VCF Operations orchestrator URL - Cloud Extensibility proxy deployed.
VCF Operations	<code>vrops</code>	A local or non-local VCF Operations login account with read-only privileges to the vCenter adapter instance of the vSphere endpoint. Note:

Integration	Type	What do I need to create an integration?
		For non-local account login, username format is username@domain@authenticated-source, for example jdoe@company.com@workspaceone

Create an Integration with Github

To create an integration with GitHub, you make a POST request. The request body includes properties specific to GitHub.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have a personal access token for authentication to GitHub. See https://docs.gitlab.com/ee/user/profile/personal_access_tokens.html.

1. Assign variables for GitHub.

```
private_key='<your_GitHub_personal_access_token>'
```

2. Submit a request to create a GitHub integration.

```
curl -X POST \
  "$url/iaas/api/integrations?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "integrationType": "com.github.saas",
    "name": "<your_github_integration>",
    "privateKey": "'$private_key'",
    "integrationProperties": {
      "url": "https://api.github.com"
    },
  }' | jq "
```

The response includes a selfLink.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Integration creation/update",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

3. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

4. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "
```

When the request completes successfully, the response includes a resource with the integration ID.

```
{
  "progress": 100,
  "status": "FINISHED",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

```

"resources": [
  "/iaas/api/integrations/example-integration-id-string"
],
"name": "Integration creation/update",
"id": "example-selfLink-alphanumeric-string",
"selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}

```

5. Assign the integration ID variable.

```
integration_id='example-integration-alphanumeric-string'
```

6. Use the integration ID variable to list the integration.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/integrations/$integration_id?apiVersion=$api_version" | jq "
```

7. Examine the response and verify that the name and ID of the integration that you created is listed.

Create a GitHub integration

Assign the required variables.

```

$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ private_key='8bc9401b5d28f4ec126929af0dc4e99dd0792b0f'

```

Create the integration.

```

$ curl -X POST \
"$url/iaas/api/integrations?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "integrationType": "com.github.saas",
  "name": "Git integration example",
  "privateKey": "'$private_key'",
  "integrationProperties": {
    "url": "https://api.github.com"
  },
}' | jq "
```

The response includes a selfLink.

```

{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Integration creation/update",
  "id": "a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597",
  "selfLink": "/iaas/api/request-tracker/a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597"
}

```

Assign the selfLink variable

```
selfLink_id='a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

When the request completes successfully, the response includes a resource with the integration ID.

```
{
  "progress": 100,
  "status": "FINISHED",
  "resources": [
    "/iaas/api/integrations/e5dda941-bb17-4f19bd15-7db0b8eab88c"
  ],
  "name": "Integration creation/update",
  "id": "a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597",
  "selfLink": "/iaas/api/request-tracker/a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597"
}
```

Assign the integration ID variable

```
integration_id='e5dda941-bb17-4f19bd15-7db0b8eab88c'
```

Use the integration ID variable to list the integration.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/integrations/$integration_id?apiVersion=$api_version" | jq "."
```

When the request completes successfully, a snippet of the response shows integration details.

```
...
{
  "integrationType": "com.github.saas",
  "integrationProperties": {
    "url": "https://api.github.com"
  },
  "name": "Git integration example",
  "id": "e5dda941-bb17-4f19bd15-7db0b8eab88c"
  "createdAt": "2022-04-02"
  "updatedAt": "2022-04-02"
  "orgId": "ce811934-ea1a-4f53-6bec-4656ca7d126",
  "_links": {
    "self": {
      "href": "/iaas/api/integrations/e5dda941-bb17-4f19bd15-7db0b8eab88c"
    }
  }
}
...
}
```

Delete an Integration

To delete an integration, you make a DELETE request with the ID of the integration.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the integration that you want to delete. See [Create an Integration with Github](#).

The following procedure shows how to delete an integration including an optional step to list the integration details before deleting the integration. It is a good practice to check the details of the integration so that you delete the correct integration.

1. Assign your integration ID variable.

```
integration_id='<your_integration_id>'
```

2. Optional: List the integration before deleting.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/integrations/$integration_id?apiVersion=$api_version" | jq "
```

3. Optional: Examine the response to verify the integration details, such as integration name and integration type.

4. Submit a request to delete the integration.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/integrations/$integration_id?apiVersion=$api_version" | jq "
```

The response includes a selfLink.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Integration deletion",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

5. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

6. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id | jq "
```

When the request completes successfully, the response shows the status as `FINISHED`.

```
{
  "progress": 100,
  "message": "Deleted",
  "status": "FINISHED",
  "name": "Integration deletion",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

Delete an integration

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ integration_id='e5dda941-bb17-4f19bd15-7db0b8eab88c'
```

Delete the integration.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/integrations/$integration_id?apiVersion=$api_version" | jq "
```

The response includes a selfLink.

```
{
  "progress": 0,
  "status": "INPROGRESS",
```

```

    "name": "Integration deletion",
    "id": "fe472e75-f346-4de7-bbc4-5edddefd9dfa",
    "selfLink": "/iaas/api/request-tracker/fe472e75-f346-4de7-bbc4-5edddefd9dfa"
  }

```

Assign the selfLink variable

```
selfLink_id='fe472e75-f346-4de7-bbc4-5edddefd9dfa'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id | jq "."
```

When the request completes successfully, the response shows the status as `FINISHED`.

```

{
  "progress": 100,
  "message": "Deleted",
  "status": "FINISHED",
  "name": "Integration deletion",
  "id": "fe472e75-f346-4de7-bbc4-5edddefd9dfa",
  "selfLink": "/iaas/api/request-tracker/fe472e75-f346-4de7-bbc4-5edddefd9dfa"
}

```

How do I import an IPAM package

To import an IPAM package using the IaaS API, you reserve enough space for the package then you make multiple PATCH requests to import smaller pieces of the package. After importing all the pieces, you use a POST request to upload the file onto the server.

- Verify that all general prerequisites and prerequisites for the VCF Automation IaaS API have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you know the size of the package that you want to upload. If the package is large, divide it into smaller packages. The following example divides the large package into smaller packages that are each 9000000 bytes in size.

```
split -b 9000000 <your_package>.zip <your_package>.zip_split
```

- If using the API to complete the IPAM integration with a cloud account, verify that you have the following parameters for your cloud account:
 - `privateKeyId` used to create your cloud account.
 - `privateKey` used to create your cloud account.
 - `hostName` used to create your cloud account.
 - `faasProviderEndpointID` is your cloud account ID.

The IaaS API implements the TUS RFC protocol which provides a mechanism for resumable file uploads. By dividing a large package into smaller ZIP packages and importing the packages individually, you ensure that you will not exceed the server's file size limit with your upload.

For more information about the TUS RFC protocol, see <https://github.com/tus/tus-resumable-upload-protocol/blob/main/protocol.md>.

This procedure shows how to import your IPAM package using the integrations IPAM APIs. Then you complete the integration using the VCF Automation UI or the integrations API.

1. Reserve space for the package with `Upload-Length` equal to the size of the package in bytes.

```

curl -X POST \
  "$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \

```

```
-H "Upload-Length: <package_size_in_bytes>" \
-H "Tus-Resumable: 1.0.0" | jq "."
```

Examine the response header to get the location for the package.

Note: You must complete the package import process before the location reservation expires in approximately two hours.

2. Assign a variable for the location ID.

```
location_id = '<package_location>'
```

3. Import the first small package with Upload-Offset equal to zero.

```
curl -X PATCH \
"$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Tus-Resumable: 1.0.0" \
-H "Content-Type: application/offset+octet-stream" \
-H "Upload-Offset: 0" | jq "."
```

A successful response returns 204 No Content and includes an upload offset number. Use that number in the PATCH request for the next package.

4. Import the next small package.

- You must import the small packages in a sequential order.
- Upload-Offset is the offset value from the response header of the previous PATCH request.

```
curl -X PATCH \
"$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Tus-Resumable: 1.0.0" \
-H "Content-Type: application/offset+octet-stream" \
-H "Upload-Offset: <upload_offset_from_previous_response>" | jq "."
```

Verify that the successful response returns 204 No Content and a larger upload offset number. Continue making PATCH requests with the upload offset numbers from the previous response until all the smaller packages have been imported.

Note: If you do not know the current upload offset number, make a HEAD request with the location ID.

```
curl -X HEAD \
"$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Tus-Resumable: 1.0.0" | jq "."
```

Use the upload offset number in the response for the next PATCH request.

5. After all the small packages have been imported, upload the file.

The input body includes:

- bundleId for the location ID.
- "option": "OVERWRITE" to overwrite any existing package.

```
curl -X POST \
"$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Content-Type: application/json" \
-H "Tus-Resumable: 1.0.0" \
-d '{
  "bundleId": "$location_id",
```

```

    "option" : "OVERWRITE"
  }
  | jq "."

```

A successful response includes the Provider ID, Provider name, and Provider version. To complete an IPAM integration, you can use the VCF Automation UI or the integrations endpoint in the IaaS API.

6. To complete an integration using the VCF Automation UI, select **Infrastructure > Integrations > Add integration > IPAM**. On the **New Integration** page that appears, select **Manage IPAM Providers** and choose the package with the Provider name and Provider version from the API response.

7. Optional: To complete an integration using the API:

- a) Assign a variable for the provider ID.

```
provider_id = '<provider_id_from_package_upload>'
```

- b) Create a new IPAM integration.

```

curl -X POST \
  "$url/iaas/api/integrations?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H "Content-Type: application/json" \
  -d '{
    "integrationProperties": {
      "providerId" : "'$provider_id'",
      "faasProviderEndpointId": "<your_provider_endpoint_ID>",
      "privateKeyId": "<your_privateKeyId>",
      "privateKey": "<your_privateKey>",
      "hostName": "<your_hostName>",
      "dcId": "onprem"
    },
    "customProperties": {"isExternal": "true"},
    "integrationType": "ipam",
    "associatedCloudAccountIds": [],
    "associatedMobilityCloudAccountIds": {},
    "privateKey": "<your_privateKey>",
    "privateKeyId": "<your_privateKeyId>"
  } | jq "."

```

The response includes a self link to track the request.

- c) Assign the selfLink variable.

```
selfLink_id='<example_selfLink_alphanumeric_string>'
```

- d) Use the selfLink variable to track the request.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."

```

The integration is created with the response includes "status": "FINISHED".

Import an IPAM Package

This example shows how to import an Infoblox IPAM package with a total size of 73342782 bytes. To import incrementally using smaller packages, you divide the larger package into smaller packages. This example assumes that you have divided the package into multiple smaller packages each 9000000 bytes in size.

After importing the IPAM package, you create an integration with a cloud account using the the VCF Automation UI or the integrations endpoint in the IaaS API. This example shows how to integrate the uploaded IPAM package with a cloud account that has the following parameters:

- faasProviderEndpointId: "f46337e9-e0c2-4ada-9244-766a8e54da48"

- `hostName: "infoblox.sof-mbu.eng.mycompany.com"`
- `privateKey: "Password!23"`
- `privateKeyId: "administrator@mycompany.local"`

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
```

Reserve space for the package.

```
$ curl -X POST \
"$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Upload-Length: 73342782" \
-H "Tus-Resumable: 1.0.0" | jq "."
```

Examine the response to get the location ID.

```
status: 201
body: empty
headers: {location:/iaas/api/integrations-ipam/package-import/8a81db06-f27f-4c-
cb-a7c5-960c35107ed6, ...}
```

Assign the location ID variable.

```
$ location_id='8a81db06-f27f-4ccb-a7c5-960c35107ed6'
```

Import the first package with a content length of 9000000 bytes and a zero upload offset value.

```
$ curl -X PATCH \
"$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Content-Length: 9000000" \
-H "Tus-Resumable: 1.0.0" \
-H "Content-Type: application/offset+octet-stream" \
-H "Upload-Offset: 0" | jq "."
```

Examine the header in the response to get the upload offset value.

```
status: 204
body: empty
headers: {upload-offset:9000000, ...}
```

Import the next package.

```
$ curl -X PATCH \
"$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Content-Length: 9000000" \
-H "Tus-Resumable: 1.0.0" \
-H "Content-Type: application/offset+octet-stream" \
-H "Upload-Offset: 9000000" | jq "."
```

Examine the header in the response again to get the next upload offset value.

```
status: 204
body: empty
headers: {upload-offset:18000000, ...}
```

Import the next package.

```
$ curl -X PATCH \
"$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Content-Length: 9000000" \
-H "Tus-Resumable: 1.0.0" \
-H "Content-Type: application/offset+octet-stream" \
-H "Upload-Offset: 18000000" | jq "."
```

Repeat the process to get the upload offset value from the header response and use that value to import the next package.

After importing the final small package, upload the complete package.

```
$ curl -X POST \
"$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Tus-Resumable: 1.0.0" \
-d '{
  "bundleId": "$location_id",
  "option" : "OVERWRITE"
}' | jq "."
```

A successful response includes the provider ID, provider name, and the provider version

```
{
  "providerId": "86801580-9042-49b6-879d-5a7361d33519",
  "providerName": "Infoblox",
  "providerVersion": "1.6"
  "logoIcon": "iVBORw...",
  ...
}
```

To create an IPAM integration using the VCF Automation UI, select **Infrastructure > Integrations > Add integration > IPAM**. On the **New Integration** page that appears, select **Manage IPAM Providers** and choose the package with the Provider name **Infoblox** and Provider version **1.6**.

To create an IPAM integration using the API, assign the provider ID variable.

```
$ provider_id = "86801580-9042-49b6-879d-5a7361d33519"
```

Create the integration.

```
$ curl -X POST \
"$url/iaas/api/integrations?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" \
-H "Content-Type: application/json" \
-d '{
  "integrationProperties": {
    "providerId" : "'$provider_id'",
    "faasProviderEndpointId": "f46337e9-e0c2-4ada-9244-766a8e54da48",
    "privateKeyId": "administrator@mycompany.local",
    "privateKey": "Password!23",
    "hostName": "infoblox.sof-mbu.eng.mycompany.com",
    "dcId": "onprem"
  },
  "customProperties": {"isExternal": "true"},
}
```

```

    "integrationType": "ipam",
    "associatedCloudAccountIds": [],
    "associatedMobilityCloudAccountIds": {},
    "privateKey": "Password!23",
    "privateKeyId": "administrator@mycompany.local"
  } | jq "."

```

Examine the response.

```

{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Integration creation/update",
  "id": "117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc",
  "selfLink": "/iaas/api/request-tracker/117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc"
}

```

Assign the selfLink variable.

```
$ selfLink_id='117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc'
```

Use the selfLink for tracking.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."

```

The integration is complete when the response includes "status": "FINISHED".

```

{
  "progress": 100,
  "status": "FINISHED",
  "resources": [
    {
      "id": "117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc",
      "name": "Integration creation/update",
      "selfLink": "/iaas/api/request-tracker/117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc"
    }
  ]
}

```

Using VCF Automation APIs to Build your Resource Infrastructure

You use the VCF Automation APIs to add a project and set up an infrastructure for the team members in VCF Automation so that new workloads can be provisioned.

Create a Cloud Zone

To create a cloud zone, you first make GET request to obtain a region ID with a cloud account ID as input. Then you make a POST request with the region ID.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the cloud account you added. See [Add an Amazon Web Services Cloud Account](#).

1. Assign the cloud account ID variable.

```
cloud_account_id='<your_cloud_account_id>'
```

2. Look up the IDs for the region named `us-east-1` that is associated with the cloud account.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'"$cloud_account_id'" | jq "
```

3. To obtain a region ID, examine the response.

4. Assign the region ID variable.

```
region_id='<your_region_id>'
```

5. Create a cloud zone.

```
curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" -d '{ "name": "Demo-zone-1", "description": "This zone is for Demo", "regionId": "'$region_id'", "placementPolicy": "DEFAULT" }' "$url/iaas/api/zones?apiVersion=$api_version" | jq "
```

6. To obtain a cloud zone ID, examine the response.

7. Assign the zone ID variable.

```
zone_id='<your_cloud_zone_id>'
```

8. List all cloud zones.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/zones?apiVersion=$api_version" | jq "
```

9. Optional: List the cloud zone with your cloud zone ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/zones/$zone_id?apiVersion=$api_version" | jq "
```

10. Examine the output to verify that the cloud zone you created is listed.

Create a Cloud Zone

Assign variables, specify a cloud account and look up IDs for the region named `us-east-1` associated with the cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-f8e705d89b2569e1aac66c6d00bf4fc7e-f4b1c44100f0e944af31eb8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0'
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'"$cloud_account_id'" | jq "
```

A snippet of the response from your request shows your cloud account ID with a region ID.

```
...
"externalRegionId": "us-east-1",
"cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-f8e705d89b2569e1aac66c6d00bf4fc7e-f4b1c44100f0e944af31eb8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"id": "4965d34c3bfe0275574bc5fd858e8",
"updatedAt": " ",
...
```

Specify a region ID and create a cloud zone.

```
$ region_id='4965d34c3bfe0275574bc5fd858e8'
```



```
$ curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" -d
'{ "name": "Demo-zone-1", "description": "This zone is for Demo", "regionId": "'$region_id'",
  "placementPolicy": "DEFAULT" }' "$url/iaas/api/zones?apiVersion=$api_version" | jq "
```

A snippet of the response from your request shows the zone ID.

```
...
"name": "Aws / us-east-1",
"id": "4965d34c3bfe0275574bc5fd8782a",
"updatedAt": " ",
...
```

Create a Cloud Zone with a Folder

When creating a cloud zone, you can specify a folder with a path that relates to a datacenter, so that when you deploy a machine you deploy it to that folder.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

Note: Users with the following RBAC permissions can also use the folder API to create a cloud zone:

- provisioning_cloud-zones_read
- provisioning_cloud-zones_manage
- Verify that you have the cloud account ID for the cloud account with the folder that you want to use. See [Add avSphere Cloud Account](#).

To create a cloud zone with a folder, you first obtain an external region ID with a cloud account ID as input. Only the vSphere, VMC, and VCF cloud accounts support folders. Then you use the Folders API to get the external ID for the folder to specify.

The following procedure shows how to find a folder that you can use when creating a cloud zone.

1. Assign the cloud account ID variable.

```
cloud_account_id='<your_cloud_account_id>'
```

2. List the info for the cloud account.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
cloud-accounts/$cloud_account_id/?apiVersion=$api_version" | jq "
```

3. To find the region where you want to create a cloud zone, examine the `enabledRegions` section of the response and note the external region ID and its ID.

4. Assign the external region ID variable.

```
external_region_id='<your_external_region_id>'
region_id='<your_region_id>'
```

5. List all folders associated with the external region and the cloud account.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
folders?externalRegionid=$external_region_id&cloudaccountid=$cloud_account_id/?apiVersion=$api_version" |
jq "
```

Note: Query parameters are optional:

- By providing the cloud account ID, you filter the folders by cloud account.
 - By providing the external region ID, you filter the folders by datacenter.
6. Examine the response and note the external ID of the folder that you want to use.
Any of the folders listed in the response can be used in the cloud specification.

7. Assign the external ID variable.

```
external_id='<your_external_id>'
```

8. To create a cloud zone with a folder in the datacenter, specify the external ID as the value for the folder.

```
curl -X POST \
  "$url/iaas/api/zones?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "regionId": "'$region_id'",
    "name": "<cloud_zone_with_folder>",
    "folder": "'$external_id'"
  } | jq "
```

The response shows the folder and cloud zone ID.

Any resources that are deployed in the cloud zone are deployed to the folder.

Create a Cloud Zone with a Folder

Assign variables and specify an ID for a vSphere cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='2636b316-b514-47f3-93b8-78f18df51a29'
```

List the information for the vSphere cloud account.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts/$cloud_account_id/?apiVersion=$api_version" | jq "
```

Examine the response to find the region where you want to create a cloud zone. A snippet of the response shows the enabledRegions with an external region ID and your cloud account ID.

```
...
"enabledRegions": [
  {
    "externalRegionId": "Datacenter:datacenter-3",
    "name": "SDDC-Datacenter",
    "cloudAccountId": "2636b316-b514-47f3-93b8-78f18df51a29",
    "id": "8d633554-90ca-4f8a-8a0f-1cd3cd90a522",
    "updatedAt": " ",
    ...
  }
]
```

Assign the external region ID variable.

```
$ external_region_id= 'Datacenter:datacenter-3'
$ region_id= '8d633554-90ca-4f8a-8a0f-1cd3cd90a522'
```

List folders associated with the external region and cloud account.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/folders?externalRegionid=$external_region_id&cloudaccountid=$cloud_account_id/?apiVersion=$api_version" | jq "
```

A snippet of the response shows the folders associated with the external region and the cloud account. The external ID is the value that you assign to the folder when creating the cloud zone.

```
...
```

```

    "cloudAccountIds": [
      "2636b316-b514-47f3-93b8-78f18df51a29"
    ]
    "externalId": "dbuyukliiska",
    "name": "dbuyukliiska",
    "id": "3863f2cc-0340-483f-b24e-d351b7c53c8c",
    "createdAt": " ",
    "updatedAt": " ",
    ...

```

Assign the external ID.

```
$ external_id='dbuyukliiska'
```

Create a cloud zone with a specified folder.

```

curl -X POST \
  "$url/iaas/api/zones?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "regionId": "'$region_id'",
    "name": "Demo-zone-with-folder-1",
    "folder": "'$external_id'"
  }' | jq "."

```

A snippet of the response from your request shows the folder and cloud zone ID.

```

...
  "folder": "dbuyukliiska",
  "externalRegionId": "Datacenter:datacenter-3",
  "cloudAccountId": "2636b316-b514-47f3-93b8-78f18df51a29",
  "name": "Demo-zone-with-folder-1",
  "id": "258bccb1-a337-4b63-9a8a-44a414761d93",
  "updatedAt": "2021-11-22",
  ...

```

Create a Project to use in VCF Automation

As a VCF Automation administrator, you make a POST request with a project name to create a project. Then you add members and cloud zones to the project so that project members can deploy blueprints to the associated zones.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that the project roles that you plan to assign have sufficient permissions to perform project-related tasks.

Note:

A user with the project administrator or project member role can perform a limited number of project-related tasks. For a complete list of tasks and roles required, see [What are the VCF Automation for VM Apps user roles](#).

- Prepare parameters including the project name, description, and email addresses for administrators, members, or viewers.

1. Assign the project name variable.

```
project_name='<your_project_name>'
```

your_project_name is a name that you choose.

2. Create a project.

```
curl -X POST \
  "$url/iaas/api/projects?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'$project_name'",
    "description" : "your-description",
    "administrators" : [{ "email" : "<admin_email>", ["type" : <"user" | "group">]}],
    "members" : [{ "email" : "<member_email>", ["type" : <"user" | "group">]}],
    "viewers" : [{ "email" : "<viewer_email>", ["type" : <"user" | "group">]}],
  }' | jq "."
```

- *admin_email*, *member_email*, and *viewer_email* are email addresses of an administrator, member, and viewer user or name of the group in the project.
- The type parameter is optional. It assigns the administrator, member, or viewer to a user or group type. If unspecified, the value defaults to user.

3. Get a list of projects and filter for the project with *your_project_name*.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects?apiVersion=$api_version&'"$filter'"=name%20eq%20'$project_name'" | jq "."
```

4. Examine the response and record the ID of your newly created project.

Create a Project to use in VCF Automation

Create a project named `Example-Assembler-project` with administrators, members, and viewers at `mycompany.com`. This example assumes that `Example-project` does not exist.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_name='Example-Assembler-project'
```

Create a project for VCF Automation.

```
$ curl -X POST \
  "$url/iaas/api/projects?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'$project_name'",
    "description" : "This is an example project for VCF Automation",
    "administrators" : [{"email" : "admin1@mycompany.com", "type" : "user"}],
    "members" : [{"email" : "member1@mycompany.com", "type" : "user"}],
    "viewers" : [{"email" : "viewer1@mycompany.com", "type" : "user"} ]
  }' | jq "."
```

The response shows the administrators, members, and viewers related to the project and the project ID.

```
{
  "administrators": [
    {
```

```

        "email": "admin1@mycompany.com",
        "type": "user"
    }
],
"members": [
    {
        "email": "member1@mycompany.com",
        "type": "user"
    }
],
"viewers": [
    {
        "email": "viewer1@mycompany.com",
        "type": "user"
    }
],
"sharedResources": true,
"name": "Example-Assembler-project",
"description": "This is an example project for VCF Automation",
"id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
"orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
}

```

Add a cloud zone to your project. See [Add a Cloud Zone to Your Project](#). If you want to add an administrator or user, see [Add Users to Your Project](#).

Add Users to Your Project

As a VCF Automation user with the project administrator role, you can use PATCH requests to add users and assign roles in your project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the project administrator role in your project and you have the project ID. See [Create a Project to use in VCF Automation](#).
- Prepare parameters including additional email addresses for administrators, members, or viewers that you want to add to the project.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

your_project_id is the ID of the new project you created.

2. List the details of your project.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects/$project_id?apiVersion=$api_version" | jq "."
```

3. Examine the response to see the administrators and users who are already in your project.

4. Submit a request to add a new administrator that includes the existing administrator for the project.

Note: If the call does not include existing administrators for the project, the PATCH request removes those administrators from the project. Specifying the administrator type is optional.

```
curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
```

```
-H "Authorization: Bearer $access_token" \
-d '{
  "administrators" : [
    {"email" : "<your_new_administrator_email>", "type" : "user"},
    {"email" : "<existing_administrator>", "type" : "user"}
  ]
}' | jq "
```

5. Submit a request to add a new member that includes the existing users for the project.

Note: If the call does not include existing members for the project, the PATCH request removes those members from the project. Specifying the member type is optional.

```
curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "members" : [
      {"email" : "<your_new_member_email>", "type" : "user"},
      {"email" : "<existing_member>", "type" : "user"}
    ]
  }' | jq "
```

Add Users to Your VCF Automation Project

For the project `Example-Assembler-project`, add another administrator and member at `mycompany.com`.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='5944aacb-91de-4541-bb9e-ef2a5403f81b'
```

List the details of your project.

```
$ curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/projects/$project_id?apiVersion=$api_version" | jq "
```

A snippet of the response shows existing administrators, members, and viewers.

```
...
"administrators": [
  {
    "email": "admin1@mycompany.com",
    "type": "user"
  }
],
"members": [
  {
    "email": "member1@mycompany.com",
    "type": "user"
  }
],
"viewers": [
  {
    "email": "viewer1@mycompany.com",
    "type": "user"
  }
]
```

```

    }
  ],
  ...

```

Add the administrator. Include the existing administrator `admin1@mycompany.com` in the PATCH request.

```

$ curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "administrators" : [
      {"email" : "newadministrator@mycompany.com", "type" : "user"},
      {"email" : "admin1@mycompany.com", "type" : "user"}
    ]
  }' | jq "."

```

Add the member. Include the existing member `member1@mycompany.com` in the PATCH request.

```

$ curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "members" : [
      {"email" : "newmember@mycompany.com", "type" : "user"},
      {"email" : "member1@mycompany.com", "type" : "user"}
    ]
  }' | jq "."

```

The response shows the project with administrators, members, and viewers, including the new administrator and new member.

```

{
  "administrators": [
    {
      "email": "newadministrator@mycompany.com",
      "type": "user"
    },
    {
      "email": "admin1@mycompany.com",
      "type": "user"
    }
  ],
  "members": [
    {
      "email": "newmember@mycompany.com",
      "type": "user"
    },
    {
      "email": "member1@mycompany.com",
      "type": "user"
    }
  ],
  "viewers": [

```

```
{
  "email": "viewer1@mycompany.com",
  "type": "user"
},
"sharedResources": true,
"name": "Example-Assembler-project",
"description": "This is an example project for VCF Automation",
"id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
"orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
}
```

Add a Cloud Zone to Your Project

As a VCF Automation administrator, you use the `PATCH iaas/api/projects` request to attach a cloud zone to a project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have added a project and you have the project ID. See [Create a Project to use in VCF Automation](#).

If the project already has cloud zones attached, review them to ensure that all zones needed for the project are included in the PATCH request to add a new cloud zone.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

your_project_id is the ID of the new project you created.

2. List all cloud zones.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/zones?apiVersion=$api_version" | jq ""
```

3. To obtain a cloud zone ID, examine the response and find the ID of the zone that you want to attach to your project. For a snippet of the response, see [Create a Cloud Zone](#).

4. Assign the cloud zone variable.

```
zone_id='<your_zone_id>'
```

5. List the details of your project.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects/$project_id?apiVersion=$api_version" | jq ""
```

6. Examine the response to see the zones already attached to your project.
7. Submit a request to attach and configure a new cloud zone that includes the ID of existing cloud zones for the project.

Note: If the call does not include existing cloud zones that are already attached to the project, the PATCH request removes those cloud zones from the project.

```
curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "zoneAssignmentConfigurations" : [
```



```

    {
      "zoneId" : "'$zone_id'",
      "priority": 1,
      "maxNumberInstances": 50
    },
    {
      "zoneId" : "<existing_cloud_zone_id>",
      "priority": 2,
      "maxNumberInstances": 100
    }
  ]
}' | jq "."

```

Attach a Cloud Zone to Your Project

For the new project `Example-project`, add a cloud zone.

```

$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='5944aacb-91de-4541-bb9e-ef2a5403f81b'

```

List all cloud zones.

```

$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/zones?apiVersion=$api_version" | jq "."

```

Examine the response to find the cloud zone you want and assign the zone ID variable.

```

$ zone_id='4965d34c3bfe0275574bc5fd8782a'

```

List the details of your project.

```

$ curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects/$project_id?apiVersion=$api_version" | jq "."

```

A snippet of the response shows an existing cloud zone.

```

...
"zones": [
  {
    "zoneId": "3cc2ecb989eee87557b0d532d4bb0",
    "priority": 0,
    "maxNumberInstances": 0
  }
]
...

```

Add the new cloud zone. Include the existing cloud zone `3cc2ecb989eee87557b0d532d4bb0` in the PATCH request.

```

$ curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "zoneAssignmentConfigurations" : [
      {
        "zoneId" : "'$zone_id'",
        "priority": 1,

```

```

        "maxNumberInstances": 50
    },
    {
        "zoneId" : "3cc2ecb989eee87557b0d532d4bb0",
        "priority": 2,
        "maxNumberInstances": 100
    }
]
}' | jq "."

```

The response after adding a cloud zone lists the project with its administrators, members, and zone.

```

{
  "administrators": [
    {
      "email": "admin1@mycompany.com"
    }
  ],
  "members": [
    {
      "email": "user1@mycompany.com"
    }
  ],
  "zones": [
    {
      "zoneId": "4965d34c3bfe0275574bc5fd8782a",
      "priority": 1,
      "maxNumberInstances": 50
    },
    {
      "zoneId": "3cc2ecb989eee87557b0d532d4bb0",
      "priority": 2,
      "maxNumberInstances": 100
    }
  ],
  "sharedResources": true,
  "name": "Example-project",
  "description": "This is an example project",
  "id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
  "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
  "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
  "_links": {
    "self": {
      "href": "/iaas/api/projects/edfd6f26-5d82-428f-96b0-b10ac5e4aca9"
    }
  }
}

```

Create Flavor Mappings

To create a flavor mapping, you make a POST request with a region ID associated with a cloud account. The cloud account can be an AWS, vSphere, Azure, or GCP cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).

Cloud vendors use flavors, or instance types, to express standard deployment sizings such as small (1 CPU, 2 GB RAM) or large (2 CPU, 8 GB RAM) for compute resources. When you build a blueprint, you pick a flavor that fits your needs and map a flavor name to a value for each account or region.

The same API calls create a flavor profile for an AWS, vSphere, Azure, or GCP cloud account. However, the flavor mapping used to create the flavor profile varies for each type of cloud account. This procedure provides the steps to create a flavor profile for an AWS cloud account. Additional examples show how to create flavor profiles for vSphere and Azure cloud accounts.

1. Assign the cloud account ID variable.

```
cloud_account_id='<your_cloud_account_id>'
```

2. Look up region IDs associated with the cloud account and in the external region ID us-east-1.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'"$cloud_account_id'" | jq "."
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
region_id='<your_region_id>'
```

5. List all fabric flavors.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-flavors/?apiVersion=$api_version" | jq "."
```

6. To select fabric flavor names with resources that fit your needs, examine the response.

7. Create a flavor profile for an AWS account that uses the fabric flavor names in your flavor mapping.

```
curl -X POST \
  $url/iaas/api/flavor-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your_flavor_profile>",
    "description": "Example AWS Compute flavors",
    "flavorMapping": {
      "small": {
        "name": "<flavor_name1_from_response>"
      },
      "medium": {
        "name": "<flavor_name2_from_response>"
      },
      "large": {
        "name": "<flavor_name3_from_response>"
      }
    },
    "regionId": "'$region_id'"
  }' | jq "."
```

8. To obtain the flavor profile ID, examine the response.

9. Assign the flavor profile ID variable.

```
flavor_profile_id='<your_flavor_profile_id>'
```

10. Optional: Look up the flavor profile you created with your flavor profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
flavor-profiles/$flavor_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the flavor profile you created.

Note: Using the external region ID and the cloud account ID, you can also filter for the flavor profile with a query that does not require the flavor profile ID. See [Filtering Resources by Region ID](#).

Create flavor mappings for different cloud accounts

Create a flavor mapping for an AWS cloud account.

1. Assign the required variables including the cloud account ID for an AWS cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

2. Look up region IDs associated with the cloud account and in the external region ID us-east-1.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-
east-1'%20and%20cloudAccountId%20eq%20'"$cloud_account_id"' | jq "."
```

A snippet of the response shows the region ID.

```
...
  "externalRegionId": "us-east-1",
  "cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d",
  "id": "37d6c1acf4a8275586468873c739",
  "updatedAt": " ",
...

```

3. Assign the AWS region ID.

```
$ aws_region_id='37d6c1acf4a8275586468873c739'
```

4. List all fabric flavors.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/fabric-flavors/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows a fabric flavor name with its resource size.

```
...
{
  "id": "t2.micro",
  "name": "t2.micro",
  "cpuCount": 1,
  "memoryInMB": 1024,
  "storageType": "EBS",
  "networkType": "Low to Moderate"
},
...

```

5. Select fabric flavor names with resources that fit your needs and create an AWS flavor profile named aws-flavor-profile.

```
$ curl -X POST \
```

```
$url/iaas/api/flavor-profiles?apiVersion=$api_version \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name": "aws-flavor-profile",
  "description": "Example AWS Compute flavors",
  "flavorMapping": {
    "small": {
      "name": "t2.micro"
    },
    "medium": {
      "name": "t2.medium"
    },
    "large": {
      "name": "t2.large"
    }
  },
  "regionId": "'$aws_region_id'"
}' | jq "
```

A snippet of the response shows the flavor profile ID.

```
...
"externalRegionId": "us-east-1",
"name": "aws-flavor-profile",
"description": "Example AWS Compute flavors",
"id": "835249077934b47558eca5963e068",
"updatedAt": " ",
...
```

Create a flavor mapping for a vSphere cloud account.

1. Assign the required variables including the cloud account ID for a vSphere cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

2. Look up region IDs associated with the cloud account and in the external region ID

Datacenter: datacenter-2.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'="'externalRegionId%20eq%20'Datacen-
ter: datacenter-2'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq "
```

A snippet of the response shows the region ID.

```
...
"externalRegionId": "Datacenter: datacenter-2",
"cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-d06bf79904ce5096492a2a2fc557f-
b0457d7d3c5b5e7ae20b29957788812bb3d-d5a5e16bdc3eec7557245925e1b08",
"id": "2aaf79b789eee8755724592b06d39",
"updatedAt": " ",
...
```

3. Assign the vSphere region ID.

```
$ vsphere_region_id='2aaf79b789eee8755724592b06d39'
```

4. Create a vSphere flavor profile named vcenter-flavor-profile.

```
$ curl -X POST \
```

```
$url/iaas/api/flavor-profiles?apiVersion=$api_version \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name": "vcenter-flavor-profile",
  "description": "Example vSphere Compute flavors",
  "flavorMapping": {
    "small": {
      "cpuCount": 1,
      "memoryInMB": 1024
    },
    "medium": {
      "cpuCount": 2,
      "memoryInMB": 2048
    },
    "large": {
      "cpuCount": 4,
      "memoryInMB": 4096
    }
  },
  "regionId": "'$vsphere_region_id'"
}' | jq "
```

A snippet of the response shows the flavor profile ID.

```
...
"externalRegionId": "Datacenter:datacenter-2",
"name": "vcenter-flavor-profile",
"description": "Example vSphere Compute flavors",
"id": "cfb7246505319275572e9e68372d0",
"updatedAt": " ",
...
```

Create a flavor mapping with an Azure cloud account ID.

1. Assign the required variables including the cloud account ID for an Azure cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

2. Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'="externalRegionId%20eq%20'us-
east-1'%20and%20cloudAccountId%20eq%20'"$cloud_account_id'" | jq "
```

A snippet of the response shows the region ID.

```
...
"externalRegionId": "us-east-1",
"cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d",
"id": "37d6c1acf4a8275586468873c739",
"updatedAt": " ",
...
```

3. Assign the Azure region ID.

```
$ azure_region_id='37d6c1acf4a8275586468873c739'
```

4. List all fabric flavors.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/fabric-flavors/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows a fabric flavor name with its resource size.

```
...
{
  "id": "Standard_A0",
  "name": "Standard_A0",
  "cpuCount": 1,
  "memoryInMB": 768,
  "bootDiskSizeInMB": 1047552,
  "dataDiskSizeInMB": 20480,
  "dataDiskMaxCount": 1
},
...
```

5. Select fabric flavor names with resources that fit your needs and create an Azure flavor profile named azure-flavor-profile.

```
$ curl -X POST \
  $url/iaas/api/flavor-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "azure-flavor-profile",
    "description": "Example Azure Compute flavors",
    "flavorMapping": {
      "small": {
        "name": "Standard_A0"
      },
      "medium": {
        "name": "Standard_A1"
      },
      "large": {
        "name": "Standard_A2"
      }
    },
    "regionId": "'$azure_region_id'"
  }' | jq "."
```

A snippet of the response shows the flavor profile ID.

```
...
"externalRegionId": "us-east-1",
"name": "azure-flavor-profile",
"description": "Example Azure Compute flavors",
"id": "4965d34c3bfe0275574bc6e505b78",
"updatedAt": " ",
...
```

Create Image Mappings

To create an image mapping, you make a POST request with a region ID associated with a cloud account. The cloud account can be an AWS, vSphere, Azure, or GCP cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).

Cloud vendors use images to configure a VM based on OS settings, such as an ubuntu-16 configuration. When you build a cloud template, you pick an image that fits your needs and map an image name to a value for each account or region. You can also add constraints and configuration scripts to further control resource placement.

The same API calls create an image profile for an AWS, vSphere, Azure, or GCP cloud account. The following example shows how to create an image profile for a vSphere cloud account with the external region ID

Datacenter:datacenter-3.

1. Assign the cloud account ID variable.

```
cloud_account_id='<your_cloud_account_id>'
```

2. Look up region IDs associated with the cloud account and in the external region ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'Datacenter:datacen-
ter-3'%20and%20cloudAccountId%20eq%20'"$cloud_account_id'" | jq "."
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
region_id='<your_region_id>'
```

5. Create an image profile with an image mapping that specifies the OVA/OVF links.

```
curl -X POST \
  $url/iaas/api/image-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your_image_profile>",
    "description": "<your_image_profile_description>",
    "imageMapping": {
      "ubuntu": {
        "externalId": "https://cloud-images.ubuntu.com/releases/16.04/release-20220305/ubuntu-16.04-serv-
er-cloudimg-amd64.ova"
      }
    },
    "regionId": "'$region_id'"
  }' | jq "."
```

6. To obtain the image profile ID, examine the response.

Create image mapping

Assign the required variables including a cloud account ID.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='ff4e7585-4197-41fb-89cb-179ef4d24779'
```

Look up region IDs associated with the cloud account and in the external region ID

Datacenter:datacenter-3.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'Datacenter:datacen-
ter-3'%20and%20cloudAccountId%20eq%20'"$cloud_account_id'" | jq "."
```


A snippet of the response shows the region ID.

```
...
    "externalRegionId": "Datacenter:datacenter-3",
    "name": "w01-vc08-DC",
    "cloudAccountId": "ff4e7585-4197-41fb-89cb-179ef4d24779",
    "id": "9b148b38-fc7c-4560-b413-5f47b30e57d8",
...

```

Assign the region ID.

```
$ region_id='9b148b38-fc7c-4560-b413-5f47b30e57d8'
```

Create an image profile named `example-image-profile`.

```
$ curl -X POST \
  $url/iaas/api/image-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-image-profile",
    "description": "Example image profile",
    "imageMapping": {
      "ubuntu": {
        "externalId": "https://cloud-images.ubuntu.com/releases/16.04/release-20220305/ubun-
tu-16.04-server-cloudimg-amd64.ova"
      }
    },
    "regionId": "'$region_id'"
  }' | jq "
```

A snippet of the response shows the image profile ID.

```
...
    "externalRegionId": "Datacenter:datacenter-3",
    "cloudAccountId": "ff4e7585-4197-41fb-89cb-179ef4d24779",
    "name": "example-image-profile",
    "id": "f670fd6c-66d6-4689-9793-d524e7066d1e-9b148b38-fc7c-4560-b413-5f47b30e57d8",
...

```

Working with Networks

As a VCF Automation administrator, you can use the IaaS APIs to create network profiles and get information about data collected and networks.

Create Network Profiles

To create a network profile, you make a POST request with a region ID associated with a cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).

A VCF Automation network profile describes the behavior of the network to be deployed. For example, a network might need to be Internet facing versus internal only. Networks and their profiles are cloud-specific.

The networks in this example are used for provisioning to existing or public networks. If you are working with on-demand or deployment networks, see [Using the Network APIs](#).

If you are provisioning to a private network, or outbound networks with one-way access to upstream networks, you create a network profile with isolation enabled by either subnet or security group. See [Create a Network Profile with Isolation](#).

If you want to add firewall rules to all machines provisioned with a network profile, you create a network profile with security groups. See [Create a Network Profile with Security Groups](#).

1. Assign the cloud account ID variable.

```
cloud_account_id='<your_cloud_account_id>'
```

2. Look up regions associated with the cloud account ID and with the region name us-east-1.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'="name%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
region_id='<your_region_id>'
```

5. Filter for fabric networks associated with the cloud account ID and in the external region ID us-east-1.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-networks?apiVersion=$api_version&"'$filter'="externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."
```

For details on how to construct a filter, see [Filtering Resources by Region ID](#).

6. Examine the response to find the IDs for the public networks that you want to include in your network profile.

7. Create a network profile.

```
curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your-network-profile>",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "fabricNetworkIds": [
      "<network_id1_from_response>",
      "<network_id2_from_response>"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "."
```

8. To obtain the network profile ID, examine the response.

9. Assign the network profile ID variable.

```
network_profile_id='<your_network_profile_id>'
```

10. Optional: Look up the network profile you created with your network profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/network-profiles/$network_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the network profile you created.

Create a network profile

Assign the required variables including a cloud account ID.

```
url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'$cloud_account_id'" | jq "
```

A snippet of the response shows the region ID.

```
...
  "externalRegionId": "us-east-1",
  "cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d",
  "id": "37d6c1acf4a8275586468873c739",
  "updatedAt": " ",
...
```

Assign the region ID.

```
$ region_id='37d6c1acf4a8275586468873c739'
```

Filter for fabric networks associated with the cloud account ID and in the external region ID `us-east-1`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/fabric-networks?apiVersion=$api_version | jq "
```

A snippet of the response shows the fabric network ID for a public network that you can include in your network profile.

```
...
  "isPublic": true,
  "isDefault": true,
  "cidr": "172.31.16.0/20",
  "externalRegionId": "us-east-1",
  "tags": [
    {
      "key": "vmware.enumeration.type",
      "value": "ec2_subnet"
    }
  ],
  "cloudAccountIds": [
    "c8c3c9bfdb449475-7f703c5265a63d87-f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31e-
b8ba3d2a5a-5a45a4b9d5c72475575931611aa28",
    "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d"
  ],
  "name": "subnet-0130834a",
  "id": "d43efed364ef18755759316540e3f",
...
```

Select the IDs of fabric networks that you want to include in your profile and create a network profile named `example-network-profile`.

```
$ curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-network-profile",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "fabricNetworkIds": [
      "d43efed364ef18755759316540e3d",
      "d43efed364ef18755759316540e3f"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "
```

A snippet of the response shows the network profile ID.

```
...
  "name": "example-network-profile",
  "description": "Example Network Profile",
  "id": "9cb2d11c768927558f043ec13d70",
  "updatedAt": " ",
  ...
```

Create a Network Profile with Isolation

To create either private networks without access to outside networks or outbound networks with one-way access to upstream networks, you create a network profile with isolation and specify the isolation type.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).
- Verify that you have the region ID for the regions you want to include in the profile. See the procedure in [Create Network Profiles](#).
- For simplicity, examples use `us-east-1` as the external region ID.
- Verify that you have the IDs for the non-public fabric networks you want to include in the profile. See the procedure in [Create Network Profiles](#).

This procedure provides the steps to create a network that supports isolation using a subnet, and includes optional steps that show how to create the network using an external subnet, security groups, or a VLAN transport zone. The network profile that uses a VLAN transport zone only supports private networks.

1. Assign the cloud account ID variable.

```
cloud_account_id='<your_cloud_account_id>'
```

2. Assign the region ID variable.

```
region_id='<your_region_id>'
```

3. Filter for network domains associated with the cloud account ID and in the external region ID `us-east-1`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
network-domains?apiVersion=$api_version&"'$filter="'externalRegionId%20eq%20'us-east-1'%20and%20cloudAc-
countId%20eq%20'$cloud_account_id'" | jq "
```

4. Examine the response to find the IDs for the network domain that you want to include in your network profile. If you are creating a network profile with a VLAN transport zone, select a network domain that has the custom property `"__transportZoneTrafficType": "VLAN_BACKED"`.
5. Create a network profile that supports isolation using a subnet and IDs for a non-public network.

```
curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your-network-profile-with-isolation-by-subnet>",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "isolationType": "SUBNET",
    "isolationNetworkDomainId": "<network_domain_id_from_response>",
    "isolatedNetworkCIDRPrefix": "27",
    "fabricNetworkIds": [
      "<non_public_network_id1>",
      "<non_public_network_id2>"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "
```

The response shows the name and ID for the network profile you created.

6. Optional: Create a network profile that supports isolation using an external subnet.

```
curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your-network-profile-with-isolation-by-external-subnet>",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "isolationType": "SUBNET",
    "isolationNetworkDomainId": "<network_domain_id_from_response>",
    "isolatedNetworkCIDRPrefix": "27",
    "isolationExternalFabricNetworkId": "<non_public_network_id1>",
    "fabricNetworkIds": [
      "<non_public_network_id1>",
      "<non_public_network_id2>"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "
```

The response shows the name and ID for the network profile you created.

7. Optional: Create a network profile that supports isolation using security groups.

```
curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your-network-profile-with-isolation-by-security-group>",
    "description": "Example Network Profile",
```

```

    "regionId": "'$region_id'",
    "isolationType" : "SECURITY_GROUP",
    "fabricNetworkIds": [
        "<non_public_network_id1>",
        "<non_public_network_id2>"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
} | jq ".

```

The response shows the name and ID for the network profile you created.

Create various types of network profiles with isolation

The following examples include the requests used to create a network profiles that support isolation using:

- A subnet.
- An external subnet.
- Security groups.
- A VLAN transport zone.

Assign the required variables including a cloud account ID and a region ID.

```

$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfd449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
$ region_id='37d6c1acf4a8275586468873c739'

```

Filter for network domains associated with the cloud account ID and in the external region ID us-east-1.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-domains?apiVersion=$api_version&"'$filter="externalRegionId%20eq%20'us-
east-1'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq ".

```

A snippet of the response shows the ID for a network domain that you can include in your network profile.

```

...
    "externalId": "vpc-4511a53d",
    "name": "rainpole-dev",
    "id": "233df662ec3b4875575931653ef00",
    "createdAt": " ",
    "updatedAt": " ",
    "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
    "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
    "_links": {
        "cloud-accounts": {
            "hrefs": [
                "/iaas/api/cloud-accounts/c8c3c9bfd-
b449475-7f703c5265a63d87-f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31e-
b8ba3d2a5a-5a45a4b9d5c72475575931611aa28",
                "/iaas/api/cloud-accounts/c8c3c9bfd449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d"
            ]
        },
        "self": {
            "href": "/iaas/api/network-domains/233df662ec3b4875575931653ef00"
        }
    }
...

```

To create a network profile with a VLAN transport zone, ensure that the network domain you choose includes a custom property for the transport zone.

```
...
  "customProperties": {
    "__path": "/infra/sites/default/enforcement-points/default/trans-
port-zones/9a358e99-5734-4926-b718-37cf4862f4bf",
    "__host_identifier": "[\"host-16\", \"host-23\", \"host-21\"]",
    "__cluster_identifier": "[\"domain-c8\"]",
    "__transportZoneTrafficType": "VLAN_BACKED",
    "path": "/infra/sites/default/enforcement-points/default/trans-
port-zones/9a358e99-5734-4926-b718-37cf4862f4bf"
  }
...

```

With the IDs of fabric networks that you want to include in your profile and the network domain ID you want to include, create a network profile named `example-network-profile-with-isolation-by-subnet`.

```
$ curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-network-profile-with-isolation-by-subnet",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "isolationType": "SUBNET",
    "isolationNetworkDomainId": "233df662ec3b4875575931653ef00",
    "isolatedNetworkCIDRPrefix": "27",
    "fabricNetworkIds": [
      "c19bd2921af95075575931654066a",
      "8fe650cc09d0627558d55c9ba1793"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "."

```

A snippet of the response shows the network profile ID.

```
...
  "name": "example-network-profile-with-isolation-by-subnet",
  "description": "Example Network Profile",
  "id": "2065036880e1c47558f1693558870",
  "updatedAt": " ",
...

```

Provide the `isolationExternalFabricNetworkId` to create a network profile with isolation using an external subnet.

```
$ curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-network-profile-with-isolation-by-external-subnet",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",

```

```

    "isolationType" : "SUBNET",
    "isolationNetworkDomainId" : "233df662ec3b4875575931653ef00",
    "isolatedNetworkCIDRPrefix" : "27",
    "isolationExternalFabricNetworkId": "c19bd2921af95075575931654066a",
    "fabricNetworkIds": [
        "c19bd2921af95075575931654066a",
        "8fe650cc09d0627558d55c9ba1793"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
}' | jq "."

```

A snippet of the response shows the network profile ID.

```

...
"name": "example-network-profile-with-isolation-by-external-subnet",
"description": "Example Network Profile",
"id": "2065036880e1c47558f16bd085288",
"updatedAt": " ",
...

```

Use the "isolationType": "SECURITY_GROUP" to create a network profile with isolation using a security group. Because this isolation does not use a subnet, this request does not use a network domain ID.

```

$ curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-network-profile-with-isolation-by-security-group",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "isolationType" : "SECURITY_GROUP",
    "fabricNetworkIds": [
        "c19bd2921af95075575931654066a",
        "8fe650cc09d0627558d55c9ba1793"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
}' | jq "."

```

A snippet of the response shows the network profile ID.

```

...
"name": "example-network-profile-with-isolation-by-security-group",
"description": "Example Network Profile",
"id": "bdab0d4c28af6e7558f16c78f5468",
"updatedAt": " ",
...

```

Create a network profile with isolation using a VLAN transport zone.

```

$ curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-network-profile-with-VLAN-transport-zone",

```



```

    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "isolationNetworkDomainId" : "233df662ec3b4875575931653ef00",
    "isolatedNetworkCIDRPrefix" : "27",
    "fabricNetworkIds": [
        "c19bd2921af95075575931654066a",
        "8fe650cc09d0627558d55c9ba1793"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
} ' | jq ".

```

A snippet of the response shows the network profile ID.

```

...
  "name": "example-network-profile-with-VLAN-transport-zone",
  "description": "Example Network Profile",
  "id": "9cb2d111c768927558f043ec13d70",
  "updatedAt": " ",
...

```

Create a Network Profile with Security Groups

To create a network profile with security groups, you make a POST request and provide security group IDs.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).
- Verify that you have the region ID for the regions you want to include in the profile. See the procedure in [Create Network Profiles](#).
- Verify that you have the IDs for the networks you want to include in the profile. See the procedure in [Create Network Profiles](#).

You create a network profile with security groups so that you can add firewall rules to all machines provisioned with that network profile.

1. Assign the region ID variable.

```
region_id='<your_region_id>'
```

2. Filter for security groups associated with the cloud account ID and in the external region ID us-east-1.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/security-groups?apiVersion=$api_version&"'$filter="externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq ".

```

3. Examine the response to find the IDs for the security groups that you want to include in your network profile.
4. Create a network profile with security groups using network IDs for a non-public network.

```

curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-network-profile-with-security-groups",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",
    "fabricNetworkIds": [
        "<network_id1>",
        "<network_id1>"
    ]
  }

```

```

],
"securityGroupIds": [
  "<security_group_id1_from_response>",
  "<security_group_id2_from_response>"
],
"tags": [ { "key": "env", "value": "prod" } ]
}' | jq "."

```

The response shows the name and ID for the network profile you created.

Create a network profile with security groups

Assign the required variables including a cloud account ID and a region ID.

```

$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
$ region_id='37d6c1acf4a8275586468873c739'

```

Filter for security groups associated with the cloud account ID and in the external region ID us-east-1.

```

curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/security-groups?apiVersion=$api_version&"'$filter="externalRegionId%20eq%20'us-
east-1'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."

```

A snippet of the response shows the ID for a security group that you can include in your network profile.

```

...
"externalId": "sg-0305bc072a9f2727b",
"name": "OC-LB-mcm681186-113024780265_SG",
"id": "bdab0d4c28af6e7558f061f772518",
"createdAt": " ",
"updatedAt": " ",
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
"orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
"_links": {
  "cloud-accounts": {
    "hrefs": [
      "/iaas/api/cloud-accounts/c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d"
    ]
  },
  "self": {
    "href": "/iaas/api/security-groups/bdab0d4c28af6e7558f061f772518"
  }
}
...

```

With the IDs of fabric networks that you want to include in your profile and the security group IDs you want to include, create a network profile named `example-network-profile-with-security-groups`.

```

$ curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "example-network-profile-with-security-groups",
    "description": "Example Network Profile",
    "regionId": "'$region_id'",

```

```

    "fabricNetworkIds": [
      "d43efed364ef18755759316540e3d",
      "d43efed364ef18755759316540e3f"
    ],
    "securityGroupIds": [
      "bdab0d4c28af6e7558f061f772518",
      "ebdab0d4c28af6e7558efe6edd71c9"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq ".

```

A snippet of the response shows the network profile ID.

```

...
"name": "example-network-profile-with-security-groups",
"description": "Example Network Profile",
"id": "9cb2d111c768927558f1799bf9e48",
"updatedAt": " ",
...

```

Using the Network APIs

As a VCF Automation administrator, you must understand the difference between using the fabric network APIs and the network APIs.

You use the fabric network APIs such as `/iaas/api/fabric-networks/` to get information about existing networks that you use to create a network profile.

You use the network APIs such as `/iaas/api/networks/` to get information about on-demand networks that are provisioned with a deployment. If provisioned, you can check these networks in the VCF Automation UI by going to **Resources > Deployments**.

To see a list of both public and on-demand networks in the VCF Automation UI, go to **Infrastructure > Resources > Networks**.

Creating Storage Profiles

Using different input variables, you can use the VCF Automation IaaS API to create storage profiles for AWS, vSphere, or Azure cloud accounts.

A VCF Automation storage profile describes the kind of storage to be deployed. Storage is usually profiled according to characteristics such as service level or cost, performance, or purpose, such as backup.

As a cloud administrator, you organize your storage profiles under cloud-specific regions. One cloud account can have multiple regions, with multiple storage profiles under each. Using a storage profile, you define your storage for the region.

Storage profiles include disk customizations, and a means of identifying the type of storage by capability tags. Tags are matched against provisioning service request constraints to create the desired storage at deployment time.

Create an Amazon Web Services Storage Profile

To create a storage profile, you make a POST request with a region ID associated with a cloud account. The request body includes an Amazon Web Services fabric volume type.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).

As an alternative to using the `storage-profiles` API call to create an Amazon Web Services storage profile, you can also use the `storage-profiles-aws` API call. Optional procedure steps show how to use the `storage-profiles-aws` API call. The example only includes the steps required to create an Amazon Web Services storage profile using the `storage-profiles` API call.

1. Assign the cloud account ID variable.

```
aws_cloud_account_id='<your_cloud_account_id>'
```

2. Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'$aws_cloud_account_id'" | jq "."
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
aws_region_id='<your_region_id>'
```

5. List all Amazon Web Services fabric volume types.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-aws-volume-types/?apiVersion=$api_version" | jq "."
```

6. To select a volume type, examine the response.

7. Create a storage profile for the selected region.

```
curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ],
    "diskProperties": {
      "deviceType": "ebs",
      "volumeType": "<volume_type_from_response>",
      "iops": "400"
    },
    "regionId": "'$region_id'",
    "name": "<your-aws-storage-profile>",
    "description": "Example AWS storage profile"
  }' | jq "."
```

8. Optional: Create a storage profile for the selected region using the `storage-profiles-aws` API call.

```
curl -X POST \
  $url/iaas/api/storage-profiles-aws?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ]
    "deviceType": "ebs",
```

```

    "volumeType": "<volume_type_from_response>",
    "iops": "1000",
    "regionId": "'$region_id'",
    "name": "<your-aws-storage-profile>",
    "description": "Example AWS storage profile"
  }' | jq "."

```

9. To obtain the storage profile ID, examine the response.

10. Assign the storage profile ID variable.

```
aws_storage_profile_id='<your_storage_profile_id>'
```

11. Optional: Look up the storage profile you created with your storage profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$aws_storage_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the storage profile you created.

Note: Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See [Filtering Resources by Region ID](#).

12. Optional: List all storage profiles using the storage-profiles-aws API call.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles-aws?apiVersion=$api_version | jq "."
```

13. Optional: Delete an Amazon Web Services storage profile. Alternatively, you can use the storage-profiles-aws API call.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$aws_storage_profile_id?apiVersion=$api_version | jq "."
```

Create an Amazon Web Services storage profile

Assign the required variables including the cloud account ID for an Amazon Web Services cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ aws_cloud_account_id='c8c3c9b9fdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID us-east-1.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'$aws_cloud_account_id'" | jq "."
```

A snippet of the response shows the region ID.

```
...
  "externalRegionId": "us-east-1",
  "cloudAccountId": "c8c3c9b9fdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d",
  "id": "37d6c1acf4a8275586468873c739",
  "updatedAt": " ",
...

```

Assign the Amazon Web Services region ID.

```
$ aws_region_id='37d6c1acf4a8275586468873c739'
```

List all Amazon Web Services fabric volume types.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-aws-volume-types/?apiVersion=$api_version" | jq "
```

A snippet of the response shows the volume types.

```
...
{
  "volumeTypes": [
    "standard",
    "io1",
    "gp2",
    "sc1",
    "st1"
  ]
},
...
```

Select volume type and create an AWS storage profile named `aws-storage-profile`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ],
    "diskProperties": {
      "deviceType": "ebs",
      "volumeType": "io1",
      "iops": "400"
    },
    "regionId": "'$aws_region_id'",
    "name": "aws-storage-profile",
    "description": "Example AWS storage profile"
    "regionId": "'$aws_region_id'"
  }' | jq "
```

A snippet of the response shows the storage profile ID.

```
...
  "externalRegionId": "us-east-1",
  "name": "aws-storage-profile",
  "description": "Example AWS storage profile",
  "id": "3e3dc378-a090-4b7e-af41-57b1735d9526",
  "createdAt": " ",
  "updatedAt": " ",
  ...
```

Create a vSphere Storage Profile

To create a vSphere storage profile, you make a POST request with a region ID. Optional request body input includes a vSphere storage policy and a vSphere datastore.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).

As an alternative to using the `storage-profiles` API call to create a vSphere storage profile, you can also use the `storage-profiles-vsphere` API call. Optional procedure steps show how to use the `storage-profiles-vsphere` API call. The example only includes the steps required to create a vSphere storage profile using the `storage-profiles` API call.

1. Assign the cloud account ID variable.

```
vsphere_cloud_account_id='<your_cloud_account_id>'
```

2. Look up region IDs associated with the cloud account and in the external region ID `Datacenter:datacenter-10`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'Datacenter:datacenter-10'%20and%20cloudAccountId%20eq%20'$vsphere_cloud_account_id'" | jq "."
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
vsphere_region_id='<your_region_id>'
```

5. Optional: If using a vSphere storage policy, list all vSphere storage policies. If using a default storage policy, skip this step.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "."
```

Examine the response and assign the vSphere storage policy ID.

```
vsphere_storage_policy_id='<your_vsphere_storage_policy_id>'
```

6. Optional: If using a vSphere datastore, list all vSphere datastores. If provisioning any datastore or cluster, skip this step.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-vsphere-datastores/?apiVersion=$api_version" | jq "."
```

Examine the response and assign the vSphere datastore ID.

```
vsphere_datastore_id='<your_vsphere_datastore_id>'
```

7. Create a vSphere storage profile.

```
curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "<your-vsphere-storage-profile>",
    "description": "Example vSphere storage profile",
    "defaultItem": true,
    "supportsEncryption": true,
    "tags": [ { "key" : "env", "value": "dev" } ],
    "diskProperties": {
      "provisioningType": "thin",
      "independent": "true",
      "persistent": "true",
      "sharesLevel": "low",
      "shares": "500",
      "limitIops": "500"
    },
    "diskTargetProperties": {
```

```

    "storagePolicyId": "'$vsphere_storage_policy_id'",
    "datastoreId": "'$vsphere_datastore_id'" },
    "regionId": "'$vsphere_region_id'"
  }' | jq "."

```

8. To obtain the storage profile ID, examine the response.

9. Assign the storage profile ID variable.

```
vsphere_storage_profile_id='<your_storage_profile_id>'
```

10. Optional: Look up the storage profile you created with your storage profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$vsphere_storage_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the storage profile you created.

Note: Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See [Filtering Resources by Region ID](#).

11. Optional: List all storage profiles using the storage-profiles-vsphere API call. API.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles-vsphere?apiVersion=$api_version | jq "."
```

12. Optional: Delete a vSphere storage profile. Alternatively, you can use the storage-profiles-vsphere API call..

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$vsphere_storage_profile_id?apiVersion=$api_version | jq "."
```

Create vSphere storage profile

Assign the required variables including the cloud account ID for a vSphere cloud account.

```

$ url='https://appliance.domain.com'
$ api_version=' '
$ vsphere_cloud_account_id='515684cce-
bafde75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1dala285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a

```

Look up region IDs associated with the cloud account and in the external region ID

Datacenter:datacenter-10.

```

$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&""'$filter'="externalRegionId%20eq%20'Datacenter:datacen-
ter-10'%20and%20cloudAccountId%20eq%20'$vsphere_cloud_account_id'" | jq "."

```

A snippet of the response shows the region ID.

```

...
    "externalRegionId": "Datacenter:datacenter-10",
    "cloudAccountId": "c8c3c9b9fdb449475-7f703c5265a63d87-809fe6fef311fd-
d63aa6dac546574aa898213265e988e34cc851db19b8c05b96-f405bb370210c875572d26445252e",
    "id": "cfb7246505319275572d26466a749",
...

```

Assign the vSphere region ID.

```
$ vsphere_region_id='cfb7246505319275572d26466a749'
```

If using a vSphere storage policy, perform the following steps.

1. List all vSphere storage policies.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the storage policy.

```
...
    "externalId": "f31f2442-8247-4517-87c2-8d69d7a6c696",
    "name": "Management Storage Policy - Stretched",
    "description": "Management Storage policy used for VMC stretched cluster",
    "id": "4aad51f0b02b5275572d264d28490",
    ...
```

2. Examine the response to assign the vSphere storage policy ID.

```
$ vsphere_storage_policy_id='4aad51f0b02b5275572d264d28490'
```

If provisioning a specific datastore or cluster, perform the following steps.

1. List all vSphere datastores.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/fabric-vsphere-datastores/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the datastore.

```
...
    "externalId": "WorkloadDatastore",
    "name": "WorkloadDatastore",
    "id": "c4f1dd4741d05e75572d264dcc590",
    "createdAt": " ",
    "updatedAt": " ",
    ...
```

2. Examine the response to assign the vSphere datastore ID.

```
$ vsphere_datastore_id='c4f1dd4741d05e75572d264dcc590'
```

Create a vSphere storage profile named `vsphere-storage-profile`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "vsphere-storage-profile",
    "description": "Example vSphere storage profile",
    "defaultItem": true,
    "supportsEncryption": true,
    "tags": [ { "key" : "env", "value": "dev" } ],
    "diskProperties": {
      "provisioningType": "thin",
      "independent": "true",
      "persistent": "true",
      "sharesLevel": "low",
      "shares": "500",
      "limitIops": "500"
    },
    "diskTargetProperties": {
      "storagePolicyId": "'$vsphere_storage_policy_id'",
      "datastoreId": "'$vsphere_datastore_id'"
    },
  },
```

```
    "regionId": "'$vsphere_region_id'"
  }' | jq "
```

A snippet of the response shows the storage profile ID.

```
...
  "externalRegionId": "Datacenter:datacenter-10",
  "name": "vsphere-storage-profile",
  "description": "Example vSphere storage profile",
  "id": "b4fbd25e-a2dd-4fde-9186-0f7bd34a1df2",
  "createdAt": " ",
  "updatedAt": " ",
  ...
```

Create a vSphere Storage Profile for a First Class Disk

To create a vSphere Storage Profile that supports First Class Disk (FCD) storage, you make a POST request with a region ID and you include first class as the disk type property. Optional request body input includes a vSphere storage policy and a vSphere datastore.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the vSphere cloud account that you added. See [Add avSphere Cloud Account](#).

1. Assign the cloud account ID variable.

```
vsphere_cloud_account_id='<your_cloud_account_id>'
```

2. Look up region IDs associated with the cloud account and in the external region ID Datacenter:datacenter-3.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&''$filter'"=externalRegionId%20eq%20'Datacenter:datacenter-3'%20and%20cloudAccountId%20eq%20'$vsphere_cloud_account_id'" | jq "
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
vsphere_region_id='<your_region_id>'
```

5. Optional: If using a vSphere storage policy, list all vSphere storage policies. If using a default storage policy, skip this step.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "
```

Examine the response and assign the vSphere storage policy ID.

```
vsphere_storage_policy_id='<your_vsphere_storage_policy_id>'
```

6. Optional: If using a vSphere datastore, list all vSphere datastores. If provisioning any datastore or cluster, skip this step.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-vsphere-datastores/?apiVersion=$api_version" | jq "
```

Examine the response and assign the vSphere datastore ID.

```
vsphere_datastore_id='<your_vsphere_datastore_id>'
```

7. Create a vSphere Storage Profile with the FCD property.

```
curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
```

```
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name": "<your_vsphere_storage_profile_with_FCD>",
  "description": "Example First Class Disk vSphere Storage Profile",
  "defaultItem": true,
  "provisioningType": "thin",
  "diskType": "firstClass",
  "regionId": "'$vsphere_region_id'",
  "tags": [ { "key": "type", "value": "fcd" } ]
}' | jq "
```

8. Examine the response.

- "defaultItem": true indicates that this storage profile is the default for the region.
- Tags help you to locate, manage, and work with the infrastructure resources.

Create vSphere Storage Profile with FCD storage

Assign the required variables including the cloud account ID for a vSphere cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ vsphere_cloud_account_id='683c647b-413d-4673-a236-08b3694cd652'
```

Look up region IDs associated with the cloud account and in the external region ID

Datacenter:datacenter-3.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'Datacenter:datacen-
ter-10'%20and%20cloudAccountId%20eq%20'$vsphere_cloud_account_id'" | jq "
```

A snippet of the response shows the region ID.

```
...
  "externalRegionId": "Datacenter:datacenter-3",
  "name": "Example external region name",
  "cloudAccountId": "683c647b-413d-4673-a236-08b3694cd652",
  "id": "0f182edc-1155-4df1-a53a-2c46be7bc373",
...
```

Assign the vSphere region ID.

```
$ vsphere_region_id='0f182edc-1155-4df1-a53a-2c46be7bc373'
```

If using a vSphere storage policy, perform the following steps.

1. List all vSphere storage policies.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "
```

A snippet of the response shows the storage policy.

```
...
  "externalId": "f31f2442-8247-4517-87c2-8d69d7a6c696",
  "name": "Management Storage Policy - Stretched",
  "description": "Management Storage policy used for VMC stretched cluster",
  "id": "4aad51f0b02b5275572d264d28490",
...
```

2. Examine the response to assign the vSphere storage policy ID.

```
$ vsphere_storage_policy_id='4aad51f0b02b5275572d264d28490'
```

If provisioning a specific datastore or cluster, perform the following steps.

1. List all vSphere datastores.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
"$url/iaas/api/fabric-vsphere-datastores/?apiVersion=$api_version" | jq "
```

A snippet of the response shows the datastore.

```
...
    "externalId": "WorkloadDatastore",
    "name": "WorkloadDatastore",
    "id": "c4f1dd4741d05e75572d264dcc590",
    "createdAt": " ",
    "updatedAt": " ",
...

```

2. Examine the response to assign the vSphere datastore ID.

```
$ vsphere_datastore_id='c4f1dd4741d05e75572d264dcc590'
```

Create a vSphere Storage Profile named `vsphere-storage-profile-with-FCD`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "vsphere-storage-profile-with-FCD",
    "description": "Example First Class Disk vSphere Storage Profile",
    "defaultItem": true,
    "provisioningType": "thin",
    "diskType": "firstClass",
    "regionId": "'$vsphere_region_id'",
    "tags": [ { "key": "type", "value": "fcd" } ]
  }' | jq "
```

Included with storage profile ID, a snippet of the response shows the tags that you defined for the storage profile.

```
{
  "defaultItem": true,
  "tags": [
    {
      "key": "type",
      "value": "fcd"
    }
  ],
  "provisioningType": "thin",
  "externalRegionId": "Datacenter:datacenter-3",
  "cloudAccountId": "683c647b-413d-4673-a236-08b3694cd652",
  "diskType": "firstClass",
  "name": "vsphere-storage-profile-with-FCD",
  "description": "Example First Class Disk vSphere Storage Profile",
  "id": "6037ac02-83e0-41bb-ba6e-ed5784ae1101",
  "createdAt": " ",
  "updatedAt": " ",
}
```

...

Use the tags to create a First Class Disk. See [Create a First Class Disk](#).

Create a Microsoft Azure Storage Profile

To create a Microsoft Azure storage profile, you make a POST request with a region ID. The request body includes a Microsoft Azure fabric storage account ID.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).

As an alternative to using the `storage-profiles` API call to create a Microsoft Azure storage profile, you can also use the `storage-profiles-azure` API call. Optional procedure steps show how to use the `storage-profiles-azure` API call. The example only includes the steps required to create a Microsoft Azure storage profile using the `storage-profiles` API call.

1. Assign the cloud account ID variable.

```
azure_cloud_account_id='<your_cloud_account_id>'
```

2. Look up region IDs associated with the cloud account.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId%20eq%20'"$azure_cloud_account_id'" | jq "."
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
azure_region_id='<your_region_id>'
```

5. List all Azure fabric storage accounts.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/fabric-azure-storage-accounts/?apiVersion=$api_version" | jq "."
```

6. To select a storage account ID, examine the response.

7. Create a storage profile for the selected region.

```
curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ],
    "diskProperties": {
      "azureOsDiskCaching": "None",
      "azureDataDiskCaching": "None"
    },
    "diskTargetProperties": { "storageAccountId": "<storage_account_id_from_response>" },
    "regionId": "'$azure_region_id'",
    "name": "<your-azure-storage-profile>",
    "description": "Example Azure storage profile"
  }' | jq "."
```

8. Optional: Create a storage profile for the selected region using the storage-profiles-azure API call.

```
curl -X POST \
  $url/iaas/api/storage-profiles-azure?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "osDiskCaching": "None",
    "dataDiskCaching": "None",
    "storageAccountId": "<storage_account_id_from_response>",
    "regionId": "'$azure_region_id'",
    "name": "<your-azure-storage-profile>",
    "description": "Example Azure storage profile"
    "tags": [ { "key": "env", "value": "dev" } ]
  }' | jq "."
```

9. To obtain the storage profile ID, examine the response.

10. Assign the storage profile ID variable.

```
azure_storage_profile_id='<your_storage_profile_id>'
```

11. Optional: Look up the storage profile you created with your storage profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$azure_storage_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the storage profile you created.

Note: Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See [Filtering Resources by Region ID](#).

12. Optional: List all Azure storage profiles using the storage-profiles-azure API call.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles-azure?apiVersion=$api_version | jq "."
```

13. Optional: Delete a Microsoft Azure storage profile. Alternatively, you can use the storage-profiles-azure API call.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$azure_storage_profile_id?apiVersion=$api_version | jq "."
```

Create a Microsoft Azure storage profile

Assign the required variables including the cloud account ID for a Microsoft Azure cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ azure_cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254
c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID us-east-1

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'$azure_cloud_account_id'" | jq "."
```

A snippet of the response shows the region ID.

```
...
```

```

    "externalRegionId": "us-east-1",
    "cloudAccountId": "c8c3c9bdfdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d",
    "id": "37d6c1acf4a8275586468873c739",
    "updatedAt": " ",
    ...

```

Assign the Azure region ID.

```
$ azure_region_id='37d6c1acf4a8275586468873c739'
```

List all fabric storage accounts.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/fabric-azure-storage-accounts/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the storage accounts.

```

...
    "externalId": "/subscriptions/b8ef63a7-a5e3-44fa-8745-lead33fa1f25/resourceGroups/default-rg/
providers/Microsoft.Storage/storageAccounts/azbasicsa80370",
    "name": "azbasicsa80370",
    "id": "f81c26bf-51b1-49cc-865c-de2ab3821c1d",
    ...

```

Select storage account ID and create an Azure storage profile named azure-storage-profile.

```

$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ],
    "diskProperties": {
      "azureOsDiskCaching": "None",
      "azureDataDiskCaching": "None"
    },
    "diskTargetProperties": { "storageAccountId": "f81c26bf-51b1-49cc-865c-de2ab3821c1d" },
    "regionId": "'$azure_region_id'",
    "name": "azure-storage-profile",
    "description": "Example Azure storage profile"
  }' | jq "."

```

A snippet of the response shows the storage profile ID.

```

...
    "externalRegionId": "us-east-1",
    "name": "azure-storage-profile",
    "description": "Example Azure storage profile",
    "id": "f83d0fd4-45de-4ca7-a699-c98bc141ecaa",
    "createdAt": " ",
    "updatedAt": " ",
    ...

```

Create a Microsoft Azure Storage Profile for a Managed Disk

To create a Microsoft Azure storage profile for a managed disk, you make a POST request with a region ID and include disk properties to specify the managed disk type.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud account ID for the new cloud account that you added. See [Adding Cloud Accounts](#).

1. Assign the cloud account ID variable.

```
azure_cloud_account_id='<your_cloud_account_id>'
```

2. Look up region IDs associated with the cloud account and in the external region ID eastus.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'eastus'%20and%20cloudAccountId%20eq%20"$azure_cloud_account_id"' | jq "."
```

3. Examine the response to find the ID for the region that you want.

4. Assign the region ID variable.

```
azure_region_id='<your_region_id>'
```

5. Create a storage profile for the selected region.

```
curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "type", "value": "managed" } ],
    "diskProperties": {
      "azureManagedDiskType": "Standard_LRS",
      "azureOsDiskCaching": "ReadWrite",
      "azureDataDiskCaching": "ReadWrite"
    },
    "regionId": "'$azure_region_id'",
    "name": "<your-azure-managed-disk-storage-profile>",
    "description": "Example Azure managed disk"
  }' | jq "."
```

6. To obtain the storage profile ID, examine the response.

7. Assign the storage profile ID variable.

```
azure_storage_profile_id='<your_storage_profile_id>'
```

8. Optional: Look up the storage profile you created with your storage profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/storage-profiles/$azure_storage_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the storage profile you created.

Note: Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See [Filtering Resources by Region ID](#).

Create a Microsoft Azure storage profile

Assign the required variables including the cloud account ID for a Microsoft Azure cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ azure_cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254
c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID eastus

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'="externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'$azure_cloud_account_id'" | jq -."
```

A snippet of the response shows the region ID.

```
...
  "externalRegionId": "eastus",
  "cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478d-
f36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d",
  "id": "20d6c1abc4a8275586468873c721",
  "updatedAt": " ",
...
```

Assign the Azure region ID.

```
$ azure_region_id='20d6c1abc4a8275586468873c721'
```

Create a Microsoft Azure storage profile named azure-with-managed-disks-storage-profile.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "type", "value": "managed" } ],
    "diskProperties": {
      "azureManagedDiskType": "Standard_LRS",
      "azureOsDiskCaching": "ReadWrite",
      "azureDataDiskCaching": "ReadWrite"
    },
    "regionId": "'$azure_region_id'",
    "name": "azure-with-managed-disks-storage-profile",
    "description": "Example Azure with managed disks storage profile"
  }' | jq -."
```

A snippet of the response shows the storage profile ID.

```
...
  "externalRegionId": "eastus",
  "name": "azure-with-managed-disks-storage-profile",
  "description": "Example Azure with managed disks storage profile",
  "id": "f83d0fd4-45de-4ca7-a699-c20bc121abcd",
  "createdAt": " ",
  "updatedAt": " 2021-08-02",
```

...

Managing Your Projects Using the Project APIs

You use the Project APIs to manage projects that can be used in any of the VCF Automation component services. In addition to basic operations such as creating, updating, or deleting projects, you can use the Project APIs to modify principals assigned to a project or modify a project cost.

Create a Project with the Project Service API

Using the Project Service API, you can create a project. You can also modify or delete a project and list all projects in an organization.

- Verify that all general prerequisites and prerequisites for the Project service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that the project roles that you plan to assign have sufficient permissions to perform project-related tasks.

Note:

A user with the project administrator or project member role can perform a limited number of project-related tasks.

- Prepare parameters including the project name, description, and email addresses for administrators, members, or viewers.

Before creating a project, it is a good practice to get a list of projects so that you can verify that the project you plan to create does not exist. Then you create the project with users assigned to project roles.

1. Get a list of projects.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/project-service/api/projects?apiVersion=$api_version" | jq "."
```

2. To verify that the project you plan to create is not already listed, examine the response.

3. Assign the project name variable.

```
project_name='<your_project_name>'
```

your_project_name is a name that you choose.

4. Create a project.

```
curl -X POST \
"$url/project-service/api/projects?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name" : "'$project_name'",
  "description" : "your-description",
  "administrators" : [{ "email" : "<admin_email>", ["type" : <"user" | "group">]}],
  "members" : [{ "email" : "<member_email>", ["type" : <"user" | "group">]}],
  "viewers" : [{ "email" : "<viewer_email>", ["type" : <"user" | "group">]}],
}' | jq "."
```

- *admin_email*, *member_email*, and *viewer_email* are email addresses of an administrator, member, and viewer user or name of the group in the project.
- The type parameter is optional. It assigns the administrator, member, or viewer to a user or group type. If unspecified, the value defaults to `user`.

Create a Project

Create a project named `Example-project` with administrators, members, and viewers at `mycompany.com`. This example assumes that `Example-project` does not exist.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_name='Example-project'
```

Create a project with an administrator, member, and viewer assigned to user type roles.

```
$ curl -X POST \
"$url/project-service/api/projects?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name" : "'$project_name'",
  "description" : "This is an example project",
  "administrators" : [{"email" : "adminX@mycompany.com", "type" : "user"}],
  "members" : [{"email" : "memberX@mycompany.com", "type" : "user"}],
  "viewers" : [{"email" : "viewerX@mycompany.com", "type" : "user"}]
}' | jq "."
```

The response shows the administrators, members, and viewers related to the project and the project ID.

```
{
  "id": "094a2fab-7715-4844-94f9-71b45452da27",
  "name": "Example-project",
  "description": "This is an example project",
  "orgId": "f670fdcf-66d6-4689-9793-d524e7066d1e",
  "administrators": [
    {
      "email": "adminX@mycompany.com",
      "type": "user"
    }
  ],
  "members": [
    {
      "email": "memberX@mycompany.com",
      "type": "user"
    }
  ],
  "viewers": [
    {
      "email": "viewerX@mycompany.com",
      "type": "user"
    }
  ],
  "supervisors": [],
  "constraints": {
    "network": {
      "conditions": []
    }
  },
  "properties": {},
  "cost": {
```

```

    "cost": 0,
    "costSyncTime": "2019-05-13T12:47:10.624Z",
    "costUnit": "USD"
  },
  "operationTimeout": 0,
  "sharedResources": true
},
...

```

Add Users to Your Project Using the Project Service API

As a service administrator, you can use a PATCH request to add, modify, or remove a project user.

- Verify that all general prerequisites and prerequisites for the Project service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the project administrator role in your project and you have the project ID. See [Create a Project with the Project Service API](#).
- Verify that the project roles that you plan to assign have sufficient permissions to perform project-related tasks.

Note:

A user with the project administrator or project member role can perform a limited number of project-related tasks.

- Prepare parameters including additional email addresses for administrators, members, or viewers that you want to add to the project.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

your_project_id is the ID of the new project you created.

2. List the details of your project.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/project-service/api/projects/$project_id?apiVersion=$api_version" | jq "."
```

3. Examine the response to see the administrators and users who are already in your project.

4. Submit a request to add a new project administrator.

```

curl -X PATCH \
  "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "modify" : [
      {
        "email" : "<your_new_administrator_email>",
        "role" : "administrator",
        "type" : "user"
      }
    ]
  }' | jq "."

```

5. Submit a request to add a new project member.

```

curl -X PATCH \
  "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \

```

```
-H "Authorization: Bearer $access_token" \
-d '{
  "modify" : [
    {
      "email" : "<your_new_member_email>",
      "role" : "member",
      "type" : "user"
    }
  ]
}' | jq "."
```

Add Users to Your Project

For the project `Example-project`, add a user with the administrator role and a user with the member role at `mycompany.com`.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='094a2fab-7715-4844-94f9-71b45452da27'
```

List the details of your project.

```
$ curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/project-
service/api/projects/$project_id?apiVersion=$api_version" | jq "."
```

The response shows existing administrators, members, and viewers.

```
{
  "id": "094a2fab-7715-4844-94f9-71b45452da27",
  "name": "Example-project",
  "description": "This is an example project",
  "orgId": "f670fd6c-66d6-4689-9793-d524e7066d1e",
  "administrators": [
    {
      "email": "adminX@mycompany.com",
      "type": "user"
    }
  ],
  "members": [
    {
      "email": "memberX@mycompany.com",
      "type": "user"
    }
  ],
  "viewers": [
    {
      "email": "viewerX@mycompany.com",
      "type": "user"
    }
  ],
  "supervisors": [],
  "constraints": {
    "network": {
      "conditions": []
    }
  },
  "properties": {},
}
```

```

    "cost": {
      "cost": 0,
      "costSyncTime": "2019-05-13T12:47:10.624Z",
      "costUnit": "USD"
    },
    "operationTimeout": 0,
    "sharedResources": true
  },
  ...

```

Add the administrator.

```

curl -X PATCH \
  "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "modify" : [
      {
        "email" : "newadminX@mycompany.com",
        "role" : "administrator",
        "type" : "user"
      }
    ]
  }' | jq "."

```

Add the member.

```

curl -X PATCH \
  "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "modify" : [
      {
        "email" : "newmemberX@mycompany.com",
        "role" : "member",
        "type" : "user"
      }
    ]
  }' | jq "."

```

The response lists the project including the added administrator and added member.

```

{
  "id": "094a2fab-7715-4844-94f9-71b45452da27",
  "name": "Example-project",
  "description": "This is an example project",
  "orgId": "f670fd6c-66d6-4689-9793-d524e7066d1e",
  "administrators": [
    {
      "email": "newadminX@mycompany.com",
      "type": "user"
    }
  ],
  ...
}

```

```

        "email": "adminX@mycompany.com",
        "type": "user"
    }
    "members": [
        {
            "email": "newmemberX@mycompany.com",
            "type": "user"
        }
        {
            "email": "memberX@mycompany.com",
            "type": "user"
        },
    ],
    "viewers": [
        {
            "email": "viewerX@mycompany.com",
            "type": "user"
        }
    ]
    "supervisors": [],
    "constraints": {
        "network": {
            "conditions": []
        }
    },
    "properties": {},
    "cost": {
        "cost": 0,
        "costSyncTime": "2019-05-13T12:47:10.624Z",
        "costUnit": "USD"
    },
    "operationTimeout": 0,
    "sharedResources": true
}

```

Working with Blueprints/Cloud Templates

To create and update cloud templates, version cloud templates, and deploy cloud templates, you use the VCF Automation Blueprint APIs. Blueprint is the term used in the API specifications. In the product, blueprints are renamed to VCF Automation Templates.

Create and Update a Cloud Template

To create a cloud template, you make a POST request. The request body includes the name of the new cloud template and the project ID of an existing project. To update a cloud template, you make a PUT request that changes one of the properties of the cloud template.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have a project ID for the project that includes cloud zones configured to support the resource requirements of your cloud template. See [Create a Project to use in VCF Automation](#).

Before creating a cloud template, you get a list of cloud templates to verify that the cloud template you plan to create does not already exist. After a cloud template is created, you can update it to change the cloud template definition.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. Assign your cloud template name variable.

```
cloud_template_name='<your_cloud_template_name>'
```

your_cloud_template_name is a name that you choose.

3. Get a list of cloud templates.

```
curl -X GET $url/blueprint/api/blueprints -H "Authorization: Bearer $access_token" | jq "."
```

4. To verify that the cloud template you plan to create is not already listed, examine the response.

5. Validate the cloud template before creating it.

```
curl -X POST \
  $url/blueprint/api/blueprint-validation?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic Cloud Machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n    description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    properties:\n      name: BasicCloudMachine\n      flavor: '\"${input.flavor}\"'\n      image: '\"${input.image}\"'\n      count: '\"${input.count}\"'\n    projectId : \"$project_id\",\n    requestScopeOrg: false
  }' | jq "."
```

6. Examine the response to confirm that you see "valid":true.

7. Create a new cloud template.

```
curl -X POST \
  $url/blueprint/api/blueprints?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic Cloud Machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n    description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    properties:\n      name: BasicCloudMachine\n      flavor: '\"${input.flavor}\"'\n      image: '\"${input.image}\"'\n      count: '\"${input.count}\"'\n    projectId : \"$project_id\",\n    requestScopeOrg: false
  }' | jq "."
```


8. Examine the response and record the ID of your newly created cloud template.

9. Assign the cloud template ID variable.

```
cloud_template_id='<your_cloud_template_id>'
```

your_cloud_template_id is the ID of the new cloud template that you created.

10. To verify that the cloud template has been created, get a list of cloud templates and filter for *your_cloud_template_name*.

```
curl -X GET $url/blueprint/api/blueprints?name=$cloud_template_name -H "Authorization: Bearer $access_token" | jq "."
```

11. Optional: To update the cloud template, use a PUT request and specify *your_cloud_template_id*.

```
curl -X PUT \
  $url/blueprint/api/blueprints/$cloud_template_id?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic Cloud Machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n    description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    properties:\n      name: BasicCloudMachine\n      flavor: '$${input.flavor}'\n      image: '$${input.image}'\n      count: '$${input.count}'\n      tags: [\n        {\n          \"key\": \"env\", \n          \"value\": \"prod\"\n        }\n      ]\n    ",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
  }' | jq "."
```

Create a Cloud Template and Update it

Create a cloud template named `MyExampleCloudTemplate`. This example assumes that `MyExampleCloudTemplate` does not already exist.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbfb51'
$ cloud_template_name='MyExampleCloudTemplate'
```

Validate the cloud template.

```
$ curl -X POST \
  $url/blueprint/api/blueprint-validation?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic cloud machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n    description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n    "
```

```

    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n  prop-
erties:\n    name: BasicCloudMachine\n    flavor: '\''${input.flavor}\''\n    image:
'\''${input.image}\''\n    count: '\''${input.count}\'',
    "projectId" : "'$project_id'",
    "requestScopeOrg": false
}' | jq "."

```

Examine the response to confirm that "valid":true.

```

{
  "valid": true,
  "validationMessages": []
}

```

Create the cloud template.

```

$ curl -X POST \
  $url/blueprint/api/blueprints?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'$cloud_template_name'",
    "description" : "Basic cloud machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n
    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n  prop-
erties:\n    name: BasicCloudMachine\n    flavor: '\''${input.flavor}\''\n    image:
'\''${input.image}\''\n    count: '\''${input.count}\'',
    "projectId" : "'$project_id'",
    "requestScopeOrg": false
}' | jq "."

```

The response from your request to create the cloud template shows the cloud template ID.

```

{
  "id": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  "createdAt": "2019-10-10T23:43:27.001Z",
  ...
  "selfLink": "/blueprint/api/blueprints/1f170637-81a3-4257-b1cd-b2219ee8034c"
  ...
}

```

Assign the cloud template ID variable.

```
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
```

Update the cloud template to set a tag on the machine properties in the cloud template.

```

$ curl -X PUT \
  $url/blueprint/api/blueprints/$cloud_template_id?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'$cloud_template_name'",
    "description" : "Basic cloud machine cloud template",

```

```

    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n  description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n  title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    properties:\n      name: BasicCloudMachine\n      flavor: '{{input.flavor}}'\n      image: '{{input.image}}'\n      count: '{{input.count}}'\n      tags: [\n        {\n          \"key\": \"env\", \n          \"value\": \"prod\" \n        } \n      ], \n      \"projectId\" : \"\"$project_id\"\", \n      \"requestScopeOrg\": false\n    }' | jq ."

```

Use the cloud template ID to version and release the cloud template to a catalog.

Setting up Policies

As a cloud administrator, you can use the VCF Automation APIs to create policies with rules or parameters that you apply to deployments in VCF Automation.

Create an Approval Policy

To add a layer of governance to deployment requests before they are run, you can create an approval policy using the Policies API. The policy controls who must agree to a deployment or Day 2 action before a request is provisioned.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Assign an API version variable for the Approvals API.

```
api_version_approval=' '
```

Note: The Approvals APIs and Policies APIs have different API version values. You set the API version value for the Policies APIs when you satisfied the general prerequisites.

- Verify that you know the name of the Cloud Template to which you want to apply the approval policy.

You create an approval policy based on certain deployment criteria, such as deployments created from a specific cloud template. For example, if you specify a cloud template ID, you can create a policy that requires a specified level of approval for deployments created from that cloud template.

The following procedure shows how to use the Approval API to get the cloud template ID and list approval actions for a deployment before creating the approval policy using the Policy API.

1. List the cloud templates.

```

curl -X GET \
  $url/approval/api/policy/data/blueprints?apiVersion=$api_version_approval \
  -H "Authorization: Bearer $access_token" | jq ."

```

2. Examine the response to find the ID of the cloud template for the approval policy.

3. Assign the cloud template variable.

```
cloudtemplateId = "<your_cloud_template_ID>"
```

4. Optional: If you do not know the actions to specify in the policy, list the IDs of deployment actions.

```

curl -X GET \
  $url/approval/api/policy/data/actions?apiVersion=$api_version_approval&search=Deployment \
  -H "Authorization: Bearer $access_token" | jq ."

```

Note the action IDs.

5. Create an approval policy with hard enforcement that is applied to deployments created from the cloud template with `cloudtemplateId`.

- For an approval policy, you specify `"typeId": "com.vmware.policy.approval"`
- `autoApprovalExpiry` specifies the number of days that the approvers have to act before triggering the `autoApprovalDecision`
- `level` specifies the order in which the policy is applied with values 1-99. For example, level 1 approvals are applied first, followed by level 2 approvals and so forth.

```
curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "<your_approval_policy_name>",
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.approval"
    "definition": {
      "level": <policy_level>,
      "approverType": "USER",
      "approvalMode": "ALL_OF",
      "autoApprovalDecision": "APPROVE",
      "approvers": [
        "USER:<approver1_ID>",
        "USER:<approver2_ID>"
      ],
      "autoApprovalExpiry": <number_of_days>,
      "actions": [
        "<actionID_1>",
        "<actionID_2>",
        "<actionID_3>"
      ]
    },
    "criteria": {
      "matchExpression": [
        {
          "key": "blueprintId",
          "operator": "eq",
          "value": "'$cloudtemplateId'"
        }
      ]
    }
  }' | jq "."
```

Create an approval policy

Create a policy named `Sample Approval Policy` to apply to deployments created from a cloud template named `template-1`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_approval=' '
```

List the cloud templates.

```
curl -X GET \
  $url/approval/api/policy/data/blueprints?apiVersion=$api_version_approval \
  -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the cloud template named `template-1`.

```
...
{
  "id": "77265efc-6d06-428e-9fad-3ad8f31441f3",
  "name": "template-1",
  "description": ""
}
...
```

Assign the cloud template ID variable.

```
$ cloudtemplateId = "77265efc-6d06-428e-9fad-3ad8f31441f3"
```

List the deployment actions.

```
curl -X GET \
  $url/approval/api/policy/data/actions?apiVersion=$api_version_approval&search=Deployment \
  -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the IDs of the deployment actions that you want to specify in your approval policy.

```
...
{
  "id": "Deployment.Create",
  "name": "Create",
  "description": "Create a deployment",
  "resourceType": "Deployment"
},
...
{
  "id": "Cloud.Azure.Machine.PowerOn",
  "name": "Power On",
  "description": "Power on a machine",
  "resourceType": "Cloud.Azure.Machine"
},
...
{
  "id": "Cloud.Azure.Machine.PowerOff",
  "name": "Power Off",
  "description": "Power off a machine",
  "resourceType": "Cloud.Azure.Machine"
},
...
```

Use the cloud template ID to create the approval policy of level 2 with hard enforcement named `Sample Approval Policy`. When a deployment is requested, users listed will act as approvers for the actions: provision, power on, and power off an Azure machine. If approvers do not act within three days, then the deployment actions are automatically approved.

```
$ curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
```

```
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
-d '{
  "name": "Sample Approval Policy",
  "enforcementType": "HARD",
  "typeId": "com.vmware.policy.approval"
  "definition": {
    "level": 2,
    "approverType": "USER",
    "approvalMode": "ALL_OF",
    "autoApprovalDecision": "APPROVE",
    "approvers": [
      "USER:mary@mycompany.com",
      "USER:susan@mycompany.com"
    ],
    "autoApprovalExpiry": 3,
    "actions": [
      "Deployment.Create",
      "Cloud.Azure.Machine.PowerOn",
      "Cloud.Azure.Machine.PowerOff"
    ]
  },
  "criteria": {
    "matchExpression": [
      {
        "key": "blueprintId",
        "operator": "eq",
        "value": "'$cloudtemplateId'"
      }
    ]
  }
}' | jq "
```

The response shows the approval policy.

```
{
  "id": "62ad2f02-0b2a-4ed8-a739-a6c40d761e49",
  "name": "Sample Approval Policy",
  "typeId": "com.vmware.policy.approval",
  "enforcementType": "HARD",
  "orgId": "d2994f92-bd52-45b1-9220-686b20944c2c",
  "definition": {
    "level": 2,
    "approverType": "USER",
    "approvalMode": "ALL_OF",
    "autoApprovalDecision": "APPROVE",
    "approvers": [
      "USER:mary@mycompany.com",
      "USER:susan@mycompany.com"
    ],
    "autoApprovalExpiry": 3,
    "actions": [
      "Deployment.Create",
      "Cloud.Azure.Machine.PowerOn",
```

```

        "Cloud.Azure.Machine.PowerOff"
    ]
},
"criteria": {
    "matchExpression": [
        {
            "key": "blueprintId",
            "operator": "eq",
            "value": "77265efc-6d06-428e-9fad-3ad8f31441f3"
        }
    ]
},
"createdAt": " T09:45:38.108885Z",
"createdBy": "admin@mycompany.com",
"lastUpdatedAt": " T09:45:38.108885Z",
"lastUpdatedBy": "admin@mycompany.com"
}

```

How to Create Resource Quota Policies

To limit the resource consumed by each user, project, or organization, you can create a resource quota policy using the Policies API. Applying the policy ensures that users do not consume all available resources.

With the policy defined, any new deployment requests are evaluated against the resource quota. If a resource request exceeds the quota, the deployment request fails with a message that indicates the reason for the failure. The resource quota does not apply to deployment requests that are already in-progress when the policy is defined.

In cases where multiple policies are defined for a resource at the same scope level, the request is evaluated against the quota with the lowest limit value.

For information about policy scope, see [How do I configure policy scope in for VM Apps](#).

The following examples show how to define a resource quota policy with:

- Organization level limits.
- Project level limits.

For both definitions, you use `POST /policy/api/policies` to create the policy with the input parameter `"typeId": "com.vmware.policy.resource.quota"`.

Prerequisites for creating a resource quota policy

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you know the resource quotas that you want to apply.

Resource quota policy with organizational level limits

This example shows how to define a resource quota policy with hard enforcement and limits on resource consumption by the organization and each user within the organization.

```

curl -k -X POST \
  "$url/policy/api/policies?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Resource Quota Policy 1",

```

```

"enforcementType": "HARD",
"typeId": "com.vmware.policy.resource.quota",
"definition": {
  "orgLevel": {
    "limits": {
      "cpu": {
        "value": 50
      },
      "storage": {
        "value": 150,
        "unit": "GB"
      },
      "memory": {
        "value": 150,
        "unit": "GB"
      },
      "instances": {
        "value": 50
      }
    },
    "userLevel": {
      "limits": {
        "cpu": {
          "value": 4
        },
        "storage": {
          "value": 10,
          "unit": "GB"
        },
        "memory": {
          "value": 10,
          "unit": "GB"
        },
        "instances": {
          "value": 2
        }
      }
    }
  }
}
}' | jq "."

```

The response shows the resource quota policy with organization and organization user level limits.

```

{
  "id": "52f67297-3e15-4e0a-9336-35894d2be0bc",
  "name": "Sample Resource Quota Policy 1",
  "typeId": "com.vmware.policy.resource.quota",
  "enforcementType": "HARD",
  "orgId": "74a191fc-27e0-4f09-af0d-6acd04d60832",
  "definition": {
    "orgLevel": {
      "limits": {
        "cpu": {

```



```

        "value": 50
      },
      "storage": {
        "value": 150,
        "unit": "GB"
      },
      "memory": {
        "value": 150,
        "unit": "GB"
      },
      "instances": {
        "value": 50
      }
    },
    "userLevel": {
      "limits": {
        "cpu": {
          "value": 4
        },
        "storage": {
          "value": 10,
          "unit": "GB"
        },
        "memory": {
          "value": 10,
          "unit": "GB"
        },
        "instances": {
          "value": 2
        }
      }
    }
  }
},
"createdAt": " T11:51:47.742311Z",
"createdBy": "admin@mycompany.com",
"lastUpdatedAt": " T11:51:47.742311Z",
"lastUpdatedBy": "admin@mycompany.com"
}

```

Resource quota policy with project level limits

This example shows how to define a resource quota policy with hard enforcement and limits on resource consumption by a project and each project user.

```

curl -k -X POST \
  "$url/policy/api/policies?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Resource Quota Policy 2",
    "enforcementType": "HARD",
    "projectId": "6df55bb1-7444-4c13-9997-4004ba0a321d",

```

```

"scopeCriteria": null,
"typeId": "com.vmware.policy.resource.quota",
"definition": {
  "projectLevel": {
    "limits": {
      "cpu": {
        "value": 50
      },
      "storage": {
        "value": 80,
        "unit": "GB"
      },
      "memory": {
        "value": 80,
        "unit": "GB"
      },
      "instances": {
        "value": 40
      }
    },
    "userLevel": {
      "limits": {
        "cpu": {
          "value": 5
        },
        "storage": {
          "value": 8,
          "unit": "GB"
        },
        "memory": {
          "value": 8,
          "unit": "GB"
        },
        "instances": {
          "value": 4
        }
      }
    }
  }
}
}' | jq "."

```

The response shows the resource quota policy with project and project user level limits.

```

{
  "id": "5809ec88-16fb-4553-98c3-5b588ebff322",
  "name": "Sample Resource Quota Policy 2",
  "typeId": "com.vmware.policy.resource.quota",
  "enforcementType": "HARD",
  "orgId": "74a191fc-27e0-4f09-af0d-6acd04d60832",
  "projectId": "6df55bb1-7444-4c13-9997-4004ba0a321d",
  "definition": {
    "projectLevel": {
      "limits": {

```

```

        "cpu": {
            "value": 50
        },
        "storage": {
            "value": 80,
            "unit": "GB"
        },
        "memory": {
            "value": 80,
            "unit": "GB"
        },
        "instances": {
            "value": 40
        }
    },
    "userLevel": {
        "limits": {
            "cpu": {
                "value": 5
            },
            "storage": {
                "value": 8,
                "unit": "GB"
            },
            "memory": {
                "value": 8,
                "unit": "GB"
            },
            "instances": {
                "value": 4
            }
        }
    }
}

}

},
"createdAt": " T12:09:34.160569Z",
"createdBy": "admin@mycompany.com",
"lastUpdatedAt": " T12:09:34.160569Z",
"lastUpdatedBy": "admin@mycompany.com"
}

```

Create a Deployment Limit Policy

To limit resource consumption when users deploy cloud templates in VCF Automation and request catalog items in VCF Automation, you can create a limit policy using the Policies API. The policy applies limits to all deployments in an organization by default.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See [Prerequisites for API Use Case Examples](#).

- Assign an API version variable for the Deployment Limit API.

```
api_version_deploymentlimit=' '
```

Note: The Deployment Limit APIs and Policies APIs have different API version values. You set the API version value for the Policies APIs when you satisfied the general prerequisites. The Deployment Limit APIs are grouped with the Approvals services. See <https://techdocs.broadcom.com/us/en/vmware-cis/aria/aria-automation/8-16/what-is-the-api-and-how-do-i-use-it.html>.

- Verify you know the resource criteria that you want to use to restrict the policy.

You create a deployment limit policy based on certain resource criteria, such as account names, account types, cloud templates, cloud zones, flavors, and many more. For example, if you specify a cloud template ID as the resource criteria, you can restrict the policy so that it only applies limits to deployments created from a specific cloud template.

Note: If approval policy or resource quota policy definitions affect deployments within the policy scope, deployment limits are enforced before the other policy types.

The following procedure shows how to use the Deployment limit API to get the cloud template ID before creating the deployment limit policy using the Policy API.

- List the cloud templates.

```
curl -X GET \
  $url/deploymentlimit/api/policy/data/blueprints?apiVersion=$api_version_deploymentlimit \
  -H "Authorization: Bearer $access_token" | jq "."
```

- Examine the response to find the ID of the cloud template used to create the deployments where you want to limit resource usage.

- Assign the cloud template variable.

```
cloudtemplateId = "<your_cloud_template_ID>"
```

- Create a deployment limit policy with hard enforcement that is applied to deployments created from the cloud template with `cloudtemplateId`. For the deployment limit policy, you specify `"typeId": "com.vmware.policy.deployment.limit"`.

```
curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "<your_limit_policy_name>",
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.deployment.limit"
    "definition": {
      "deploymentLimits": {
        "cpu": {
          "value": 6
        },
        "memory": {
          "unit": "GB",
          "value": 5
        },
        "instances": {
          "value": 3
        },
        "storage": {
          "unit": "GB",
          "value": "20"
        }
      }
    }
  }'
```

```

    }
  },
  "deploymentResourceLimits": {
    "resources": [
      {
        "name": "vSphere-Machine-Limits",
        "limits": {
          "cpu": {
            "value": 2
          },
          "memory": {
            "unit": "GB",
            "value": 2
          },
          "storage": {
            "unit": "GB",
            "value": "20"
          }
        }
      },
      {
        "matchExpression": [
          {
            "key": "type",
            "value": "Cloud.vSphere.Machine",
            "operator": "eq"
          }
        ]
      }
    ]
  },
  "criteria": {
    "matchExpression": [
      {
        "key": "blueprintId",
        "operator": "eq",
        "value": "'$cloudtemplateId'"
      }
    ]
  }
}
| jq "."

```

Create a deployment limit policy

Create a deployment limit policy named `Sample Limit Policy` that is applied to limit resource usage in deployments created from a cloud template named `template2`.

Assign variables.

```

$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_deploymentlimit='2020-11-01'
$ orgId='394a4ccb-22c6-4ef0-8c75-8b77efbfb51'

```

List the cloud templates.

```
$ curl -X GET \
  $url/deploymentlimit/api/policy/data/blueprints?apiVersion=$api_version_deploymentlimit \
  -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the cloud template named `template2`.

```
...
{
  "id": "3d3c714f-0aeb-423d-a494-97e85e4a8566",
  "name": "template2",
  "description": "Cloud template for example deployment"
},
...
```

Assign the cloud template ID variable.

```
$ cloudtemplateId = "3d3c714f-0aeb-423d-a494-97e85e4a8566"
```

Use the cloud template ID to create the deployment limit policy with hard enforcement named `Sample Limit Policy`. The value for the type ID is fixed as `com.vmware.policy.deployment.limit`.

```
$ curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Limit Policy",
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.deployment.limit"
    "definition": {
      "deploymentLimits": {
        "cpu": {
          "value": 6
        },
        "memory": {
          "unit": "GB",
          "value": 5
        },
        "instances": {
          "value": 3
        },
        "storage": {
          "unit": "GB",
          "value": "20"
        },
      },
    },
    "deploymentResourceLimits": {
      "resources": [
        {
          "name": "vSphere-Machine-Limits",
          "limits": {
            "cpu": {
              "value": 2
            },

```

```

        "memory": {
          "unit": "GB",
          "value": 2
        },
        "storage": {
          "unit": "GB",
          "value": "20"
        }
      },
      "criteria": {
        "matchExpression": [
          {
            "key": "type",
            "value": "Cloud.vSphere.Machine",
            "operator": "eq"
          }
        ]
      }
    }
  ],
  "criteria": {
    "matchExpression": [
      {
        "key": "blueprintId",
        "operator": "eq",
        "value": "'$cloudtemplateId'"
      }
    ]
  }
}
| jq "."

```

The response shows the deployment limit policy.

```

{
  "id": "62ad2f02-0b2a-4ed8-a739-a6c40d761e49",
  "name": "Sample Limit Policy",
  "typeId": "com.vmware.policy.deployment.limit",
  "enforcementType": "HARD",
  "orgId": "d2994f92-bd52-45b1-9220-686b20944c2c",
  "definition": {
    "deploymentLimits": {
      "cpu": {
        "value": 6
      },
      "memory": {
        "unit": "GB",
        "value": 5
      },
      "instances": {
        "value": 3
      }
    }
  }
}

```

```

    "storage": {
      "unit": "GB",
      "value": 20
    }
  },
  "deploymentResourceLimits": {
    "resources": [
      {
        "name": "vSphere-Machine-Limits",
        "limits": {
          "cpu": {
            "value": 2
          },
          "memory": {
            "unit": "GB",
            "value": 2
          },
          "storage": {
            "unit": "GB",
            "value": "20"
          }
        },
        "criteria": {
          "matchExpression": [
            {
              "key": "type",
              "value": "Cloud.vSphere.Machine",
              "operator": "eq"
            }
          ]
        }
      }
    ]
  },
  "criteria": {
    "matchExpression": [
      {
        "key": "blueprintId",
        "operator": "eq",
        "value": "7950795a-4f66-451c-a79f-be9ef6bd723c"
      }
    ]
  },
  "createdAt": " 8T09:45:38.108885Z",
  "createdBy": "user@mycompany.com",
  "lastUpdatedAt": " T09:45:38.108885Z",
  "lastUpdatedBy": "user@mycompany.com"
}

```

Create a Content Sharing Policy

As a VCF Automation administrator, you can create a content sharing policy that entitles all VCF Automation users in a project to shared content defined in the policy.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See [Prerequisites for API Use Case Examples](#).

Note: The Policies API and the Catalog API have the same API version. You set the API version value for both services when you satisfy the prerequisites for the Policies service.

- Assign an API version variable for the Projects API.

```
api_version_projects=' '
```

Note: The Projects APIs and Policies APIs have different API version values. You set the API version value for the Policies APIs when you satisfied the prerequisites for the Policies service.

The following procedure shows how to use the Policies API to create a content sharing policy that defines shared content for users or groups that are part of the same project. To get the input required for the project, project users, and project groups, you use the Projects API. To get the catalog sources or catalog items to share in the policy, you use the Catalog API.

The steps to get catalog sources or items to share are optional, because a content sharing policy may not share both sources and items. However, the policy must share at least one catalog source or item.

1. Get a list of all projects in your organization.

```
curl -X GET "$url/project-service/api/projects?apiVersion=$api_version_projects" -H "Authorization: Bearer $access_token" | jq ""
```

Examine the response to get the ID of the project for the shared policy. The project must include the users or groups for which you want to entitle content. See [Create a Project with the Project Service API](#).

2. Assign the project ID variable.

```
project_id='<your_project_id>'
```

3. Get a list of users in the project.

```
curl -X GET \
$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version_projects&expandGroups=true& \
'$filter="(substringof('{}',tolower(acct)))" \
-H "Authorization: Bearer $access_token" | jq ""
```

Examine the response to get the user email addresses for the project users. Users selected in the policy are entitled to items shared with the project.

4. Get a list of groups in the project.

```
curl -X GET \
$url/project-service/api/projects/$project_id/groups?apiVersion=$api_version_projects&page=0 \
'$filter="(substringof('{}',tolower(displayName)))" \
-H "Authorization: Bearer $access_token" | jq ""
```

Examine the response to get the group display name for the project users that are part of the group. Groups selected in the policy are entitled to items shared with the project.

5. Optional: Get a list of catalog sources for your project.

```
curl -X GET $url/catalog/api/admin/sources?projectId=$project_id&apiVersion=$api_version -H "Authorization: Bearer $access_token" | jq ""
```

Examine the response to find catalog sources that you want to share.

6. Optional: If you are sharing a catalog source, assign the catalog source ID variable.

```
catalog_source_id='<shared_catalog_source_id>'
```

7. Optional: Get a list of catalog items for your project.

```
curl -X GET $url/catalog/api/items?projectId=$project_id&apiVersion=$api_version -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find catalog items that you want to share.

8. Optional: If you are sharing a catalog item, the catalog item ID variable.

```
catalog_item_id='<shared_catalog_item_id>'
```

9. Create a content sharing policy with hard enforcement.

- The value for user type is always `USER`.
- The group's display name is always in the format `groupDisplayName@groupDomain`.
- The `typeId` for the content sharing policy is always `com.vmware.policy.catalog.entitlement`.

```
curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "<your_content_sharing_policy_name>",
    "projectId": "'$project_id'",
    "definition": {
      "entitledUsers": [
        {
          "userType": "USER",
          "principals": [
            {
              "type": "USER",
              "referenceId": "<user1_email_address>"
            },
            {
              "type": "USER",
              "referenceId": "<user2_email_address>"
            }
          ],
          {
            "type": "PROJECT",
            "referenceId": "$project_id"
          },
          {
            "type": "GROUP",
            "referenceId": "<groupDisplayName@groupDomain>"
          }
        ]
      },
      "items": [
        {
          "id": "'$catalog_source_id'",
          "type": "CATALOG_SOURCE_IDENTIFIER"
        },
        {
          "id": "'$catalog_item_id'",
          "type": "CATALOG_ITEM_IDENTIFIER"
        }
      ]
    }
  }
```

```

    }
  ]
},
"enforcementType": "HARD",
"typeId": "com.vmware.policy.catalog.entitlement"
}
}' | jq "."

```

Create a content sharing policy

Create a policy named Sample Sharing Policy.

Assign variables.

```

$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_projects=' '

```

List the projects in your organization.

```

$ curl -X GET "$url/project-service/api/projects?apiVersion=$api_version_projects" -H "Authorization: Bearer $access_token" | jq "."

```

Examine the response to find the project that you want to use for the shared content policy.

```

{
  "id": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e",
  "name": "Example-project",
  "description": "This is an example project",
  "orgId": "f670fdcf-66d6-4689-9793-d524e7066d1e",
  ...
}

```

Assign the project ID.

```

$ project_id='1d0bcd42-4d8f-4a8f-8b31-f34a4707533e'

```

List the users in the project.

```

$ curl -X GET \
  $url/project-service/api/projects/$project_id/principals?apiVersion=$api_version_projects&expand-Groups=true& \
  '$filter="(substringof('{}',tolower(acct)))" \
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows the email addresses for the users in the project.

```

...
  "content": [
    {
      "id": "ab373898-d29b-4e3b-8703-58023cadd140",
      "acct": "user1@mycompany.com",
      "domain": "mycompany.com"
    }
  ],
  ...
}

```

Get a list of groups in the project.

```

$ curl -X GET \
  $url/project-service/api/projects/$project_id/groups?apiVersion=$api_version_projects&page=0 \

```

```
'$filter='"(substringof('{',tolower(displayName)))" \
-H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the group display name for the users that are part of the group.

```
...
"content": [
  {
    "id": "a9da96e7-ba19-47d4-9f38-dd1983e29424",
    "displayName": "test@mycompany.com",
    "groupType": "USER_GROUP",
    "usersCount": 2
  },
  ...
]
```

Get a list of catalog sources for your project.

```
$ curl -X GET $url/catalog/api/admin/sources?projectId=$project_id&apiVersion=$api_version -H "Au-
thorization: Bearer $access_token" | jq "."
```

Examine the response to find the catalog sources that you want to share.

```
...
"content": [
  {
    "id": "600026c6-3155-4395-a990-580ff1159e82",
    "name": "BpContent-Quality Engineering",
    "description": "For Project-Quality Engineering",
    "typeId": "com.vmw.blueprint",
    "createdAt": "2022-10-12T10:37:01.751799Z",
    "createdBy": "admin@mycompany.com",
    "lastUpdatedAt": "2022-10-17T05:06:33.976796Z",
    "lastUpdatedBy": "system-user",
    "config": {
      "sourceProjectId": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e"
    }
  },
  ...
]
```

Assign the ID of the catalog source to share.

```
$ catalog_source_id='600026c6-3155-4395-a990-580ff1159e82'
```

Get a list of catalog items for your project.

```
$ curl -X GET $url/catalog/api/admin/items?projectId=$project_id&apiVersion=$api_version -H "Autho-
rization: Bearer $access_token" | jq "."
```

Examine the response to find the catalog item.

```
...
"content": [
  {
    "id": "b2d0fba7-5f62-3c79-b1b8-a2aa7d38063b",
    "name": "CF-SQAVC67-Centos-MultiMachine",
    "description": "CF-SQAVC67-Centos-MultiMachine vsphere components, disk, networks",
    "sourceId": "600026c6-3155-4395-a990-580ff1159e82",
    "sourceName": "BpContent-Quality Engineering",
    ...
  }
]
```

Assign the ID of the catalog item to share.

```
$ catalog_item_id='b2d0fba7-5f62-3c79-b1b8-a2aa7d38063b'
```

Create a content sharing policy with hard enforcement.

The following example shows the group's display name which is of the format groupDisplayName@groupDomain where:

- groupDisplayName is test@mycompany.com
- groupDomain is mycompany.com

```
$ curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Sharing Policy",
    "projectId": "'$project_id'",
    "definition": {
      "entitledUsers": [
        {
          "userType": "USER",
          "principals": [
            {
              "type": "USER",
              "referenceId": "user1@mycompany.com"
            },
            {
              "type": "PROJECT",
              "referenceId": "'$project_id'"
            },
            {
              "type": "GROUP",
              "referenceId": "test@mycompany.com@mycompany.com"
            }
          ]
        },
        {
          "id": "'$catalog_source_id'",
          "type": "CATALOG_SOURCE_IDENTIFIER"
        },
        {
          "id": "'$catalog_item_id'",
          "type": "CATALOG_ITEM_IDENTIFIER"
        }
      ]
    },
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.catalog.entitlement"
  }'
```

The response shows the content sharing policy.

```

{
  "id": "0cecca9a-d778-47b5-acdf-c08248406052",
  "name": "Sample Sharing Policy",
  "projectId": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e",
  "definition": {
    "entitledUsers": [
      {
        "userType": "USER",
        "principals": [
          {
            "type": "USER",
            "referenceId": "user1@mycompany.com"
          },
          {
            "type": "PROJECT",
            "referenceId": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e"
          },
          {
            "type": "GROUP",
            "referenceId": "test@mycompany.com@mycompany.com"
          }
        ],
        "items": [
          {
            "id": "600026c6-3155-4395-a990-580ff1159e82",
            "type": "CATALOG_SOURCE_IDENTIFIER"
          },
          {
            "id": "b2d0fba7-5f62-3c79-b1b8-a2aa7d38063b",
            "type": "CATALOG_ITEM_IDENTIFIER"
          }
        ]
      }
    ]
  },
  "enforcementType": "HARD",
  "typeId": "com.vmware.policy.catalog.entitlement",
  "orgId": "10ea6be1-7723-4bf0-a221-8b4f3c7a26f7",
  "createdAt": "2022-10-24T07:52:22.731448Z",
  "createdBy": "admin@mycompany.com",
  "lastUpdatedAt": "2022-10-24T07:52:22.731448Z",
  "lastUpdatedBy": "admin@mycompany.com"
}

```

Version and Release a Cloud Template to a VCF Automation Catalog

After creating a cloud template, you version and release your cloud template using a POST request. The request body includes the ID of an existing cloud template and the number of the version to release.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See [Prerequisites for API Use Case Examples](#).

- Verify that you have the cloud template ID for the cloud template you want to version and release. See [Create and Update a Cloud Template](#).
- Verify that you know the version of the cloud template that you want to create and release to the catalog.

By versioning and releasing the cloud template, you mark a cloud template version as ready to be consumed by VCF Automation. To show the released cloud template in VCF Automation, you must have a catalog source.

1. Assign the cloud template ID variable.

```
cloud_template_id='<your_cloud_template_id>'
```

2. Assign a cloud template version variable.

```
cloud_template_version='<your_cloud_template_version>'
```

your_cloud_template_version is the version that you want to create.

3. To version the cloud template without releasing it, submit the request with "release": false .

```
curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "changeLog": "Creating a version '$cloud_template_version'",
    "description": "Creating a version from the current draft",
    "release": false,
    "version": "'$cloud_template_version'"
  }' | jq "
```

4. Release the cloud template.

```
curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/actions/release?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' | jq "
```

Version and Release a Cloud Template

Release version 5 of your cloud template with ID 1f170637-81a3-4257-b1cd-b2219ee8034c.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
$ cloud_template_version='v5'
```

Version the cloud template without releasing it.

```
$ curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "changeLog": "Creating a version '$cloud_template_version'",
    "description": "Creating a version from the current draft",
    "release": false,
    "version": "'$cloud_template_version'"
  }' | jq "
```

Release the cloud template.

```
$ curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/actions/release?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' | jq "."
```

A snippet of the response shows the new cloud template version with a **RELEASED** status.

```
...
"blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
"name": "MyExampleCloudTemplate",
"description": "Basic Cloud Machine cloud template",
"version": "v5",
"tags": [],
"content": "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n    description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    properties:\n      name: BasicCloudMachine\n      flavor: '${input.flavor}'\n      image: '${input.image}'\n      count: '${input.count}'\n      tags: [\n        {\n          \"key\": \"env\", \n          \"value\": \"prod\" \n        } \n      ] \n",
"status": "RELEASED",
"versionDescription": "Creating a version from the current draft",
"versionChangeLog": "Creating a version v5",
"valid": true
}
```

Edit and Version a Custom Form in Your Cloud Template

After creating a cloud template in VCF Automation, versioning it, and importing it into your VCF Automation content, you can add a custom request form. Then you can use the form designer to edit and version the form, so that you can choose the version and use it later.

- Verify that you have satisfied all general prerequisites and prerequisites for the VCF Automation Catalog service. See [Prerequisites for API Use Case Examples](#).
- Verify that you have versioned and released your cloud template in VCF Automation. See [Version and Release a Cloud Template to a VCF Automation Catalog](#).
- Verify that you have imported your cloud template into VCF Automation. See [Add VCF Automation templates to the VCF Automation catalog](#).

The following procedure assumes that your VCF Automation content includes a versioned VCF Automation cloud template. Using the Catalog service API, you find the ID of the cloud template and get the cloud template schema for one of the template versions. To create a custom form from the schema, you use the Form Service API. Additional steps show how to customize and version the custom form, and how to list and restore a versioned form.

For more information about Custom Forms, see [Learn more about custom forms in VCF Automation for VM Apps](#).

1. List the items in your catalog.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/catalog/api/admin/items?apiVersion=$api_version" | jq "."
```

Examine the response to find the catalog ID of the cloud template that you want to use.

2. Assign a variable for the cloud template catalog ID.

```
item_id='<cloud_template_catalog_ID>'
```

3. Get information about the catalog item.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/catalog/api/admin/items/$item_id?apiVersion=$api_version" | jq "."
```

Examine the result to choose the version of the template that you want to use for the custom form.

4. Assign a variable for the cloud template version ID.

The form that you create and customize a form will be associated with this cloud template version ID.

```
version_id='<your_cloud_template_version_ID>'
```

5. Using the catalog ID and version ID, get the schema for the cloud template.

```
$ schema='curl -k -X GET \
"$url/catalog/api/admin/items/$item_id/versions/$version_id \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' | jq -r .schema'
```

6. Use the schema to create a custom form.

```
curl -k -X POST \
"$url/form-service/api/forms/designer/request?sourceType=com.vmw.blueprint.version&sourceId=$item_id/$version_id&formType=requestForm" \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
-d '$schema' | jq "."
```

You have a current version of your custom form.

7. Customize the form.

- *your_modified_form_json_as_string* Used to update the form field properties. For information about form field properties, see [Custom form designer field properties in VCF Automation for VM Apps](#).
- *your_modified_styles_string* Used to update a cascading style sheet written in CSS Syntax Module Level 3. For information about the CSS file format, see [Learn more about VCF Automation custom forms](#).

```
curl -k -X POST \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
"$url/form-service/api/forms" \
-d '{
  "name": "Demo Blueprint / 2.0.0",
  "form": "<your_modified_form_json_as_string>",
  "styles": "<your_modified_styles_string>",
  "status": "ON", // set ON to enable the custom form, OFF to disable
  "type": "requestForm",
  "sourceId": "$item_id/$version_id",
  "sourceType": "com.vmw.blueprint.version"
}' | jq "."
```

A response with the status 201 'Created' indicates a successful form update.

8. To save the current form with a new version name, you version the form.

```
curl -k -X POST \
"$url/form-service/api/forms/versions" \
-H "Authorization: Bearer $token" \
```

```
-H 'Content-Type: application/json' \
-d '{
  "name": "<your_form_version_name>",
  "sourceId": "$item_id/$version_id",
  "sourceType": "com.vmw.blueprint.version",
  "formType": "requestForm"
}' | jq "
```

The form is saved with version name.

To create more versions, you repeat the steps to customize and version the form. If you customize without versioning, the changes you make are only applied to the current form.

9. Optional: If you have at least one form version in addition to the current form, you can list and restore the version that you want to use.

- To list form versions:

```
curl -k -X GET \
"$url/form-service/api/forms/versions?sourceType=com.vmw.blueprint.version&sourceId=$item_id/$version_id&formType=requestForm" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json' | jq "
```

The response lists the names of stored form versions and their IDs or `formVersionId`.

- To restore a previously versioned form, use the `formVersionId`:

```
curl -k -X PATCH \
"$url/form-service/api/forms/versions/$formVersionId/restore" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json' | jq "
```

Edit and Version a Custom Form

Edit the custom form for a cloud template with the name `Demo Blueprint`. Then version the custom form it so that you can save it to use later.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2020-08-25'
```

List the items in your catalog.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/catalog/api/admin/items?apiVersion=$api_version" | jq "
```

Examine the response to find the cloud template named `Demo Blueprint` and get the catalog ID.

```
[
{
  "id": "890275e3-cbc6-3778-af0d-d9eddf4664a0",
  "name": "Demo Blueprint",
  "description": "",
  "sourceId": "d4c24bba-68cc-4923-9155-338c4bf3e663",
  "sourceName": "template-content-source",
  "type": {
    "id": "com.vmw.blueprint",
    "link": "/catalog/api/types/com.vmw.blueprint",
    "name": "VMware Aria Automation Templates"
  },
}
```

```

"createdAt": "2023-05-03T15:15:02.805617Z",
"createdBy": "user@mycompany.com",
"lastUpdatedAt": "2023-05-03T15:15:02.805617Z",
"iconId": "1495b8d9-9428-30d6-9626-10ff9281645e",
"bulkRequestLimit": 1
}
]

```

Assign the item ID variable with the catalog ID of the cloud template.

```
$ item_id='890275e3-cbc6-3778-af0d-d9eddf4664a0'
```

Get catalog information about the cloud template.

```

$ curl -X GET \
"$url/catalog/api/admin/items/$item_id?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq .content

```

A snippet of the response lists the imported versions.

```

[
{
  "id": "2.0.0",
  "description": "Added numberInput field",
  "createdAt": "2023-03-29T06:19:48.748349Z",
  "externalId": "/blueprint/api/blueprints/bbbb30b8-a73a-44bb-92db-27a504e0ec85"
},
{
  "id": "1.0.0",
  "description": "Added stringInput field",
  "createdAt": "2023-03-29T06:19:30.845516Z",
  "externalId": "/blueprint/api/blueprints/bbbb30b8-a73a-44bb-92db-27a504e0ec85"
}
]

```

The cloud template in this example has two versions in the catalog. To get the schema and create a custom form for version 2.0.0, assign a variable.

```
$ version_id='2.0.0'
```

Use the catalog ID and the version ID to get the schema for the cloud template.

```

$ schema='curl -k -X GET \
"$url/catalog/api/admin/items/$item_id/versions/$version_id" \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' | jq -r .schema'

```

Use the schema to create a custom form.

```

$ curl -k -X POST \
"$url/form-service/api/forms/designer/request?sourceType=com.vmw.blueprint.ver-
sion&sourceId=$item_id/$version_id&formType=requestForm" \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
-d '$schema' | jq ."

```

The response shows the schema for Demo Blueprint version 2.0.0. It also shows the form ID or 4e383140-51a4-44a7-afcd-26296079596d.

```
{
  "tenant": "733178c1-0620-478c-96b4-2b04ece1478d",
  "id": "4e383140-51a4-44a7-afcd-26296079596d",
  "name": "Demo Blueprint / 2.0.0",
  "form": "{ \"layout\": { \"pages\": [ { \"id\": \"page_general\", \"title\": \"General\", \"sections\": [ { \"id\": \"section_project\", \"fields\": [ { \"id\": \"project\", \"display\": \"dropDown\", \"signpostPosition\": \"right-middle\" } ] }, { \"id\": \"section_deploymentName\", \"fields\": [ { \"id\": \"deploymentName\", \"display\": \"textField\", \"signpostPosition\": \"right-middle\" } ] }, { \"id\": \"section_stringInput\", \"fields\": [ { \"id\": \"stringInput\", \"display\": \"textField\", \"state\": { \"visible\": true, \"read-only\": false }, \"signpostPosition\": \"right-middle\" } ] }, { \"id\": \"section_e0bcef-b5\", \"fields\": [ { \"id\": \"numberInput\", \"display\": \"dropDown\", \"state\": { \"visible\": true, \"read-only\": false }, \"signpostPosition\": \"right-middle\" } ] } ] }, \"schema\": { \"project\": { \"label\": \"Project\", \"type\": { \"dataType\": \"string\", \"isMultiple\": false }, \"valueList\": { \"id\": \"projects\", \"type\": \"scriptAction\" }, \"constraints\": { \"required\": true }, \"numberInput\": { \"label\": \"numberInput\", \"type\": { \"dataType\": \"integer\", \"isMultiple\": false }, \"valueList\": { \"id\": \"bdimov/actionThatReturnsLong\", \"type\": \"scriptAction\", \"parameters\": [ ] }, \"constraints\": { \"required\": true }, \"stringInput\": { \"label\": \"stringInput\", \"type\": { \"dataType\": \"string\", \"isMultiple\": false }, \"constraints\": { \"required\": true }, \"deploymentName\": { \"label\": \"Deployment Name\", \"type\": { \"dataType\": \"string\", \"isMultiple\": false }, \"constraints\": { \"required\": true, \"max-value\": 900 } } }, \"options\": { \"externalValidations\": [ ] } }, \"sourceType\": \"com.vmw.blueprint.version\", \"sourceId\": \"890275e3-cbc6-3778-af0d-d9eddf4664a0/2.0.0\", \"type\": \"requestForm\", \"status\": \"ON\", \"formFormat\": \"JSON\", \"styles\": \"\" } }
```

Update the form schema to change the visibility of the Deployment Name from true to false.

```
$ curl -k -X POST \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
"$url/form-service/api/forms" \
-d '{
  "name": "Demo Blueprint / 2.0.0",
  "form": '{ \"layout\": { \"pages\": [ { \"id\": \"page_general\", \"sections\": [ { \"id\": \"section_project\", \"fields\": [ { \"id\": \"project\", \"display\": \"dropDown\", \"signpostPosition\": \"right-middle\", \"state\": { \"visible\": true, \"read-only\": false } } ] }, { \"id\": \"section_deploymentName\", \"fields\": [ { \"id\": \"deploymentName\", \"display\": \"textField\", \"state\": { \"visible\": \"false\", \"read-only\": false }, \"signpostPosition\": \"right-middle\" } ] } ] }, \"schema\": { \"project\": { \"label\": \"Project\", \"type\": { \"dataType\": \"string\", \"isMultiple\": false }, \"valueList\": { \"id\": \"projects\", \"type\": \"scriptAction\" }, \"constraints\": { \"required\": true }, \"deploymentName\": { \"label\": \"Deployment Name\", \"type\": { \"dataType\": \"string\", \"isMultiple\": false }, \"constraints\": { \"required\": true, \"max-value\": 900 } }, \"options\": { \"externalValidations\": [ ] } } }, \"status\": \"ON\", // set ON to enable the custom form, OFF to disable \"type\": \"requestForm\", \"sourceId\": \"$item_id/$version_id\", \"sourceType\": \"com.vmw.blueprint.version\" } ' }
```

If you want to see the updated form, use the form ID from the schema.

```
curl -k -X GET "$url/form-service/api/forms/4e383140-51a4-44a7-afcd-26296079596d" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json' | jq "."
```

Version the form and give it a name that will help you to identify it in a list later.

```
$ curl -k -X POST \
"$url/form-service/api/forms/versions" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json' \
-d '{
  "name": "Version1_Deployment_Visibility_FALSE",
  "sourceId": "$item_id/$version_id",
  "sourceType": "com.vmw.blueprint.version",
  "formType": "requestForm"
}' | jq "."
```

You can repeatedly update and version the form.

- Each time you update the form, you are updating the current version.
- Each time you version the form, you save the updated form with a new name.

If you want to restore a version of the form, list your form versions to find the version of the form that you want.

```
$ curl -k -X GET \
"$url/form-service/api/forms/versions?sourceType=com.vmw.blueprint.version&sourceId=$item_id/$version_id&formType=requestForm" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json'
```

The response includes the IDs and names of the custom forms.

```
[
{
  "id": "4bfdd822-8975-4883-a8b2-22396ddf9395",
  "createdDate": "2023-05-17T14:54:07.187+0000",
  "createdBy": "user@mycompany.com",
  "name": "Version1_Deployment_Visibility_FALSE"
},
{
  "id": "bed58877-e408-41ce-ad20-3bc48c569d84",
  "createdDate": "2023-05-17T14:39:16.267+0000",
  "createdBy": "user@mycompany.com",
  "name": "Version2_TextField_Added"
}
]
```

Assign the ID of the form you want to restore to the form version ID variable.

```
$ formVersionID='4bfdd822-8975-4883-a8b2-22396ddf9395'
```

Restore the form.

```
$ curl -k -X PATCH \
"$url/form-service/api/forms/versions/$formVersionId/restore" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json'
```

You have learned how to create, update, version, and restore a custom form.

Remove a Cloud Template Version from a VCF Automation Catalog

If you want to remove a cloud template version, you make a POST request. The body of the input indicates the version to remove.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the cloud template ID for the cloud template you want to remove. See [Create and Update a Cloud Template](#).
- Verify that you have the cloud template version that you want to remove from the catalog. See [Version and Release a Cloud Template to a VCF Automation Catalog](#).

If you have a versioned and released cloud template in VCF Automation, it appears in the catalog source. If you do not want that cloud template version to be available for deployment, you must remove it from the catalog.

1. Assign the cloud template ID variable.

```
cloud_template_id='<your_cloud_template_id>'
```

2. Assign the cloud template version variable.

```
cloud_template_version='<your_cloud_template_version>'
```

your_cloud_template_version is the version that you want to remove.

3. Remove a version of your cloud template from the catalog.

```
curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/action/unre-
  lease?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' | jq "."
```

4. To see the change in the VCF Automation UI, select the **Content & Policies** tab.

- a) Select **Content Sources**.
- b) Select the name of the content source with your cloud template.
- c) On the Content Source Details screen that appears, click **Save and Import**.

The cloud template version you specified is removed from the VCF Automation catalog. Any other released cloud template versions remain listed.

Remove a Cloud Template Version

Remove version 5 of your cloud template with ID `fa6b42d5-ac46-451d-8917-b2f7e527b785`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='fa6b42d5-ac46-451d-8917-b2f7e527b785'
$ cloud_template_version='v5'
```

Remove the cloud template version.

```
$ curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/action/unre-
  lease?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
```

```
-H 'Content-Type: application/json' | jq "."
```

A snippet of the response shows the cloud template version with a **VERSIONED** status.

```
...
"blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
"name": "MyExampleCloudTemplate",
"description": "Basic Cloud Machine cloud template",
"version": "v5",
"tags": [],
"content": "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n    de-
scription: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n
  title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    prop-
erties:\n      name: BasicCloudMachine\n      flavor: '${input.flavor}'\n      image: '${in-
put.image}'\n      count: '${input.count}'\n      tags: [\n        {\n          \"key\": \"env\",
          \"value\": \"prod\"\n        }\n      ]\n",
"status": "VERSIONED",
"versionDescription": "Creating a version from the current draft",
"versionChangeLog": "Creating a version v5",
"valid": true
}
```

Test Your Cloud Template Deployment

To test the deployment of a cloud template, you use the Blueprint APIs to make a POST request with the cloud template ID as input.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that the flavor mapping and image mapping specified in the released VCF Automation cloud template version exist in your cloud account. See [Create Flavor Mappings](#) and [Create Image Mappings](#).
- Verify that you have the ID for the cloud template you want to deploy. See [Create and Update a Cloud Template](#).

Before deploying a cloud template, you can test the syntax and placement of your cloud template to ensure deployment viability. If errors are reported in the test, you must fix the errors and test again before deploying the cloud template.

1. Assign the cloud template ID variable.

```
cloud_template_id='<your_cloud_template_id>'
```

2. Assign image mapping and flavor mapping variables for the cloud template you intend to deploy.

```
image_mapping='<your_image_mapping_name>'
flavor_mapping='<your_flavor_mapping_name>'
```

The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

3. Test the cloud template deployment.

```
curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "simulate":true,
    "blueprintId": "'$cloud_template_id'",
    "inputs": {
```

```

        "count": 2,
        "image": ""$image_mapping"",
        "flavor": ""$flavor_mapping""
    }
} | jq ". "

```

4. Examine the response and assign the cloud template request ID.

```
cloud_template_request_id='<your_cloud_template_request_id>'
```

5. Get the status of the test request.

```

curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq ". "

```

Test a Deployment

For a cloud template with ID 1f170637-81a3-4257-b1cd-b2219ee8034c, test the deployment with image mapping set to ubuntu and flavor mapping set to small.

Assign variables.

```

$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
$ image_mapping='ubuntu'
$ flavor_mapping='small'

```

Test the cloud template deployment.

```

$ curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "simulate":true,
    "blueprintId": ""$cloud_template_id"",
    "inputs": {
      "count": 2,
      "image": ""$image_mapping"",
      "flavor": ""$flavor_mapping""
    }
  }' | jq ". "

```

A snippet of the response shows the cloud template request ID.

```

{
  "id": "5c33355e-fc52-4a30-97c3-3752cf9b644e",
  "createdAt": "2019-10-11T00:11:55.544Z",
  ...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  ...
}

```

Assign the cloud template request ID variable.

```
$ cloud_template_request_id='5c33355e-fc52-4a30-97c3-3752cf9b644e'
```

Request the status of the deployment.


```
$ curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the status of the deployment test request.

```
...
"blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
"inputs": {
  "count": 2,
  "image": "ubuntu",
  "flavor": "small"
},
"status": "FINISHED",
...
```

If your test deployment is successful, you are ready to deploy your cloud template.

Deploy Your Cloud Template

To request the deployment of a cloud template, you use the Blueprint APIs to make a POST request with the cloud template ID as input.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have a project ID for the project that includes cloud zones configured to support the resource requirements of your cloud template. See [Create a Project to use in VCF Automation](#).
- Verify that the flavor mapping and image mapping specified in the released VCF Automation cloud template version exist in your cloud account. See [Create Flavor Mappings](#) and [Create Image Mappings](#).
- Verify that you have the ID for the cloud template you want to deploy. See [Create and Update a Cloud Template](#).
- Verify that you have tested your cloud template deployment to ensure deployment viability. See [Test Your Cloud Template Deployment](#).

The prerequisites of this task call for verifying that you have the cloud template ID of the cloud template you want to deploy. In addition, this procedure includes an optional step to deploy a cloud template without a cloud template ID by providing contents inline instead.

1. Assign the cloud template ID variable.

```
cloud_template_id='<your_cloud_template_id>'
```

2. To deploy a cloud template, assign variables for image mapping and flavor mapping.

```
image_mapping='<your_image_mapping_name>'
flavor_mapping='<your_flavor_mapping_name>'
```

The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

3. Request a deployment of a cloud template.

```
curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "requesting deployment from cloud template",
    "blueprintId": "'$cloud_template_id'",
    "inputs": {
```

```

        "count": 2,
        "image": ""$image_mapping"",
        "flavor": ""$flavor_mapping""
    }
} | jq ". "

```

4. Examine the response to get the cloud template request ID and the deployment ID.

5. Assign the cloud template request ID and the deployment ID.

```

cloud_template_request_id='<your_cloud_template_request_id>'
deployment_id='<your_deployment_id>'

```

6. Optional: If you do not have a cloud template ID, you can also request a cloud template deployment with contents inline.

a) Validate the cloud template before creating it.

```

curl -X POST \
  $url/blueprint/api/blueprint-validation?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : ""$cloud_template_id"",
    "description" : "Basic Cloud Machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n  de-
scription: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n  description: Image
Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n    title:
Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    properties:\n
  name: BasicCloudMachine\n    flavor: '$${input.flavor}$' \n    image: '$${input.im-
age}$'\n    count: '$${input.count}$'",
    "projectId" : ""$project_id"",
    "requestScopeOrg": false
  }' | jq ". "

```

b) Examine the response to confirm that you see "valid":true.

c) Request the cloud template deployment.

```

curl -X POST \
  $url/blueprint/api/blueprint-requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "requesting deployment from inline cloud template",
    "projectId": ""$project_id"",
    "inputs": {
      "count": "2",
      "image": ""$image_mapping"",
      "flavor": ""$flavor_mapping""
    },
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n  de-
scription: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n  description: Image
Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum: 2\n    title:
Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n    properties:\n
  name: BasicCloudMachine\n    flavor: '$${input.flavor}$' \n    image: '$${input.im-
age}$'\n    count: '$${input.count}$'"
  }' | jq ". "

```

7. Look up the status of the cloud template deployment.

```
curl -X GET \
  $url/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Deploy a Cloud Template

For a cloud template with ID `1f170637-81a3-4257-b1cd-b2219ee8034c`, request the deployment with image mapping set to `ubuntu` and flavor mapping set to `small`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
$ image_mapping='ubuntu'
$ flavor_mapping='small'
```

Request the deployment of a cloud template.

```
$ curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "requesting deployment from cloud template",
    "blueprintId": "'$cloud_template_id'",
    "inputs": {
      "count": 2,
      "image": "'$image_mapping'",
      "flavor": "'$flavor_mapping'"
    }
  }' | jq "."
```

A snippet of the response shows the cloud template request ID, the deployment ID, and the cloud template ID.

```
{
  "id": "889f95a8-79a3-4b2f-b19e-32d1536dd69a",
  "createdAt": "2019-10-11T00:11:55.544Z",
  ...
  "projectName": "Example-project",
  "deploymentId": "15454178-63fc-42ea-b4ad-7ed8a5cdb128",
  "requestTrackerId": "889f95a8-79a3-4b2f-b19e-32d1536dd69a",
  ...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  ...
}
```

Assign the cloud template request ID variable.

```
$ cloud_template_request_id='889f95a8-79a3-4b2f-b19e-32d1536dd69a'
```

Request the status of the deployment.

```
$ curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the cloud template request ID and the cloud template ID with the status of the deployment request. If the deployment fails, the failure message indicates the reason for the failure.

```
{
  "id": "889f95a8-79a3-4b2f-b19e-32d1536dd69a"
  ...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  "inputs": {
    "count": 2,
    "image": "ubuntu",
    "flavor": "small"
  },
  "status": "FINISHED",
  ...
}
```

Use the deployment ID to look up resource information for your deployment.

Specify SCSI disk placement using the VCF Automation API

When attaching a disk object to a vSphere VM, you can specify the SCSI controller and the logical unit number (LUN) disk properties so that you can identify the disk when taking day 2 actions.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).

To attach a disk to a VM, you need the block device ID of the disk and the ID of the VM. The following procedure includes steps to:

- List all machines in your environment to find the ID of the vSphere VM that you want to attach a disk to.
- List all block devices in your environment to find the disk you want to attach.
- Attach the disk specifying the SCSI controller and LUN.

1. List all the machines in your environment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/machines?apiVersion=$api_version | jq "
```

To identify the vSphere VM where you want to attach your disk, look for the name and the region ID in the response. The `self:href` path includes the machine ID.

```
{
  "powerState": "OFF",
  "externalRegionId": "Datacenter:datacenter-2",
  "cloudAccountIds": [
    "e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bdf1c802400819ad45a341c-ba1f3-b37e594a2e813475574a7c3c42b5d"
  ],
  "provisioningStatus": "READY",
  "customProperties": {
    "osType": "LINUX",
    "vcUuid": "1f9678f0-90d1-4347-82ee-f1ac2fac4216",
    "memoryGB": "1",
    "datacenter": "Datacenter:datacenter-2",
    "instanceUUID": "503a00ea-5ce5-3ae8-db3d-ebbd537eed5f",
    "softwareName": "Ubuntu Linux (64-bit)",
    "cpuCount": "2",
    "memoryInMB": "1024"
  },
  "externalId": "503a00ea-5ce5-3ae8-db3d-ebbd537eed5f",
}
```

```

    "name": "wordpress-mcm827-142063808276-ovf-backing",
    "id": "0355cee4-5d88-36b6-9a87-5f37b5baa6e2",
    "createdAt": "2022-04-02",
    "updatedAt": "2022-04-02",
    "organizationId": "f670fd6c-66d6-4689-9793-d524e7066d1e",
    "orgId": "f670fd6c-66d6-4689-9793-d524e7066d1e",
    "_links": {
      "network-interfaces": {
        "hrefs": [
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/network-inter-
faces/1c31ee02-c83c-3229-84ec-929f3592494a"
        ]
      },
      "cloud-accounts": {
        "hrefs": [
          "/iaas/api/cloud-accounts/e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306c-
c791422547ead9153c3bdf1c802400819ad45a341cba1f3-b37e594a2e813475574a7c3c42b5d"
        ]
      },
      "operations": {
        "hrefs": [
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/operations/power-on",
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/operations/snapshots",
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/operations/resize",
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks",
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/{id}"
        ]
      },
      "disks": {
        "hrefs": [
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/eff-
b94a6-41ad-3553-bb9a-22896785a283",
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/0815ed4c-0a33-3058-
a5f5-3032a426ded4",
          "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/3804a2b2-99fc-3fca-bd2f-
f0cfc3845b54"
        ]
      },
      "self": {
        "href": "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2"
      }
    }
  },

```

In this response example, the machine ID is 0355cee4-5d88-36b6-9a87-5f37b5baa6e2 .

2. Assign the machine ID.

```
machine_id='example-machineID-alphanumeric-string'
```

3. List the block devices in your environment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices?apiVersion=$api_version | jq "."
```

Examine the response to find the block device that you want to attach. The block device must be available and in the same external region as the vSphere VM. The `self:href` path includes the block device ID.

```
...
{
  "capacityInGB": 30,
  "status": "AVAILABLE",
  "type": "HDD",
  "persistent": false,
  "externalRegionId": "Datacenter:datacenter-2",
  "cloudAccountIds": [
    "e0f23c91d5ec-
ca75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-6b4f9990d36ee87558f04e6e8e0ca
  ],
  "provisioningStatus": "READY",
  "customProperties": {
    "diskKind": "Unmanaged"
  },
  "externalId": "74213f6d-ee11-4549-996a-772c0621f7d1",
  "name": "Hard disk 1",
  "id": "02c39b60-32f5-4a7e-9317-88bf8a3fe20c",
  "createdAt": "2022-04-02",
  "updatedAt": "2022-04-02",
  "organizationId": "f670fdcf-66d6-4689-9793-d524e7066d1e",
  "orgId": "f670fdcf-66d6-4689-9793-d524e7066d1e",
  "_links": {
    "cloud-accounts": {
      "hrefs": [
        "/iaas/api/cloud-accounts/e0f23c91d5ec-
ca75-7f703c5265a63d87-e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-6b4f9990d36ee87558f04e6e8e0ca
      ]
    },
    "operations": {
      "hrefs": [
        "/iaas/api/block-devices/02c39b60-32f5-4a7e-9317-88bf8a3fe20c?capacityInGB={capacityInGB}"
      ]
    },
    "self": {
      "href": "/iaas/api/block-devices/02c39b60-32f5-4a7e-9317-88bf8a3fe20c"
    }
  }
},
...

```

In this response example, the block device ID is `02c39b60-32f5-4a7e-9317-88bf8a3fe20c`.

4. Assign the block device ID variable.

```
block_device_id='example-blockdeviceID-alphanumeric-string'
```

5. Attach the disk to the VM, specifying both the SCSI controller number and the LUN disk properties.

- For the SCSI controller number, you can specify any of four values: `SCSI_Controller_0`, `SCSI_Controller_1`, `SCSI_Controller_2`, `SCSI_Controller_3`.
- For the LUN, you can specify any integer value from 0 through 15. Unit 0 is the boot disk. Disks with higher LUN values attach after disks with lower LUN values.

Note: If you do not specify a LUN, the disk attaches to the first available unit number. If you specify neither the SCSI controller nor the LUN, the disk attaches to the first available SCSI controller and first available unit number.

This request example uses `"scsiController": "SCSI_Controller_0"` and `"unitNumber": "0"`.

```
curl -X POST \
  $url/iaas/api/machines/$machine_id/disks?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "blockDeviceId": "'$block_device_id'",
    "scsiController": "SCSI_Controller_0",
    "name": "BootDisk ",
    "description": "Unit 0 is the boot disk",
    "unitNumber": "0"
  }' | jq "
```

The response includes a `selfLink` value.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

6. Assign the `selfLink` variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

7. Use the `selfLink` variable to track the progress of the disk attachment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "
```

After the request completes successfully, the response includes a list of resources with a machine that has your machine ID in the path.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/machines/example-machineID-alphanumeric-string"
  ],
  ...
}
```

8. Optional: To specify SCSI controller and LUN when attaching additional disks, repeat steps 5 to 7.

If you deploy a blueprint that includes a VM with multiple vSphere disk objects, the disks are assigned to the SCSI controller and LUN that you specified. In this way, you can identify the disks when taking day 2 actions such as formatting or resizing a disk.

How to Create Custom Naming Templates

If you enroll in custom naming in the VCF Automation UI, you can create custom naming templates for your organization or project using the IaaS API. Custom naming provides a way for you to name deployed resources using conventions that you define.

If you create a custom name using `POST /iaas/api/naming`, any project that does not already have a template applied can use the naming template. An organization-level naming template applies to any project that is added to an organization.

A maximum of two resource naming templates can be applied to a project: one project-level naming template and one organization-level naming template. Each can have multiple resource types. If naming templates exist at both the project and organization level, the project uses the naming convention used in the project-level naming template.

Note: If you created a project and specified a custom naming template as part of a previous project definition, that template is deprecated when you enroll in custom naming. To replace that template, create a custom name at the project level and add a new naming template.

The following examples show how to define custom naming templates for an organization and a project.

Prerequisites for creating a custom name

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have enrolled in Custom Naming.
For information about enrolling in custom naming, see [Custom naming deployed resources in for VM Apps](#).
- Verify that you know your organization ID. If you do not know your organization ID, perform the following steps:
 - a. Log in to the organization console of your VCF Automation appliance, for example `https://appliance.domain.com/csp/gateway/portal/`
 - b. Click the drop-down arrow by your name to display the organization ID below the organization name.
 - c. Right click the icon next to the organization ID to copy the long string.
- If you are adding a project-level template, verify that you know the name and ID of the project that you want to assign to the template. For information on how to get a list of projects, see [Create a Project to use in VCF Automation](#).

How to create a custom name with organization scope

This example shows how to define an organization-level custom name with a naming template for a machine resource type in `"orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"`.

```
curl -X POST \
  "$url/iaas/api/naming?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "org-level custom name",
    "description": "Example organization-level custom name",
    "projects": [
      {
        "defaultOrg": true,
```



```

    "active": true,
    "projectName": "*",
    "projectId": "*",
    "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
  }
],
"templates": [
  {
    "uniqueName": true,
    "staticPattern": "",
    "counters": [],
    "incrementStep": 1,
    "pattern": "myvm-{{$#}}",
    "startCounter": 1,
    "resourceTypeName": "Machine",
    "resourceType": "COMPUTE",
    "resourceDefault": true
  }
]
}' | jq ".

```

The response shows the organization-level custom name with the custom name ID.

```

{
  "name": "org-level custom name",
  "description": "Example organization-level custom name",
  "id": "6ca7be62-627b-41f0-9505-fc29c1349a85",
  "_links": {
    "self": {
      "href": "/iaas/api/naming/6ca7be62-627b-41f0-9505-fc29c1349a85"
    }
  }
}

```

How to create a custom name with project scope

This example shows how to define a project-level custom name with naming templates for network and machine resource types, and assigned to a project that has not already been added to a custom name with project scope.

```

curl -X POST \
  "$url/iaas/api/naming?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "proj-level custom name ",
    "description": "Example project-level custom name",
    "projects": [
      {
        "defaultOrg": false,
        "active": true,
        "projectName": "Example-CA-project",
        "projectId": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
        "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
      }
    ]
  }'

```

```

],
"templates": [
  {
    "uniqueName": true,
    "staticPattern": "",
    "counters": [],
    "incrementStep": 1,
    "pattern": "myCAproject-${##}",
    "startCounter": 1,
    "resourceTypeName": "Network",
    "resourceType": "NETWORK",
    "resourceDefault": true
  },
  {
    "uniqueName": true,
    "staticPattern": "",
    "counters": [],
    "incrementStep": 1,
    "pattern": "myCAproject-${##}",
    "startCounter": 1,
    "resourceTypeName": "Machine",
    "resourceType": "COMPUTE",
    "resourceDefault": true
  }
]
}' | jq "."

```

The response shows the project-level custom name with the custom name ID.

```

{
  "name": "proj-level custom name",
  "description": "Example project-level custom name",
  "id": "9afa9636-9536-4867-a325-fc70eb073a86",
  "_links": {
    "self": {
      "href": "/iaas/api/naming/9afa9636-9536-4867-a325-fc70eb073a86"
    }
  }
}

```

Requesting a Deployment from a Catalog Item Using VCF Automation APIs

To create catalog items and request deployments, you use the VCF Automation Catalog and Deployment APIs.

In this use case, you create a catalog source with a project that you previously used to create a blueprint, so that you can request a deployment from the catalog item. Optionally, if you want to expire your deployment, you can create a lease policy.

Create a Catalog Source and List Discovered Items

To create a catalog source, you make a POST request with a project ID that has a blueprint version released to the project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Catalog service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID for a project that has the blueprint versioned and released to it.

Because you are requesting the deployment of a blueprint from the catalog, this example lists the steps required to create a VCF Automation Templates source type.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. List all your catalog sources.

```
curl -X GET \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

3. Examine the response to confirm that the name of the catalog source that you plan to create is not listed.

4. List all catalog source types.

```
curl -X GET \
  $url/catalog/api/types?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

5. Examine the response to find the catalog source type that you want to create.

6. Assign the catalog item type ID variable for the VCF Automation Templates source type.

```
catalog_type_id='com.vmw.blueprint'
```

7. Create a catalog source for your blueprint.

```
curl -X POST \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H "Content-Type: application/json" \
  -d '{
    "config":{
      "sourceProjectId":"'$project_id'"
    },
    "typeId":"'$catalog_type_id'",
    "name": "<your_catalog_source_name>"
  }' | jq "."
```

8. To obtain the catalog source ID, examine the response.

9. List all items discovered in the project.

```
curl -X GET \
  $url/catalog/api/admin/items?projectId=$project_id&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

10. To obtain the catalog item ID, examine the response.

11. Optional: Assign the blueprint name variable.

```
cloud_template_name='<your_cloud_template_name_that_was_released_to_catalog>'
```

12. Optional: To get the catalog item ID, you can also list discovered items by name.

```
curl -X GET \
  $url/catalog/api/admin/items?projectId=$project_id&search=$cloud_template_name&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Create a catalog source and list discovered items

Create a catalog source for a blueprint named BasicCloudMachine.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
```

List all available sources in your catalog.

```
$ curl -X GET \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "
```

Examine the response to confirm that the name of the catalog source that you plan to create is not listed. The following snippet shows that you cannot create a catalog source with the name Catalog Source from Blueprintecho s.

```
...
  "id": "753d24a3-e2b0-4d5e-bba6-9e32e5964c69",
  "name": "Catalog Source from Blueprintecho s",
...
```

List all available catalog source types.

```
$ curl -X GET \
  $url/catalog/api/types?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "
```

Examine the response to find the ID for a VCF Automation Templates source type.

```
...
  "id": "com.vmw.blueprint",
  "name": " VCF Automation Template",
  "baseUri": "http://catalog-service:8000/catalog/api/provider/blueprint",
  "createdBy": "deploymentsservice",
...
```

Assign the source type to the catalog item type variable.

```
$ catalog_type_id='com.vmw.blueprint'
```

Create a catalog source of the VCF Automation Templates source type.

```
$ curl -X POST \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H "Content-Type: application/json" \
  -d '{
    "config":{
      "sourceProjectId":"' $project_id '"
    },
    "typeId":"' $catalog_type_id '",
    "name":"Catalog Source from VCF Automation Templates"
  }' | jq "
```

A snippet of the response shows the catalog source ID.

```
{
  "id": "753d24a3-e2b0-4d5e-bba6-9e32e5964c69",
  "name": "Catalog Source from VCF Automation Templates",
  "typeId": "com.vmw.blueprint",
  "createdAt": " T22:02:33.553Z",
  ...
}
```

Assign the catalog source ID.

```
$ catalog_source_id='753d24a3-e2b0-4d5e-bba6-9e32e5964c69'
```

List items discovered in your project.

```
$ curl -X GET \
  $url/catalog/api/admin/items?projectId=$project_id&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the catalog item ID with your blueprint name.

```
{
  "id": "718917c0-1e02-3141-8142-11da5acaed8f",
  "name": "BasicCloudMachine",
  "description": "Basic Cloud Machine blueprint",
  "sourceId": "753d24a3-e2b0-4d5e-bba6-9e32e5964c69",
  ...
}
```

Assign the catalog item ID.

```
$ catalog_item_id='718917c0-1e02-3141-8142-11da5acaed8f'
```

Before requesting a deployment, you use the `catalog_source_id` to create entitlement for the catalog source. Or you can use the `catalog_item_id` to create entitlement for the blueprint item.

Request Deployment

To request a deployment from a catalog item, you make a POST request with a project ID that has a blueprint version released to the project. The request body includes the ID of the catalog item from which you are requesting the deployment, and the version of the released blueprint.

- Verify that all general prerequisites and prerequisites for both the VCF Automation Catalog service and the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that the flavor mapping and image mapping specified in the released blueprint version exist in your cloud account. See [Create Flavor Mappings](#) and [Create Image Mappings](#).
- Verify that you have the ID for a project that has the blueprint versioned and released to it.
- Verify that you have the ID of the catalog item from which you plan to request a deployment. See [Create a Catalog Source and List Discovered Items](#).
- Verify that you have the version of a blueprint released to the project that you want to request for deployment.
- Verify that you have created an entitlement for your catalog item or create one.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. Assign the catalog item ID variable.

```
catalog_item_id='<your_catalog_item_id>'
```

3. List the available versions of the catalog item that can be requested.

```
curl -X GET \
```

```
$url/catalog/api/items/$catalog_item_id/versions \
-H "Authorization: Bearer $access_token" | jq "."
```

4. Examine the response to verify the version of the item that you want has been published to the catalog.

5. Assign the catalog item version.

```
catalog_item_version='<your_catalog_item_version>'
```

6. Assign your deployment name variable.

```
deployment_name='<your_deployment_name>'
```

If your deployment name includes spaces, use double quotes as in the following example.

```
deployment_name="This deployment name includes spaces"
```

a) To ensure that the deployment name you plan to use does not already exist, list all deployments.

```
curl -X GET \
-G --data-urlencode "name=$deployment_name" \
$url/deployment/api/deployments?apiVersion=$api_version \
-H "Authorization: Bearer $access_token" | jq "."
```

b) Examine the response. If your deployment name appears, create a new name and reassign your deployment name variable.

7. To deploy a blueprint, assign variables for image mapping and flavor mapping.

```
image_mapping='<your_image_mapping_name>'
flavor_mapping='<your_flavor_mapping_name>'
```

The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

8. Request the deployment from a catalog item.

```
curl -X POST \
$url/catalog/api/items/$catalog_item_id/request?apiVersion=$api_version \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
-d '{
  "deploymentName": ""$deployment_name"",
  "projectId": ""$project_id"",
  "inputs": {
    "count": 1,
    "image": ""$image_mapping"",
    "flavor": ""$flavor_mapping""
  },
  "version": ""$catalog_item_version""
}' | jq "."
```

The `inputs` field includes values for request time variables such as size, image, or password.

9. To obtain the deployment ID, examine the response.

10. Assign the deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

11. Get the status of the deployment.

```
curl -X GET \
$url/deployment/api/deployments/$deployment_id?apiVersion=$api_version \
-H "Authorization: Bearer $access_token" | jq "."
```

Request Deployment of a Blueprint from a Catalog Item

Request the deployment of a blueprint with catalog item ID 718917c0-1e02-3141-8142-11da5acaed8f. When requesting the deployment, set image mapping set to `ubuntu` and flavor mapping set to `small`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
$ catalog_item_id='718917c0-1e02-3141-8142-11da5acaed8f'
```

List available versions.

```
$ curl -X GET \
  $url/catalog/api/items/$catalog_item_id/versions \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows version numbers.

```
...
{
  "id": "v2",
  "description": "Creating a version from the current draft",
  "createdAt": " 7T19:33:04.445Z"
},
{
  "id": "v1",
  "description": "Creating a version from the current draft",
  "createdAt": " T19:25:43.327Z"
}
...
```

Assign the catalog item version number.

```
$ catalog_item_version='v2'
```

Assign a deployment name and check to ensure that it does not already exist.

```
$ deployment_name="Example Deployment of blueprint"
$ curl -X GET \
  -G --data-urlencode "name=$deployment_name" \
  $url/deployment/api/deployments?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows existing deployments. Example Deployment of blueprint is not listed.

```
{
  "id": "c14e787f-60ee-4cce-a5b5-c9440bf181ab",
  "name": "Not Example Deployment",
  "orgId": "c9258a19-fef0-4431-a999-d711e1741c60",
  "catalogItemId": "947b9db2-cf89-3b83-8035-bbcf83bd4c34",
  ...
}
```

To deploy a blueprint, you must assign image mapping and flavor mapping variables.

```
$ image_mapping='ubuntu'
$ flavor_mapping='small'
```

Request a deployment from the catalog item.

```
$ curl -X POST \
  $url/catalog/api/items/$catalog_item_id/request?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "deploymentName": "$deployment_name",
    "projectId": "$project_id",
    "inputs": {
      "count": 1,
      "image": "$image_mapping",
      "flavor": "$flavor_mapping"
    },
    "version": "$catalog_item_version"
  }' | jq "
```

The response provides the deployment ID.

```
{
  "deploymentId": "3721d9e2-fce3-48eb-96e5-d8f381354610",
  "deploymentName": "Example Deployment of blueprint"
}
```

Assign the deployment ID.

```
$ deployment_id='3721d9e2-fce3-48eb-96e5-d8f381354610'
```

Get the deployment status.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id?apiVersion=$api_version"
  -H "Authorization: Bearer $access_token" | jq "
```

A snippet of the response shows the deployment status.

```
{,
  "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbfb51",
  "status": "CREATE_SUCCESSFUL"
}
```

Create a Lease Policy

To create a lease policy for your deployment, you make a POST request with a project ID that has a blueprint version released to the project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID for a project that has the blueprint versioned and released to it.

Creating a lease policy is optional. For example, you can create a lease policy to specify when you want a deployment to expire. You specify the policy with either a soft or hard lease enforcement type.

- If specified with soft enforcement, the policy can be overridden and will have lower priority than policies with hard enforcement.
- If specified with hard enforcement, the policy must be enforced. If strict enforcement is not possible, for example in cases of conflicting policies, the policy can be overridden but VCF Automation will report an error.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```


2. Assign a lease policy with soft enforcement to your project.

```
curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "<your_lease_policy_name>",
    "projectId": "'$project_id'",
    "definition": {
      "leaseGrace": 1,
      "leaseTermMax": 10,
      "leaseTotalTermMax": 100
    },
    "enforcementType": "SOFT",
    "typeId": "com.mycompany.policy.deployment.lease"
  }' | jq "."
```

Create a lease policy with soft enforcement

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
```

Create the soft lease policy named Sample Lease.

```
$ curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Lease",
    "projectId": "'$project_id'",
    "definition": {
      "leaseGrace": 1,
      "leaseTermMax": 10,
      "leaseTotalTermMax": 100
    },
    "enforcementType": "SOFT",
    "typeId": "com.vmware.policy.deployment.lease"
  }' | jq "."
```

The response shows the lease policy.

```
{
  "id": "49893797-208c-4322-8ed5-061467674d54",
  "name": "Sample Lease",
  "typeId": "com.mycompany.policy.deployment.lease",
  "enforcementType": "SOFT",
  "orgId": "c9258a19-fef0-4431-a999-d711e1741c60",
  "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbefb51",
  "definition": {
    "leaseGrace": 1,
    "leaseTermMax": 10,

```

```

    "leaseTotalTermMax": 100
  },
  "createdAt": " T02:29:07.936Z",
  "createdBy": "admin@mycompany.com",
  "lastUpdatedAt": " T02:29:07.936Z",
  "lastUpdatedBy": "admin@mycompany.com"
}

```

Working with Deployments and Resources

To work with deployments, you use the VCF Automation and VCF Automation Deployment APIs. You also use the VCF Automation Deployment APIs to manage or get information about resources in your deployment.

These use cases include examples of procedures you can follow to reconfigure your deployment after initial blueprint deployment. The first example deploys a new blueprint with a deployment ID. In subsequent examples, you use the deployment ID to update the deployment or reconfigure the blueprint components.

If you want to onboard a machine that was deployed outside of VCF Automation, you use the Relocation Service APIs to create an onboarding plan so that you can onboard the machine. Onboarding use cases show how to create an onboarding plan and onboard a machine with or without a blueprint.

Deploy a Cloud Template with Contents Inline

To request a deployment of a cloud template with contents inline, you use the Blueprint APIs to make a POST request with a project ID. Before requesting the deployment, you use the Deployment API to ensure that the deployment name does not already exist.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID for a project that is associated with the deployment that you are managing. See [Request Deployment](#).
- Verify that the flavor mapping and image mapping specified in the cloud template to be deployed exist in your cloud account. See [Create Flavor Mappings](#) and [Create Image Mappings](#).
- Verify that the cloud zone that you are deploying into is associated with your project. See [Add a Cloud Zone to Your Project](#).
- Verify that a network profile is configured for the cloud account associated with the project. See [Create Network Profiles](#).
- Assign an API version variable for the Blueprint API.

```
api_version_blueprint=' '
```

Note: The VCF Automation Deployment service and the VCF Automation Blueprint service have different API version values. You set the API version value for the VCF Automation Deployment service when you satisfied the general prerequisites.

In this example, you deploy a cloud template by providing contents inline instead of providing a cloud template ID. The new cloud template has different content from previously deployed cloud templates including a load balancer component, a virtual machine component, and a network component.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. Assign your deployment name variable.

```
deployment_name='<your_deployment_name>'
```

If your deployment name includes spaces, use double quotes as in the following example.

```
deployment_name="This deployment name includes spaces"
```

a) To ensure that the deployment name you plan to use does not already exist, list all deployments.

```
curl -X GET \
  -G --data-urlencode "name=$deployment_name" \
  $url/deployment/api/deployments?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

b) To verify that the deployment name does not already exist, examine the response. If your deployment name appears, create a new name and reassign your deployment name variable.

3. To deploy a cloud template, assign variables for image mapping and flavor mapping.

```
image_mapping='<your_image_mapping_name>'
flavor_mapping='<your_flavor_mapping_name>'
```

The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

4. Request deployment of a cloud template with contents inline.

```
curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version_blueprint \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "deploymentName": "\"$deployment_name\"",
    "description": "requesting deployment with contents inline",
    "projectId": "\"$project_id\"",
    "inputs": {
      "flavor": "\"$flavor_mapping\"",
      "image" : "\"$image_mapping\""
    },
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n    description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\nresources:\n  cloud-vm:\n    type: Cloud.AWS.EC2.Instance\n    properties:\n      name: cloudvm\n      flavor: '\"${input.flavor}\"'\n      image: '\"${input.image}\"'\n      networks:\n        - name: '\"${resource.Cloud_Network_1.name}\"'\n        network: '\"${resource.Cloud_Network_1.id}\"'\n      Provider_LoadBalancer_1:\n        type: Cloud.LoadBalancer\n        properties:\n          name: OC-LB\n          routes:\n            - protocol: HTTP\n              port: '\"80\"'\n              instanceProtocol: HTTP\n              instancePort: '\"80\"'\n              healthCheckConfiguration:\n                protocol: HTTP\n                port: '\"80\"'\n                urlPath: /index.html\n                intervalSeconds: 60\n                timeoutSeconds: 5\n                unhealthyThreshold: 5\n                healthyThreshold: 2\n              network: '\"${resource.Cloud_Network_1.name}\"'\n              instances:\n                - '\"${resource[\"cloud-vm\"].id}\"'\n              internetFacing: false\n              Cloud_Network_1:\n                type: Cloud.Network\n                properties:\n                  name: provider\n                  networkType: public\n            }\n  } | jq "."
```

5. To obtain the cloud template request ID and the deployment ID, examine the response.

6. Assign the cloud template request ID variable.

```
cloud_template_request_id='<your_cloud_template_request_id>'
```

7. Get the status of the request.

```
curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version_blueprint \
  -H "Authorization: Bearer $access_token" | jq "."
```

Deploy a Cloud Template with Contents Inline

Request a deployment named Deployment with Cloud Template Contents Inline. When requesting the deployment, set image mapping set to ubuntu and flavor mapping set to small.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_blueprint=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbfb51'
$ deployment_name="Deployment with Cloud Template Contents Inline"
```

To request deployment, you must assign image mapping and flavor mapping variables.

```
$ image_mapping='ubuntu'
$ flavor_mapping='small'
```

Request deployment of a cloud template with contents inline.

```
$ curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version_blueprint \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "deploymentName": "'$deployment_name'",
    "description": "requesting deployment with contents inline",
    "projectId": "'$project_id'",
    "inputs": {
      "flavor": "'$flavor_mapping'",
      "image" : "'$image_mapping'"
    },
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n  description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description: Image Mapping Name\nresources:\n  cloud-vm:\n    type: Cloud.AWS.EC2.Instance\n    properties:\n      name: cloudvm\n      flavor: '$${input.flavor}'\n      image: '$${input.image}'\n    networks:\n      - name: '$${resource.Cloud_Network_1.name}'\n        network: '$${resource.Cloud_Network_1.id}'\n        Provider_LoadBalancer_1:\n          type: Cloud.LoadBalancer\n          properties:\n            name: OC-LB\n            routes:\n              - protocol: HTTP\n                port: '$80'\n            instanceProtocol: HTTP\n            instancePort: '$80'\n            healthCheckConfiguration:\n              protocol: HTTP\n              port: '$80'\n              urlPath: /index.html\n              intervalSeconds: 60\n              timeoutSeconds: 5\n              unhealthyThreshold: 5\n              healthyThreshold: 2\n            network: '$${resource.Cloud_Network_1.name}'\n          instances:\n            - '$${resource["cloud-vm"].id}'\n            internetFacing: false\n          Cloud_Network_1:\n            type: Cloud.Network\n            properties:\n              name: provider\n              networkType: public\n            lic\n  }' | jq "."
```

A snippet of the response provides the cloud template request ID and the deployment ID.

```
...
"type": "blueprint-request",
```

```

    "id": "bec37f54-3de5-451d-b484-a110c0ed8772",
    ...
    "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbefb51",
    "projectName": "example-project",
    "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
    "requestTrackerId": "bec37f54-3de5-451d-b484-a110c0ed8772",
    "deploymentName": "Deployment with Cloud Template Contents inline",
    ...

```

Assign the cloud template request ID.

```
$ cloud_template_request_id='bec37f54-3de5-451d-b484-a110c0ed8772'
```

Get the deployment status.

```

$ curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version_blue-
print \
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows the deployment status.

```

...
"inputs": {
  "image": "ubuntu",
  "flavor": "small"
},
"status": "FINISHED",
...

```

You can use the deployment ID to perform other operations on your deployment. Other operations include, changing the lease on your deployment, fetching deployment resources, reconfiguring a load balancer, or adding a disk to a machine and power it off.

Look up Deployment Details

To look up deployment details such as the resources provisioned for each deployment, you use the Deployment APIs to make a GET request that displays all available resources. Then you use the resource ID in the output to make a GET request that returns the details of a particular resource.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have a deployment ID for the deployment that you requested.

1. Assign the deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

2. Display all the available resources that are provisioned in your deployment.

```

curl -X GET \
  $url/deployment/api/deployments/$deployment_id?expand=resources&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."

```

3. Examine the response to find the ID of the resource for which you want details.

4. Assign the deployment resource ID.

```
deployment_resource_id='<your_deployment_resource_id>'
```

5. Display the details of that resource.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$deployment_resource_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

6. List the deployment events.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/events?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

In case of deployment failures, listing deployment events can help with debugging.

Look up the details of a provisioned resource in your deployment

Display the resources provisioned in your deployment.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='15454178-63fc-42ea-b4ad-7ed8a5cdb128'
```

Look up deployment details.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id?expand=resources&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the details for each resource details including a deployment resource ID.

```
...
  "resources": [
    {
      "id": "3994a33e-bd93-4eea-87f1-f99ff17717ce",
      "name": "BasicCloudMachine[0]",
    }
  ]
...

```

Assign the deployment resource ID variable for the BasicCloudMachine resource.

```
$ deployment_resource_id='3994a33e-bd93-4eea-87f1-f99ff17717ce'
```

Display the details of that resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$deployment_resource_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the details of the single resource.

```
{
  "id": "3994a33e-bd93-4eea-87f1-f99ff17717ce",
  "name": "BasicCloudMachine[0]",
  "type": "Cloud.Machine",
  "dependsOn": [],
  "createdAt": " T17:56:09.463Z",
  "properties": {
    "id": "/resources/compute/3114189206b1763d",
  }
}
```

```

    "name": "BasicCloudMachine",
    ...
    "service": "EC2",
    "storage": {
      "disks": [
        {
          "name": "boot-disk",
          "type": "HDD",
        }
      ]
    },
    ...
    "networks": [
      {
        "name": "BasicCloudMachine_nic",
        "address": "172.16.1.98",
        "assignment": "dynamic"
      }
    ],
    ...
    "__ext:ComputeReservationTaskState:STARTED:SELECTED": "true",
    "__ext:ComputeAllocationTaskState:STARTED:START_COMPUTE_ALLOCATION": "true"
  },
  "state": "OK"
}

```

List events from the deployment.

```

$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/events?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows successful events.

```

...
  "totalTasks": 3,
  "status": "SUCCESSFUL",
  "inputs": {
    "count": 2,
    "image": "ubuntu",
    "flavor": "small"
  }
...

```

Get Deployment Resource IDs

To get IDs of the resources in your deployment, you use the Deployment APIs to make a GET request.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the deployment you want to reconfigure. See [Deploy a Cloud Template with Contents Inline](#).

To perform operations on the load balancer or virtual machine in your deployment, you need the IDs of those resources.

1. Assign your deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

2. List the resources in your deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

3. Examine the response.

- Find the resource named `Provider_LoadBalancer_1` and copy the value to assign to the load balancer ID.
- Find the resource named `cloud-vm` and copy the value to assign to the virtual machine ID.

4. Assign variables for the resources.

```
load_balancer_id='<your_load_balancer_id>'
virtual_machine_id='<your_virtual_machine_id>'
```

Get Deployment Resource IDs

Get the resource IDs for your deployment with ID `5551a299-8b67-45e3-909e-a638d11b0d9f`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'
```

List the resources in your deployment.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the ID for the resource `Provider_LoadBalancer_1` and the ID for the resource `cloud-vm`.

```
...
{
  "id": "d5b4569d-2234-4fc4-a594-45e6b0251588",
  "name": "Provider_LoadBalancer_1",
  "type": "Cloud.LoadBalancer",
  ...
{
  "id": "42f49781-1490-4a08-ae21-8baf383a72ac",
  "name": "cloud-vm",
  "type": "Cloud.AWS.EC2.Instance",
  ...
```

Assign the load balancer ID and virtual machine ID variables.

```
$ load_balancer_id='d5b4569d-2234-4fc4-a594-45e6b0251588'
$ virtual_machine_id='42f49781-1490-4a08-ae21-8baf383a72ac'
```

Use the load balancer ID to reconfigure your load balancer. Use the virtual machine ID to add a disk to the VM and power off the VM.

Change the Lease on Your Deployment

To change the lease on your deployment, you use the Deployment APIs to make a POST request with a new lease expiration date.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the deployment you want to reconfigure. See [Deploy a Cloud Template with Contents Inline](#).

The lease on your deployment is set to never expire by default. The following procedure shows how to add a lease expiration date. It also includes an optional step that shows how to reset the lease on your deployment back to never expire.

1. Assign your deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

2. Get a list of actions available for your deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

3. Examine the response.

- Confirm that you see the action "name": "ChangeLease".
- "valid":true indicates that the action is valid for the deployment.

4. Assign the action ID variable for the action "name": "ChangeLease".

```
action_id='Deployment.ChangeLease'
```

5. List the deployment actions for the action ID .

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions/$action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

6. Examine the response. The schema field shows the format of the input for an action on the deployment.

7. To change the lease on the deployment, assign the lease expiry date using the format specified in the schema as in the following example.

```
lease_expiry_date=2021-05-15T23:11:00Z
```

8. Change the lease expiration date.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId":"Deployment.ChangeLease",
    "inputs": {
      "Lease Expiration Date": "'$lease_expiry_date'"
    }
  }' | jq "."
```

9. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

10. Check the status of the request.

```
curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
```

```
-H "Authorization: Bearer $access_token" | jq "."
```

If the request is successful, the response shows "status": "SUCCESSFUL".

11. Optional: To change the lease on the deployment back to never expire, use the same `action_id='Deployment.ChangeLease'` but leave inputs empty.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId": "Deployment.ChangeLease",
    "inputs": {}
  }' | jq "."
```

Use the request ID from the response to check the status of the request. The lease is changed when the response shows "status": "SUCCESSFUL".

Change the Lease on Your Deployment

Change the lease on your deployment with ID 5551a299-8b67-45e3-909e-a638d11b0d9f.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'
```

List the actions available for your deployment.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows `Deployment.ChangeLease`.

```
...
{
  "id": "Deployment.ChangeLease",
  "name": "ChangeLease",
  "displayName": "Change Lease",
  "description": "Set a deployment's expiration date",
  "valid": true,
  "actionType": "RESOURCE_ACTION"
}
...
```

Assign the action ID variable.

```
$ action_id='Deployment.ChangeLease'
```

To get the schema for your action, list the deployment actions by ID.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions/$action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response provides the schema for the lease expiration date.

```
...
```

```

    "properties": {
      "Deployment expires in": {
        "type": "string",
        "readOnly": true,
        "default": "9d 21h 27m"
      },
      "Lease Expiration Date": {
        "type": "string",
        "title": "Lease Expiration Date",
        "description": "The lease can be extended by up to 90d 0h 0m",
        "format": "date-time",
        "formatMinimum": "2021-02-12T21:47:00Z",
        "formatMaximum": "2021-05-20T19:15:00Z",
        "default": "2021-02-22T19:15:00Z"
      }
    },
    ...

```

Assign the lease expiry date variable in the proper format.

```
$ lease_expiry_date=2021-05-15T23:11:00Z
```

Submit a request to change the lease expiration.

```

$ curl -X POST \
  $url/deployment/api/deployments/$deployment_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId": "Deployment.ChangeLease",
    "inputs": {
      "Lease Expiration Date": "'"$lease_expiry_date"'"
    }
  }' | jq "."

```

A snippet of the response shows request ID.

```

...
  "id": "6b9b5534-0d84-4f07-9941-5c3cc26f7e3b",
  "name": "Change Lease",
  "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
  ...

```

Assign the request ID variable.

```
$ request_id='6b9b5534-0d84-4f07-9941-5c3cc26f7e3b'
```

Check the status of the request.

```

$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows that the request was successful.

```

...
  "actionId": "Deployment.ChangeLease",
  "completedTasks": 1,
  "totalTasks": 1,
  "status": "SUCCESSFUL"

```

```
}
```

Delete Your Deployment

To delete a deployment, you use the VCF Automation Deployment API and if needed, the VCF Automation IaaS API. You make a DELETE request to remove the deployment and clean up the associated resources from the cloud provider.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the VCF Automation administrator service role.
- Verify that you know the deployment that you want to delete.
- Assign an API version variable for the IaaS API.

```
api_version_iaas='2021-07-15'
```

Note: The VCF Automation Deployment service and the VCF Automation Infrastructure as a Service (IaaS) service have different API version values. You set the API version value for the VCF Automation Deployment service when you satisfied the general prerequisites.

If using the VCF Automation Deployment service fails to delete the deployment, you can use the VCF Automation IaaS service to force the deletion. The following procedure shows how to delete a deployment when both API services are used.

1. List all deployments.

```
curl -X GET "$url/deployment/api/deployments?apiVersion=$api_version" -H "Authorization: Bearer $access_token" | jq "."
```

2. Examine the response to find the name and ID of the deployment that you want to delete.

3. Assign your deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

4. Get a list of actions for the deployment.

```
curl -X GET \
"$url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

5. Examine the response.

- Confirm that you see the action "id": "Deployment.Delete".
- "valid":true indicates that the action is valid for the deployment.

6. Delete the deployment.

```
curl -X DELETE \
"$url/deployment/api/deployments/$deployment_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

7. Examine the response for the ID of the deletion request.

8. Assign the ID to the request ID variable.

```
request_id='<delete_request_id>'
```

9. Get the status of the deployment request.

```
curl -X GET \
"$url/deployment/api/requests/$request_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

10. Examine the response.

- If the response shows "status": "SUCCESSFUL", then the delete request succeeded.
- If the response shows "status": "FAILED" then the delete request may have failed due to a dependency such as an unreleased IP in an AD account. If you are certain that you want to remove the deployment and all related resources, you can choose to ignore the deletion failures and submit a delete request that uses the VCF Automation IaaS service.

11. Optional: If you want to force deletion, use `forceDelete=true` in the VCF Automation IaaS service request to delete the deployment.

```
curl -X DELETE \
"$url/iaas/api/deployments/$deployment_id?forceDelete=true&apiVersion=$api_version_iaas" \
-H "Authorization: Bearer $access_token" | jq "."
```

12. Optional: Repeat steps 7, 8, and 9.

13. Optional: Examine the response to verify that "status": "SUCCESSFUL" appears.

Delete Your Deployment

This example shows how to delete a deployment with the VCF Automation Deployment service and when that deletion fails, force the deletion with the VCF Automation IaaS service.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_iaas=' '
```

List all deployments.

```
curl -X GET "$url/deployment/api/deployments?apiVersion=$api_version" -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the deployment that you want to delete.

```
{
  "id": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
  "name": "example_deployment",
  "description": "",
  ...
},
```

Assign your deployment variable.

```
deployment_id='164f2d4e-1755-491e-b0a0-583f0ed4ae3e'
```

List the actions available for your deployment.

```
$ curl -X GET \
"$url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the action "id": "Deployment.Delete" with "valid": true so you can delete the deployment.

```
...
{
  "id": "Deployment.Delete",
  "name": "Delete",
```

```

"displayName": "Delete",
"description": "Delete a deployment",
"valid": true,
"actionType": "RESOURCE_ACTION"
}
...

```

Delete the deployment.

```

curl -X DELETE \
"$url/deployment/api/deployments/$deployment_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows the ID which is the request ID.

```

...
"id": "d9541db3-2806-42aa-bde0-fb870d114833",
"name": "Delete",
"requestedBy": "user@mycompany.com",
"actionId": "Deployment.Delete",
"deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
"resourceIds": [
...
],
"status": "PENDING",
...

```

Assign the request ID variable.

```
$ request_id='d9541db3-2806-42aa-bde0-fb870d114833'
```

Check the status of the request.

```

$ curl -X GET \
"$url/deployment/api/requests/$request_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows that the request failed.

```

{
"id": "d9541db3-2806-42aa-bde0-fb870d114833",
"name": "Delete",
"requestedBy": "user@mycompany.com",
"actionId": "Deployment.Delete",
"deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
"resourceIds": [
...
],
"status": "FAILED",
...
}

```

If you are certain that you want to delete the deployment, force the deletion using the VCF Automation IaaS service.

```

curl -X DELETE \
"$url/iaas/api/deployments/$deployment_id?forceDelete=true&apiVersion=$api_version_iaas" \
-H "Authorization: Bearer $access_token" | jq "."

```

Examine the response to get the new request ID.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Delete",
  "id": "a1b674a2-09aa-4b14-9459-35ddddd9bbc1",
  "selfLink": "/iaas/api/request-tracker/a1b674a2-09aa-4b14-9459-35ddddd9bbc1"
  ...
}
```

Assign the new request ID.

```
$ new_request_id='a1b674a2-09aa-4b14-9459-35ddddd9bbc1'
```

Use the VCF Automation Deployment service to check the status of the request.

```
$ curl -X GET \
"$url/deployment/api/requests/$new_request_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to verify that the action is in progress.

```
{
  "id": "a1b674a2-09aa-4b14-9459-35ddddd9bbc1",
  "name": "Delete",
  "requestedBy": "user@mycompany.com",
  "actionId": "Deployment.Delete",
  "deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
  "resourceIds": [
    ...
  ],
  "status": "INPROGRESS",
  ...
}
```

Continue to check the status of the request.

```
$ curl -X GET \
"$url/deployment/api/requests/$new_request_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

The response shows when the deployment deletion request is successful.

```
{
  "id": "a1b674a2-09aa-4b14-9459-35ddddd9bbc1",
  "name": "Delete",
  "requestedBy": "user@mycompany.com",
  "actionId": "Deployment.Delete",
  "deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
  "resourceIds": [
    ...
  ],
  "status": "SUCCESSFUL",
  ...
}
```

Reconfigure Load Balancer

To reconfigure the load balancer in your deployment, you use the Deployment APIs to make a POST request with the ID of the load balancer to update.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the deployment you want to reconfigure. See [Deploy a Cloud Template with Contents Inline](#).
- Verify that you have the ID of the load balancer in your deployment. See [Get Deployment Resource IDs](#).

1. Assign your deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

2. Assign your load balancer ID variable.

```
load_balancer_id='<your_load_balancer_id>'
```

3. Get a list of actions available for the load balancer in your deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

4. Examine the response.

- Confirm that you see the action "name": "LoadBalancer.Reconfigure".
- "valid":true indicates that the action is valid for the deployment resource.

5. Assign the action ID variable for the reconfigure action "name": "LoadBalancer.Reconfigure".

```
reconfigure_action_id='Cloud.LoadBalancer.LoadBalancer.Reconfigure'
```

6. List the resource actions for the action ID .

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions/$reconfigure_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

7. Examine the response. The schema field shows the format of the input for an action on the load balancer resource.

8. Reconfigure the load balancer with input inline.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
    "inputs": {
      "routes": [
        {
          "port": "81",
          "protocol": "TCP",
          "instancePort": "81",
          "instanceProtocol": "TCP",
          "healthCheckConfiguration": {
            "port": "81",
```



```

        "urlPath": "/index.html",
        "protocol": "HTTP",
        "timeoutSeconds": 5,
        "intervalSeconds": 60,
        "healthyThreshold": 2,
        "unhealthyThreshold": 5
    }
}
]
}
}' | jq "."

```

9. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

10. Check the status of the request.

```

curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."

```

If the request is successful, the response shows "status": "SUCCESSFUL".

Reconfigure the Load Balancer in Your Deployment

For your deployment with ID 5551a299-8b67-45e3-909e-a638d11b0d9f, reconfigure the load balancer with resource ID d5b4569d-2234-4fc4-a594-45e6b0251588.

Assign variables.

```

$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'

```

Assign the load balancer ID.

```
$ load_balancer_id='d5b4569d-2234-4fc4-a594-45e6b0251588'
```

List the actions available for the load balancer resource.

```

$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows `LoadBalancer.Reconfigure` action.

```

...
{
  "id": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
  "name": "LoadBalancer.Reconfigure",
  "displayName": "Reconfigure",
  "description": "Reconfigure Load Balancer",
  "valid": true,
  "actionType": "RESOURCE_ACTION"
}
...

```

Assign the action ID variable.

```
$ reconfigure_action_id='Cloud.LoadBalancer.LoadBalancer.Reconfigure'
```

To get the schema for your action, list the resource actions by ID.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions/$reconfigure_action_id?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response provides the schema for the load balancing action.

```
...
"schema": {
  "type": "object",
  "title": "Reconfigure Load Balancer",
  "description": "Request schema for updating routes of load balancer resource",
  "properties": {
    "routes": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "protocol": {
            "type": "string",
            "title": "Protocol",
            "description": "The communication protocol for an incoming request to the load balancer. HTTP, HTTPS, or TCP.",
            "enum": [
              "HTTP",
              "HTTPS",
              "TCP"
            ]
          },
          "port": {
            "type": "string",
            "title": "Port",
            "description": "The listening port for an incoming request to the load balancer.",
            "pattern": "^\\d+$"
          },
          "instanceProtocol": {
            "type": "string",
            "title": "Instance protocol",
            "description": "The communication protocol used between the load balancer and the machines in the pool. HTTP, HTTPS, or TCP.",
            "enum": [
              "HTTP",
              "HTTPS",
              "TCP"
            ]
          }
        }
      }
    }
  }
}
...
```

Submit a request to reconfigure the load balancer with new route properties.

```
$ curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/requests \
```

```
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
-d '{
  "actionId": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
  "inputs": {
    "routes": [
      {
        "port": "81",
        "protocol": "TCP",
        "instancePort": "81",
        "instanceProtocol": "TCP",
        "healthCheckConfiguration": {
          "port": "81",
          "urlPath": "/index.html",
          "protocol": "HTTP",
          "timeoutSeconds": 5,
          "intervalSeconds": 60,
          "healthyThreshold": 2,
          "unhealthyThreshold": 5
        }
      }
    ]
  }
}' | jq "
```

A snippet of the response shows request ID.

```
...
  "id": "7342a348-65e0-4376-9472-94be56b928a9",
  "name": "Reconfigure",
  "deploymentId": "13c04d0a-fd81-4bcc-99b1-ac499fb1821d",
...
```

Assign the request ID variable.

```
$ request_id='342a348-65e0-4376-9472-94be56b928a9'
```

Check the status of the request.

```
$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" | jq "
```

A snippet of the response shows that the request was successful.

```
...
  "actionId": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
  "completedTasks": 1,
  "totalTasks": 1,
  "status": "SUCCESSFUL",
}
```

Add a Disk to a Machine and Power It Off

To add a disk to a machine in your deployment, you use the Deployment APIs to make a POST request with the ID of the virtual machine to update. To power off the machine, you make a POST request and specify the action to perform.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID of the deployment you want to reconfigure. See [Deploy a Cloud Template with Contents Inline](#).
- Verify that you have the ID of the virtual machine in your deployment. See [Get Deployment Resource IDs](#).

1. Assign your deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

2. Assign your virtual machine ID variable.

```
virtual_machine_id='<your_virtual_machine_id>'
```

3. Get a list of actions available for the virtual machine in your deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions?apiVer-
  sion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

4. Examine the response.

- Confirm that you see the action `Add.Disk` with `"valid":true`. Copy the value to assign to the add disk action ID.
- Confirm that you see the action `PowerOff` with `"valid":true`. Copy the value to assign to the power off action ID.

`"valid":true` indicates that each action is valid for the deployment resource.

5. Assign variables for the resources.

```
add_disk_action_id='<your_add_disk_id>'
poweroff_machine_action_id='<your_poweroff_action_id>'
```

6. List the resource actions for the add disk action ID.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions/$reconfigure_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

7. Examine the response. The schema field shows the format of the input for an action on the virtual machine resource.

8. Attach a disk of size 1 GB to the machine.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId":"Cloud.AWS.EC2.Instance.Add.Disk",
    "inputs":{
      "name":"disk1",
      "capacityGb":1,
      "type":"Cloud.Volume"
    }
  }' | jq "."
```

9. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

10. Check the status of the request.

```
curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

If the request is successful, the response shows "status": "SUCCESSFUL".

11. List the resource actions for the power off action ID.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions/$poweroff_ma-
chine_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

12. Examine the response. No schema field indicates that no inputs field is required for this action on the virtual machine resource.**13. Power off the machine.**

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId": "Cloud.AWS.EC2.Instance.PowerOff"
  }' | jq "."
```

14. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

15. Check the status of the request.

```
curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

If the request is successful, the response shows "status": "SUCCESSFUL".

Add a Disk and Power Off Your Virtual Machine

For your deployment with ID 5551a299-8b67-45e3-909e-a638d11b0d9f, reconfigure the virtual machine with resource ID 42f49781-1490-4a08-ae21-8baf383a72ac by adding a disk and powering it off.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'
```

Assign the virtual machine ID.

```
$ virtual_machine_id='42f49781-1490-4a08-ae21-8baf383a72ac'
```

List the actions available for the virtual machine resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions?apiVer-
sion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the actions to Add.Disk and PowerOff.

```

...
{
  "id": "Cloud.AWS.EC2.Instance.Add.Disk",
  "name": "Add.Disk",
  "displayName": "Add Disk",
  "description": "Add a disk to the machine",
  "valid": true,
  "actionType": "RESOURCE_ACTION"
},
...
{
  "id": "Cloud.AWS.EC2.Instance.PowerOff",
  "name": "PowerOff",
  "displayName": "Power Off",
  "description": "Power off a machine",
  "valid": true,
  "actionType": "RESOURCE_ACTION"
},
...

```

Assign the action ID variables to add a disk and power off the virtual machine.

```

$ add_disk_action_id='Cloud.AWS.EC2.Instance.Add.Disk'
$ power_off_action_id='Cloud.AWS.EC2.Instance.PowerOff'

```

Get the add disk action for the virtual machine resource.

```

$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions/$add-
  disk_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response provides the schema to add a disk.

```

...
"properties": {
  "name": {
    "type": "string",
    "title": "Name",
    "description": "Disk Name",
    "minLength": 1
  },
  "capacityGb": {
    "type": "integer",
    "title": "Size(GB)",
    "description": "Disk Capacity in GB",
    "minimum": 1
  },
  "type": {
    "type": "string",
    "title": "Type",
    "description": "Disk Resource Type.",
    "readOnly": true,
    "default": "Cloud.Volume"
  },
}

```

```
...
```

Follow the schema and submit a request to add a 1 GB disk to the virtual machine.

```
$ curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId": "Cloud.AWS.EC2.Instance.Add.Disk",
    "inputs": {
      "name": "disk1",
      "capacityGb": 1,
      "type": "Cloud.Volume"
    }
  }' | jq "
```

A snippet of the response shows request ID.

```
...
  "id": "17dec8d9-2e2a-4c29-9067-ce41c37be7a3",
  "name": "Add Disk",
  "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
...
```

Assign the request ID variable.

```
$ request_id='17dec8d9-2e2a-4c29-9067-ce41c37be7a3'
```

Check the status of the request.

```
$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "
```

A snippet of the response shows that the request was successful.

```
...
  "actionId": "Cloud.AWS.EC2.Instance.Add.Disk",
  "completedTasks": 3,
  "totalTasks": 3,
  "status": "SUCCESSFUL",
}
```

Get the power off action for the virtual machine resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions/$poweroff_machine_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "
```

The complete response shows that there is no schema for the power off action.

```
{
  "id": "Cloud.AWS.EC2.Instance.PowerOff",
  "name": "PowerOff",
  "displayName": "Power Off",
  "description": "Power off a machine",
  "dependents": [
```

```

    "Provider_LoadBalancer_1"
  ],
  "valid": true,
  "actionType": "RESOURCE_ACTION"
}

```

Power off the virtual machine.

```

$ curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId": "Cloud.AWS.EC2.Instance.PowerOff"
  }' | jq "."

```

A snippet of the response shows request ID.

```

...
  "id": "ab7d3aec-f850-4b0e-9c1c-47378c182a00",
  "name": "Power Off",
  "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
...

```

Assign the request ID variable.

```
$ request_id='ab7d3aec-f850-4b0e-9c1c-47378c182a00'
```

Check the status of the request.

```

$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."

```

A snippet of the response shows that the request was successful.

```

...
  "actionId": "Cloud.AWS.EC2.Instance.PowerOff",
  "completedTasks": 1,
  "totalTasks": 1,
  "status": "SUCCESSFUL",
  "inputs": {}
}

```

Viewing Billable Objects

As an VCF Automation or VCF Automation administrator, you can use the VCF Automation Deployment APIs to view billable objects that are used in your organization.

Prerequisites for viewing billable objects

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See [Prerequisites for API Use Case Examples](#).

How to get a summary of billable objects

The following request retrieves a categorized summary or total count of VCF Automation billable objects in your organization.

```
curl -X GET \
$url/deployment/api/billing-metrics?apiVersion=$api_version \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" | jq "."
```

The following sample response shows all billable objects.

```
{
  "billingMetrics": [
    {
      "id": "vSphereManagedVMCount",
      "displayName": "vSphere Managed VM Count",
      "value": 19
    },
    {
      "id": "vSphereCpuCount",
      "displayName": "vSphere CPU Count",
      "value": 20
    },
    {
      "id": "vSphereCpuCoreCount",
      "displayName": "vSphere CPU Core Count",
      "value": 52
    },
    {
      "id": "vmcManagedVMCount",
      "displayName": "VMC Managed VM Count",
      "value": 0
    },
    {
      "id": "vmcCpuCount",
      "displayName": "VMC CPU Count",
      "value": 0
    },
    {
      "id": "vmcCpuCoreCount",
      "displayName": "VMC CPU Core Count",
      "value": 0
    },
    {
      "id": "publicCloudManagedVMCount",
      "displayName": "Public Cloud Managed VM Count",
      "value": 58
    }
  ]
}
```

How to get information about billable objects

By applying the filter `billable=true`, the following request limits the amount of output from the call to include only the billable objects that are in your organization.

```
curl -X GET \
"$url/deployment/api/resources?apiVersion=$api_version&billable=true" \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
| jq "."
```

The following sample response snippet provides the details about one of the billable objects.

```
{
  "content": [
    {
      "id": "9bc16a26-bd81-4660-9aa6-6eadfe72456d",
      "name": "Cloud_Machine_1-mcm1157065-230404373562",
      "type": "Cloud.AWS.EC2.Instance",
      "properties": {
        "hostName": "",
        "resourceId": "9bc16a26-bd81-4660-9aa6-6eadfe72456d",
        "externalLink": "https://us-east-1.console.aws.amazon.com/ec2/v2/home?region=us-east-1#Instances:instanceId=i-0c7ec71b0aa26e080;sort=instanceState",
        "project": "7a90a6fb-f78d-4b91-9b3c-d88d3dd897fd",
        "storage": {
          "disks": [
            {
              "iops": "100",
              "name": "Cloud_Machine_1-mcm1157065-230404373562-boot-disk",
              "type": "SSD",
              "service": "ebs",
              "regionId": "us-east-1",
              "bootOrder": 1,
              "encrypted": false,
              "capacityGb": 30,
              "deviceName": "/dev/sda1",
              "deviceType": "ebs",
              "persistent": false,
              "snapshotId": "snap-00bbf435c29dcdf5c",
              "volumeType": "gp2",
              "endpointType": "aws",
              "resourceLink": "/resources/disks/3525c26f-30e9-4070-bb1d-a53430594f84",
              "SourceTaskLink": "/provisioning/resource-enumeration-tasks",
              "existingResource": "false"
            }
          ]
        }
      }
    },
    {
      "networks": [
        {
          "id": "/resources/network-interfaces/25ff21a4-8f79-4d77-861f-03148e532823",
          "name": "public-subnet-us-east-1c",
          "tags": [],
          "address": "172.31.84.7",
          "assignment": "dynamic",

```

```

        "deviceIndex": 0,
        "mac_address": "12:f0:45:3a:bd:1f",
        "resourceName": "public-subnet-us-east-1c",
        "securityGroupNames": [
            "photon-model-sg"
        ]
    },
    ],
    "powerState": "OFF",
    "zone": "us-east-1c",
    "environmentName": "Amazon Web Services",
    "providerId": "i-0c7ec71b0aa26e080",
    "osType": "WINDOWS",
    "id": "/resources/compute/9bc16a26-bd81-4660-9aa6-6eadfe72456d",
    "cpuCount": 1,
    "isSimulate": "false",
    "image": "small",
    "totalMemoryMB": 1024,
    "componentType": "Cloud.AWS.EC2.Instance",
    "awsVpcId": "vpc-adf7f2d5",
    "imageId": "ami-0bad85cc4b10fde5e",
    "endpointType": "aws",
    "address": "",
    "endpointId": "f2a31ae2-6256-4d2e-b72e-4687d57ce246",
    "externalId": "i-0c7ec71b0aa26e080",
    "resourceName": "Cloud_Machine_1-mcm1157065-230404373562",
    "tags": [
        {
            "key": "ll"
        },
        {
            "key": "sdf"
        }
    ],
    ],
    "rootDeviceType": "ebs",
    "primaryMAC": "12:f0:45:3a:bd:1f",
    "flavor": "aws-flavor",
    "service": "ec2",
    "name": "Cloud_Machine_1",
    "flavorRef": "t2.micro",
    "accounts": [
        "aws-vaidehi",
        "aws-skarwa"
    ],
    ],
    "region": "us-east-1",
    "flavorMappingName": "aws-flavor",
    "account": "aws-vaidehi"
},
"createdAt": "2023-04-20T17:12:54.348099Z",
"syncStatus": "SUCCESS",
"origin": "DEPLOYED",
"deploymentId": "f588a723-65f4-402f-b548-a13cdd1b74d0",
"projectId": "7a90a6fb-f78d-4b91-9b3c-d88d3dd897fd",

```

```

    "orgId": "79395484-98e2-450e-b1ab-095d6dae71ef",
    "billable": true
  },
  ...

```

Onboarding virtual machines

To onboard machines that are not yet managed by an VCF Automation project, you use the Onboarding APIs.

The onboarding process begins with creating an onboarding plan that locates the machines to be onboarded and creates a deployment for those machines when the plan runs. When you add deployments to the plan, you have the option of adding the deployment with a blueprint that allows all day 2 actions on the deployment.

Onboard machines as a single deployment

As an administrator, you can use the Onboarding APIs to onboard unmanaged machines as a single VCF Automation deployment so that you can manage the machines. To onboard machines, you:

- Verify that all general prerequisites and prerequisites for the Onboarding service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the ID for the cloud account with cloud zones where the machines to be onboarded are located. See [Adding Cloud Accounts](#) and [Create a Cloud Zone](#).
- Verify that you have the ID for a project with at least one user and with access to the cloud zones. See [Create a Project to use in VCF Automation](#)
- Create an onboarding plan.
- Query for unmanaged machines.
- Create a bulk deployment that adds the unmanaged machines.
- Run the plan.

When you run the plan you can check the plan progress to see if the plan completes successfully and onboards the machines.

The following procedure shows how to onboard machines and create a deployment in VCF Automation without using a blueprint. This is the fastest way to onboard machines. However, to run the **Update** day 2 action on the deployment after onboarding, you must create the deployment with a blueprint. See [How do I add a cloud template to my onboarding plan](#).

1. Create your onboarding plan without placements.

In this example,

- project ID: 4c4f8a47-d746-43f4-b88c-ea94ac8bd573
- cloud account ID: 0cbc8179-d202-4a30-9460-c25d2653a677

```

curl -X POST \
  $url/relocation/onboarding/plan \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "'<your_plan_name>' ",
    "projectId": "4c4f8a47-d746-43f4-b88c-ea94ac8bd573",
    "endpointIds": [
      "0cbc8179-d202-4a30-9460-c25d2653a677"
    ],
    "usePlacements": false
  }'

```

```
} ' | jq -."
```

Note: To create an onboarding plan with placements, set `"usePlacements": true`. With placement turned on, the onboarded workload considers resource limits defined in your project cloud zones or resource quota policy and takes longer to run.

Examine the response to verify the status, the project ID, the cloud ID, and to get the `documentSelfLink`, for example:

```
{
  "status": "OK",
  "nextRefreshTimeMicros": 0,
  "refreshIntervalMicros": 0,
  "name": "Plan-782",
  "projectId": "4c4f8a47-d746-43f4-b88c-ea94ac8bd573",
  "endpointIds": [
    "0cbc8179-d202-4a30-9460-c25d2653a677"
  ]
  ...
  "documentSelfLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
  ...
}
```

The value for the `documentSelfLink` is used for the `planLink` in subsequent steps.

2. Use the `planLink` to discover the compute resources representing unmanaged machines.

```
curl -X POST \
  $url/relocation/api/wo/query-unmanaged-machine \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
    "expandFields": [
      "id",
      "documentSelfLink",
      "name",
      "powerState",
      "address",
      "creationTimeMicros",
      "expandedTags",
      "tagLinks",
      "endpointLinks"
    ],
    "optionExcludePlanMachines": true
  }' | jq -."
```

Examine the response to get the snippet for unmanaged machines with `"powerState": "ON"`, for example:

```
...
"/resources/compute/8bff887d-56fd-3752-bb99-d3f32a35abde": {
  "address": "10.10.10.17",
  "creationTimeMicros": 1717409843457000,
  "documentSelfLink": "/resources/compute/dd646fdc-e6db-3b16-a72e-d1c905f7a9c8",
  "endpointLinks": "/resources/endpoints/aa6a5445-e63a-4659-a277-00073a2645c8",
  "expandedTags": [],
  "id": "5017d984-aa61-e00e-dfa6-a986403c01b8",
  "name": "Machine-76",
  "powerState": "ON",
```

```

    "tagLinks": []
  }
  "/resources/compute/9bc16a26-bd81-4660-9aa6-6eadfe72456d": {
    "address": "10.10.2.243",
    "creationTimeMicros": 1717409843457000,
    "documentSelfLink": "/resources/compute/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
    "endpointLinks": "/resources/endpoints/2d57f1c4-fbe0-46ce-a061-3a5f1396f37b",
    "expandedTags": [],
    "id": "a6f324e4-101c-33f6-ac51-d9aaaa020123",
    "name": "Machine-25",
    "powerState": "ON",
    "tagLinks": []
  }...

```

3. Use the planLink, documentSelfLink, and name to create a single deployment for the unmanaged machines.

```

curl -X POST \
  $url/relocation/onboarding/task/create-deployment-bulk \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
    "deployments": [
      {
        "name": "<your_deployment_name>",
        "resources": [
          {
            "link": "/resources/compute/dd646fdc-e6db-3b16-a72e-d1c905f7a9c8",
            "name": "Machine-76"
          },
          {
            "link": "/resources/compute/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
            "name": "Machine-25"
          }
        ]
      }
    ]
  }' | jq "."

```

The response shows that the discovered machines have been assigned to the deployment.

```

{
  "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
  "deployments": [
    {
      "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
      "name": "Deployment-562",
      "resources": [
        {
          "link": "/resources/compute/dd646fdc-e6db-3b16-a72e-d1c905f7a9c8",
          "name": "Machine-76"
        },
        {
          "link": "/resources/compute/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
          "name": "Machine-25"
        }
      ]
    }
  ]
}

```

```

    }
  ],
  "tenantLink": "/core/tenants/dc96003b28356c75",
  "documentVersion": 0,
  "documentUpdateTimeMicros": 0,
  "documentExpirationTimeMicros": 0
}
],
"tenantLink": "/core/tenants/dc96003b28356c75"
}

```

4. Use the planLink to run the plan and onboard the machines.

```

curl -X POST \
  $url/relocation/api/wo/execute-plan \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
  }' | jq "."

```

5. Use the planLink to check the run.

```

curl -X GET \
  $url/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json'
| jq "."

```

The status in the response changes as the plan moves through `executing` or `updating`. If the run fails, the status shows `"status": "error"`.

```

{
  "status": "OK",
  "nextRefreshTimeMicros": 0,
  "refreshIntervalMicros": 0,
  "name": "Plan-782",
  "projectId": "4c4f8a47-d746-43f4-b88c-ea94ac8bd573",
  "endpointIds": [
    "0cbc8179-d202-4a30-9460-c25d2653a677"
  ],
  "createdBy": "example_admin@example_company.com",
  "enableExtensibilityEvents": true,
  "organizationId": "174adb59-8132-46f0-9cd8-2b2361e9cb2c",
  "customProperties": {},
  "usePlacements": false,
  "isQuick": false,
  "tenantLink": "/core/tenants/dc96003b28356c75",
  "documentVersion": 0,
  "documentKind": "com:vmware:relocation:services:onboarding:plan:OnboardingPlanService:OnboardingPlanS-
tate",
  "documentSelfLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
  "documentUpdateTimeMicros": 1718190629415000,
  "documentUpdateAction": "POST",
  "documentExpirationTimeMicros": 0,
  "documentAuthPrincipalLink": "/core/authz/users/example_admin@example_company.com"
}

```

```
}
```

How do I add a cloud template to my onboarding plan

When creating an onboarding plan, you can attach an existing VMware cloud template (VCT) to the deployment. The cloud template allows you to iteratively update the deployment and avoid the need to delete onboarded resources and create new ones while gradually scaling out your deployment.

With a cloud template attached to your deployment, you can run all day 2 actions including the **Update** day 2 action provided that the YAML of the selected cloud template accepts user input.

Note: You can choose any cloud template but depending on the cloud template details, an **Update** day 2 action might delete and re-create original resources.

By attaching a cloud template with resource mapping, you can create an onboarding deployment that maps VMs to template resources in the same way as the deployment that was originally provisioned in VCF Automation using the same cloud template. The only restriction is that the onboarding deployment must have the same number of machines as the selected cloud template and the admin must specify the VM to template resource mapping.

A deployment that uses a cloud template with resource mapping includes the **Update** day 2 action if the selected template accepts user input and the update only applies changes to existing resources. It will not re-create the original resources unless there are major changes between the current version of cloud template and original deployment. Onboarded deployments can also be iteratively updated without deletion and re-creation of the onboarded machines.

Prerequisites for adding deployments with a cloud template

- Verify that all general prerequisites and prerequisites for the Onboarding service have been satisfied. See [Prerequisites for API Use Case Examples](#).
- Verify that you have the `planLink` for an existing onboarding plan. For example, to list onboarding plans use `GET /relocation/api/onboarding/plan`. See [Onboard machines as a single deployment](#).
- Verify that you have the IDs and names of the resources that you are planning to onboard. For example, to list unmanaged machines, use `POST /relocation/api/wo/query-unmanaged-machine`. See [Onboard machines as a single deployment](#).
- Verify that the VMware cloud template that you plan to attach exists. For example, to list cloud templates use `GET /blueprint/api/blueprints`.
- If adding a cloud template with mapping, verify that you have the correct template component name to link to each machine. For example:
 - To get template component names, use `GET /blueprint/api/blueprints/<blueprint_id>` where `blueprint_id` is the ID for your cloud template. In the response, the `blueprint content` field lists the machine component names.
 - To list unmanaged machines, use `POST /relocation/api/wo/query-unmanaged-machine`.

Example payloads include the following input:

- planLink for existing onboarding plan


```
"planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
```
- Template to attach if onboarding with template but without mapping


```
"name": "singleMachines"
"link": "/blueprint/api/blueprints/b79612a9-6e2f-4b48-a775-88eb7676dc2c"
```
- Template to attach if onboarding with template and with mapping


```
"template": {
  "name": "vc01",
  "link": "/blueprint/api/blueprints/74bbcdac-b945-4a95-ac86-8bed0a75e2c2",
```



```

    "components":
    {
        "/resources/compute/bf07d26a-38cd-392e-b3d6-0004cf36168a": "Cloud_vSphere_Machine_1",
        "/resources/compute/cc628c21-5531-414d-a09e-5f35916ce689": "Cloud_vSphere_Machine_2"
    }
}

```

How do I add a deployment with a cloud template

The following example adds a deployment with the template `singleMachine` to bring the VMs `East-centos-small-000011` and `est` under management.

```

curl -X POST \
  $url/relocation/onboarding/task/create-deployment-bulk \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "deployments": [
      {
        "resources": [
          {
            "link": "/resources/compute/b82870f1-575b-308b-ac2e-874fad85703b",
            "name": "East-centos-small-000011",
            "tagLinks": []
          },
          {
            "link": "/resources/compute/e12fefb6-2d0a-3ee1-bb7f-1ceceb3d1ca6",
            "name": "est",
            "tagLinks": []
          }
        ],
        "template": {
          "name": "singleMachine",
          "link": "/blueprint/api/blueprints/b79612a9-6e2f-4b48-a775-88eb7676dc2c"
        }
      }
    ],
    "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
  }' | jq "."

```

How do I add a deployment with a cloud template and mapping

The following example adds a deployment with the template `vc01` to bring:

- VMs `VM-20` and `VM-21` under management with:
 - `VM-20` mapped to `Cloud_vSphere_Machine_1` in the template.
 - `VM-21` mapped to `Cloud_vSphere_Machine_2` in the template.
- VMs `VM-0014` and `VM-0015` under management with:
 - `VM-0014` mapped to `Cloud_vSphere_Machine_1` in the template.
 - `VM-0015` mapped to `Cloud_vSphere_Machine_2` in the template.

```

curl -X POST \
  $url/relocation/onboarding/task/create-deployment-bulk \
  -H "Authorization: Bearer $access_token" \

```

```

-H 'Content-Type: application/json' \
-d '{
  "deployments": [
    {
      "template": {
        "name": "vc01",
        "link": "/blueprint/api/blueprints/74bbcdade-b945-4a95-ac86-8bed0a75e2c2",
        "components":
chine_1",
          {
            "/resources/compute/bf07d26a-38cd-392e-b3d6-0004cf36168a": "Cloud_vSphere_Ma-
          {
            "/resources/compute/cc628c21-5531-414d-a09e-5f35916ce689": "Cloud_vSphere_Machine_2"
          }
        },
      "resources": [
        {
          "link": "/resources/compute/bf07d26a-38cd-392e-b3d6-0004cf36168a",
          "name": "VM-20",
          "tagLinks": []
        },
        {
          "link": "/resources/compute/cc628c21-5531-414d-a09e-5f35916ce689",
          "name": "VM-21",
          "tagLinks": []
        }
      ]
    },
    {
      "template": {
        "name": "vc01",
        "link": "/blueprint/api/blueprints/74bbcdade-b945-4a95-ac86-8bed0a75e2c2",
        "components":
chine_1",
          {
            "/resources/compute/2ce64191-867e-3bb2-9017-6f5aa5d3a3c2": "Cloud_vSphere_Ma-
            "/resources/compute/c4f46940-e4e1-45fb-bd30-58794398a50c": "Cloud_vSphere_Ma-
chine_2"
          }
        },
      "resources": [
        {
          "link": "/resources/compute/2ce64191-867e-3bb2-9017-6f5aa5d3a3c2",
          "name": "VM-0014",
          "tagLinks": []
        },
        {
          "link": "/resources/compute/c4f46940-e4e1-45fb-bd30-58794398a50c",
          "name": "VM-0015",
          "tagLinks": []
        }
      ]
    }
  ],
}
```

```
"planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
}' | jq ""
```

VCF Automation and All Apps API

Service providers can perform many VCF Automation tasks using an extensive API based on REST calls. After the provider administrator creates a tenant organization, organization administrators and DevOps programmers can automate many tasks using REST calls or Kubernetes commands.

Users who sign into the VCF Automation appliance have two choices of interface, both at the same fully qualified domain name (FQDN).

- Provider UI at <https://FQDN/provider> for the service provider administrator
- Automation UI at <https://FQDN/automation> for tenant organization users

It is the service provider administrator who sets up a region, organization, first user, IP spaces, provider gateway, and other networking resources for tenant organizations. The first user of a tenant organization can then act as administrator to create other users, projects, namespaces, policies, and content libraries.

Rather than employ the user interface to set up everything, certain frequently performed tasks can be automated with REST calls or Kubernetes commands.

About VCF Automation Applications

For those unfamiliar with VMware automation products, this topic offers an introduction to various use cases and configurations of VCF Automation.

VMware Cloud Foundation (VCF) helps customers build a software defined data center including compute, storage, network, management, and automation of private or hybrid clouds. Based on this infrastructure foundation, VCF Automation provides multi-cloud infrastructure to configure and automate deployment of traditional and modern applications, with best practices to enforce cost, security, and performance.

Formerly called Aria Automation, VCF Automation for VM Apps is only slightly changed and remains available for all VCF customers. Additionally, VCF 9.0 introduced VCF Automation for All Apps, with additional support for so-called modern applications (containers and custom resources) and continued support for traditional applications based on virtual machines.

VCF Automation for All Apps provides strong isolation for tenant applications, while VCF Automation for VM Apps implies weak isolation sufficient for guest applications. All Apps covers situations where users do not trust each other, requiring increased security and reduced resource sharing, to guard against data exfiltration and denial of service. VM Apps should be adequate for applications running in virtual machines, which provide inherent separation, but this is less satisfactory for micro-services running in Kubernetes containers.

REST calls for Service Provider Administration

REST calls for service providers are grouped by similarity into the categories below. Compared to VCF Automation for VM Apps, new categories were established for Custom Resource Types & Actions, and Instances.

For more information see the [VCF Automation for All Apps APIs](#) reference documentation.

Category	Description
Access Control	Identity operations for enabling groups, users, and roles.
Aggregator	Approvals, deployment limits, and work items.
Approvals	Enforce policies controlling who must agree to a deployment.
Blueprint	Create, validate, and provision blueprints.

Category	Description
Catalog	Access items and catalog sources for content sharing.
Content Gateway	Manage and synchronize external content sources.
Custom Forms	Define dynamic form rendering in automation services.
Custom Resource Types & Actions	Define form schemas for custom resources and actions.
Instances	Access instances and platforms or deployed blueprints.
Orchestrator Gateway	Run workflows and actions to automate complex tasks.
Policies	Interact with policies created in the Catalog.
Projects	Provide visibility and isolation of provisioned resources.
Provisioning Service	Enable integration and centralize configuration properties.

Missing categories that are available with VCF Automation for VM Apps are: ABX (action based extensibility), Deployment, Deployment Metrics, Identity, and Onboarding.

After you sign into the VCF Automation user interface, you can browse available REST calls by clicking **admin > API Help Center > Automation APIs** in the upper right corner.

Kubernetes and REST Calls for Organization Management

Kubernetes commands and REST calls for DevOps programmers in tenant organizations are grouped into the categories below. Administrators and developers can act on various resources in the Kubernetes service.

Depending on the kind of resource, administrators and programmers may call the API to perform the following actions.

Resource kind	Administrator action verbs	DevOps action verbs
Project	create, get, update, patch, delete, list	get, list
ProjectRole	get, list	get, list
ProjectRoleBinding	create, get, update, patch, delete, list	get, list
Region	get, list	
SupervisorNamespace	create, get, delete, list	get, list
SupervisorNamespaceClass	create, get, update, patch, delete, list	
SupervisorNamespaceClassConfig	create, get, update, patch, delete, list	

Here is an example of YAML for an organization administrator or DevOps programmer to create a supervisor namespace with `kubectl`, if the `SupervisorNamespaceClassConfig e2e-small` contains a `Zones` section:

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespace
metadata:
  generateName: test
  namespace: default-project
spec:
  regionName: e2e-region
  className: e2e-small
  vpcName: e2e-region-default-vpc
```

Here is a full list of the API resources for DevOps programmers calling `kubectl`. Abbreviations and API version path have been shortened.

```
% kubectl --context cci api-resources
```

NAME	APIVERSION	NAMESPACED	KIND
projectrolebindings	authorization...v1alpha1	true	ProjectRoleBinding
projectroles	authorization...v1alpha1	false	ProjectRole
regionmonitortokenrequests	infrastructure...v1alpha1	false	RegionMonitorTokenRequest
regionstorageclassquotas	infrastructure...v1alpha1	false	RegionStorageClassQuota
regionvirtualmachineclassquotas	infrastructure...v1alpha1	false	RegionVirtualMachineClassQuota
regionvirtualmachineclasssummaries	infrastructure...v1alpha1	false	RegionVirtualMachineClassSummary
supervisornamespaceclassconfigs	infrastructure...v1alpha2	false	SupervisorNamespaceClassConfig
supervisornamespaceclasses	infrastructure...v1alpha2	false	SupervisorNamespaceClass
supervisornamespaces	infrastructure...v1alpha2	true	SupervisorNamespace
virtualmachineremoteconsolerequests	infrastructure...v1alpha1	true	VirtualMachineRemoteConsoleRequest
vkscredentialrequests	infrastructure...v1alpha1	false	VksCredentialRequest
projects	project...v1alpha2	false	Project
regions	topology...v1alpha2	false	Region
zones	topology...v1alpha1	false	Zone

In addition to the `kubectl` command, organization administrators and DevOps programmers can use the `vcf` CLI. You can find a command reference for it in the manual *Installing and Using VCF CLI v9.0*, section "VCF CLI Context, Architecture and Configuration."

Provider Portal for Service Provider Management

The following topics describe a set of REST commands for setting up a service provider, emphasizing tasks that might be repeated frequently.

In the context of VCF Automation, a provider offers services to tenant organizations running virtual machines, developing operations software and microservices, or performing other tasks in a shared computing environment.

The provider interface creates the required organizations and configures underlying infrastructure including regions, IP spaces, gateways, storage, quotas, and users.

When the VCF Automation user interface (UI) first starts, it shows an inventory of VCF resources, a series of steps to get started, and Overview menus on the left for Infrastructure, Services, and Administration. Using REST calls, the provider administrator can automate tasks that might be repeated frequently, or tasks that are easier to accomplish from the command line.

Advantages of Service Provider Accounts

In a production environment with virtual machines and container based microservices, a service account is recommended, rather than a normal user account.

VMware introduced service provider accounts in 2022 as an alternative to user accounts, to enable standalone access for automation plug-ins and third-party applications. Some service provider accounts have system access only through an API, with no access to a user interface. (VCF Automation provider accounts can access a user interface.)

Service provider accounts have several advantages. They cannot be impersonated by anyone else, unless credentials are disclosed. Provider management accounts authenticate using API access tokens, which rotate after each use. Access tokens normally last one hour, and are based on a refresh token, which normally lasts 90 days but can be revoked earlier than that, and can be optionally rotated for each access.

When authenticating with a service provider account, applications normally cannot perform user management tasks, create API refresh tokens, or manage other provider management accounts.

Provider management accounts are created by a system administrator with privileges View Service Accounts and Manage Service Accounts. System administrators can create provider management accounts either from the VCF Automation user interface, or with the API using a REST call (POST ... register).

Provider management accounts are like user accounts. If allowed, they can create assets such as virtual machines, catalogs, templates, media, and automation sequences. Tasks initiated by provider accounts are properly attributed. User accounts can share assets with a provider account, and vice versa. System administrators can change provider account attributes, revoke previously granted access, and even delete the provider account, hopefully after checking to ensure assets are transferred if need be.

Before VCF Automation, provider accounts were used for tenant management and the tenant portal in VMware Cloud Director, which is now discontinued.

Prerequisites for Service Provider Portal

Although REST commands exist for attaching a vCenter and registering NSX Managers, such operations are usually accomplished using VCF Installer and Operations, while specifying the FQDN of connection endpoints.

You can verify vCenter and NSX Manager status using the VCF Automation UI:

1. Click Connections. The vCenter tab appears initially.
2. Name shows the vCenter endpoint, followed by other information such as License Status.
3. Click the tab for NSX Manager. When the tab appears, Name shows the NSX endpoint.
4. On the left, click Supervisors. The default Supervisor was created by vCenter registration.

The Supervisor shows its Region affiliation. Each NSX Manager (or load balancing set of NSX Managers) will be associated with a Region, as outlined below.

Before running any VCF Automation tasks, the provider automation script must sign in for proper authentication and authorization. See [Generating API Tokens for Service Provider Account](#) below.

Note: in commands below, `vcfa.vcf.example.com` represents the VCF Automation appliance's FQDN. Assume that all REST commands below are on a single line, or marked for continuation with backslash (Linux and MacOS shells) or backquote (PowerShell).

Create a Region

In VCF Automation, a Region is a set of one or more supervisors from one or more vCenter instances, all served by the same NSX Manager or load balancing NSX Managers. Regions are often geographic based, for example `us-west` and `us-east`. If two vCenters are associated with different NSX instances, you cannot add their supervisors in the same region.

This REST command associates `us-west` with an NSX Manager endpoint, including the default Supervisor.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/vcf/regions
-H 'Authorization: <value>' -H 'Content-Type: application/json'
-d '{"name":"us-west","nsxManager":"https://nsx-west.vcf.example.com",
  "supervisors":["supervisor"], "storagePolicies":["vSAN Default Storage Policy"]}'
```

When creating a Region, you must specify its name as a string, the endpoint URL of an NSX Manager instance, the endpoint URL of one or more supervisors, and one or more storagePolicies (for example, vSAN ESA Default Storage Policy).

By default, the Region contains a Zone specifying memory and CPU capacity. Zones are aggregated from all supervisors in a Region. The Region inherits a collection of VM Classes, giving a choice of VM sizes and guaranteed or best-effort performance. A Region starts with one Storage Class, but you can add more.

Networking Resources

A Provider does not require Internet connectivity, but connectivity is almost always useful.

For wide area networking, the Provider needs an Edge Cluster, an IP address Space with CIDR blocks (IPv4 only), and a Provider Gateway associated with the Region. Such a connected Organization is later set to `networkingTenancyEnabled`.

As a result of associating the Region with NSX, an Edge Cluster is auto-discovered. The Edge Cluster is a logical grouping (often two) of VCF networking Edge transport nodes, which provide network services and north-south routing for workloads. This command lists the nodes. Number of pages and page size are mandatory.

```
curl -X GET https://vcfa.vcf.example.com/cloudapi/vcf/edgeClusters?page=1&pageSize=128
-H 'Authorization: <value>'
```

You should know the IP address range that tenant organizations can use for Internet connections. The IP Spaces command sets that range for internal scope, and a security address for external reach.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/vcf/ipSpaces
-H 'Authorization: <value>' -H 'Content-Type: application/json'
-d '{"name":"us-west","externalScopeCidr":"0.0.0.0/0",
  "internalScopeCidrBlocks":[{"id":"us-west","name":"us-west","cidr":"23.215.0.0/16"}]}'
```

Later, you can set quotas for an organization limiting the number of single IPs, the number of CIDRs, and maximum subnet size. Alternatively, you can leave these unlimited.

After the region is given an IP address space, it must get a Provider Gateway. You specify a name for the gateway, a region to which it applies, and an available Tier-0 gateway for physical backing.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/vcf/providerGateways
-H 'Authorization: <value>' -H 'Content-Type: application/json'
-d '{"name":"us-west","regionRef":{"name":"us-west","id":"us-west"},
  "backingRef":{"id":"ContainerT0","name":"containerT0"}}'
```

Configure NSX Networking with the vSphere UI

As of VCF 9.1, administrators can manage and operate NSX networking through the vSphere user interface. Virtual public clouds (VPCs) are visible in the vCenter network inventory. Administrators can connect them to each other, or to an external network, using a transit gateway (TGW).

A virtual private cloud (VPC) is a configurable set of resources allocated in a cloud environment, providing isolation between different organizations sharing a cluster. A VPC is somewhat like a subnet, but a software construct, and a VPC can itself contain virtual subnets.

A transit gateway (TGW) connects one VPC to another, or to an external network such as the Internet. The external connection can be centralized (CTGW) going through an edge router, or distributed (DTGW) being directly connected to multiple VPCs.

The network span is a logical construct that defines which VPC and non-VPC clusters can connect to each other. By default, all vCenter clusters belong to the default network span. A cluster can belong to multiple network spans. A span can be made exclusive, so that a cluster set can be dedicated to a workload and not shared across the cloud environment.

To configure NSX networking with the vSphere Client user interface:

1. In the network view (globe on upper left) click *Datacenter* > Virtual Private Clouds > ACTIONS > New Transit Gateway. Enter a gateway name for the TGW. Accept the default span, or choose one that has been established. Select either None for internal connections, Centralized for CTGW, or Distributed for DTGW. Click SAVE.
2. To establish a network span, click Virtual Private Clouds > Configure > Settings > Network Spans > ADD NETWORK SPAN. Enter a name, choose Exclusive if desired, and select vSphere clusters in the span. Click SAVE.

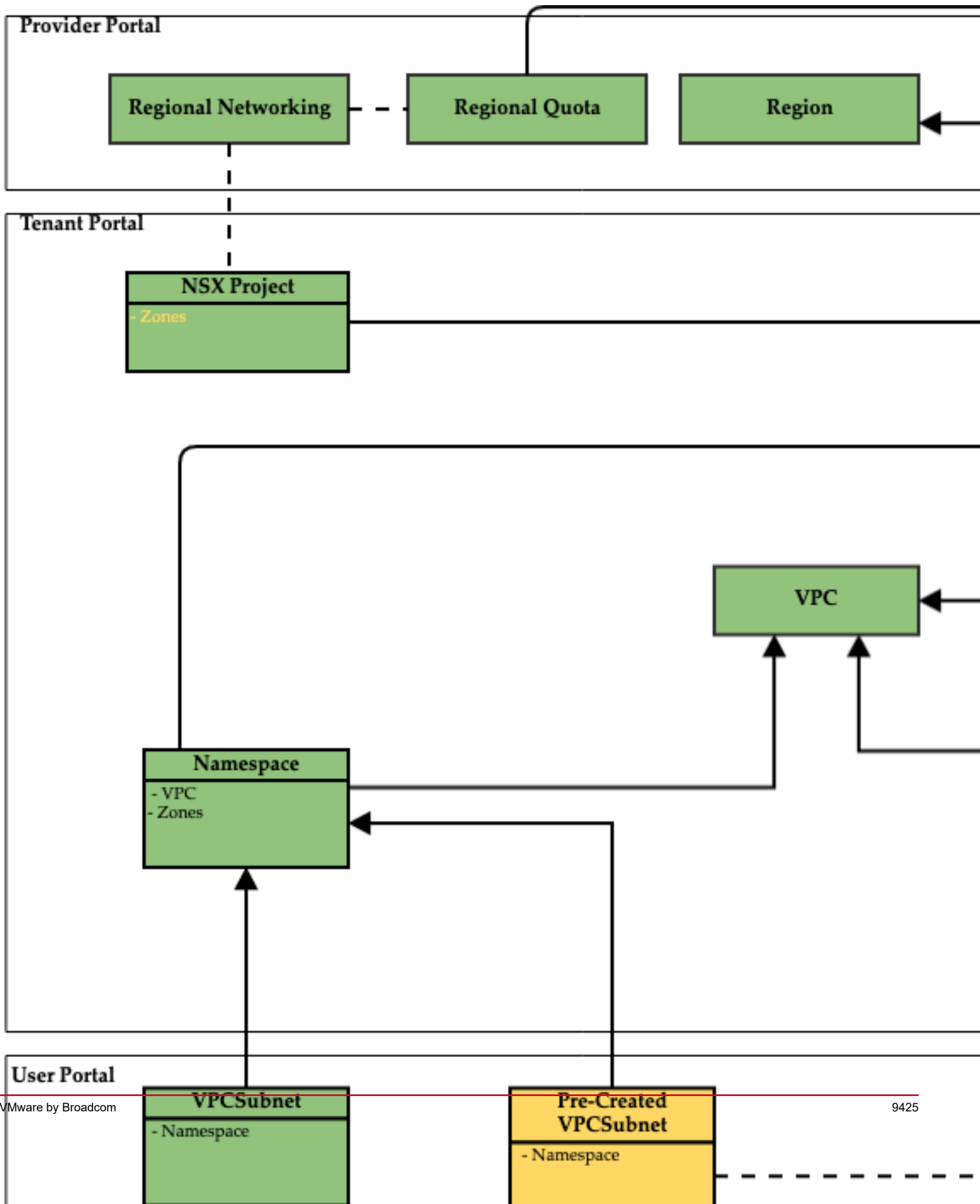
3. To configure VPC connectivity policy, click a VPC on far left, such as Default Transit Gateway. Then click Configure > Routing and Forwarding > Connectivity Policies > ADD POLICY. Enter a Name, choose a policy Type (Community, Isolated, Promiscuous), and select VPC Members that are connected to the TGW. Click SAVE.
4. Once you have configured NSX networking, you can troubleshoot VPC connectivity with live traffic analysis. Click Virtual Private Clouds > Monitor > Troubleshooting > Traceflow.
5. To set up cluster vNetworking, go to the Cluster view (rectangles on upper left). Click *Datacenter* > *Cluster* > Configure > [scroll] vNetworking > Network Configuration. If correctly configured, the cluster will be prepared with the default overlay transport zone for VPC readiness.

Provisioning a Distributed Transit Gateway

As of VCFA 9.1, organizations that require high-performance distributed routing and VLAN based connectivity can set up a distributed transit gateway (DTGW). In VCFA 9.0, organizations had to be configured with centralized transit gateway (CTGW).

The service provider administrator can offer distributed transit gateway (DTGW) as a networking option for organization tenants. The service provider can assign multiple VLAN connections to the same tenant, ensuring that the connection span overlaps with the zones assigned to the tenant's regional quota. Tenant organizations can then create multiple DTGWs.

Figure 662: Network diagram with distributed transit gateway



The distributed VLAN connection defines information about the external VLAN attachment used by a tenant's DTGWs, including the external VLAN ID and gateway address.

Regional network settings automatically provision either DTGW or CTGW based on the external connection and networking infrastructure that the provider assigned to the tenant. If virtual network appliance clusters exist in the region, a distributed VLAN connection can be selected as default for regional network settings. The provider can add additional ones, but the default distributed VLAN connection cannot be removed or replaced.

The following REST calls manage distributed VLAN connections. JSON describes a VLAN connection.

```
GET, POST          /cloudapi/vcf/distributedVlanConnections
GET, PUT, DELETE   /cloudapi/vcf/distributedVlanConnection/{id}

{
  "status": "REALIZED",
  "id": "urn:vcloud:distributedVlanConnection:94994a69-cc52-44b1-990d-f434d0d21847",
  "name": "dist1",
  "description": "",
  "vlanId": 111,
  "gatewayCidr": "10.1.1.24/24",
  "regionRef": {
    "name": "region2",
    "id": "urn:vcloud:region:6dd82079-aae0-4cbd-98f5-1d635816722f"
  },
  "zoneRefs": null,
  "ipSpaceRef": {
    "name": "mp-ipl-region2",
    "id": "urn:vcloud:ipSpace:d141d0bd-7048-46ea-824e-c71129244786"
  },
  "subnetExclusive": false,
  "backingId": "dist1",
  "dedicatedOrgRef": null,
  "dedicatedSharedSubnetRef": null
}
```

The following calls view and remove regional network settings. JSON describes regional network settings.

```
GET, PUT, POST, DELETE /vcf/regionalNetworkingSettings/urn:vcloud:regionalNetworkingSetting:<UUID>

{
  "status": "REALIZED",
  "id": "urn:vcloud:regionalNetworkingSetting:8381aebb-ab5a-4637-badd-7ca667bc540f",
  "name": "mp1-STFTTestOrgregion2",
  "regionRef": {
    "name": "region2",
    "id": "urn:vcloud:region:6dd82079-aae0-4cbd-98f5-1d635816722f"
  },
  "orgRef": {
    "name": "mp1-STFTTestOrg",
    "id": "urn:vcloud:org:c612alb8-c882-409a-bf71-5d5c7a97b3e5"
  },
  "providerGatewayRef": null,
  "distributedVlanConnectionRef": {
    "name": "dist1",
    "id": "urn:vcloud:distributedVlanConnection:94994a69-cc52-44b1-990d-f434d0d21847"
  },
  "serviceEdgeClusterRef": null,
}
```

```

"virtualNetworkApplianceClusterRef": {
  "name": "vna-cluster3",
  "id": "urn:vcloud:virtualNetworkApplianceCluster:27433d75-ca0d-4a51-a331-c4160a4c5fd3"
},
"defaultVpcPrivateSubnetCidrOverride": null,
"projectTgwPrivateSubnetCidrOverride": null
}

```

The following calls view and update distributed VLAN connections assigned to a region, one of which is default. The <id> is the NSX ID of the distributed VLAN connection. JSON describes a distributed VLAN connection.

```

GET, POST /vcf/regionalNetworkingSettings/urn:vcloud:regionalNetworkingSetting:<UUID>/distributedVlanConnections
GET, DELETE /vcf/regionalNetworkingSettings/urn:vcloud:regionalNetworkingSetting:<UUID>/distributedVlanConnections/urn:vcloud:distributedVlanConnections:<UUID>
{
  "status": "REALIZED",
  "id": "dist1",
  "distributedVlanConnectionRef": {
    "name": "dist1",
    "id": "urn:vcloud:distributedVlanConnection:94994a69-cc52-44b1-990d-f434d0d21847"
  },
  "isDefault": true,
  "vlanId": 111,
  "gatewayCidr": "10.1.1.24/24",
  "ipSpaceRef": {
    "name": "mp-ip1-region2",
    "id": "urn:vcloud:ipSpace:d141d0bd-7048-46ea-824e-c71129244786"
  }
}

```

Virtual network appliance (VNA) clusters are used to support stateful services for DTGWs. In the UI, it is the Virtual Network Appliance Cluster. (Edge clusters function similarly for CTGWs.) The first call is an empty POST to synchronize service clusters in tenant manager. The GET lists all service clusters in the system, as shown in the sample JSON return.

```

POST /vcf/serviceClusters/sync
GET /vcf/serviceClusters
{
  "status": null,
  "id": "urn:vcloud:virtualNetworkApplianceCluster:27433d75-ca0d-4a51-a331-c4160a4c5fd3",
  "name": "vna-cluster3",
  "description": null,
  "regionRef": {
    "name": "region2",
    "id": "urn:vcloud:region:6dd82079-aae0-4cbd-98f5-1d635816722f"
  },
  "nodeCount": 2,
  "orgCount": 0,
  "healthStatus": "UP",
  "defaultQosConfig": null,
  "backingRef": {
    "id": "vna-cluster3",
    "name": "vna-cluster3"
  }
}

```

```
}
```

Avi Load Balancer Operations

Service providers can perform day 2 operations for load balancing using REST calls to the Avi load balancer (formerly NSX Advanced load balancer)).

Avi load balancer performs local and global server load balancing, web application firewall security, and ingress services for container-based environments. Avi load balancer was incorporated into NSX networking years ago, and now it is incorporated into VMware Cloud Foundation.

When active, VCF Automation discovers Avi controllers and their associations with NSX Manager. VCF Automation also discovers provider-managed service groups from the Avi controller. Assignment of a service group to a Region is then determined by the load balancer's association with NSX manager.

The load balancer service engine (SE) is an appliance that requires an IP address in each virtual service network. Service engines are created within a group, which defines how SEs are sized, placed, and made available. A service engine group is a collection of SEs that share properties such as network access and failover. Applications are guaranteed data plane isolation when deployed in separate SE groups.

To query the discovered Avi controllers, the following GET call returns a paged list of Avi controllers:

```
curl --request GET \
  --url https://<vcfa.ip>:8443/cloudapi/1.0.0/loadBalancer/controllers \
  --header 'accept: application/json;version=40.0' \
  --header 'authorization: Bearer <token>' \
  --header 'content-type: application/json;version=40.0'
[response includes]
  "id": "urn:vcloud:loadBalancerController:4887b3c0-7870-492f-8169-7758bc83c76f"
```

Once you have the URN of a particular load balancer, you can call `controllers` again to obtain more information about that controller.

```
GET /1.0.0/loadBalancer/controllers/{aviUrnID}
```

This call adds and registers an Avi load balancer, given a JSON definition.

```
POST /1.0.0/loadBalancer/controllers
```

This call updates an Avi load balancer, given a JSON definition.

```
PUT /1.0.0/loadBalancer/controllers/<aviUrnID>
```

This call deletes an Avi load balancer.

```
DELETE /1.0.0/loadBalancer/controllers/<aviUrnID>
```

As of VCF 9.1, the Service Engine Group name (`segName` field) in `SupervisorNamespace.spec` can be updated with PATCH, after the namespace is created.

Create an Organization

Organization is the VCF Automation term for tenant organization.

To query organizations that already exist, run the following command. Number of pages and page size are mandatory.

```
curl -X GET https://vcfa.vcf.example.com/cloudapi/1.0.0/orgs?page=1&pageSize=128
-H 'Authorization: <value>'
```

To create an organization, specify an org name and its display name. Other optional data parameters include description and ID, which is auto-filled if not specified.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/1.0.0/orgs
-H 'Authorization: <value>' -H 'Content-Type: application/json'
-d '{"displayName":"Test One","name":"test1"}'
```

As you would expect, there are also REST commands to get organization details, update details, and delete an organization. Organization details include `canManageOrgs`, `creationStatus`, `directlyManagedOrgCount`, `isClassicTenant`, `isEnabled`, `isProviderConsumptionOrg`, `managedBy`, and `maskedEventTaskUsername`.

The newly created organization needs a first-user (org admin) with the Organization Administrator role. The first-user is always local, with appliance-specific password rather than identity provider account. For subsequent users, you call a REST command that specifies both organization ID (`test1`) and role.

```
curl -X POST https://{api_host}/cloudapi/1.0.0/users
-H 'Authorization: <value>' -H 'Content-Type: application/json'
-d '{"username":"admin","orgEntityRef":"test1","roleEntityRefs":["OrgAdmin"]}'
```

The organization can be associated with a regional network by assigning a quota policy to it, while specifying the quota name and the organization ID.

```
curl -X POST https://{api_host}/cloudapi/1.0.0/quotaPolicies
-H 'Authorization: <value>' -H 'Content-Type: application/json'
-d '{"name":"tlq","orgId":"test1","quotaPoolDefinitions":[{"resourceType":"quota","quota":0}]}'
```

Create a VM Apps Organization

To create a VM Apps organization using the API, you use the Provider Management APIs. See [Org APIs](#).

Before performing the task to create the VM Apps organization, obtain an access token. See [Generating API Tokens for Service Provider Account](#).

The following example shows how to create a VM Apps organization with:

- **Name:** `vmApp-org-1`
- **Display name:** `vmApp-org-1 tenant`
- **Description:** `Additional VM App Org Tenant`

To create multiple organizations, repeat the command.

```
curl POST \
  'https://<vcfa.vcf.example.com>/cloudapi/1.0.0/orgs'\
-H 'accept: application/json;version=10.0.0.0-alpha'
-H 'authorization: Bearer $access_token'
-H 'content-type: application/json'
-d '{
  "name":"vmApp-org-1",
  "displayName":"vmApp-org-1 tenant",
  "description":"Additional VM App Org Tenant",
  "isEnabled":true,
  "isClassicTenant":true
}'
```

Infrastructure Compute Policies

Infrastructure policies define requirements and constraints for resource placement and configuration across the regions and zones that are allocated to your organization. At least one compatible policy must be available to each namespace.

Service providers can list, create, and delete policies with the Policies operations. The REST call `Get Assignments` returns a paginated list of policy assignments, which can be filtered by policy IDs, target names, policy types, project ID, or assignment phase. The following `Create Policy` request body sets compute policy for higher performance virtual machines.

```
{
  "name": "High Performance VM Policy",
  "description": "Compute policy for high performance virtual machines",
  "vcComputePolicyName": "HighPerformanceVMPolicy",
  "isMandatory": true,
  "policyRules": [
    { "key": "cpu.cores", "operator": ">=", "values": ["4"] },
    { "key": "vm.class", "operator": "in", "values": ["guaranteed-4xlarge", "guaranteed-8xlarge"] }
  ]
}
```

When a role or object refers to a policy that is subsequently removed with `Delete Policy`, the policy does not go into effect.

Content Library

Somewhat underutilized, the Content Library was introduced long ago in vSphere 6. Content libraries can contain ISO images, OVF files, VM templates, vApps, scripts, applications, supervisors, and other resources. A populated Content Library is intended for sharing across an enterprise, helping reduce storage space and administrator overhead.

It is a good use of automation to refresh and share a pre-existing Content Library.

To determine what Content Libraries are accessible to an organization, run this command:

```
curl -X GET https://vcfa.vcf.example.com/cloudapi/vcf/contentLibraries?page=1&pageSize=128
-H 'Authorization: <value>'
```

To refresh the accessible copy of a distributed catalog, run this command, where `cat.vcf.example.com` is the URN of the accessible catalog:

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/1.0.0/catalogs/cat.vcf.example.com/refresh
-H 'Authorization: <value>'
```

Consumers of provider services will be able to access the refreshed Content Library with the all-apps REST command **Get Catalog Items**. All-apps consumers can also publish, republish, and unpublish existing catalog items.

Blueprints

In addition to the content library, tenants can create their own blueprints. If you have a pre-existing organizational blueprint, you can enable it for a tenant.

The VCF Automation user interface offers a drag-and-drop method to create blueprints.

Identity Management

VMware Cloud Foundation 9 has unified identity management that permits single sign-on over a wider range of assets than previously, and offering a decent choice of identity providers.

To help run scheduled updates, the following command begins the LDAP sync task for the identity manager (IDM).

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/1.0.0/ldap/sync
-H 'Authorization: <value>'
```

Suppose certain users in an organization prefer to use LDAP instead of SAML authentication. This can be set with the `providerType` argument of the **Update User** REST command, or the similar **Update Group** REST command. IDM choices are: Local, LDAP, SAML, and OAuth.

You can choose from modern identity providers Okta, Ping ID, Entra ID, ADFS, and SAML, or from directory-based identity providers AD/LDAP and OpenLADP.

Custom Branding

For a unique look and feel, interface branding may be applied for all tenants of a Provider service.

The VCF Automation user interface (UI) has a Branding page where you can see resources such as logo, title, and background. Settings can then be assigned as a custom theme to organization pages either shared or private. For details, see topic "Changing the VCF Automation Portal Branding" in the VCF *Provider Management* guide.

An API for `brandingThemes` is also available. To see available themes for an organization, run this command:

```
curl -X GET https://vcfa.example.com/cloudapi/vcf/brandingThemes?page=1&pageSize=128
-H 'Authorization: <value>'
```

To create a branding theme with the API requires two calls. The first call changes the portal branding name but the branding object remains busy while waiting for the resource Zip to upload. After the second call completes, the branding theme becomes usable again.

```
https://vcfa.example.com/cloudapi/vcf/brandingThemes/urn:vcloud:entity:broadcom:theme:UUID/resources
https://vcfa.example.com/transfer/UUID/resources.zip
```

An easier option is to run a VCF Operations Orchestrator workflow (formerly vRO) to create and install branding themes. Visit [this Broadcom developer Tools webpage](#) to download a generic branding workflow that you can modify and run.

To set an organization branding theme, a sample Python script follows the script outline. You will have to change specific values, such as URL and user credentials.

1. Before you run the script, create a `resources.zip` file containing `background.png`, `favicon.ico`, `styles.css`, and `customLinks.json`, for both light and dark themes.
2. In the Python script, get an access token by calling `cloudapi sessions/provider`.
3. Define the URL of an API endpoint that can receive the `resources.zip` file.
4. Upload the `resources.zip` file using an HTTP PUT request.

```
import requests
import os

# 1. Get an access token
print("__Step 1__")
url = 'https://vcfa.example.com/cloudapi/1.0.0/sessions/provider'
custom_headers = {
    'Accept': 'application/json;version=41.0.0-alpha',
    'Content-Type': 'application/json',
}
username = 'admin@system'
password = 'Super!Secret9'
response = requests.post(url, verify=False, auth=(username, password), headers=custom_headers)
print(url)
if response.status_code == 200:
    # Convert the JSON response body to a Python dictionary/list
    user_data = response.json()
    print("Request successful. User data:")
```

```

    # Use print for a readable output of the dictionary, print(user_data)
    YourAccessToken = response.headers.get('x-vmware-vcloud-access-token')
    print(YourAccessToken)
else:
    print(f"Error: Request failed with status code {response.status_code}")

# 2. Define the URL of the API endpoint that will receive the file
print("__Step 2__")
url = 'https://vcfa.example.com/cloudapi/vcf/brandingThemes'
file_path = 'resources.zip'
custom_headers = {
    'Accept': 'application/json;version=41.0.0-alpha',
    'Content-Type': 'application/json',
    'Authorization': f"Bearer {YourAccessToken}"
}
response = requests.get(url, verify=False, headers=custom_headers)
print(url)
if response.status_code == 200:
    # Convert the JSON response body to a Python dictionary/list
    user_data = response.json()
    print("Request successful. User data:")
    # Use print for a readable output of the dictionary, print(user_data)
else:
    print(f"Error: Request failed with status code {response.status_code}")

# 3. Specify path and contents of your Zip file
print("__Step 3__")
url = 'https://vcfa.example.com/cloudapi/vcf/brandingThemes/urn:vcloud:entity:example:theme:5b11f1ca-40e9-4ab9-aad6-cb7c9c62ec4f/resources'
payload = {
    "fileName": "resources.zip",
    "size": 317455,
    "logo": "logo.png",
    "favicon": "favicon.ico",
    "background": "background.png",
    "styles": "styles.css",
    "customLinks": "customLinks.json",
    "portalName": "test233"
}
response = requests.put(url, verify=False, headers=custom_headers, json=payload)
print(url)
if response.status_code <= 202:
    # Convert the JSON response body to a Python dictionary/list
    # user_data = response.json()
    Link = response.headers.get('link')
    print("Request successful.");
    # Use print for a readable output of the dictionary
    url = Link.split(";")[0]
    url = url[1:-1]
    print(url)
else:
    print(f"Error: Request failed with status code {response.status_code}")

```



```
# 4. Upload Zip file
print("__Step 4__")
print(url)
custom_headers = {
    'Accept': 'application/json;version=41.0.0-alpha',
    'Content-Type': 'application/zip',
    'Authorization': f"Bearer {YourAccessToken}"
    #'Accept-Encoding': 'gzip, deflate, br, zstd'
}
def upload_file_put(url, file_path, content_type='application/octet-stream'):
    """
    Uploads a file using an HTTP PUT request.
    Args:
        url (str): The target URL for the PUT request.
        file_path (str): The path to the file to upload.
        content_type (str): The Content-Type of the file (e.g., 'image/jpeg').
    """
    # Open the file in binary read mode
    with open(file_path, 'rb') as f:
        # Set the headers, specifically the Content-Type
        headers = {'Content-Type': content_type}
        # Send the PUT request with the file data in the 'data' parameter
        response = requests.put(url, verify=False, data=f, headers=headers)
        # Check the response
        if response.status_code in [200, 201, 204]:
            print(f"File uploaded successfully! Status code: {response.status_code}")
            print(response.text)
        else:
            print(f"File upload failed. Status code: {response.status_code}")
            print(response.text)
    file_mime_type = 'application/pdf'
    upload_file_put(url, file_path, file_mime_type)
print("Done!")
```

Metadata Management

As of VCF 9.1, administrators can manage metadata for both provider and tenant resources. You can locate resources by searching for related metadata, with role-based access control permissions.

Additionally, legacy VMware Cloud Director (VCD) entities such as a virtual data center (VDC) can transfer metadata to VCF Automation, as shown in the following table.

VCD to VCFA Resource	Migration Method	Comments
Organization to Organization	Migrate by VcdOrgId	Metadata writable by Provider but tenants lack access to Organization.
Provider VDC to Region	Migrate provider VDC	Provider only.
Organization VDC to Region Quota	Migrate organization VDC	Metadata writable by Provider but tenants lack access to Region Quota.
vApp to Namespace	Transfer vApp into Namespace	Provider metadata migrated to RegionQuota. Tenant metadata migrated to Namespace.

VCD to VCFA Resource	Migration Method	Comments
VM to VM Service VM	Transfer VM into VM Service	Provider metadata migrated to RegionQuota. Tenant metadata migrated to VM.

The metadata REST calls for the service provider follow the same structure as for defined entities. Reference documentation is available in the API explorer online help.

Retrieve all metadata for the {id} designated entity:

```
GET https://{vcfa}/1.0.0/orgs/{id}/metadata
```

Create a new metadata entry in the {id} designated entity:

```
POST https://{vcfa}/1.0.0/orgs/{id}/metadata
```

Replace all metadata entries of the organization with this desired state:

```
PUT https://{vcfa}/1.0.0/orgs/{id}/metadata
```

Retrieve a single metadata entry in the {id} designated entity:

```
GET https://{vcfa}/1.0.0/orgs/{id}/metadata/{metadataID}
```

Update the value of a single key-value metadata entry:

```
PUT https://{vcfa}/1.0.0/orgs/{id}/metadata/{metadataID}
```

Remove a single metadata entry:

```
DELETE https://{vcfa}/1.0.0/orgs/{id}/metadata/{metadataID}
```

Generating API Tokens for Service Provider Account

You can sign into the service provider account in the VCF Automation UI to generate a refresh token and, from that, generate an access token to authorize API calls.

REST calls require OAuth style HTTP authentication in request headers. The access token is a one-hour expiring token used to authenticate all API calls, including those made from a Provider management account. To acquire an access token, you first generate a refresh token using an account in the VCF Automation interface, then you generate an access token using the token API.

1. Go to the VCF Automation administrator interface at a URL such as `vcfa9.example.com/provider`. Enter the organization name, such as `system`, and click `Continue`. If necessary, switch from `LDAP` to `Local authentication`.
2. Log in with the service administrator's username and password.
3. In the upper right corner, click **v** (down arrow) > `My Account`.
4. On the `My Account` page, click `API Tokens` > `NEW`.
5. In the `Create General Token` box, type the "client" administrator's user name, the one you just logged in with, and click `CREATE`.

Make sure to copy and paste the resulting Refresh token, which looks somewhat like this:

```
XhWKaESdLzr0Uo7Nd1JBn2AJZvAC0g07.
```

6. Run the following Curl command to get the device code, user code, and verification URL. Supply the FQDN of the VCF Automation appliance, the organization name that you entered in step 1, and the service administrator's user name.

```
curl -k 'https://{vcfa.url}/tm/oauth/tenant/<organization>/device_authorization' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'client_id=<serviceAdminUserName>'
```

7. Curl returns a response body with `device_code`, `user_code`, etc. The interval indicates frequency of verification; `expires_in` indicates duration of the access token in seconds. Copy and save the `user_code` and `device_code` for verification in steps below.

```
{
  "device_code": "qA9Ak5Jc130cw_UkXDUOA93WJnh-fU7btAGfInL-A0",
  "user_code": "QSD1-9B64",
  "verification_uri": "https://vcfa.10.163.14.191.vcfd.broadcom.net/automation/#/infrastructure/access",
  "expires_in": 3600,
  "interval": 60
}
```

8. On the Service Accounts tab, click Review Access Requests. Paste the `user_code` that you copied from the response body. Click the LOOKUP button and then the GRANT button to approve the access request.
9. Using the Refresh token that you saved in step 5 above, and the `device_code` that you save in step 7, run another Curl command to generate an access token.

```
curl -k 'https://<vcfa.url>/tm/oauth/tenant/<organization>/token' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=urn:ietf:params:oauth:grant-type:device_code' \
--data-urlencode 'refresh_token=<refresh_token>' --data-urlencode 'client_id=<username>' \
--data-urlencode 'device_code=<device_code>'
```

10. The response body included the `access_token`, which is very long. Additional lines include one-hour expiration and the Refresh token recapitulated.

```
{
  "access_token":
    "About 18 lines of letters and numbers..."
  "token_type": "Bearer",
  "expires_in": "3600",
  "refresh_token": "XhWKaESdLzr0Uo7Nd1JBn2AJZvAC0g07"
}
```

After the access token expires, there will be a different grant type (`refresh_token`) for getting another access token. Here is the Curl command to acquire a new access token after it expires.

```
curl --location 'https://<vcfa.url>/oauth/tenant/<org>/token' \
--header 'Accept: application/*' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=refresh_token' \
--data-urlencode 'refresh_token=<refreshToken>'
```

If the service provider account was created without rotation required, you can use the same refresh token to get new access tokens until the refresh token itself expires. If rotation is activated, when you make the `refresh_token` grant call, the response body will contain the new access token as well as a new refresh token that you should retain (the original will be invalidated).

Automation Portal for Organization Management

In VCF Automation, the Provider creates services for tenant organizations so they can run virtual machines, develop operations software and microservices using containers, and perform other tasks in a shared computing environment.

Kubernetes and REST commands help automate the creation of projects, supervisor namespaces, virtual machines, content library items, and so forth. Sections below describe some actions that can be performed with the `kubectl` command line and REST calls.

Administrators of tenant organizations can assign regions for secure boundaries, namespace class for how to allocate resources, and virtual private clouds (VPC) for network connections that protect regional isolation. Blueprints can be

created so DevOps programmers can easily create an appropriate self-service namespace without having to create to complicated YAML descriptions.

VCF CLI and Kubernetes Setup

The `vcf` command line interface is required to set up the VCF Automation context, cluster plug-in, and cluster authentication. You can find it for download on the overview page of the VCF Automation UI. The `vcf` command is also available in various Linux package managers, Homebrew for MacOS, and Chocolatey for Windows.

For example, to download and install on MacOS, type these commands:

```
% brew tap vmware/homebrew-vcfcli
% brew install vcf-cli
```

If you don't already have Kubernetes control, it too is available using Linux package managers, Homebrew for MacOS, or Chocolatey for Windows. For example, to download and install on MacOS, type this command:

```
% brew install kubectl
```

Obtain API Token using the UI

The VCF Automation service Provider will have created an account for your tenant Organization.

After you sign in, click the $\vee$ down arrow in the upper right corner, then click My Account > API Tokens > New. Where it says Client Name, click the tenant account name you signed in with.

The token will look something like this: FIH7L3SXiGAqYZTW784ZcTjQsnvhLIP4.

Initial Setup of Kubernetes Cluster

Access to Kubernetes requires the token obtained above, a certificate for the VCF Automation appliance, and various designations for namespace and project.

Follow these steps to set up the cluster:

1. Fetch the VCF Automation certificate and send it to a PEM file.

```
openssl s_client -connect fqdn:443 -showcerts </dev/null 2>/dev/null | openssl x509 -outform PEM > vc-
fa-ca-cert.pem
```

2. Create the VCF Automation context.

```
vcf context create cci -e fqdn -t cci --api-token <VCF_CLI_API_TOKEN> --tenant-name tenant --ca-certifi-
cate vcfa-ca-cert.pem
Provide Tenant Name: tenant
```

3. Use the VCF Automation context.

```
vcf context use cci:namespaceName:projectName
```

4. Install the VCF CLI cluster plugin.

```
vcf plugin install cluster
```

5. Register the Kubernetes cluster JWT authenticator.

6. `vcf cluster register vcfa-jwt-authenticator`

7. Generate the Kubernetes cluster configuration.

```
vcf cluster kubeconfig get resource.cluster.object.metadata.name --export-file resource.cluster.objec-
t.metadata.name-kubeconfig
```

Follow these steps to access the cluster:

1. Set environment variable `VCF_CLI_VCFA_API_TOKEN` with the API token to circumvent interactive mode (avoid having to copy-paste it repeatedly).

```
export VCF_CLI_VCFA_API_TOKEN=FlH7L3SXIGaQYZTW784ZcTjQsnvhLlP4
```

2. Run the Kubernetes control command.

```
kubectl --kubeconfig resource.cluster.object.metadata.name get namespaces
```

Note:

Project name should comply with Kubernetes standards. Administrators can change a project name, while the underlying project ID remains the same.

Creating and Managing Namespaces

This topic shows the YAML for some `kubectl` create commands to manage namespaces. Using the Tenant portal UI, organization users can also create a namespace in the Provider dialog.

Before you create a namespace, a namespace class is required. You could create a custom namespace class, or use one of the pre-packaged namespace classes: small, medium, or large.

Here is an example that creates a namespace, in the case when the Supervisor Namespace Class Config (e2e-small) contains a Zones array section:

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespace
metadata:
  generateName: test
  namespace: default-project
spec:
  regionName: e2e-region
  className: e2e-small
  vpcName: e2e-region-default-vpc
```

Here is the corresponding namespace class config with the Zones specification. Note that Zones must be set using REST, not Kubernetes.

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespaceClassConfig
metadata:
  name: e2e-small
spec:
  defaultLimits:
    cpuLimit: 1000000M
    cpuReservation: 1000M
    memoryLimit: 25000Mi
    memoryReservation: 1000Mi
  storageClasses:
    - limit: 20480Mi
      name: '*'
  vmClasses:
    - name: '*'
  zones:
    - cpuLimit: 100000M
      cpuReservation: 100M
      memoryLimit: 2500Mi
      memoryReservation: 100Mi
```

```
name: vcfa-e2e-wl-vc04-cl1-zone1
```

If there are no Zones in the Supervisor Namespace Class Config, here is an example with override:

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespace
metadata:
  generateName: test
  namespace: default-project
spec:
  regionName: e2e-region
  className: e2e-small
  vpcName: e2e-region-default-vpc
  initialClassConfigOverrides:
    zones:
    - cpuLimit: 100000M
      cpuReservation: 100M
      memoryLimit: 2500Mi
      memoryReservation: 100Mi
      name: vcfa-e2e-wl-vc04-cl1-zone1
```

The Region is set by the Service Provider administrator, and is read-only for the organization tenant, who can see it described.

```
% kubectl describe region
Name:          e2e-region
Namespace:
Labels:        <none>
Annotations:   <none>
API Version:   topology.cci.vmware.com/v1alpha2
Kind:          Region
Metadata:
  UID:         f3c633b6-2014-4f65-8736-281ef8cc9a0d
Status:
  Load Balancer Type:  NSX_LB
Events:             <none>
```

Likewise, the Zone is set by the Service Provider administrator, and is read-only for the organization tenant, who can see it described.

```
% kubectl describe zones vcfa-e2e-wl-vc04-cl1-zone1
Name:          dd487fed-b8f1-5696-812a-772a4a607453
Namespace:
Labels:        <none>
Annotations:   <none>
API Version:   topology.cci.vmware.com/v1alpha1
Kind:          Zone
Metadata:
  UID:         dd487fed-b8f1-5696-812a-772a4a607453
Spec:
  Cpu Limit:          1000000M
  Cpu Reservation:    10000M
  Memory Limit:       250000Mi
  Memory Reservation: 20000Mi
  Region Name:        e2e-region
```

```

Zone Name:          vcfa-e2e-w1-vc04-cl1-zone1
Status:
  Cpu Reservation Used:    200M
  Cpu Used:                201000M
  Memory Reservation Used: 200Mi
  Memory Used:             7500Mi
Events:                 <none>

```

Developer Self-Service with Virtual Machine Groups

As organization tenants, DevOps programmers can create a virtual machine group, which is useful to associate machines that can be managed together. For example, a multi-machine application might have a management back end, a dedicated database, and a web interface front end.

This functionality is similar to the traditional vApp in vSphere.

By creating a virtual machine group, you can perform operations on a set of VMs at an appropriate time. For instance, you can define the order of power operations so that your user interface does not start prematurely. You can power off all VMs to synchronize backup of the virtual machine group. You can power off an individual VM without affecting the power state of other VMs in the group.

You may add existing or new VMs to a VM group. You may create a VM group at the same time as its constituent VMs. VM groups may include other VM groups. The virtual machine group must have a namespace, it should contain at least two virtual machines, and the VM Service must be available.

In the YAML definition, a VM group is defined as kind (type) `VirtualMachineGroup`. Affinity and anti-affinity rules are defined in kind `VirtualMachineAffinity`. The snapshot details of a virtual machine group for backup, or other purposes, is defined in kind `VirtualMachineSnapshotSpec`.

A new VIM task `syncVolume` (or `CnsSyncVolume`) was implemented in VCF 9.1 to synchronize the container native storage (CNS) volumes used for storage by the VM group. It gets called behind the scenes during desired state operations.

Region VM Class Quotas

Regions are associated with quotas for resource use (CPU, memory, storage), with VM classes based on size, and with storage classes based on capacity. CPU and memory have both a maximum limit and a minimum reserve allocation.

You can run Kubernetes commands to determine a region's VM class summaries, VM class quotas, and storage class quotas.

```

% kubectl get regions
NAME
paif-region
% kubectl get regionvirtualmachineclasssummaries --field-selector spec.regionName=paif-region
% kubectl get regionvirtualmachineclassquotas --field-selector spec.regionName=paif-region
% kubectl get regionstorageclassquotas --field-selector spec.regionName=paif-region

```

Networking for All Apps API

Software defined networking with VMware NSX offers a way to increase security and isolation of clients with VCF Automation All Apps tenancy.

VCF Automation involves three contexts: the Console context of VCF, the Supervisor context created with vCenter, and the Kubernetes context for Supervisor clusters. Network static routes and subnets are defined as "kinds" of custom resource definitions (CRD) in the Supervisor context. These are the important ones.

- `networkinfos.crd.nsx.vmware.com`
NetworkInfo is used to report the network information for a namespace.
- `subnets.crd.nsx.vmware.com`
Subnet is the resource schema for the subnets API. SubnetPortSpec defines the desired state of SubnetPort. SubnetPortStatus defines the observed state of SubnetPort.
- `subnetsets.crd.nsx.vmware.com`
SubnetSet is the resource schema for the subnetsets API. SubnetSetSpec defines the desired state of SubnetSet. SubnetSetStatus defines the observed state of SubnetSet.
- `staticroutes.crd.nsx.vmware.com`
StaticRoute is the resource schema for the staticroutes API. StaticRouteCondition defines the condition of StaticRoute. StaticRouteSpec defines static routes configuration on the virtual private cloud. StaticRouteStatus defines the observed state of StaticRoute.

Virtual private cloud information in VCF is obtained from the NetworkInfo custom resource, which is read-only. This example shows a Kubernetes command retrieving network information so that subnets and static routes can later be applied to this network.

```
# kubectl get networkinfo -o yaml
apiVersion: v1
items:
- apiVersion: crd.nsx.example.com/v1alpha1
  kind: NetworkInfo
  metadata:
    creationTimestamp: "2024-11-04T08:11:13Z"
    generation: 2
    name: svc-tkg-domain-c10
    namespace: svc-tkg-domain-c10
    resourceVersion: "10279"
    uid: 93ee3688-d6f1-477c-9914-18bdab05b2db
  vpcs:
  - defaultSNATIP: 192.168.0.4
    name: vmware-system-supervisor-services-vpc_ef56697a-4a16-4090-8299-6cdb48caab05
    privateIPs:
    - 172.17.0.0/16
```

This example shows a Kubernetes command creating a private subnet named `subnet-sample` served by DHCP with 64 nodes.

```
$ cat <<EOF | kubectl create -f -
apiVersion: crd.nsx.example.com/v1alpha1
kind: Subnet
metadata:
  name: subnet-sample
spec:
  accessMode: Private
  ipv4SubnetSize: 64
  subnetDHCPConfig:
    mode: DHCPServer
EOF
```

Once subnets are created, they can be organized into a subnetSet.

This example shows a Kubernetes command creating a private subnet set named `subnetset-sample` with size 64.

```
$ cat <<EOF | kubectl create -f -
```



```

apiVersion: crd.nsx.example.com/v1alpha1
kind: SubnetSet
metadata:
  name: subnetset-sample
spec:
  accessMode: Private
  ipv4SubnetSize: 64
EOF

```

This example shows a Kubernetes command creating a 192 private network with static routing through 10.10.0.1 or 10.10.02.

```

$ cat <<EOF | kubectl create -f -
apiVersion: crd.nsx.example.com/v1alpha1
kind: StaticRoute
metadata:
  name: route1
spec:
  network: 192.168.2.0/24
  nextHops:
  - ipAddress: 10.10.0.2
  - ipAddress: 10.10.0.1
EOF

```

Help and Support for VCF SDKs, APIs, and VCF PowerCLI

This guide outlines the available resources and procedures for troubleshooting and obtaining technical assistance for VCF SDKs, APIs, and VCF PowerCLI cmdlets.

Self-Help Resources

For Self-Help use the available documentation and community resources to search for a solution to your issue. The following resources are recommended:

- [Broadcom Developer Portal](#) : Access official technical documentation, VCF SDKs and API references, VCF PowerCLI cmdlets and integration guides.
- [VCF Code Community](#): Engage with peer experts and engineers to discuss use cases and community-driven solutions.

Official Technical Support

If an issue cannot be resolved through self-help, customers with a valid Broadcom VCF support contract can open a formal support case.

Our specialized VCF API/SDK Support Team provides comprehensive assistance across the following areas:

- **Installation and Setup** - Installing and troubleshooting issues for VCF SDKs, APIs, or VCF PowerCLI modules.
- **Functional Issues** - Investigating and resolving instances where VCF SDKs, APIs, or VCF PowerCLI cmdlets do not perform according to documentation.
- **Architectural Guidance** - Offering design advice on VCF SDKs/APIs to ensure scalable integrations, best practices, and alignment with the lifecycle of the components.
- **Performance Optimization** - Analyzing execution logic and VCF API call patterns to identify bottlenecks, reduce latency, and improve resource efficiency in scripts/workflows.

- Use Case Clarification - Providing architectural guidance and clarifying the supported scope for specific automation tasks.
- Scripting Support - Troubleshooting and assistance for custom or sample scripts is limited to those using VCF SDKs, APIs, or VCF cmdlets.
- Bug Reporting - Processing and escalating verified defects in VCF SDKs/APIs/VCF cmdlets to engineering teams.

Filing a Support Case

Note:

The VCF API, SDK, and VCF PowerCLI are not listed as standalone product selections in the Broadcom Support Portal.

When filing a case, the component should be set to the primary product in use, and then the sub-component should be set to API/SDK Software Development Kit (API, Automation).

You can find more information on how to engage with the VCF API/SDK Support Team in the Broadcom knowledge base article: [436447](#).

Program Information

For more information regarding the VCF API/SDK support program and scope of service visit:

- [VMware® SDKs and APIs Solution Brief](#)
- [Supported Product Matrix for VCF SDK/API](#)
- [Broadcom Developer Portal](#)

Advanced Services

You can extend your platform with add-on functionality from advanced services. The following advanced services are available for VCF and vSphere Foundation.

Add-Ons	Supported Platform Types	Description
Protection and Recovery	- VCF - vSphere Foundation	Use protection and recovery to build powerful cyber and data resiliency for VCF. It provides ransomware and disaster recovery in a unified management experience, accelerated recovery, and simplified consumption with flexible licensing across use cases and clouds. Protection and recovery for VCF Automation provides protection and recovery for regular and multi-tenant providers that want to replicate workload VMs between a pair of Supervisors, within two region quotas, in the same VCF Automation organization.
VMware Data Services Manager	VCF	Use VMware Data Services Manager to build enterprise-hardened data services through a purpose-built solution for VCF with high availability, performance, backup, and monitoring.
VMware® vDefend™	VCF	Use VMware vDefend for built-in zone and application microsegmentation, threat detection and response, lateral security, ransomware and APT protection, reduced attack surface, and zero trust enforcement across VCF environments.
VMware® Avi™ Load Balancer	- VCF - vSphere Foundation	VMware Avi Load Balancer is the industry's first and only software-defined platform deeply integrated with VCF private cloud, delivering local and global load balancing, Kubernetes ingress, web application and API security and application analytics.
VMware Tanzu® Platform VMware Tanzu® Data	- VCF - vSphere Foundation	- Use VMware Tanzu Platform to accelerate ideas to code and code to production by simplifying and integrating the processes and tools used by developers and IT operations. - VMware Tanzu Data to maximize your data with solutions built to store, process, query, and analyze critical data resources in real-time and at a massive scale.
Network Observability	VCF	Network Observability by Broadcom extends the VCF management capabilities with end-to-end visibility into the network overlay, underlay, and applications hosted in a public or hybrid cloud. The solution provides actionable insights to automatically pinpoint problems, resulting in lower internal resource costs and faster issue resolution, quickly getting end-users back to revenue-producing activities.

Documentation Legal Notice

This Documentation, which includes embedded help systems and electronically distributed materials, (hereinafter referred to as the “Documentation”) is for your informational purposes only and is subject to change or withdrawal by Broadcom at any time. This Documentation is proprietary information of Broadcom and may not be copied, transferred, reproduced, disclosed, modified or duplicated, in whole or in part, without the prior written consent of Broadcom.

If you are a licensed user of the software product(s) addressed in the Documentation, you may print or otherwise make available a reasonable number of copies of the Documentation for internal use by you and your employees in connection with that software, provided that all Broadcom copyright notices and legends are affixed to each reproduced copy.

The right to print or otherwise make available copies of the Documentation is limited to the period during which the applicable license for such software remains in full force and effect. Should the license terminate for any reason, it is your responsibility to certify in writing to Broadcom that all copies and partial copies of the Documentation have been returned to Broadcom or destroyed.

TO THE EXTENT PERMITTED BY APPLICABLE LAW, BROADCOM PROVIDES THIS DOCUMENTATION “AS IS” WITHOUT WARRANTY OF ANY KIND, INCLUDING WITHOUT LIMITATION, ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NONINFRINGEMENT. IN NO EVENT WILL BROADCOM BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY LOSS OR DAMAGE, DIRECT OR INDIRECT, FROM THE USE OF THIS DOCUMENTATION, INCLUDING WITHOUT LIMITATION, LOST PROFITS, LOST INVESTMENT, BUSINESS INTERRUPTION, GOODWILL, OR LOST DATA, EVEN IF BROADCOM IS EXPRESSLY ADVISED IN ADVANCE OF THE POSSIBILITY OF SUCH LOSS OR DAMAGE.

The use of any software product referenced in the Documentation is governed by the applicable license agreement and such license agreement is not modified in any way by the terms of this notice

The manufacturer of this Documentation is Broadcom Inc.

Provided with “Restricted Rights.” Use, duplication or disclosure by the United States Government is subject to the restrictions set forth in FAR Sections 12.212, 52.227-14, and 52.227-19(c)(1) - (2) and DFARS Section 252.227-7014(b) (3), as applicable, or their successors.

Copyright © 2005–2025 Broadcom. All Rights Reserved. The term “Broadcom” refers to Broadcom Inc. and/or its subsidiaries. All trademarks, trade names, service marks, and logos referenced herein belong to their respective companies.

